

MODULE 1 TOFA USER & ADMIN MODULE

TOFA USER & ADMIN MODULE

Authentication, User Management & Admin Creation

This module covers:

- User authentication
- User account lifecycle
- Email verification
- Password management
- Referral system
- User profile data
- Admin creation
- Account status management

1. USER AUTHENTICATION METHODS

Users can authenticate using:

1. Email & Password
2. Google OAuth

2. AUTHENTICATION FLOW

EMAIL SIGNUP FLOW

User submits signup form

↓

Account created

↓

OTP sent to email

↓

User verifies email

↓

Account becomes ACTIVE

GOOGLE SIGNUP FLOW

User clicks Continue with Google

↓

Google authenticates user
↓
Account created automatically
↓
User redirected to platform
↓
Terms of Use modal appears
↓
User must accept Terms
↓
Account becomes ACTIVE

3. USER TABLE STRUCTURE

Table: **users**

Field	Type	Notes
id	uuid	primary key
firstName	varchar	
lastName	varchar	
email	varchar	unique
phoneNumber	varchar	
passwordHash	varchar	nullable for Google users
termsOfUse	boolean	
merchantTerms	boolean	
companyName	varchar	
regNumber	varchar	
storeName	varchar	
companyAddress	varchar	
pickUpAddress	varchar	

yearsOfBusiness	integer	
businessType	varchar	
deliveryAddress	varchar	
userType	enum	
isEmailVerified	boolean	
isCompanyVerified	boolean	
country	varchar	
companyLogo	varchar	
companyBio	text	
selectedLanguage	varchar	
totalReviewCount	integer	
totalAverageReviews	decimal	
totalPoints	integer	reward system
referral	varchar	referral code used
referralCode	varchar	user's own referral code
createdAt	timestamp	
updatedAt	timestamp	
status	enum	

USER TYPE ENUM

buyer
seller

A user can later **upgrade from buyer** → **seller**.

USER STATUS ENUM

active
inactive
disabled
deleted

STATUS RULES

Status	Meaning
active	Email verified, full access
inactive	Email not verified
disabled	User blocked by admin
deleted	User deleted account

Important Notes:

- **inactive users cannot login**
- **disabled users can login but cannot perform actions**
- **deleted users cannot access account**

4. SIGN UP

Endpoint

POST /auth/signup

Payload

```
{  
  "email": "user@email.com",  
  "password": "Password123",  
  "referralCode": "ABC123",  
  "termsOfUse": true  
}
```

Validation Rules

Rule	Description
Email	must be unique
Password	minimum 8 characters
termsOfUse	must be true
referralCode	optional

Backend Processing

1. Validate payload
2. Check email uniqueness
3. Hash password
4. Generate referralCode for user
5. Store referral relationship if referralCode provided
6. Generate OTP
7. Send verification email
8. Generate JWT token

Signup Response

```
{  
  "token": "JWT_TOKEN",  
  "user": {  
    "id": "usr_921",  
    "email": "user@email.com",  
    "status": "inactive",  
    "isEmailVerified": false  
  }  
}
```

}

Note:

Token is returned so **frontend can use it when verifying OTP**.

EMAIL SENT AFTER SIGNUP

Subject:

Verify Your Email - Traders of Africa

Email Template

Hello {FirstName},

Welcome to Traders of Africa!

To complete your account registration, please verify your email using the One-Time Password (OTP) below.

Your OTP Code:

{OTP}

This code will expire in 10 minutes.

If you did not create this account, please ignore this email.

Best Regards,
Traders of Africa Team

5. VERIFY EMAIL

Endpoint

POST /auth/verify-email

Payload

```
{  
  "token": "JWT_TOKEN",  
  "otp": "123456"  
}
```

Backend Processing

1. Decode token
2. Retrieve user
3. Validate OTP
4. Check expiry (10 minutes)
5. Update:

```
isEmailVerified = true  
status = active
```

Response

```
{  
  "message": "Email verified successfully",  
  "status": "active"  
}
```

6. GOOGLE SIGNUP FLOW

Flow

User clicks Continue with Google

↓

Google authenticates user

↓

Platform receives Google profile

↓

Account created

↓

Terms of Use modal shown

↓

User must accept terms

GOOGLE AUTH ENDPOINT

POST /auth/google

Payload

```
{  
  "googleToken": "GOOGLE_ID_TOKEN"  
}
```

Backend Processing

1. Verify Google token
2. Extract email
3. Check if user exists
4. If not → create account
5. Generate referralCode

6. Return user token

Response

```
{
  "token": "JWT_TOKEN",
  "user": {
    "id": "usr_992",
    "email": "user@gmail.com",
    "termsOfUse": false
  }
}
```

Frontend must show **Terms Modal**.

ACCEPT TERMS

Endpoint

PATCH /auth/update-terms/:userId

Payload

```
{
  "termsOfUse": true
}
```

Response

```
{
  "message": "Terms accepted successfully"
}
```

7. LOGIN

Endpoint

POST /auth/login

Payload

```
{  
  "email": "user@email.com",  
  "password": "Password123"  
}
```

Special Rule

If:

password correct
email correct
but email not verified

Then:

OTP sent to email
user redirected to verify email

Response

```
{  
  "token": "JWT_TOKEN",  
  "requiresEmailVerification": true  
}
```

LOGIN SUCCESS RESPONSE

```
{  
  "token": "JWT_TOKEN",  
  "user": {  
    "id": "usr_112",  
    "firstName": "Victor",  
    "lastName": "Ejiogu",  
    "phoneNumber": "200938362",  
    "email": "victor@email.com",  
    "userType": "seller",  
    "status": "active",
```

```
"isEmailVerified": true,  
"isCompanyVerified": true,  
"totalPoints": 120,  
"companyName": "Victor Agro Export Ltd",  
"storeName": "Victor Agro Export",  
"country": "Nigeria",  
"totalReviewCount": 12,  
"totalAverageReviews": 4.7,  
"selectedLanguage": "en"  
"createdAt": "2026-03-10"  
"deliveryAddress": "Lekki Lagos",  
"pickupAddress": "Lekki Lagos",  
}  
}
```

LOGIN SECURITY RULES

Rule	Value
OTP expiry	10 minutes
Max login attempts	5 attempts
Account lock	15 minutes

8. FORGOT PASSWORD

Endpoint

POST /auth/forgot-password

Payload

```
{  
"email": "user@email.com"  
}
```

Backend Processing

1. Generate OTP
2. Store reset token
3. Send reset email

Email Template

Subject

Password Reset Request

Body

Hello,

We received a request to reset your password.

Your password reset code is:

{OTP}

This code expires in 10 minutes.

If you did not request this, please ignore this email.

Regards,
Traders of Africa

9. RESET PASSWORD

Endpoint

POST /auth/reset-password

Payload

```
{  
  "otp": "123456",  
  "password": "NewPassword123"  
}
```

Response

```
{  
  "message": "Password reset successful"  
}
```

10. CHANGE PASSWORD

Endpoint

PATCH /auth/change-password

Payload

```
{  
  "oldPassword": "OldPass123",  
  "newPassword": "NewPass123"  
}
```

Rules

minimum 8 characters
must match complexity rules

Response

```
{  
  "message": "Password changed successfully"  
}
```

11. UPDATE USER PROFILE

Endpoint

PATCH /users/profile

Payload

```
{  
  "firstName": "Victor",  
  "lastName": "Ejiogu",  
  "phoneNumber": "+2348012345678",  
  "selectedLanguage": "en",  
  "deliveryAddress": "Lekki Lagos",  
  "companyBio": "Leading agro exporter"  
}
```

Backend Processing

Only update **fields provided in payload**.

Response

```
{  
  "message": "Profile updated successfully"  
}
```

12. GET CURRENT USER

This endpoint returns **complete user data** including seller status.

GET CURRENT USER

Endpoint

GET /users/me

Authentication

JWT required

Response

```
{  
  "id": "usr_112",  
  "firstName": "Victor",  
  "lastName": "Ejiogu",  
  "phoneNumber": "200938362",  
  "email": "victor@email.com",  
  "userType": "seller",  
  "status": "active",  
  "isEmailVerified": true,  
  "isCompanyVerified": true,  
  "totalPoints": 120,  
  "companyName": "Victor Agro Export Ltd",  
  "storeName": "Victor Agro Export",  
  "country": "Nigeria",  
  "totalReviewCount": 12,  
  "totalAverageReviews": 4.7,  
  "selectedLanguage": "en",  
  "deliveryAddress": "Lekki Lagos",  
  "createdAt": "2026-03-10"  
}
```

13. UPDATE USER STATUS

Endpoint

PATCH /auth/update-user-status

Payload

```
{  
  "status": "delete",  
  "reason": "I don't need account anymore"  
}
```

Backend Processing

status = deleted

The account is **soft deleted**.

Response

```
{  
  "message": "Account deleted successfully"  
}
```

14. ADMIN USERS

Admins are created **only by Super Admin**.

ADMIN TABLE

Table: **admins**

Field	Type
id	uuid
email	varchar
firstName	varchar
lastName	varchar
phoneNumber	varchar
passwordHash	varchar
role	enum

createdBy	uuid
createdAt	timestamp
status	Enum

ADMIN ROLE ENUM

admin
super_admin
support_agent
finance

CREATE ADMIN

Endpoint

POST /admin/create-admin

Authentication

Super Admin Only

Payload

```
{  
  "firstName": "victor",  
  "lastName": "ejiogu",  
  "phoneNumber": "23",  
  "email": "admin@tofa.com",  
  "password": "AdminPass123",  
  "role": "admin"  
}
```

Response

```
{
```

```
"message": "Admin created successfully"  
}
```

EMAIL SENT WHEN ADMIN CREATED

Subject

Admin Access Granted

Body

Hello,

You have been granted administrative access to the Traders of Africa platform.

You can now log in to the admin dashboard using the credentials provided.

Please ensure your credentials remain secure.

Best Regards,
Traders of Africa

LOGIN ADMIN

Endpoint

POST /admin/login

Authentication

Super Admin Only

Payload

```
{
```

```
"email": "admin@tofa.com",
"password": "AdminPass123",
}
```

Response

```
{
  "message": "Admin created successfully",
  "token": "JWT_TOKEN",
  "admin": {
    "id": "usr_112",
    "firstName": "Victor",
    "lastName": "Ejiogu",
    "phoneNumber": "0706955353",
    "email": "victor@email.com",
    "role": "seller",
    "status": "active",
    "createdAt": "2026-03-10"
  }
}
```

ADMIN DISABLE USER

Endpoint

PATCH /admin/disable-user/:userId

Payload

```
{
  "reason": "Fraudulent activities"
}
```

Response

```
{
  "status": "disabled"
}
```



```
}
```

ADMIN DEACTIVATE ADMIN

Super Admin can deactivate admins.

DEACTIVATE ADMIN

Endpoint

PATCH /admin/deactivate-admin/:adminId

Authentication

Super Admin only

Payload

```
{  
  "reason": "Left the organization"  
}
```

Backend Processing

status = inactive

Response

```
{  
  "message": "Admin deactivated successfully"  
}
```

ADMIN STATUS ENUM

active
inactive
suspended

IMPORTANT NOTES

totalPoints

Used for:

- review reward system
- buyer engagement
- loyalty rewards

Example triggers:

Event	Points
Leave review	+10
Successful trade	+20
Referral signup	+30

SECURITY NOTES

OTP expiration

10 minutes

Password hashing

bcrypt

JWT expiration

24 hours

USER EVENTS (EMAIL TRIGGERS)

Event

Signup

Login unverified

Forgot password

Password reset

Admin disable

Email

Email verification

OTP verification

Reset code

Confirmation

Account restriction notice

MODULE 2 SELLER / BUSINESS VERIFICATION MODULE

SELLER / BUSINESS VERIFICATION MODULE

This module handles:

- upgrading users to sellers
- company information
- business verification
- document submission
- admin approval process

SELLER ONBOARDING FLOW

User registers as buyer

↓

User decides to sell

↓

User upgrades to seller

↓

User submits company information

↓

User uploads verification documents

↓

Admin reviews submission

↓

Admin approves or rejects

↓

Seller becomes verified

DATABASE STRUCTURE

Your **users** table already stores many company fields, but we add a **dedicated verification table**.

TABLE: company_verifications

Field	Type	Notes
id	uuid	
userId	uuid	
companyName	varchar	
registrationNumber	varchar	
businessType	varchar	
yearsOfBusiness	integer	
companyAddress	varchar	
pickupAddress	varchar	
country	varchar	
companyLogo	varchar	
companyBio	text	
verificationStatus	enum	
rejectionReason	text	
submittedAt	timestamp	
reviewedAt	timestamp	
reviewedBy	uuid	

VERIFICATION STATUS ENUM

pending
approved
rejected

SELLER UPGRADE

A user must **explicitly upgrade to a seller**.

UPGRADE USER TO SELLER / SUBMIT COMPANY VERIFICATION

Endpoint

POST /users/upgrade-to-seller

Content-Type

multipart/form-data

Payload

```
{
  "storeName": "Victor Agro Export Ltd",
  "companyName": "Victor Agro Export Ltd",
  "registrationNumber": "RC123456",
  "businessType": "Agriculture",
  "yearsOfBusiness": 5,
  "companyAddress": "Lagos Nigeria",
  "pickupAddress": "Oshodi warehouse",
  "companyBio": "Exporter of premium agricultural commodities",
  "country": "Nigeria"
  "companyLogo" : File
}
```

Backend Processing

1. Validate user exists
2. Create verification record
3. Set verificationStatus
verificationStatus = pending
4. Update userType

userType = seller

5. Save store details

6. Notify admin moderation queue

Response

```
{  
  "message": "Verification submitted successfully",  
  "status": "pending"  
}
```

ADMIN REVIEW SELLER

Admins review verification requests.

GET PENDING SELLERS

Endpoint

GET /admin/seller-verifications

Query

status=pending

page

limit

Response

```
{  
  "data": [  
    {  
      "verificationId": "ver_221",  
      "companyName": "Victor Agro Export",  
      "userId": "usr_112",
```



```
"submittedAt": "2026-03-16"  
}  
]  
}
```

GET PENDING SELLERS BY ID

Endpoint

GET /admin/seller-verifications/:verificationId

Response

```
{  
  "status": "pending",  
  "submittedAt": "2026-03-16T12:00:00Z"  
}
```

APPROVE SELLER

Endpoint

PATCH /admin/approve-seller/:verificationId

Payload

```
{  
  "notes": "Documents verified"  
}
```

Backend Processing

1. Update verificationStatus

approved

2. Update user

isCompanyVerified = true

Response

```
{  
"message": "Seller approved successfully"  
}
```

EMAIL SENT AFTER APPROVAL

Subject

Your Business Has Been Verified

Body

Hello {FirstName},

Congratulations!

Your business has been successfully verified on Traders of Africa.

You can now start listing products, responding to RFQs, and trading with buyers across the marketplace.

We wish you great success on the platform.

Best Regards,
Traders of Africa

REJECT SELLER

Endpoint

PATCH /admin/reject-seller/:verificationId

Payload

```
{  
"reason": "Invalid registration document"  
}
```

Backend Processing

verificationStatus = rejected

EMAIL SENT AFTER REJECTION

Subject

Business Verification Update

Body

Hello {FirstName},

Thank you for submitting your business verification.

Unfortunately, we could not approve your verification at this time for the following reason:

{Reason}

Please update your information and resubmit your verification.

Best Regards,
Traders of Africa

RELATIONSHIP BETWEEN MODULES

This module connects to:

Products
RFQ
Quotes
Orders
Reviews
Messaging
Admin Moderation

Seller verification must be **approved before:**

creating products
responding to RFQs
receiving orders

MODULE 3 PRODUCT MANAGEMENT

3.1 Overview

The Product Management module allows verified sellers to create, manage, publish, update, archive, deactivate, and remove products from the TOFA Marketplace.

Products serve as the primary inventory units that buyers can discover, purchase directly, or use to initiate Requests for Quotations (RFQs).

The module supports both simple products and products with multiple variations while ensuring consistency across inventory, pricing, ordering, and RFQ workflows.

This module integrates with:

- Seller Verification
- Categories
- Cart
- Orders
- RFQs
- Search
- Reviews
- Subscription Ranking
- Analytics

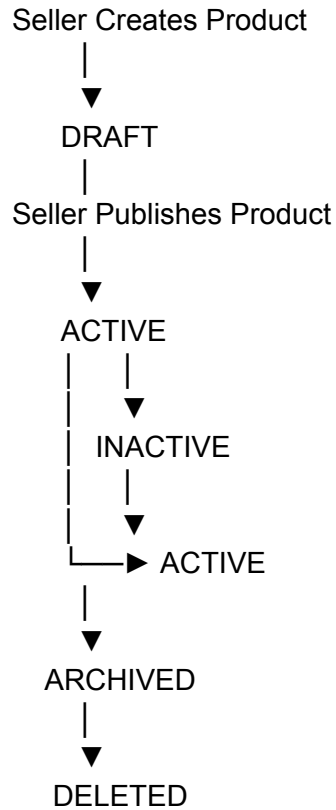
3.2 Business Objectives

The Product Management module is designed to:

- Allow verified sellers to list products.
- Support multiple product categories.
- Support both fixed-price and variable products.
- Allow products to be negotiated through RFQs.
- Support inventory tracking.
- Support multilingual product information.
- Provide a consistent structure for Orders, RFQs, Reviews, and Search.
- Allow sellers to temporarily deactivate products without deleting them.

3.3 Product Lifecycle

Every product follows the lifecycle below.



The backend must validate status transitions rather than accepting arbitrary changes. For example, a deleted → active transition should not be allowed through the normal seller API.

3.4 Product Status Definitions

The agreed product statuses are **draft**, **active**, **inactive**, **archived**, and **deleted**.

These should use lowercase values consistently in the API and database (the headings below use uppercase only for readability).

DRAFT

A draft product is a product that has been created by the seller but has not yet been published.

Characteristics

- Visible only to the seller.
- Fully editable.
- Cannot receive RFQs.
- Cannot receive Orders.
- Not searchable.

Typical Use Cases

- The product is still being completed.
- The seller has not uploaded all required information.
- Seller intends to publish later.

The backend automatically assigns **status: draft** when a new product is created; the frontend should therefore **not send status** when creating a product.

ACTIVE

An active product is publicly available.

Characteristics

- Visible to buyers.
- Searchable.
- Can receive Orders.
- Can receive RFQs.
- Included in the seller storefront.
- Included in recommendations.

INACTIVE

An inactive product is temporarily unavailable.

Characteristics

- Hidden from buyers.
- Cannot receive Orders.
- Cannot receive RFQs.
- Retains all historical information.
- Can be activated again at any time.

Typical Use Cases

- Temporary stock shortage.
- Seasonal availability.

- Seller pauses sales.

ARCHIVED

Archived products are products the seller no longer actively sells but wishes to retain for historical purposes.

Characteristics

- Hidden from buyers.
- Cannot receive Orders.
- Cannot receive RFQs.
- Preserves historical Orders, Reviews, and Analytics.
- May optionally be restored by the seller.

Typical Use Cases

- Discontinued products.
- Historical reporting.
- Product replaced with a newer version.

DELETED

Deleted products are removed from the seller's active catalog.

Characteristics

- Hidden from buyers.
- Hidden from the seller product list by default.
- Historical Orders continue referencing the product snapshot.
- Recommended implementation is **soft delete**.

Recommended backend behaviour: set `status = deleted` and `deletedAt = current timestamp`. The database record itself should remain, since existing Orders, Reviews, Payments, and RFQs may still reference the product.

3.5 Product Types

The marketplace supports two product types.

SIMPLE

Simple products contain only one purchasable option.

Example

Cement (50kg Bag)

The product has

- one price
- one inventory
- one SKU

No variants exist.

VARIABLE

Variable products contain multiple purchasable options.

Each option may have:

- different pricing
- different stock
- different images
- different SKU

Example

Cashew Nuts

50kg Bag

100kg Bag

250kg Bag

Backend Rule

A product **must be either SIMPLE or VARIABLE.**

It cannot be both.

If

productType = SIMPLE

then

variants = []

If

productType = VARIABLE

then

variants.length >= 1

3.6 Product Categories

Unlike many marketplaces, TOFA allows a product to belong to multiple categories.

Example

Agriculture

Export

Food Processing

This improves product discoverability.

Backend Rules

- Minimum categories = 1
- Maximum categories = 5 (recommended)

Every category must exist before assignment.

3.7 Product Languages

Product information supports multiple languages.

The backend stores all supported translations.

Example

{

```

"name": {
  "en": "Premium Cashew Nuts",
  "fr": "Noix de Cajou Premium",
  "ar": "كاجو ممتاز",
  "sw": "Koroshu Bora",
  "pt": "Castanha de Caju Premium"
}
}

```

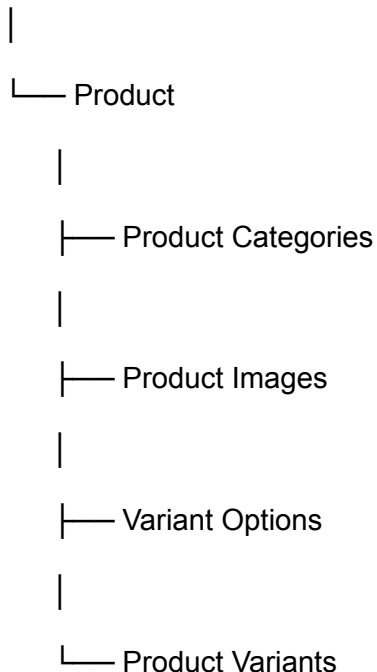
The API returns only the user's preferred language, falling back to English when a translation is unavailable.

3.8 PRODUCT DATABASE STRUCTURE

The Product Management module should separate the main product record from records that can occur multiple times, such as variants, images and categories.

At a high level:

Seller



A product belongs to one seller but may belong to multiple categories.

A product may either have a single price or multiple variants.

3.8.1 Product

The **Product** record contains information that applies to the entire product rather than an individual variation.

Field	Type	Required	Description
id	UUID/String	Yes	Unique product identifier generated by backend
sellerId	UUID/String	Yes	Seller that owns the product
productName	Translation Object	Yes	Multilingual product name
productDescription	Translation Object	Yes	Multilingual product description
countryOfOrigin	String	Yes	Country where product originates
currency	String	Yes	Currency used for product and all its variants
price	Decimal / null	Conditional	Price when product has no variants
discount	Decimal / null	No	Percentage discount for a non-variant product

productType	Enum	Yes	Determines whether the product is SIMPLE or VARIABLE (i.e., whether it uses variants)
barcode	String / null	No	Product-level barcode where applicable
supplyCapacity	Decimal	Yes	Maximum quantity seller indicates they can supply
unitForSupplyCapacity	String	Yes	Unit for supply capacity
minOrdersAllowed	Decimal	Yes	Minimum quantity buyer can purchase
unitForMinOrder	String	Yes	Unit associated with minimum order
minDuration	Integer	Yes	Minimum fulfilment/lead-time duration
maxDuration	Integer	Yes	Maximum fulfilment/lead-time duration
durationUnit	String	Yes	Unit for lead time, recommended day , week , etc.
status	Enum	Yes	Product lifecycle status

inventoryStatus	Enum	Yes	Computed inventory availability
totalStock	Decimal	Yes	Computed available inventory
createdAt	Timestamp	Yes	Creation timestamp
updatedAt	Timestamp	Yes	Last update timestamp
deletedAt	Timestamp / null	No	Used for soft deletion

Important

`price` and `discount` are product-level values only when:

`productType = SIMPLE`

When:

`productType = VARIABLE`

the product-level `price` and `discount` must be `null`. Pricing and inventory are instead controlled by each variant.

This prevents two competing sources of pricing information.

3.9 INVENTORY STATUS

Product status and inventory status represent different things.

This distinction is important.

`status` answers:

Is this product supposed to be available on the marketplace?

`inventoryStatus` answers:

Does this product currently have stock?

Supported inventory statuses:

`in_stock`

`out_of_stock`

The frontend must not send `inventoryStatus`.

It is calculated by the backend.

For products without variants:

`totalStock > 0`

→ `in_stock`

`totalStock = 0`

→ `out_of_stock`

For products with variants:

at least one variant quantity `> 0`

→ `in_stock`

all variant quantities `= 0`

→ `out_of_stock`

Important product-status rule

Running out of stock should **not automatically change**:

`active` → `inactive`

Instead:

`status = active`

inventoryStatus = out_of_stock

This allows the product page to remain available, for example with an **Out of Stock** label, while preventing checkout.

This distinction also preserves SEO, reviews and product history.

3.10 PRODUCT CATEGORIES

A product can belong to more than one category.

The relationship is therefore:

Product

|

| many-to-many

▼

Category

A join table is recommended.

ProductCategory

Field	Type
id	UUID
productId	UUID
categoryId	UUID
createdAt	Timestamp

Example:

Premium Cashew Nuts

Categories:

Agriculture

Food & Beverage

Export Commodities

API representation:

```
{  
  "categoryIds": [  
    "cat_agriculture",  
    "cat_food",  
    "cat_export"  
  ]  
}
```

Backend must validate that every supplied category exists.

A duplicate category ID should be ignored or rejected.

3.11 PRODUCT TRANSLATION STRUCTURE

TOFA's supported core marketplace languages are:

en — English

fr — French

ar — Arabic

sw — Swahili

pt — Portuguese

English is the default language.

Translatable product fields include at minimum:

productName

productDescription

They should be stored as translation objects.

Example:

```
{  
  "productName": {  
    "en": "Premium Cashew Nuts",  
    "fr": "Noix de cajou premium",  
    "ar": "كاجو ممتاز",  
    "sw": "Koroshu Bora",  
    "pt": "Castanha de Caju Premium"  
  }  
}
```

The same applies to:

```
{  
  "productDescription": {  
    "en": "Premium export-grade cashew nuts sourced from Nigeria.",  
    "fr": "Noix de cajou de qualité supérieure destinées à l'exportation.",  
    "ar": "...",  
    "sw": "...",  
  }  
}
```

```
"pt": "..."  
}  
}
```

Translation responsibility

The seller should not be required to manually enter five translations.

Recommended flow:

Seller enters product in English

↓

Backend saves English

↓

Translation service/process generates:

fr

ar

sw

pt

↓

Translations saved against product

The English value should always be preserved as the original/fallback value.

API language behaviour

Because `selectedLanguage` is stored on the User record, authenticated requests should use that preference.

Example:

User.selectedLanguage = fr

The public product response may return:

```
{  
  "productName": "Noix de cajou premium",  
  "productDescription": "Noix de cajou de qualité supérieure..."  
}
```

The frontend therefore does not need to manually select:

productName.fr

for normal buyer-facing endpoints.

For unauthenticated users:

default = en

The frontend may additionally send:

Accept-Language: fr

for visitors who change language before creating an account.

Fallback rule:

requested language

↓

Does translation exist?

YES → return translation

NO

↓

return English

Seller management endpoints should be able to return the complete translation object where required for editing.

3.12 PRODUCT IMAGES

Product images should be stored separately rather than as one serialized array if the backend is using a relational database.

ProductImage

Field	Type	Description
id	UUID	Image identifier
productId	UUID	Parent product
url	String	CDN/storage URL
sortOrder	Integer	Image display position
isPrimary	Boolean	Main product image
createdAt	Timestamp	Upload date

Example:

```
[
  {
    "id": "img_001",
    "url": "https://cdn.tofa.com/products/cashew-front.jpg",
    "sortOrder": 1,
    "isPrimary": true
  }
]
```

```
},  
  
{  
  "id": "img_002",  
  "url": "https://cdn.tofa.com/products/cashew-packaging.jpg",  
  "sortOrder": 2,  
  "isPrimary": false  
}  
]
```

Exactly one product image should normally be designated as the primary image.

The first uploaded image can automatically become primary unless the seller explicitly selects another image.

3.13 VARIANT ARCHITECTURE

Variants represent different **sellable configurations of the same product**.

Consider a shirt with:

Size:

S

M

L

Color:

Red

Blue

Material:

Cotton

Polyester

These are **variant options**.

A specific combination:

Size = M

Color = Blue

Material = Cotton

is a **variant**.

This distinction must be maintained consistently between frontend and backend.

3.14 VARIANT OPTIONS

The seller first defines the characteristics that can vary.

Your original structure is appropriate and should be retained with consistent naming:

```
{  
  "variantOptions": [  
    {  
      "name": "Size",  
      "values": ["S", "M", "L"]  
    },  
    {  
      "name": "Color",  
      "values": ["Red", "Blue"]  
    },  
    {
```



```
"name": "Material",  
  "values": ["Cotton", "Polyester"]  
}  
]  
}
```

Maximum:

3 variant options per product

This is an important restriction because combinations increase multiplicatively.

For example:

3 Sizes

×

2 Colors

×

2 Materials

=

12 possible variants

The frontend should show the seller the resulting number of combinations before generating them.

3.15 VARIANT OPTION DATA STRUCTURE

ProductVariantOption

Field	Type
id	UUID

productId	UUID
name	String
values	JSON/Array
sortOrder	Integer

Example:

```
{  
  "id": "option_1",  
  "name": "Color",  
  "values": [  
    "Blue",  
    "Red",  
    "Black"  
  ],  
  "sortOrder": 1  
}
```

3.16 PRODUCT VARIANT STRUCTURE

Every actual purchasable combination becomes a ProductVariant.

ProductVariant

Field	Type	Required	Description
id	UUID	Yes	Variant ID
productId	UUID	Yes	Parent product
sku	String	Yes	Backend-generated SKU
attributes	JSON	Yes	Combination represented by variant
price	Decimal	Yes	Variant selling price
discount	Decimal/null	No	Percentage discount
quantity	Decimal	Yes	Available stock
image	String/null	Conditional	Variant image
createdAt	Timestamp	Yes	
updatedAt	Timestamp	Yes	

Example:

```
{  
  "id": "var_003",  
  "sku": "PS-M-B-C",  
  "attributes": {  
    "Size": "M",  
    "Color": "Blue",  
    "Material": "Cotton"  
  },  
  "price": 12000,  
  "discount": 2,  
  "quantity": 60,  
  "image": "https://cdn.tofa.com/products/blue-shirt.jpg"  
}
```

3.17 VARIANT GENERATION GUIDE

The frontend and backend need to interpret variant options in exactly the same way.

Suppose the seller selects:

```
[  
  {  
    "name": "Size",  
    "values": ["S", "M"]  
  },  
  {
```

```
"name": "Color",  
  "values": ["Blue", "Red"]  
}  
]
```

The available combinations are:

S + Blue

S + Red

M + Blue

M + Red

The frontend may automatically generate:

```
[  
  {  
    "attributes": {  
      "Size": "S",  
      "Color": "Blue"  
    }  
  },  
  {  
    "attributes": {  
      "Size": "S",  
      "Color": "Red"  
    }  
  },  
  {
```

```
"attributes": {  
  "Size": "M",  
  "Color": "Blue"  
}  
,  
{  
  "attributes": {  
    "Size": "M",  
    "Color": "Red"  
  }  
}  
]
```

The seller then supplies the commercial information for each combination:

price

discount

quantity

image where required

Result:

```
{  
  "attributes": {  
    "Size": "M",  
    "Color": "Blue"  
  },  
  "price": 12000,
```

```
"discount": 5,  
"quantity": 40,  
"image": "https://cdn.tofa.com/products/shirt-blue.jpg"  
}
```

Important

The frontend can generate the combinations for the seller's convenience, but the **backend must independently validate them**.

The backend must never assume that combinations submitted by the frontend are valid simply because the frontend generated them.

3.18 VARIANT VALIDATION

For every variant:

1. Every attribute name must exist in `variantOptions`.
2. Every attribute value must be allowed by that option.
3. A combination cannot occur more than once.
4. Price must be greater than zero.
5. Quantity cannot be negative.
6. Discount must be between `0` and `100`, when supplied.
7. Variant currency is inherited from the product.
8. SKU is generated by the backend.

For example, if options are:

```
[  
  {  
    "name": "Size",  
    "values": ["S", "M"]  
  }  
]
```

this must be rejected:

```
{  
  "attributes": {  
    "Size": "XL"  
  }  
}
```

because **XL** was never defined.

3.19 COLOR IMAGE RULE

Your original requirement should remain.

If **Color** is one of the variant options, an image must be provided for every unique color.

For:

Red

Blue

Black

the seller should provide at least:

Red → image

Blue → image

Black → image

Variants sharing the same color may share the same image.

For example:

Small + Blue

Medium + Blue

Large + Blue

may all reference:

blue-shirt.jpg

The frontend should therefore request an image **per color**, rather than forcing the seller to upload the same image for every generated combination.

3.20 SKU GENERATION

SKU means **Stock Keeping Unit**.

It uniquely identifies a sellable variation of a product for inventory and order management.

The seller should not manually control the final SKU.

The backend generates it.

Using your example:

Product:

Vintage Shirts

Variant:

```
{  
  "Size": "M",  
  "Color": "Blue",  
  "Material": "Cotton"  
}
```

Base SKU:

VS-M-B-C

Where:

VS = Vintage Shirts

M = Medium

B = Blue

C = Cotton

However, initials alone cannot guarantee uniqueness.

For example:

Black

Blue

Brown

all produce **B**.

Therefore, **VS-M-B-C** should be considered the **human-readable base**, while the backend should append or otherwise generate a uniqueness component when necessary.

Example:

VS-M-B-C-001

VS-M-B-C-002

The database must enforce:

SKU UNIQUE

This prevents inventory, Cart and Order records from referencing an ambiguous variant.

3.21 DISCOUNT STRUCTURE

Your current rule defines a discount as a percentage.

Example:

```
{  
  "price": 5000,  
  "discount": 10  
}
```

means:

Original price = 5,000

Discount = 10%

Selling price = 4,500

The backend should calculate the final price.

The frontend should not submit:

discountedPrice

as the authoritative value.

Recommended calculation:

finalPrice =

price - (price × discount / 100)

If:

discount = null

then:

finalPrice = price

Validation:

$0 < \text{discount} < 100$

or `null`.

A 100% discount should not be allowed through the normal product discount field because it effectively creates a free product and may interfere with payment/order logic.

3.22 MINIMUM ORDER QUANTITY

The minimum order structure consists of:

minOrdersAllowed

unitForMinOrder

Example:

```
{  
  "minOrdersAllowed": 50,
```

```
"unitForMinOrder": "kg"
}
```

Meaning:

The buyer must purchase at least 50 kg.

This should affect:

- Product page
- Add to Cart
- Direct RFQ
- Checkout

If the minimum order is 50, the frontend must not allow:

```
{
  "quantity": 20
}
```

The backend must still validate this independently.

3.23 SUPPLY CAPACITY

Supply capacity describes the quantity the seller can supply within their normal business capacity.

Example:

```
{
  "supplyCapacity": 5000,
  "unitForSupplyCapacity": "kg"
}
```

This is **not the same as inventory**.

For example:

Current stock = 800 kg

Supply capacity = 5,000 kg/month

The seller may currently have 800 kg available but have operational capacity to supply 5,000 kg.

This distinction is particularly useful for B2B RFQs.

3.24 LEAD TIME

Your existing:

`minDuration`

`maxDuration`

should represent the estimated fulfillment lead time.

I recommend adding:

`durationUnit`

Example:

```
{  
  "minDuration": 5,  
  "maxDuration": 10,  
  "durationUnit": "days"  
}
```

This means:

Estimated fulfillment time: 5–10 days.

Backend validation:

`minDuration >= 0`

`maxDuration >= minDuration`

3.25 SIMPLE PRODUCT EXAMPLE

A product without variants could therefore look like:

```
{  
  "productName": {  
    "en": "Dangote Cement"  
  },  
  "categoryIds": [  
    "cat_building_materials",  
    "cat_cement"  
  ],  
  "countryOfOrigin": "Nigeria",  
  "currency": "NGN",  
  "productType": "SIMPLE",  
  "price": 9500,  
  "discount": 5,  
  "quantity": 3000,  
  "supplyCapacity": 10000,  
  "unitForSupplyCapacity": "bag",  
  "minOrdersAllowed": 10,  
  "unitForMinOrder": "bag",  
  "minDuration": 1,  
  "maxDuration": 3,  
  "durationUnit": "days",  
}
```

```
"barcode": null,  
"productDescription": {  
  "en": "High-quality cement suitable for residential and commercial construction."  
},  
"variantOptions": [],  
"variants": []  
}
```

There is one adjustment required here compared with your original model:

Non-variant products also require a quantity field.

Otherwise, there is no reliable way to determine their inventory status.

So:

productType = SIMPLE

→ product.quantity controls inventory

productType = VARIABLE

→ variants[].quantity controls inventory

3.26 VARIABLE PRODUCT EXAMPLE

A complete variable product could look like:

```
{  
  "productName": {  
    "en": "Premium Polo Shirt"  
  },  
}
```

```
"categoryIds": [  
  "cat_fashion",  
  "cat_clothing"  
],  
"countryOfOrigin": "Nigeria",  
"currency": "USD",  
"productType": "VARIABLE",  
"price": null,  
"discount": null,  
"quantity": null,  
"supplyCapacity": 5000,  
"unitForSupplyCapacity": "pieces",  
"minOrdersAllowed": 10,  
"unitForMinOrder": "pieces",  
"minDuration": 5,  
"maxDuration": 10,  
"durationUnit": "days",  
"barcode": null,  
"productDescription": {  
  "en": "Premium cotton polo shirts available in different sizes, colours and materials."  
},  
"variantOptions": [  
  {  
    "name": "Size",
```



```
    "values": ["S", "M", "L"]
  },
  {
    "name": "Color",
    "values": ["Red", "Blue"]
  },
  {
    "name": "Material",
    "values": ["Cotton", "Polyester"]
  }
],
```

```
"variants": [
  {
    "attributes": {
      "Size": "S",
      "Color": "Red",
      "Material": "Cotton"
    },
    "price": 12,
    "discount": null,
    "quantity": 100,
    "image": "https://cdn.tofa.com/products/red-shirt.jpg"
  },
```

```
{  
  "attributes": {  
    "Size": "M",  
    "Color": "Blue",  
    "Material": "Cotton"  
  },  
  "price": 12,  
  "discount": 2,  
  "quantity": 60,  
  "image": "https://cdn.tofa.com/products/blue-shirt.jpg"  
}  
]  
}
```

The backend generates:

variant IDs

SKUs

status

inventoryStatus

totalStock

timestamps

translations

These values should not be trusted from the frontend.

3.27 PRODUCT DATA OWNERSHIP GUIDE

To prevent frontend/backend inconsistencies, responsibility should be explicitly divided.

Information	Responsible Party
Product information	Seller / Frontend
Product images	Seller / Frontend
Categories	Seller / Frontend
Variant options	Seller / Frontend
Variant combinations	Frontend generates, Backend validates
Price	Seller
Discount	Seller
Quantity	Seller
SKU	Backend
Product ID	Backend

Variant ID	Backend
Product status on creation	Backend
Inventory status	Backend
Total stock	Backend
Final discounted price	Backend
Translation generation	Backend/service
selectedLanguage	User profile
Language fallback	Backend
createdAt / updatedAt	Backend

The frontend may calculate values such as discounted price or total stock for immediate display, but those calculations must never be treated as authoritative.

3.28 RELATIONSHIP WITH FUTURE MODULES

The Product structure established here becomes the source for several other modules.

Cart

Cart references:

productId

variantId (when applicable)

quantity

The backend obtains the current price from Product/ProductVariant.

Order

An OrderItem should store a **snapshot** of product information at the time of purchase.

This is important because a seller may later:

- rename the product;
- change the image;
- change the price;
- change its variants;
- archive the product.

Historical orders must not change when the product changes.

Direct RFQ

A Direct RFQ may reference:

productId

variantId (optional)

This allows a buyer either to request a quote for the overall product or a specific variation.

Market RFQ

A Market RFQ does **not require a productId**, because the buyer may be sourcing something that is not currently listed.

It can instead reference categories and product requirements.

Reviews

Reviews reference the actual product purchased through the completed Order.

Deleting or archiving the Product must therefore never delete historical Reviews.

Subscription

Seller subscription affects **product ranking**, not product data.

For example:

Premium Seller

↓

Active Product

↓

Ranking boost

↓

Higher search visibility

It must never allow an inactive, archived, deleted or out-of-stock product to bypass eligibility rules simply because the seller has a subscription.

This establishes the **Product data architecture**. The next section should now define the actual **Product APIs** against this model: Create Product, Update Product, Get Seller Products, Get Public Products, Get Product Details, image deletion, status changes, categories, variant editing, inventory updates, search/filtering and their complete request/response structures.

MODULE 4 CATEGORY MANAGEMENT MODULE

4. CATEGORY MANAGEMENT MODULE

4.1 Overview

The Category Management module defines how product categories are created, structured, managed, translated, retrieved, and assigned to products across the TOFA Marketplace.

Categories are created and managed by Admin users. Sellers cannot create categories directly.

The marketplace already has existing categories, and these should be imported or seeded into the system before launch. Afterward, Admin users can create additional categories where necessary.

A product can belong to more than one category.

This module integrates with:

- Product Management
- Search and Filtering
- Market RFQ
- Seller Analytics
- Admin Management
- Internationalization

4.2 Business Objectives

The Category Management module is designed to:

- maintain a controlled list of marketplace categories;
- prevent duplicate or inconsistent category names;
- allow products to belong to multiple categories;
- support multilingual category names;
- allow Admin users to create, edit, deactivate, archive, and delete categories;
- provide categories for product creation and filtering;
- provide category references for Market RFQs;
- preserve category history when categories are no longer actively used.

4.3 Category Ownership and Permissions

Categories are managed only by Admin users.

Admin can:

- create categories;
- edit categories;
- activate categories;
- deactivate categories;
- archive categories;
- view deleted categories where necessary.

Super Admin can:

- perform all Admin category actions;
- permanently manage restricted category operations if required;
- restore archived/deactivated categories;
- access category audit history.

Seller can:

- fetch active categories;
- select categories while creating or editing a product.

Buyer can:

- view active categories;
- browse products by category;
- use categories as filters.

Sellers must not be able to create new categories from the product creation flow.

If a seller cannot find a suitable category, the frontend may later provide a "Request Category" feature, but this should be treated as a separate workflow and not direct category creation.

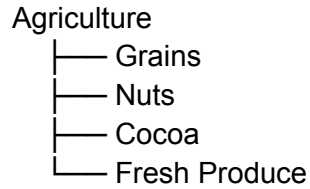
4.4 Category Structure

TOFA should support both categories and optional subcategories.

Example:

Construction Materials

- ├— Cement & Concrete
- ├— Roofing Materials
- ├— Steel & Reinforcement
- └— Plumbing Materials



The recommended implementation is a self-referencing category structure.

Each category may optionally have a `parentId`.

A category with:

`parentId = null`

is a top-level category.

A category with:

`parentId = "cat_construction"`

is a child category or subcategory.

This allows the marketplace to support deeper structures in the future without redesigning the database.

4.5 Category Database Structure

Table: Category

Field	Type	Required	Description
id	UUID/String	Yes	Unique category identifier
name	Translation Object	Yes	Multilingual category name
slug	String	Yes	URL-friendly unique category identifier
description	Translation Object/null	No	Optional category description
parentId	UUID/String/null	No	Parent category where applicable
icon	String/null	No	Optional icon

image	String/null	No	Optional category image
status	Enum	Yes	Category lifecycle status
sortOrder	Integer	Yes	Controls display order
createdBy	UUID/String	Yes	Admin that created the category
updatedBy	UUID/String/null	No	Last Admin that updated category
createdAt	Timestamp	Yes	Creation timestamp
updatedAt	Timestamp	Yes	Last update timestamp
deletedAt	Timestamp/null	No	Soft deletion timestamp

4.6 Category Status

Recommended category statuses:

active
inactive
archived
deleted

active

The category is available across the marketplace.

An active category:

- can be selected when creating products;
- appears in category filters;
- appears in category navigation;
- can be used in Market RFQs.

inactive

The category is temporarily unavailable for new usage.

An inactive category:

- cannot be selected for new products;
- should not appear in normal public category navigation;

- existing products already assigned to it retain the relationship.

This is useful when TOFA wants to temporarily stop using a category without losing historical data.

archived

The category is retired from active marketplace use.

An archived category:

- cannot be selected for products;
- is hidden publicly;
- keeps historical product relationships;
- may be restored later by Admin/Super Admin.

deleted

The category has been removed from normal use.

Deletion should use soft deletion.

Recommended backend behaviour:

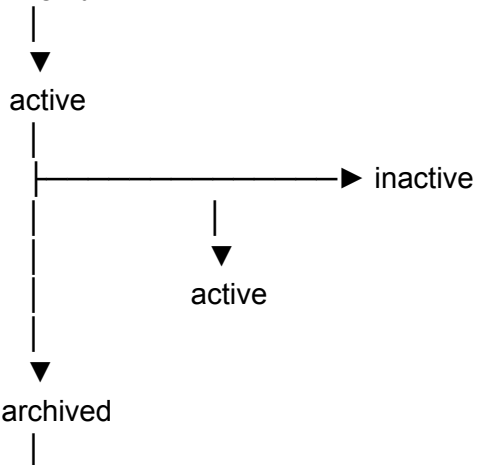
status = deleted

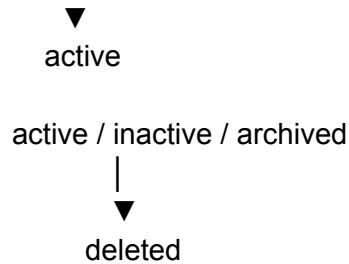
deletedAt = current timestamp

Existing historical records must continue to reference the category.

4.7 Category Status Flow

Category Created





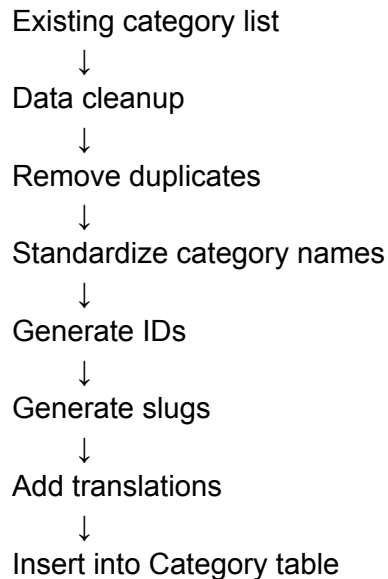
The backend must validate status transitions.

A deleted category should not normally be restored through the standard Admin endpoint unless TOFA explicitly supports restoration.

4.8 Existing Categories

Because TOFA already has an existing category list, these categories should be seeded or migrated into the system.

Recommended process:



Existing categories should not be recreated manually through the Admin dashboard one after another.

A migration or seeding script should be used.

4.9 Category Naming Rules

Category names must be consistent.

Recommended naming style:

Construction Materials
Agriculture
Food & Beverage
Electrical Equipment
Automotive
Oil & Gas

Avoid inconsistent formats such as:

construction-materials
Construction materials
CONSTRUCTION MATERIALS
construction Materials

The display value should use proper title formatting.

The slug handles URL formatting.

Example:

```
{  
  "name": {  
    "en": "Construction Materials"  
  },  
  "slug": "construction-materials"  
}
```

4.10 Category Translation Structure

Category names and descriptions should support the core TOFA marketplace languages:

en — English
fr — French
ar — Arabic
sw — Swahili
pt — Portuguese

Example:

```
{
  "name": {
    "en": "Construction Materials",
    "fr": "Matériaux de construction",
    "ar": "مواد البناء",
    "sw": "Vifaa vya Ujenzi",
    "pt": "Materiais de Construção"
  }
}
```

English remains the fallback language.

When Admin creates a category, the Admin may provide only the English value.

The backend translation process can populate the remaining supported languages.

Example creation:

```
{
  "name": "Construction Materials"
}
```

Stored form:

```
{
  "name": {
    "en": "Construction Materials",
    "fr": "Matériaux de construction",
    "ar": "مواد البناء",
    "sw": "Vifaa vya Ujenzi",
    "pt": "Materiais de Construção"
  }
}
```

4.11 Slug Generation

The backend should generate the category slug automatically.

Example:

Construction Materials

becomes:

construction-materials

The frontend should not be responsible for generating the final slug.

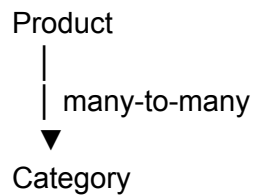
The slug must be unique.

If a duplicate category name is attempted, the backend should reject it rather than generating duplicate slugs.

4.12 Product-to-Category Relationship

A product can belong to multiple categories.

Relationship:



Recommended join table:

ProductCategory

Field	Type	Description
id	UUID/String	Relationship ID
productId	UUID/String	Product reference
categoryId	UUID/String	Category reference
createdAt	Timestamp	Relationship creation date

Example:

Product: Premium Cashew Nuts

Categories:

- Agriculture
- Food & Beverage
- Export Commodities

Product API representation:

```
{
  "categoryIds": [
    "cat_agriculture",
    "cat_food_beverage",
    "cat_export_commodities"
  ]
}
```

The backend must verify:

- every category exists;
- every selected category is active;
- duplicate category IDs are not stored.

4.13 Create Category

Endpoint

POST /admin/categories

Authentication

Admin or Super Admin JWT required.

Purpose

Creates a new category.

Request Payload

```
{
  "name": "Construction Materials",
  "description": "Products and materials used for building and construction.",
  "parentId": null,
  "icon": "building",
}
```

```
"image": "https://cdn.tofa.com/categories/construction.jpg",  
"sortOrder": 1  
}
```

Notes

`description` is optional.

`parentId` is optional.

If `parentId` is provided, the backend must verify that the parent category exists and is not deleted.

`icon` and `image` are optional.

`sortOrder` controls UI display order.

Backend Processing

The backend should:

1. validate Admin permissions;
2. validate required fields;
3. check for duplicate category names;
4. validate `parentId` where supplied;
5. generate unique category ID;
6. generate slug;
7. save English category name and description;
8. generate supported translations;
9. set:

status = active

10. save `createdBy`;
11. save timestamps.

Successful Response

```
{  
  "success": true,  
  "message": "Category created successfully.",  
  "data": {  
    "id": "cat_construction_materials",
```

```
"name": {
  "en": "Construction Materials",
  "fr": "Matériaux de construction",
  "ar": "مواد البناء",
  "sw": "Vifaa vya Ujenzi",
  "pt": "Materiais de Construção"
},
"slug": "construction-materials",
"parentId": null,
"status": "active",
"sortOrder": 1,
"createdAt": "2026-08-18T12:00:00Z"
}
}
```

Possible Error Responses

Duplicate category:

```
{
  "success": false,
  "message": "A category with this name already exists."
}
```

Invalid parent:

```
{
  "success": false,
  "message": "Selected parent category does not exist."
}
```

Unauthorized:

```
{
  "success": false,
  "message": "You are not authorized to create categories."
}
```

4.14 Get All Public Categories

Endpoint

GET /categories

Purpose

Returns active marketplace categories for buyers, sellers, filters, product creation, and public navigation.

Authentication

Not required.

Query Parameters

parentId

search

page

limit

Example:

GET /categories?parentId=cat_construction&page=1&limit=20

Public Behaviour

Only:

status = active

categories should be returned.

Response

```
{
  "success": true,
  "data": [
    {
      "id": "cat_construction",
      "name": "Construction Materials",
```

```
"slug": "construction-materials",
"icon": "building",
"image": "https://cdn.tofa.com/categories/construction.jpg",
"parentId": null,
"childrenCount": 4
},
{
  "id": "cat_agriculture",
  "name": "Agriculture",
  "slug": "agriculture",
  "icon": "leaf",
  "image": "https://cdn.tofa.com/categories/agriculture.jpg",
  "parentId": null,
  "childrenCount": 6
}
],
"pagination": {
  "page": 1,
  "limit": 20,
  "total": 18,
  "totalPages": 1
}
}
```

Language Behaviour

If authenticated:

User.selectedLanguage

determines the returned category name.

If unauthenticated:

Accept-Language

may be used.

Fallback:

English

4.15 Get Category Tree

Endpoint

GET /categories/tree

Purpose

Returns the category hierarchy in a structure suitable for:

- marketplace navigation;
- product creation;
- category menus;
- filters.

Response

```
{
  "success": true,
  "data": [
    {
      "id": "cat_construction",
      "name": "Construction Materials",
      "slug": "construction-materials",
      "children": [
        {
          "id": "cat_cement",
          "name": "Cement & Concrete",
          "slug": "cement-and-concrete",
          "children": []
        },
        {
          "id": "cat_roofing",
          "name": "Roofing Materials",
          "slug": "roofing-materials",
          "children": []
        }
      ]
    }
  ]
}
```

4.16 Get Category by ID

Endpoint

GET /categories/:categoryId

Response

```
{
  "success": true,
  "data": {
    "id": "cat_construction",
    "name": "Construction Materials",
    "description": "Products and materials used for building and construction.",
    "slug": "construction-materials",
    "parentId": null,
    "icon": "building",
    "image": "https://cdn.tofa.com/categories/construction.jpg",
    "status": "active"
  }
}
```

Public requests should only retrieve active categories.

Admin endpoints may retrieve inactive or archived categories.

4.17 Get All Categories — Admin

Endpoint

GET /admin/categories

Authentication

Admin or Super Admin required.

Query Parameters

status
search
parentId
page
limit

Example:

GET /admin/categories?status=inactive&page=1&limit=20

Response

```
{
  "success": true,
  "data": [
    {
      "id": "cat_construction",
      "name": {
        "en": "Construction Materials",
        "fr": "Matériaux de construction"
      },
      "slug": "construction-materials",
      "parentId": null,
      "status": "active",
      "productsCount": 420,
      "childrenCount": 4,
      "sortOrder": 1,
      "createdBy": {
        "id": "admin_123",
        "name": "Marketplace Admin"
      },
      "createdAt": "2026-08-18T12:00:00Z",
      "updatedAt": "2026-08-18T12:00:00Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 18,
    "totalPages": 1
  }
}
```


4.18 Update Category

Endpoint

PATCH /admin/categories/:categoryId

Authentication

Admin or Super Admin required.

Request Payload

Partial updates are allowed.

```
{
  "name": "Building & Construction Materials",
  "description": "Materials used across residential, commercial and industrial construction.",
  "parentId": null,
  "icon": "building-2",
  "sortOrder": 2
}
```

Backend Processing

The backend should:

1. validate Admin permission;
2. verify category exists;
3. reject edits to deleted categories;
4. validate duplicate names;
5. validate parent relationship;
6. prevent a category from becoming its own parent;
7. prevent circular parent relationships;
8. regenerate slug if the English name changes;
9. update translations;
10. save `updatedAt`;
11. update `updatedAt`.

Successful Response

```
{
  "success": true,
  "message": "Category updated successfully.",
}
```

```
"data": {
  "id": "cat_construction",
  "name": {
    "en": "Building & Construction Materials",
    "fr": "Matériaux de bâtiment et de construction"
  },
  "slug": "building-and-construction-materials",
  "status": "active",
  "updatedAt": "2026-08-18T13:30:00Z"
}
```

4.19 Update Category Status

Endpoint

PATCH /admin/categories/:categoryId/status

Authentication

Admin or Super Admin required.

Payload

```
{
  "status": "inactive",
  "reason": "Category temporarily removed while catalogue structure is being reviewed."
}
```

Allowed Values

active
inactive
archived
deleted

Response

```
{
  "success": true,
  "message": "Category status updated successfully.",
  "data": {
```

```
"id": "cat_construction",  
"status": "inactive"  
}  
}
```

4.20 Category Deactivation Rules

If an Admin changes a category from:

active → inactive

existing products must **not automatically become inactive**.

For example:

Product A
Categories:
- Agriculture
- Export Commodities

If **Export Commodities** becomes inactive, Product A may still remain active because it is also assigned to **Agriculture**.

The category relationship is retained for historical purposes.

However, sellers cannot assign the inactive category to new products.

4.21 Category Archive Rules

Archiving a category must not delete products associated with it.

Example:

Category archived
↓
Existing ProductCategory records retained
↓
Products remain unchanged
↓
Archived category excluded from new product selection

If an active product has only one category and that category becomes archived, the backend should not silently modify the product.

Instead, Admin reporting should flag products that no longer have an active category so they can be reassigned.

4.22 Delete Category

Category deletion should happen through the status endpoint rather than hard deletion.

Recommended:

PATCH /admin/categories/:categoryId/status

Payload:

```
{
  "status": "deleted",
  "reason": "Duplicate category"
}
```

Backend:

status = deleted
deletedAt = current timestamp

Existing:

- ProductCategory records;
- Market RFQs;
- analytics;
- historical transactions

must continue referencing the category.

4.23 Category Validation Rules

The backend must enforce the following:

- category name is required;
- English category name is mandatory;
- category name must be unique within the appropriate scope;

- slug must be globally unique;
- parent category must exist;
- parent category cannot be deleted;
- category cannot be its own parent;
- circular category relationships are prohibited;
- `sortOrder` must be zero or greater;
- sellers cannot create categories;
- products can only be assigned to active categories;
- deleted categories cannot be modified through normal Admin operations.

4.24 Frontend Implementation Guide

Admin Category Management

The Admin dashboard should provide a category management page containing:

Category Name
 Parent Category
 Status
 Products Count
 Subcategories Count
 Sort Order
 Created Date
 Actions

Supported actions:

Edit
 Activate
 Deactivate
 Archive
 Delete

The exact actions shown should depend on the current status.

Example:

active

→ Edit / Deactivate / Archive / Delete

inactive

→ Edit / Activate / Archive / Delete

archived

→ View / Restore / Delete

deleted

→ View only

Category Creation Form

Recommended fields:

Category Name *

Description

Parent Category

Icon

Image

Sort Order

The frontend should submit the seller/Admin-entered English content.

The backend controls the final translation object.

Product Creation

When creating a product, the frontend should fetch:

GET /categories/tree

or:

GET /categories

Only active categories should be selectable.

Products must support multiple category selection.

Example UI:

Categories

☒ Agriculture

☒ Food & Beverage

☐ Manufacturing

[x] Export Commodities

The frontend sends only IDs:

```
{
  "categoryIds": [
    "cat_agriculture",
    "cat_food",
    "cat_export"
  ]
}
```

The frontend should never send complete category objects as the authoritative product relationship.

4.25 Backend Implementation Guide

The backend should maintain category logic centrally rather than duplicating validation inside Product, RFQ, and Search modules.

A reusable validation service is recommended.

Conceptually:

validateCategoryIds(categoryIds)

- |
- | — categories exist?
- | — categories active?
- | — duplicates?
- | — valid relationship?

This validator can then be reused by:

Product Creation

Product Update

Market RFQ Creation

Search Filters

Admin Operations

Translation Handling

Buyer-facing endpoints should normally return translated strings:

```
{
  "name": "Matériaux de construction"
}
```

Admin endpoints should return the full translation object:

```
{
  "name": {
    "en": "Construction Materials",
    "fr": "Matériaux de construction",
    "ar": "مواد البناء",
    "sw": "Vifaa vya Ujenzi",
    "pt": "Materiais de Construção"
  }
}
```

Product Count

`productsCount` should ideally be calculated or maintained efficiently.

It should represent active/non-deleted products depending on dashboard requirements.

The database does not necessarily need a permanent `productsCount` column if it can be calculated reliably.

4.26 Search and Filtering Relationship

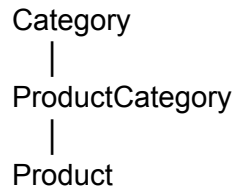
Categories are a primary search dimension.

Example:

GET /products?categoryId=cat_agriculture

Because a product may belong to multiple categories, the Product search query must use the ProductCategory relationship.

Example:



Category filtering must not assume:

`Product.categoryId`

because the agreed product architecture uses:

`Product.categoryIds[]`

through a many-to-many relationship.

4.27 Market RFQ Relationship

Market RFQs use categories to determine what sellers may respond.

Example:



A Market RFQ may support one or more category IDs depending on the final RFQ specification.

The Category module itself does not determine RFQ eligibility; it only provides the category classification used by the RFQ module.

4.28 Notification Requirements

Normal category creation, edits, activation, and deactivation do not require buyer/seller email notifications.

These are internal marketplace configuration actions.

However, the system should generate Admin activity/audit records for:

CATEGORY_CREATED
CATEGORY_UPDATED
CATEGORY_ACTIVATED
CATEGORY_DEACTIVATED
CATEGORY_ARCHIVED
CATEGORY_DELETED

Example audit record:

```
{  
  "action": "CATEGORY_ARCHIVED",  
  "adminId": "admin_123",  
  "categoryId": "cat_export",  
  "reason": "Category merged into International Trade",  
  "createdAt": "2026-08-18T14:00:00Z"  
}
```

If TOFA later introduces seller category requests, notifications and emails for that workflow should be documented separately.

4.29 Email Notification Templates

The Category Management module does not require transactional emails to buyers or sellers under the current flow because categories are created and managed internally by Admin users.

Admin category actions should use in-app confirmation and audit logs rather than email.

For example:

Category created successfully.
Category updated successfully.
Category deactivated successfully.
Category archived successfully.

This avoids unnecessary operational email traffic for routine Admin activities.

4.30 Important Implementation Notes

Categories must not be hard deleted simply because they are no longer required.

Products, RFQs, analytics, search history, and future reporting may rely on the historical category relationship.

Changing a category name must not change its database ID.

For example:

ID:
cat_construction

can remain unchanged even if:

Construction Materials

is renamed to:

Building & Construction Materials

The ID remains the stable system reference.

Slugs may change when names change, but where category pages are publicly indexed, the application should consider redirecting old slugs to the new slug.

Product assignment must always use category IDs rather than names or slugs.

The backend should treat category IDs as the authoritative relationship.

4.31 Category Module Summary

The Category Management module provides:

- Admin-controlled category creation;
- support for existing seeded categories;
- category and subcategory structures;
- multilingual category information;
- multiple categories per product;

- controlled category lifecycle;
- public and Admin category APIs;
- category hierarchy;
- search/filter integration;
- Market RFQ classification;
- backend/frontend implementation guidance;
- historical data preservation through soft deletion.

MODULE 5. CART & CHECKOUT MANAGEMENT

5. CART & CHECKOUT MANAGEMENT MODULE

5.1 Overview

The Cart & Checkout Management module manages the buyer's journey from adding products to the Cart through selecting delivery arrangements, selecting a payment method, reviewing the final payable amount, and creating a CheckoutSession that will be handed over to the Payment module.

This module does **not** process or verify payments. Its responsibility ends when the Checkout has been finalized and the system has enough information to create a Payment record.

The module is responsible for:

- adding products to Cart;
- managing Cart quantities;
- supporting simple and variable products;
- grouping Cart items by seller;
- validating product status;
- validating current product prices and discounts;
- validating inventory;
- validating minimum order quantity;
- allowing the buyer to select or add a delivery address;
- automatically using an existing default delivery address where available;
- retrieving integrated logistics quotations;
- allowing different delivery methods for different seller groups;
- allowing the buyer to apply the same integrated logistics provider to multiple eligible seller groups;
- supporting seller-arranged and buyer-arranged delivery;
- calculating product subtotals;
- calculating logistics costs payable through TOFA;
- selecting a payment method;
- generating a clear Checkout summary;
- creating one CheckoutSession;
- preparing that CheckoutSession for the Payment module.

The normal Cart flow and RFQ-based Checkout flow use the same Checkout infrastructure, but they differ in where product prices and logistics values originate.

5.2 Module Boundary

The Cart & Checkout module ends here:

Products / Accepted Quote



Checkout



Final Amount



Selected Payment Method



CheckoutSession Created

The Payment module begins here:

CheckoutSession



Payment Record



Gateway / Bank Transfer / TT / LC



Payment Verification

Payment proof upload, payment verification, gateway callbacks, Admin payment approval, and payment status management belong in the Payment Management module.

5.3 Purchase Source Types

A Checkout can originate from:

cart

direct_rfq

market_rfq

cart

The buyer is purchasing products directly from product listings.

Prices come from the current Product or ProductVariant records.

direct_rfq

The buyer accepted a quotation from a Direct RFQ sent to one seller.

Prices come from the accepted quotation.

market_rfq

The buyer accepted one quotation from a Market RFQ where multiple sellers could respond.

Prices come from the accepted winning quotation.

5.4 End-to-End Cart Checkout Flow

The normal Cart purchase flow should be:

Buyer adds product to Cart



Cart groups products by seller



Buyer proceeds to Checkout



System loads buyer's delivery address



Default address selected automatically where available



Integrated logistics options fetched



Buyer selects delivery method for each seller group



Buyer optionally applies one provider to all eligible groups



Buyer selects payment method



System shows final Checkout summary



Buyer confirms Checkout



CheckoutSession created

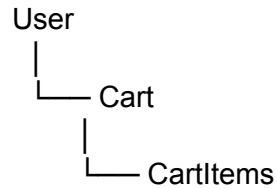


Payment module takes over

5.5 Cart Database Structure

Each buyer should have one active Cart.

Relationship:



Cart

Field	Type	Required	Description
id	UUID/String	Yes	Unique Cart ID
userId	UUID/String	Yes	Buyer that owns the Cart
createdAt	Timestamp	Yes	Cart creation date
updatedAt	Timestamp	Yes	Last update

CartItem

Field	Type	Required	Description
id	UUID/String	Yes	Cart item ID
cartId	UUID/String	Yes	Parent Cart
productId	UUID/String	Yes	Product reference
variantId	UUID/String/null	Conditional	Required for variable products
quantity	Decimal	Yes	Requested quantity
createdAt	Timestamp	Yes	Date added
updatedAt	Timestamp	Yes	Last update

CartItem should not treat price as authoritative.

Current pricing must always come from:

Product

for simple products, or:

ProductVariant

for variable products.

5.6 Simple Product Cart Rule

For:

productType = simple

the frontend sends:

```
{  
  "productId": "prod_101",  
  "variantId": null,  
  "quantity": 10  
}
```

The backend uses:

Product.price
Product.discount
Product.quantity

5.7 Variable Product Cart Rule

For:

productType = variable

the buyer must select a valid variant.

```
{  
  "productId": "prod_202",  
  "variantId": "var_003",  
  "quantity": 5  
}
```

```
}
```

The backend uses:

```
ProductVariant.price  
ProductVariant.discount  
ProductVariant.quantity
```

A variable product cannot be added without a valid `variantId`.

5.8 Seller Grouping

Cart items must be grouped by seller.

Example:

Cart

Seller A

```
|— Product 1  
|— Product 2
```

Seller B

```
|— Product 3  
|— Product 4
```

Seller C

```
|— Product 5
```

Each seller group later becomes one seller-specific Order.

Therefore:

One Seller Group

↓

One Seller Order

Delivery is also selected per seller group.

5.9 Why Delivery Is Per Seller Group

If a buyer purchases two products from Seller A, they should not automatically pay two separate delivery charges.

Example:

Seller A
├── Product 1
└── Product 2

should be treated as:

Seller A Shipment Group
├── Product 1
└── Product 2

where possible.

The logistics provider should quote for the seller shipment group rather than each individual CartItem.

This rule is important because logistics is generally based on:

- seller pickup point;
- combined shipment contents;
- combined weight/size where applicable;
- delivery address.

5.10 Delivery Types

For normal Cart Checkout:

integrated_logistics
seller_arranged
buyer_arranged

For RFQ-based Checkout, the system may additionally support:

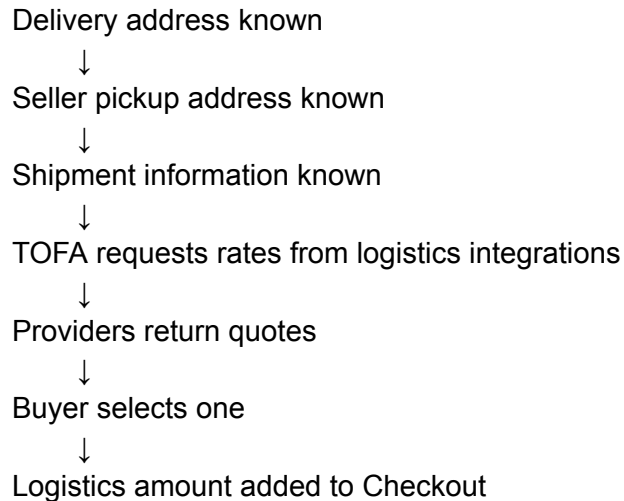
b2b_logistics

5.11 Integrated Logistics

integrated_logistics means the buyer chooses a third-party logistics provider integrated with TOFA.

TOFA itself is not the logistics provider.

Flow:



Because the amount is known before payment:

Products
+
Integrated Logistics
=
TOFA Checkout Total

5.12 Seller-Arranged Delivery — Cart

seller_arranged means the seller will arrange logistics after the Order is placed.

For normal Cart purchases:

- seller has not yet chosen the transporter;
- actual logistics cost is unknown;
- TOFA does not calculate the amount;
- TOFA does not include that logistics amount in Checkout.

Flow:

Buyer selects Seller-Arranged Delivery



Checkout contains product amount only



Buyer pays for products through TOFA



Order is created



Seller arranges logistics



Seller communicates delivery arrangement/cost



Delivery charge handled outside TOFA

Frontend wording:

Seller-arranged delivery

Description:

The seller will arrange delivery after your order is placed. Delivery charges will be communicated and paid separately outside TOFA.

The frontend must not show this as free delivery.

5.13 Buyer-Arranged Delivery

buyer_arranged means the buyer will arrange logistics independently.

TOFA does not calculate or collect the logistics amount.

Frontend wording:

I'll arrange delivery myself

Description:

You will arrange and pay for delivery separately outside TOFA.

5.14 B2B Logistics

b2b_logistics is mainly used for RFQ-generated Orders and large-volume transactions.

Examples include:

- freight;
- cross-border haulage;
- container movement;
- specialised cargo transport;
- port-to-port logistics;
- warehouse-to-warehouse logistics.

The B2B logistics amount must be known before it can be included in the Checkout total.

Full B2B logistics rules will be documented in the RFQ/Quote and Logistics modules.

5.15 Delivery Option Matrix

Delivery Type	Cart	RFQ	Included in TOFA Checkout
Integrated Logistics	Yes	Yes	Yes
Seller-Arranged	Yes	Yes	Cart: No; RFQ: Yes if quoted and accepted
Buyer-Arranged	Yes	Yes	No
B2B Logistics	Normally No	Yes	Yes once quoted

5.16 Delivery Address User Experience

The buyer should not be forced to enter a delivery address every time they Checkout.

When the buyer opens Checkout, the frontend should retrieve saved addresses.

Endpoint

GET /users/addresses

Response

```
{
  "success": true,
  "data": [
    {
      "id": "addr_001",
      "label": "Office",
      "recipientName": "Victor Ejiogu",
    }
  ]
}
```

```

    "phoneNumber": "+2348012345678",
    "addressLine1": "32 Ezekiel Street",
    "addressLine2": null,
    "city": "Lekki",
    "state": "Lagos",
    "country": "Nigeria",
    "postalCode": "105102",
    "isDefault": true
  },
  {
    "id": "addr_002",
    "label": "Warehouse",
    "recipientName": "Victor Ejiogu",
    "phoneNumber": "+2348012345678",
    "addressLine1": "12 Industrial Estate",
    "addressLine2": null,
    "city": "Ikeja",
    "state": "Lagos",
    "country": "Nigeria",
    "postalCode": null,
    "isDefault": false
  }
]
}

```

5.17 Default Address Rule

If an address exists with:

`isDefault = true`

it should automatically be selected when Checkout opens.

Example UI:

Delivery Address

Office
 32 Ezekiel Street
 Lekki, Lagos, Nigeria

Default

[Change]

The buyer does not need to select it manually.

5.18 Buyer With Multiple Saved Addresses

If the buyer selects **Change**, show saved addresses.

Select Delivery Address

- Office
32 Ezekiel Street, Lekki
Default
- Warehouse
12 Industrial Estate, Ikeja

+ Add New Address

Selecting a different Checkout address should not automatically change the user's default address.

The buyer may separately choose:

Set as default

5.19 Buyer Without Saved Address

If there is no saved address, Checkout should prompt the buyer to add one.

Recommended fields:

Label
Recipient Name
Phone Number
Address Line 1
Address Line 2
Country
State/Region
City

Postal Code
Set as Default

Endpoint

POST /users/addresses

Payload

```
{
  "label": "Office",
  "recipientName": "Victor Ejiogu",
  "phoneNumber": "+2348012345678",
  "addressLine1": "32 Ezekiel Street",
  "addressLine2": null,
  "city": "Lekki",
  "state": "Lagos",
  "country": "Nigeria",
  "postalCode": "105102",
  "isDefault": true
}
```

Response

```
{
  "success": true,
  "message": "Address added successfully.",
  "data": {
    "id": "addr_003",
    "isDefault": true
  }
}
```

Toast

Delivery address added successfully.

No email required.

5.20 Checkout Address Selection

Checkout should maintain its own selected address reference.

Recommended endpoint:

PATCH /checkout/address

Payload

```
{
  "deliveryAddressId": "addr_002"
}
```

Response

```
{
  "success": true,
  "message": "Delivery address updated successfully.",
  "data": {
    "deliveryAddressId": "addr_002",
    "requiresLogisticsRefresh": true
  }
}
```

Toast

Delivery address updated. Please review your delivery options.

5.21 Address Snapshot

When Checkout is finalized, the backend must store a snapshot of the selected address.

Example:

```
{
  "deliveryAddressSnapshot": {
    "recipientName": "Victor Ejiogu",
    "phoneNumber": "+2348012345678",
    "addressLine1": "32 Ezekiel Street",
    "city": "Lekki",
    "state": "Lagos",
    "country": "Nigeria",
    "postalCode": "105102"
  }
}
```

```
}  
}
```

Historical Orders must not change if the user later edits their saved address.

5.22 When Integrated Logistics Quotes Are Retrieved

Integrated logistics quotes should be fetched after a valid delivery address is selected.

Normally:

Buyer opens Checkout



Default address auto-selected



System automatically requests logistics quotes

If no default exists:

Buyer selects/adds address



System requests logistics quotes

The buyer does not need to separately press a button unless TOFA wants a manual refresh option.

5.23 Get Delivery Options

Endpoint

POST /checkout/delivery-options

Payload

```
{  
  "deliveryAddressId": "addr_456"  
}
```

The backend derives Cart seller groups automatically.

Backend Processing

For each seller group:

1. retrieve seller pickup address;
2. retrieve CartItems;
3. retrieve relevant product/variant shipment information;
4. retrieve buyer delivery address;
5. call eligible logistics integrations;
6. add seller-arranged option;
7. add buyer-arranged option.

Response

```
{
  "success": true,
  "data": {
    "deliveryAddress": {
      "id": "addr_456",
      "city": "Abuja",
      "state": "FCT",
      "country": "Nigeria"
    },
    "sellerGroups": [
      {
        "sellerId": "seller_1",
        "storeName": "Agro Supplies Ltd",
        "integratedOptions": [
          {
            "quoteId": "log_quote_001",
            "type": "integrated_logistics",
            "providerId": "provider_gig",
            "providerName": "GIG Logistics",
            "serviceName": "Standard Delivery",
            "amount": 12500,
            "currency": "NGN",
            "estimatedDelivery": {
              "min": 2,
              "max": 3,
              "unit": "business_days"
            },
            "expiresAt": "2026-08-19T00:30:00Z"
          }
        ]
      }
    ]
  }
}
```

```

{
  "quoteId": "log_quote_002",
  "type": "integrated_logistics",
  "providerId": "provider_dhl",
  "providerName": "DHL",
  "serviceName": "Express",
  "amount": 19000,
  "currency": "NGN",
  "estimatedDelivery": {
    "min": 1,
    "max": 2,
    "unit": "business_days"
  },
  "expiresAt": "2026-08-19T00:30:00Z"
},
{
  "sellerArrangedOption": {
    "available": true,
    "type": "seller_arranged",
    "amount": null,
    "description": "The seller will arrange delivery after the order is placed. Delivery charges will be handled separately outside TOFA."
  },
  "buyerArrangedOption": {
    "available": true,
    "type": "buyer_arranged",
    "amount": null,
    "description": "You will arrange and pay for delivery separately outside TOFA."
  }
}
]
}

```

5.24 Same Logistics Provider for Multiple Sellers

The buyer should be given a convenience option to use the same integrated logistics provider for multiple seller groups where that provider is available.

Example:

Use the same logistics provider for all eligible shipments

This does not merge the shipments.

Example:

Seller A → GIG quote A → ~~¥~~12,500

Seller B → GIG quote B → ~~¥~~8,500

Seller C → GIG quote C → ~~¥~~15,000

These remain separate logistics quotations.

The feature only saves the buyer from selecting GIG manually three times.

5.25 Same Provider Eligibility

Suppose:

Seller A:

GIG

DHL

Seller B:

GIG

ABC

Seller C:

GIG

ABC

The frontend can show:

Use GIG Logistics for all shipments

If the provider is available only for some sellers, the frontend may show:

Apply GIG Logistics where available

The remaining seller groups must still be completed manually.

5.26 Apply Same Provider Endpoint

Endpoint

POST /checkout/delivery-options/apply-provider

Payload

```
{
  "providerId": "provider_gig",
  "applyTo": "eligible_seller_groups"
}
```

Backend Processing

For each seller group:

- find a valid unexpired quote from that provider;
- apply the quote if available;
- leave seller groups without that provider unchanged;
- recalculate logistics total.

Response

```
{
  "success": true,
  "message": "GIG Logistics applied to all eligible shipments.",
  "data": {
    "appliedSellerGroups": [
      "seller_1",
      "seller_2"
    ],
    "unavailableSellerGroups": [
      "seller_3"
    ],
    "logisticsTotal": 21000
  }
}
```

Toast — All Applied

GIG Logistics selected for all eligible shipments.

Toast — Partial

GIG Logistics was applied where available. Please select delivery for the remaining shipment.

No email required.

5.27 Select Delivery Options

Endpoint

POST /checkout/delivery-options/select

Payload

```
{
  "sellerSelections": [
    {
      "sellerId": "seller_1",
      "type": "integrated_logistics",
      "quoteId": "log_quote_001"
    },
    {
      "sellerId": "seller_2",
      "type": "seller_arranged",
      "quoteId": null
    },
    {
      "sellerId": "seller_3",
      "type": "buyer_arranged",
      "quoteId": null
    }
  ]
}
```

Rules

For:

integrated_logistics

quoteId is required.

For:

seller_arranged
buyer_arranged

quoteId must be null.

The frontend must never send a trusted logistics amount.

Response

```
{
  "success": true,
  "message": "Delivery options selected successfully.",
  "data": {
    "productsTotal": 450000,
    "logisticsTotal": 15000,
    "totalAmount": 465000,
    "currency": "NGN"
  }
}
```

Toast — Integrated

Delivery option selected successfully.

Toast — Seller Arranged

Seller-arranged delivery selected. Delivery charges will be handled separately outside TOFA.

Toast — Buyer Arranged

Buyer-arranged delivery selected. You will arrange and pay for delivery separately.

No email required.

5.28 Logistics Quote Expiry

Integrated logistics quotations must contain:

quoteld
amount

expiresAt

If the selected quote expires before Checkout creation:

```
{
  "success": false,
  "code": "LOGISTICS_QUOTE_EXPIRED",
  "message": "The selected delivery price has expired. Please refresh your delivery options."
}
```

Toast

The selected delivery price has expired. Please choose a delivery option again.

No email required.

5.29 Add to Cart

Endpoint

POST /cart/items

Authentication

Authenticated user required.

Simple Product Payload

```
{
  "productId": "prod_101",
  "variantId": null,
  "quantity": 10
}
```

Variable Product Payload

```
{
  "productId": "prod_202",
  "variantId": "var_003",
  "quantity": 5
}
```

Backend Validation

Backend must verify:

1. product exists;
2. product status is **active**;
3. inventory status is **in_stock**;
4. seller account is eligible;
5. variant is required where applicable;
6. variant belongs to product;
7. quantity is available;
8. MOQ is satisfied;
9. buyer is not purchasing their own product.

Duplicate Item Rule

If the same:

productId + variantId

already exists in Cart, increase quantity rather than create a duplicate CartItem.

Response

```
{
  "success": true,
  "message": "Product added to cart.",
  "data": {
    "cartItemId": "cart_item_001",
    "productId": "prod_202",
    "variantId": "var_003",
    "quantity": 5,
    "cartItemsCount": 4
  }
}
```

Toast

Product added to cart.

No email required.

5.30 Get Cart

Endpoint

GET /cart

Response

```
{
  "success": true,
  "data": {
    "cartId": "cart_123",

    "sellerGroups": [
      {
        "sellerId": "seller_1",
        "storeName": "Agro Supplies Ltd",

        "items": [
          {
            "cartItemId": "cart_item_1",
            "productId": "prod_101",
            "variantId": "var_1",
            "productName": "Premium Polo Shirt",
            "mainImage": "https://cdn.tofa.com/products/polo.jpg",

            "attributes": {
              "Size": "M",
              "Color": "Blue"
            },

            "unitPrice": 12000,
            "discount": 5,
            "finalUnitPrice": 11400,

            "quantity": 10,
            "subtotal": 114000,
            "currency": "NGN",

            "minimumOrder": {
              "quantity": 5,
              "unit": "pieces"
            },
          }
        ]
      }
    ]
  }
}
```

```
        "availableQuantity": 60
      }
    ],

    "itemsCount": 1,
    "productsSubtotal": 114000
  }
],

"productsSubtotal": 114000,
"currency": "NGN",
"totalCount": 1
}
}
```

5.31 Update Cart Item

Endpoint

PATCH /cart/items/:cartItemId

Payload

```
{
  "quantity": 15
}
```

Response

```
{
  "success": true,
  "message": "Cart updated successfully.",
  "data": {
    "cartItemId": "cart_item_1",
    "quantity": 15,
    "subtotal": 171000
  }
}
```

Toast

Cart updated successfully.

No email required.

5.32 Remove Cart Item

Endpoint

DELETE /cart/items/:cartItemId

Response

```
{  
  "success": true,  
  "message": "Product removed from cart."  
}
```

Toast

Product removed from cart.

No email required.

5.33 Clear Cart

Endpoint

DELETE /cart

Response

```
{  
  "success": true,  
  "message": "Cart cleared successfully."  
}
```

Toast

Cart cleared successfully.

No email required.

5.34 Payment Method Selection in Checkout

Checkout must capture the buyer's selected payment method.

Checkout does not process the payment.

The selected method is passed into the Payment module after the CheckoutSession is created.

Cart Payment Methods

Typical methods may include:

direct_bank_transfer
paystack
Flutterwave
Stripe
Papss
transactworld

RFQ Payment Methods

RFQ transactions may additionally support:

letter_of_credit
telegraphic_transfer

Payment availability should be backend-controlled rather than permanently hardcoded in the frontend.

5.35 Get Available Payment Methods

Endpoint

GET /checkout/payment-methods

Purpose

Returns payment methods available for the current Checkout context.

Backend may consider:

- source type;

- amount;
- currency;
- buyer country;
- seller country;
- transaction rules;
- platform configuration.

Cart Response Example

```
{
  "success": true,
  "data": [
    {
      "imgUrl":
      "code": "direct_bank_transfer",
      "name": "Direct Bank Transfer",
      "requiresProof": true
    },
    {
      "code": "paystack",
      "name": "Paystack",
      "requiresProof": false
    },
    {
      "code": "flutterwave",
      "name": "Flutterwave",
      "requiresProof": false
    },
    {
      "code": "transactworld",
      "name": "Transactworld",
      "requiresProof": false
    }
  ]
}
```

RFQ Response Example

```
{
  "success": true,
  "data": [
    {
      "code": "direct_bank_transfer",
      "name": "Direct Bank Transfer",
```

```

    "requiresProof": true
  },
  {
    "code": "telegraphic_transfer",
    "name": "Telegraphic Transfer (TT)",
    "requiresProof": true
  },
  {
    "code": "letter_of_credit",
    "name": "Letter of Credit (LC)",
    "requiresProof": true
  },
  {
    "code": "paystack",
    "name": "Paystack",
    "requiresProof": false
  }
]
}

```

The `requiresProof` field is informational for Checkout UI.

The Payment module will define the actual proof workflow.

5.36 Select Payment Method

Endpoint

PATCH /checkout/payment-method

Payload

```

{
  "paymentMethod": "direct_bank_transfer"
}

```

Response

```

{
  "success": true,
  "message": "Payment method selected successfully.",
  "data": {

```

```
"paymentMethod": "direct_bank_transfer",
"requiresProof": true
}
}
```

Toast

Payment method selected successfully.

No email required at this stage.

5.37 Checkout Preview — Cart

Endpoint

GET /checkout/preview

Purpose

Returns the final authoritative Checkout summary before confirmation.

Response

```
{
  "success": true,
  "data": {
    "sourceType": "cart",

    "deliveryAddress": {
      "id": "addr_456",
      "label": "Office",
      "recipientName": "Victor Ejiogu",
      "addressLine1": "32 Ezekiel Street",
      "city": "Lekki",
      "state": "Lagos",
      "country": "Nigeria"
    },

    "sellerGroups": [
      {
        "sellerId": "seller_1",
        "storeName": "Agro Supplies Ltd",

```

```
"items": [  
  {  
    "productId": "prod_101",  
    "variantId": "var_1",  
    "productName": "Premium Cashew Nuts",  
    "quantity": 2,  
    "unitPrice": 50000,  
    "subtotal": 100000  
  }  
],
```

```
"productsSubtotal": 100000,
```

```
"delivery": {  
  "type": "integrated_logistics",  
  "providerName": "GIG Logistics",  
  "serviceName": "Standard Delivery",  
  "amount": 15000,  
  "estimatedDelivery": "2–3 business days"  
},
```

```
"sellerTotal": 115000  
},
```

```
{  
  "sellerId": "seller_2",  
  "storeName": "Fresh Foods Ltd",
```

```
"items": [  
  {  
    "productId": "prod_202",  
    "variantId": null,  
    "productName": "Premium Rice",  
    "quantity": 10,  
    "unitPrice": 20000,  
    "subtotal": 200000  
  }  
],
```

```
"productsSubtotal": 200000,
```

```
"delivery": {  
  "type": "seller_arranged",
```

```

      "amount": null,
      "description": "Delivery charges will be handled separately outside TOFA."
    },

    "sellerTotal": 200000
  }
],

"paymentMethod": {
  "code": "direct_bank_transfer",
  "name": "Direct Bank Transfer",
  "requiresProof": true
},

"summary": {
  "productsTotal": 300000,
  "logisticsTotal": 15000,
  "totalAmount": 315000,
  "currency": "NGN"
}
}
}

```

5.38 Checkout Summary UI

Recommended structure:

DELIVERY ADDRESS

Office
32 Ezekiel Street
Lekki, Lagos, Nigeria

[Change]

ORDER SUMMARY

AGRO SUPPLIES LTD

Premium Cashew Nuts
2 × ₦50,000 ₦100,000

Delivery
GIG Logistics ₦15,000
2–3 business days

Seller Total ₦115,000

FRESH FOODS LTD

Premium Rice
10 × ₦20,000 ₦200,000

Delivery
Seller-arranged delivery

Delivery charges will be handled separately outside TOFA.

Seller Total ₦200,000

PAYMENT METHOD

Direct Bank Transfer

[Change]

PAYMENT SUMMARY

Products ₦300,000
Integrated Logistics ₦15,000

TOTAL ₦315,000

[Confirm & Continue]

5.39 Create Cart Checkout

Endpoint

POST /checkout

Authentication

Authenticated buyer required.

Payload

Most Checkout information should already exist in Checkout state.

A minimal payload is recommended:

```
{  
  "notes": "Please deliver during business hours."  
}
```

The backend already knows:

- active Cart;
- selected address;
- selected delivery options;
- selected payment method.

Backend Processing

The backend must:

1. retrieve active Cart;
2. group CartItems by seller;
3. validate product statuses;
4. validate seller statuses;
5. validate variants;
6. validate inventory;
7. validate MOQ;
8. retrieve current prices;
9. retrieve current discounts;
10. validate selected delivery address;
11. verify every seller group has a delivery method;
12. validate integrated logistics quotes;
13. verify quote expiry;
14. calculate seller product subtotals;
15. calculate integrated logistics costs;
16. exclude seller-arranged external delivery costs;
17. exclude buyer-arranged delivery costs;
18. validate selected payment method;
19. calculate productsTotal;

20. calculate logisticsTotal;
21. calculate totalAmount;
22. create CheckoutSession;
23. create CheckoutSellerGroup records;
24. store delivery address snapshot;
25. reserve inventory;
26. assign Checkout expiration.

Response

```
{
  "success": true,
  "message": "Checkout created successfully.",
  "data": {
    "checkoutId": "checkout_456",
    "sourceType": "cart",
    "productsTotal": 300000,
    "logisticsTotal": 15000,
    "totalAmount": 315000,
    "currency": "NGN",
    "paymentMethod": "direct_bank_transfer",
    "sellerGroupsCount": 2,
    "status": "active",
    "expiresAt": "2026-08-19T23:00:00Z"
  }
}
```

Toast

Checkout created successfully. Please continue to payment.

No email required here.

Payment emails are triggered after the Payment module creates the actual Payment record.

5.40 CheckoutSession Structure

CheckoutSession

Field	Type	Required	Description
id	UUID/String	Yes	Checkout ID

userId	UUID/String	Yes	Buyer
sourceType	Enum	Yes	cart/direct_rfq/market_rfq
sourceId	UUID/String/null	No	RFQ reference
quoteId	UUID/String/null	No	Accepted Quote
quoteVersionId	UUID/String/null	No	Accepted Quote version
deliveryAddressId	UUID/String	Yes	Selected delivery address
deliveryAddressSnapshot	JSON	Yes	Historical address snapshot
paymentMethod	Enum	Yes	Selected payment method
productsTotal	Decimal	Yes	Product amount
logisticsTotal	Decimal	Yes	Logistics payable through TOFA
totalAmount	Decimal	Yes	Final payable amount
currency	String	Yes	Payment currency
status	Enum	Yes	Checkout lifecycle
expiresAt	Timestamp	Yes	Expiration time
createdAt	Timestamp	Yes	Creation date
updatedAt	Timestamp	Yes	Last update

Recommended Checkout status:

active
converted
expired
cancelled

active

Checkout has been created and can proceed to Payment.

converted

Checkout has been successfully converted into a Payment transaction.

expired

Checkout was not completed within the allowed period.

cancelled

Checkout was cancelled before conversion.

5.41 CheckoutSellerGroup Structure

CheckoutSellerGroup

Field	Type	Description
id	UUID/String	Group ID
checkoutId	UUID/String	Parent Checkout
sellerId	UUID/String	Seller
productsSubtotal	Decimal	Seller products total
logisticsAmount	Decimal	Logistics collected through TOFA
sellerTotal	Decimal	Products + platform logistics
deliveryType	Enum	Delivery method
logisticsQuoteId	UUID/String/null	Integrated quote
createdAt	Timestamp	Creation timestamp

For Cart:

seller_arranged
buyer_arranged

may use:

logisticsAmount = 0

internally, but API/UI should return:

amount = null

to avoid implying free delivery.

5.42 RFQ Checkout Relationship

RFQ Checkout reuses the Checkout infrastructure but does not use the current Cart.

It also does not use current Product pricing.

Rule:

Cart Checkout

→ current Product/ProductVariant pricing

RFQ Checkout

→ accepted Quote pricing

Flow:

RFQ

↓

Seller Quote

↓

Buyer accepts valid Quote

↓

Quote terms locked

↓

Checkout

↓

Payment

5.43 RFQ Seller-Arranged Delivery

For RFQ transactions, seller-arranged delivery may be included in TOFA Checkout because the seller can obtain and submit the logistics price while responding to the RFQ.

Example accepted Quote:

```
{  
  "pricePerUnit": 2500,  
  "quantity": 500,
```

```
"currency": "NGN",
"productsTotal": 1250000,

"delivery": {
  "type": "seller_arranged",
  "logisticsAmount": 75000,
  "estimatedDeliveryDays": 5
},

"totalAmount": 1325000,
"validUntil": "2026-08-21T23:59:59Z"
}
```

Once the buyer accepts the valid quotation:

Products
+
Quoted Seller Logistics
=
TOFA Checkout Total

This differs from normal Cart Checkout because the logistics cost has already been explicitly quoted and agreed.

5.44 RFQ Buyer-Arranged Delivery

If the buyer selected:

buyer_arranged

the seller quotes product pricing only.

No delivery fee is included in Checkout.

5.45 RFQ Integrated Logistics

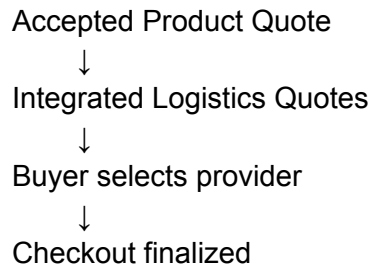
If the buyer selected:

integrated_logistics

the seller quotes the product terms.

Once the quotation is accepted, TOFA retrieves integrated logistics pricing based on the accepted quantity and delivery destination.

Flow:



5.46 RFQ B2B Logistics

For:

b2b_logistics

Checkout cannot be finalized until the B2B logistics amount has been determined where that amount is payable through TOFA.

5.47 Quote Validity

Every seller RFQ quotation must have:

validUntil

Example:

```
{
  "quoteId": "quote_901",
  "validUntil": "2026-08-21T23:59:59Z"
}
```

An expired quotation cannot be converted into Checkout.

Toast

This quotation has expired. Please request an updated quotation from the seller.

5.48 Quote Acceptance vs Checkout Expiry

These are separate concepts.

Quote validity

Determines how long the buyer has to accept the seller's offer.

Checkout expiry

Determines how long the buyer has to proceed after accepting the quotation.

Once an accepted quotation is converted into Checkout, its agreed commercial terms are locked for the Checkout period.

5.49 Quote Versioning

Updated quotations should not overwrite old quotations.

Recommended:

Quote

└── Version 1 — expired
└── Version 2 — active

Checkout must store:

quoteId
quoteVersionId

so the exact accepted terms remain auditable.

5.50 Inventory Reservation

For Cart Checkout, inventory should be reserved when CheckoutSession is created.

Example:

Current available quantity = 20
Checkout quantity = 5

available = 15
reserved = 5

If Payment succeeds:

reserved → completed sale

If Checkout/Payment expires:

reserved → released

The Payment module will define the final timing rules.

5.51 Cart After Checkout Creation

The Cart should not be permanently cleared immediately when Checkout is created.

Recommended:

Checkout created



Cart items reserved



Payment confirmed



Purchased Cart items removed

If Payment fails or expires, items may remain or be restored subject to inventory availability.

5.52 Price Changes Before Checkout

For normal Cart purchases, current price remains authoritative until Checkout creation.

If price changes, Checkout should flag it.

Example:

{

```
"priceChanges": [  
  {  
    "productId": "prod_101",  
    "previousPrice": 10000,  
    "currentPrice": 12000  
  }  
]  
}
```

Toast

The price of one or more products in your cart has changed. Please review the updated amount before continuing.

No email required.

This rule does not apply to an already accepted RFQ quotation, because the quotation terms are locked.

5.53 Inventory Changes

If a requested quantity is no longer available:

```
{  
  "success": false,  
  "code": "INSUFFICIENT_INVENTORY",  
  "message": "One or more products no longer have enough stock.",  
  "items": [  
    {  
      "productId": "prod_101",  
      "variantId": "var_1",  
      "requestedQuantity": 10,  
      "availableQuantity": 5  
    }  
  ]  
}
```

Toast

Some items in your cart no longer have the requested quantity available. Please review your cart.

No email required.

5.54 Product Becomes Unavailable

Products with:

inactive
archived
deleted

cannot proceed through normal Cart Checkout.

Toast

One or more items in your cart are no longer available and must be removed before checkout.

5.55 Address Change After Logistics Selection

If the delivery address changes:

existing integrated logistics quotes become invalid

The frontend should automatically clear those selections and retrieve fresh quotations.

Toast

Your delivery address has changed. Please review your delivery options again.

5.56 Cart Change After Logistics Selection

If the buyer changes:

- quantity;
- variant;
- products;
- number of Cart items;

the affected seller group's integrated logistics quote may become invalid.

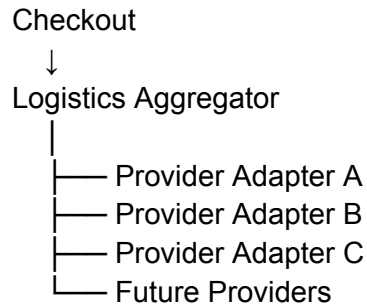
Only the affected seller group should need recalculation.

Toast

Your cart has changed. Please refresh the delivery option for the affected shipment.

5.57 Logistics Provider Abstraction

Integrated provider logic should sit behind a logistics aggregation layer.



Provider adapters may expose:

getQuote()
createShipment()
trackShipment()
cancelShipment()

where supported.

5.58 Currency Rule

A single Checkout should resolve to one payment currency.

Until formal FX conversion is introduced:

All amounts within one Checkout must resolve to one payable currency.

5.59 Frontend Implementation Guide

Recommended Checkout page:

1. Delivery Address
2. Delivery Options

3. Payment Method
4. Order Summary
5. Confirm & Continue

Delivery Address

- auto-select default address;
- allow Change;
- allow Add New Address;
- do not force repeated address entry.

Delivery Options

For each seller:

Store Name

- Integrated Logistics Provider A
- Integrated Logistics Provider B
- Seller-arranged delivery
- I'll arrange delivery myself

Where possible:

Apply Provider X to all eligible shipments

Payment Method

Show only backend-approved methods.

Final Summary

Show:

- delivery address;
- seller groups;
- product subtotals;
- platform logistics;
- externally arranged delivery notes;
- payment method;
- final total.

5.60 Backend Implementation Guide

Backend is authoritative for:

- product price;
- variant price;
- discount;
- product status;
- inventory;
- MOQ;
- seller grouping;
- default/saved address ownership;
- logistics provider eligibility;
- logistics quote;
- quote expiry;
- seller-arranged/buyer-arranged logic;
- payment-method eligibility;
- productsTotal;
- logisticsTotal;
- totalAmount;
- Checkout expiration;
- inventory reservation;
- RFQ quote validity;
- accepted RFQ quote version.

Frontend is responsible for:

- rendering Cart;
- capturing quantities;
- selecting/adding address;
- displaying logistics options;
- capturing delivery selection;
- offering apply-same-provider convenience;
- selecting payment method;
- displaying backend-calculated summary.

Frontend must never be authoritative for:

price
discountedPrice
subtotal
logisticsAmount
productsTotal
logisticsTotal
totalAmount

inventory
quote validity

5.61 Toast and Email Notification Matrix

Action	Toast		Email
Product added to Cart	Yes	No	
Cart updated	Yes	No	
Product removed	Yes	No	
Cart cleared	Yes	No	
Address added	Yes	No	
Checkout address changed	Yes	No	
Delivery selected	Yes	No	
Provider applied to multiple shipments	Yes	No	
Logistics quote expired	Yes	No	
Payment method selected	Yes	No	
Price changed	Yes	No	
Inventory changed	Yes	No	
Product unavailable	Yes	No	
Checkout created	Yes	No	
RFQ quote accepted	Yes	Yes — RFQ Module	
RFQ quote updated	Yes	Yes — RFQ Module	
Payment created	Yes	Yes — Payment Module	
Payment confirmed	Yes	Yes — Payment Module	
Order created	Yes	Yes — Order Module	

5.62 Standard Toast Content

Product Added

Product added to cart.

Cart Updated

Cart updated successfully.

Address Added

Delivery address added successfully.

Address Changed

Delivery address updated. Please review your delivery options.

Integrated Delivery Selected

Delivery option selected successfully.

Seller-Arranged Delivery

Seller-arranged delivery selected. Delivery charges will be handled separately outside TOFA.

Buyer-Arranged Delivery

Buyer-arranged delivery selected. You will arrange and pay for delivery separately.

Provider Applied

{ProviderName} selected for all eligible shipments.

Partial Provider Application

{ProviderName} was applied where available. Please select delivery for the remaining shipment.

Payment Method Selected

Payment method selected successfully.

Logistics Quote Expired

The selected delivery price has expired. Please choose a delivery option again.

Price Changed

The price of one or more products in your cart has changed. Please review the updated amount before continuing.

Inventory Changed

Some items in your cart no longer have the requested quantity available. Please review your cart.

Checkout Created

Checkout created successfully. Please continue to payment.

5.63 Internal Events

Recommended application events:

CART_ITEM_ADDED

CART_ITEM_UPDATED

CART_ITEM_REMOVED

CART_CLEARED

CHECKOUT_ADDRESS_SELECTED

CHECKOUT_ADDRESS_CHANGED

DELIVERY_OPTIONS_GENERATED

DELIVERY_OPTION_SELECTED

DELIVERY_PROVIDER_APPLIED

LOGISTICS_QUOTE_EXPIRED

CHECKOUT_PAYMENT_METHOD_SELECTED

CHECKOUT_CREATED

CHECKOUT_EXPIRED

CHECKOUT_CANCELLED

CHECKOUT_CONVERTED

5.64 Module Summary

The Cart & Checkout module establishes that:

- each buyer has one active Cart;
- Cart supports simple and variable products;
- Cart items are grouped by seller;
- multiple products from the same seller are treated as one seller delivery group where possible;
- each seller group later becomes its own Order;
- buyer delivery address is handled at Checkout;
- a valid default address is automatically selected;
- the buyer can switch to another saved address or add a new address without leaving Checkout;
- changing address invalidates integrated logistics quotes;
- integrated logistics is provided by third-party companies, not TOFA directly;
- integrated logistics cost is included in Checkout;
- seller-arranged delivery on normal Cart Orders is arranged and paid separately outside TOFA;
- buyer-arranged delivery is arranged and paid separately outside TOFA;
- unknown external logistics costs must never be presented as free delivery;
- different sellers can use different delivery methods;
- the same integrated provider can be applied across multiple eligible seller groups for convenience;
- those seller shipments remain separate even when the provider is the same;
- payment method is selected during Checkout but processed in the Payment module;
- Cart payments may support Direct Bank Transfer, Paystack, Flutterwave and Transactworld;
- RFQ Checkouts may additionally support TT and Letter of Credit;
- available payment methods are returned by backend;
- one Checkout has one final payable amount;
- one Checkout later produces one Payment;
- the one Payment can later relate to multiple seller Orders;
- Cart Checkout uses current Product pricing;
- RFQ Checkout uses accepted Quote pricing;
- accepted RFQ quotation terms are locked;
- seller-arranged logistics may be included in RFQ Checkout when the seller quoted that amount and the buyer accepted it;
- RFQ quotations have expiry periods;
- Checkout has a separate expiry period;
- backend controls all authoritative prices, totals, logistics quotations, inventory, quote validity and payment-method eligibility;
- Payment Management is the next module.

MODULE 6. PAYMENT MANAGEMENT

6. PAYMENT MANAGEMENT MODULE

6.1 Overview

The Payment Management module is the centralized payment engine for the TOFA Marketplace.

It must not be designed only around Cart or Checkout transactions.

The same Payment infrastructure should support:

- marketplace Checkout payments;
- Direct RFQ transactions;
- Market RFQ transactions;
- seller subscriptions;
- future featured-product payments;
- future advertising payments;
- future service charges;
- future account upgrades;
- future logistics-only transactions where applicable;
- other payable services introduced later.

The Payment module is responsible for:

- determining available payment methods;
- routing payment methods based on currency and transaction context;
- creating Payment records;
- generating unique payment references;
- initializing payment gateways;
- supporting PAPSS;
- handling direct bank transfer;
- supporting Telegraphic Transfer (TT);
- supporting Letter of Credit (LC);
- receiving payment proof;
- Admin verification;
- gateway callbacks/webhooks;
- payment status management;
- payment expiry;
- retrying failed payments;
- changing payment methods;
- triggering the appropriate action after successful payment.

The Payment module should therefore be reusable by any TOFA module that needs to collect money.

6.2 Core Payment Principle

Payment should not directly depend on:

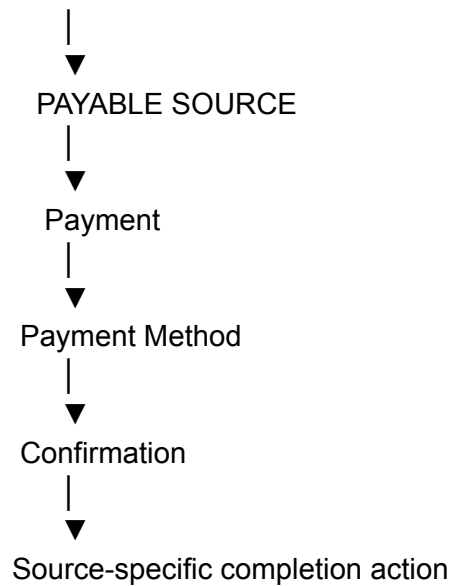
Cart
Order
Subscription
RFQ

Instead, each module creates or identifies something that needs to be paid for.

That record becomes the **payment source**.

Conceptually:

Checkout
Subscription
Featured Product
Advertising
Future Service



Examples:

Checkout Payment Confirmed
→ Create / activate Orders

Subscription Payment Confirmed

→ Activate Subscription

Featured Product Payment Confirmed

→ Activate Featured Placement

This allows future payment types to be added without redesigning the Payment module.

6.3 Payment Source Types

Recommended initial values:

checkout

subscription

Future values may include:

featured_listing

advertisement

service_fee

logistics

account_upgrade

wallet_funding

other

These values should preferably be configuration/extensible rather than deeply hardcoded throughout the application.

Recommended field:

sourceType

Examples:

```
{  
  "sourceType": "checkout",  
  "sourceId": "checkout_456"  
}
```

or:

```
{  
  "sourceType": "subscription",  
  "sourceId": "subscription_123"  
}
```

The combination:

sourceType + sourceId

identifies what is being paid for.

6.4 Payment Purpose

sourceType identifies the system entity.

A separate **purpose** field is useful for reporting and UI.

Examples:

marketplace_purchase
subscription_purchase
subscription_renewal
featured_listing
advertisement
service_payment

Example:

```
{  
  "sourceType": "subscription",  
  "sourceId": "sub_123",  
  "purpose": "subscription_renewal"  
}
```

This distinction allows a single Subscription entity to create different kinds of payments.

6.5 Payment Amount Ownership

The Payment module must never trust the frontend to decide how much the user should pay.

The source module must expose an authoritative payable amount.

Examples:

Checkout

`CheckoutSession.totalAmount`

Subscription

`SubscriptionInvoice.amount`

or the calculated subscription plan price.

Featured Listing

`FeaturedListingRequest.amount`

Payment retrieves or receives the validated source amount from the relevant backend service.

The frontend must not be allowed to submit:

```
{  
  "amount": 100  
}
```

and make that amount authoritative.

6.6 Payment Source Contract

Every module that uses Payment should provide a common internal contract.

Conceptually:

`getPayableDetails(sourceType, sourceId)`

should resolve:

```
{
  "sourceType": "subscription",
  "sourceId": "sub_123",
  "purpose": "subscription_purchase",
  "payerId": "user_123",
  "amount": 150000,
  "currency": "NGN",
  "description": "TOFA Premium Seller Subscription",
  "expiresAt": null,
  "metadata": {}
}
```

This enables Payment to remain source-agnostic.

6.7 Payment Completion Contract

After Payment is confirmed, Payment should emit a generic event:

PAYMENT_CONFIRMED

containing:

```
{
  "paymentId": "pay_123",
  "sourceType": "subscription",
  "sourceId": "sub_123",
  "purpose": "subscription_purchase"
}
```

The owning module responds.

Examples:

Checkout module

→ create/progress Orders

Subscription module

→ activate subscription

Featured Listing module
→ activate promotion

Payment itself should not contain business logic such as:

if subscription → activate plan
if cart → create orders

inside the core payment service.

That responsibility remains with each source module.

6.8 Supported Payment Methods

Recommended initial payment methods:

direct_bank_transfer
paystack
flutterwave
transactworld
papss
telegraphic_transfer
letter_of_credit

Frontend labels:

Direct Bank Transfer
Paystack
Flutterwave
Transactworld
PAPSS
Telegraphic Transfer (TT)
Letter of Credit (LC)

Not every method should be available for every source.

For example:

Subscription
→ Paystack

- Flutterwave
- Transactworld
- Direct Bank Transfer
- PAPSS where supported

while a large RFQ transaction may also allow:

TT
LC

6.9 Payment Method Eligibility

The backend determines payment options based on:

- source type;
- purpose;
- payment currency;
- payment amount;
- payer country;
- transaction countries where relevant;
- payment provider configuration;
- payment provider availability;
- minimum/maximum provider amount;
- transaction type;
- Admin/platform rules.

Recommended process:

Payment source



Amount + Currency + Context



Payment Routing Service



Eligible Providers



Recommended Payment Methods

6.10 Currency-Based Gateway Suggestion

Gateways must be suggested according to the currency being paid.

Example:

Currency = NGN

the backend might return:

Paystack
Flutterwave
Direct Bank Transfer
PAPSS

while another currency may produce a different list.

The exact currency support for Paystack, Flutterwave, Transactworld, PAPSS or future providers must live in backend provider configuration rather than being permanently documented as fixed business rules.

This allows TOFA to update provider support without redeploying frontend code.

6.11 PaymentProvider Structure

PaymentProvider

Field	Type	Description
id	UUID/String	Provider ID
code	String	Provider code
name	String	Display name
type	Enum	gateway/manual/bank_rail
supportedCurrencies	Array	Supported currencies
supportedCountries	Array/null	Country restrictions
supportedSourceTypes	Array/null	Source restrictions
supportedPurposes	Array/null	Purpose restrictions
minAmount	Decimal/null	Minimum transaction

maxAmount	Decimal/null	Maximum transaction
requiresProof	Boolean	Buyer uploads proof
supportsAutoVerification	Boolean	Provider confirms electronically
status	Enum	active/inactive
sortOrder	Integer	Display priority
createdAt	Timestamp	Creation timestamp
updatedAt	Timestamp	Last update

6.12 Payment Method Recommendation

The backend can mark one or more providers as recommended.

Example:

```
{
  "code": "paystack",
  "name": "Paystack",
  "recommended": true,
  "requiresProof": false
}
```

Recommended provider selection may consider:

- currency compatibility;
- cost;
- reliability;
- payer geography;
- payment success rate;
- source type;
- transaction amount.

Frontend simply renders the backend recommendation.

6.13 Payment Database Structure

Payment

Field	Type	Required	Description
id	UUID/String	Yes	Payment ID
paymentReference	String	Yes	Unique TOFA reference
payerId	UUID/String	Yes	User/company making payment
sourceType	String/Enum	Yes	checkout/subscription/etc.
sourceId	UUID/String	Yes	Entity being paid for
purpose	String/Enum	Yes	Business reason for payment
amount	Decimal	Yes	Authoritative payable amount
currency	String	Yes	Payment currency
paymentMethod	Enum	Yes	Selected method
providerId	UUID/String/null	No	Gateway/provider
providerReference	String/null	No	External transaction ref
status	Enum	Yes	Payment status
paymentProofUrl	String/null	No	Uploaded proof
payerTransactionReference	String/null	No	User-provided reference
failureReason	String/null	No	Failure description
rejectionReason	String/null	No	Manual rejection reason
canResubmitProof	Boolean	Yes	Proof resubmission permission
expiresAt	Timestamp/null	No	Payment expiry
proofUploadedAt	Timestamp/null	No	Proof upload time
verifiedAt	Timestamp/null	No	Verification timestamp
verifiedBy	UUID/String/null	No	Verifying Admin
paidAt	Timestamp/null	No	Payment completion timestamp

metadata	JSON/null	No	Source/provider-specific metadata
createdAt	Timestamp	Yes	Creation date
updatedAt	Timestamp	Yes	Last update

6.14 Payment Metadata

metadata can contain non-authoritative contextual information without changing the Payment schema for every new source type.

Checkout example:

```
{
  "checkoutId": "checkout_456",
  "productsAmount": 300000,
  "logisticsAmount": 15000,
  "sellerGroupsCount": 2
}
```

Subscription example:

```
{
  "subscriptionPlanId": "plan_premium",
  "billingPeriod": "yearly",
  "subscriptionName": "Premium"
}
```

The core fields:

```
amount
currency
sourceType
sourceId
```

remain authoritative.

6.15 Payment Statuses

Recommended:

pending
awaiting_payment
processing
proof_uploaded
under_review
confirmed
failed
rejected
expired
cancelled

6.16 Payment Status Definitions

pending

Payment exists but processing has not started.

awaiting_payment

Buyer must take an external action.

Examples:

- bank transfer;
- TT;
- LC;
- another manual payment process.

processing

A gateway or payment rail is processing the payment.

proof_uploaded

Payment evidence has been uploaded.

This is not confirmation.

under_review

TOFA Admin/compliance is reviewing submitted evidence.

confirmed

TOFA has verified the required payment condition.

Confirmation triggers the source module's completion flow.

failed

Gateway/payment provider attempt failed.

rejected

Manual payment evidence was rejected.

expired

Payment was not completed during the valid period.

cancelled

Payment was intentionally cancelled before confirmation.

6.17 Create Payment

Endpoint

POST /payments

Authentication

Authenticated payer required.

Payload

```
{
  "sourceType": "subscription",
  "sourceId": "subscription_123",
  "paymentMethod": "paystack"
}
```


Checkout example:

```
{  
  "sourceType": "checkout",  
  "sourceId": "checkout_456",  
  "paymentMethod": "direct_bank_transfer"  
}
```

The frontend does not send:

amount
currency

as authoritative values.

6.18 Create Payment Backend Processing

Backend should:

1. validate sourceType;
2. locate source;
3. verify source belongs to payer;
4. verify source is payable;
5. retrieve authoritative amount;
6. retrieve authoritative currency;
7. retrieve purpose;
8. check whether source already has a confirmed payment;
9. validate selected payment method;
10. validate payment method supports currency;
11. validate source/payment method compatibility;
12. generate payment reference;
13. create Payment;
14. initialize provider where applicable;
15. assign expiry;
16. return payment details.

6.19 Create Checkout Payment Example

Request

```
{
  "sourceType": "checkout",
  "sourceId": "checkout_456",
  "paymentMethod": "paystack"
}
```

Backend resolves

```
{
  "amount": 315000,
  "currency": "NGN",
  "purpose": "marketplace_purchase"
}
```

Response

```
{
  "success": true,
  "message": "Payment initialized successfully.",
  "data": {
    "paymentId": "pay_456",
    "paymentReference": "TOFA-PAY-20260821-000123",
    "sourceType": "checkout",
    "sourceId": "checkout_456",
    "purpose": "marketplace_purchase",
    "amount": 315000,
    "currency": "NGN",
    "paymentMethod": "paystack",
    "status": "processing",

    "provider": {
      "name": "Paystack",
      "authorizationUrl": "https://payment-provider.example/...",
      "providerReference": "provider_ref_001"
    }
  }
}
```

Toast

Payment initialized. Complete your payment to continue.

6.20 Create Subscription Payment Example

Suppose the seller selects:

Premium Plan
Annual Billing
₦600,000

The Subscription module creates:

subscription_123

with:

status = pending_payment

Payment request:

```
{  
  "sourceType": "subscription",  
  "sourceId": "subscription_123",  
  "paymentMethod": "flutterwave"  
}
```

Backend resolves:

```
{  
  "amount": 600000,  
  "currency": "NGN",  
  "purpose": "subscription_purchase"  
}
```

Response:

```
{  
  "success": true,  
  "message": "Subscription payment initialized successfully.",  
  "data": {
```

```
"paymentId": "pay_sub_001",
"paymentReference": "TOFA-PAY-20260821-000200",
"sourceType": "subscription",
"sourceId": "subscription_123",
"purpose": "subscription_purchase",
"amount": 600000,
"currency": "NGN",
"paymentMethod": "flutterwave",
"status": "processing",

"provider": {
  "name": "Flutterwave",
  "authorizationUrl": "https://payment-provider.example/..."
}
}
```

Toast

Subscription payment initialized. Complete your payment to activate your plan.

6.21 Subscription Direct Bank Transfer

Subscription payments should also be able to use manual methods where TOFA allows them.

Request:

```
{
  "sourceType": "subscription",
  "sourceId": "subscription_123",
  "paymentMethod": "direct_bank_transfer"
}
```

Response:

```
{
  "success": true,
  "message": "Bank transfer payment created successfully.",
  "data": {
    "paymentId": "pay_sub_002",
    "paymentReference": "TOFA-PAY-20260821-000201",
```

```
"sourceType": "subscription",
"purpose": "subscription_purchase",
"amount": 600000,
"currency": "NGN",
"paymentMethod": "direct_bank_transfer",
"status": "awaiting_payment",

"bankAccount": {
  "bankName": "{BankName}",
  "accountName": "{AccountName}",
  "accountNumber": "{AccountNumber}"
},

"requiresProof": true
}
}
```

Once Admin confirms payment:

PAYMENT_CONFIRMED

↓

Subscription Module receives event

↓

Subscription becomes active

6.22 Payment Methods Endpoint

Endpoint

GET /payments/methods

Query Parameters

sourceType

sourceId

Example:

GET /payments/methods?sourceType=subscription&sourceId=subscription_123

Backend resolves:

amount
currency
source rules
payer country

Response

```
{
  "success": true,
  "data": {
    "sourceType": "subscription",
    "sourceId": "subscription_123",
    "amount": 600000,
    "currency": "NGN",

    "methods": [
      {
        "code": "paystack",
        "name": "Paystack",
        "type": "gateway",
        "requiresProof": false,
        "recommended": true
      },
      {
        "code": "flutterwave",
        "name": "Flutterwave",
        "type": "gateway",
        "requiresProof": false,
        "recommended": false
      },
      {
        "code": "papss",
        "name": "PAPSS",
        "type": "bank_rail",
        "requiresProof": false,
        "recommended": false
      },
      {
        "code": "direct_bank_transfer",
        "name": "Direct Bank Transfer",
        "type": "manual",
        "requiresProof": true,
      }
    ]
  }
}
```

```
    "recommended": false
  }
]
}
}
```

6.23 RFQ Payment Methods

For eligible RFQ transactions, backend may additionally return:

```
telegraphic_transfer
letter_of_credit
```

Example:

```
{
  "code": "letter_of_credit",
  "name": "Letter of Credit (LC)",
  "type": "manual",
  "requiresProof": true
}
```

These methods should only appear where allowed by transaction configuration.

6.24 Direct Bank Transfer

Direct Bank Transfer is a manual/proof-based payment.

Flow:

```
Payment created
↓
TOFA bank details shown
↓
Buyer transfers money
↓
Buyer uploads proof
↓
proof_uploaded
```

↓
Admin reviews
↓
confirmed / rejected

The exact bank account displayed may depend on currency.

Example:

NGN

→ TOFA NGN account

USD

→ TOFA USD domiciliary account

Backend must choose the account based on:

Payment.currency

where configured.

6.25 Payment Account Configuration

Recommended table:

PaymentAccount

Field	Type	Description
id	UUID/String	Account ID
paymentMethod	String	direct_bank_transfer / TT etc.
currency	String	Account currency
bankName	String	Bank
accountName	String	Beneficiary
accountNumber	String	Account number

swiftCode	String/null	SWIFT
iban	String/null	IBAN where relevant
bankAddress	String/null	Bank address
country	String	Account jurisdiction
status	Enum	active/inactive

This allows TOFA to configure payment destination by currency.

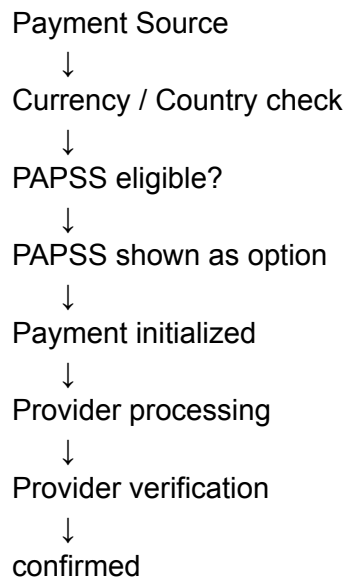
6.26 PAPSS

PAPSS should participate in the same payment routing architecture as other providers.

It should not be hardcoded into Checkout.

Eligibility is determined by the Payment Routing Service.

Flow:



If PAPSS requires manual verification under TOFA's implementation, its provider configuration can instead indicate:

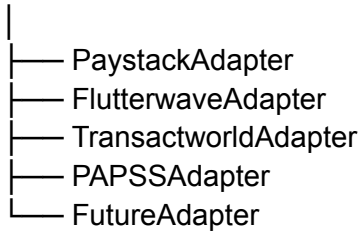
`requiresProof = true`

or another intermediate workflow.

6.27 Paystack / Flutterwave / Transactworld

All gateway integrations should use adapters.

PaymentService



Recommended adapter functions:

initializePayment()
verifyPayment()
handleWebhook()
refundPayment()

where supported.

6.28 Upload Payment Proof

Endpoint

POST /payments/:paymentId/proof

Content Type

multipart/form-data

Possible fields:

file
transactionReference

notes

Response

```
{
  "success": true,
  "message": "Payment proof uploaded successfully.",
  "data": {
    "paymentId": "pay_457",
    "status": "proof_uploaded",
    "transactionReference": "NIP123456789",
    "proofUploadedAt": "2026-08-21T10:30:00Z"
  }
}
```

Toast

Payment proof uploaded successfully. Your payment is awaiting verification.

6.29 Payment Proof Email

Subject: Payment proof received — {PaymentReference}

Preview Text: We received your payment proof and it is awaiting verification.

Hello {PayerName},

We have received the payment proof submitted for your TOFA transaction.

Payment Reference: {PaymentReference}

Payment Purpose: {PaymentPurpose}

Payment Method: {PaymentMethod}

Amount: {Currency} {Amount}

Transaction Reference: {TransactionReference}

Submitted At: {SubmittedAt}

Your payment is now awaiting verification.

You will receive another notification after the review has been completed.

Please do not make another payment using this reference unless requested by TOFA.

Best Regards,
Traders of Africa Team

CTA: View Payment

6.30 Get User Payments

Endpoint

GET /payments

Query

sourceType
purpose
status
paymentMethod
page
limit

Example:

GET /payments?sourceType=subscription&status=confirmed&page=1&limit=10

Response

```
{
  "success": true,
  "data": [
    {
      "paymentId": "pay_sub_001",
      "paymentReference": "TOFA-PAY-20260821-000200",

      "source": {
        "type": "subscription",
        "id": "subscription_123"
      },

      "purpose": "subscription_purchase",

      "amount": 600000,
      "currency": "NGN",
```

```

    "paymentMethod": "flutterwave",
    "status": "confirmed",

    "description": "Premium Seller Subscription",

    "createdAt": "2026-08-21T09:00:00Z",
    "paidAt": "2026-08-21T09:05:00Z"
  }
],

  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 5,
    "totalPages": 1
  }
}

```

6.31 Get Payment By ID

Endpoint

GET /payments/:paymentId

Checkout Payment Example

```

{
  "success": true,
  "data": {
    "paymentId": "pay_456",
    "paymentReference": "TOFA-PAY-20260821-000123",

    "source": {
      "type": "checkout",
      "id": "checkout_456"
    },

    "purpose": "marketplace_purchase",

    "amount": 315000,
    "currency": "NGN",
  }
}

```

```
"paymentMethod": "direct_bank_transfer",
"status": "proof_uploaded",

"metadata": {
  "productsAmount": 300000,
  "logisticsAmount": 15000
},

"paymentProof": {
  "url": "https://cdn.tofa.com/payments/proof123.pdf",
  "transactionReference": "NIP123456789",
  "uploadedAt": "2026-08-21T10:30:00Z"
},

"createdAt": "2026-08-21T09:00:00Z",
"expiresAt": "2026-08-22T09:00:00Z",
"verifiedAt": null
}
}
```

Subscription Payment Example

```
{
  "success": true,
  "data": {
    "paymentId": "pay_sub_001",
    "paymentReference": "TOFA-PAY-20260821-000200",

    "source": {
      "type": "subscription",
      "id": "subscription_123"
    },

    "purpose": "subscription_purchase",

    "amount": 600000,
    "currency": "NGN",

    "paymentMethod": "flutterwave",
    "status": "confirmed",

    "metadata": {
      "planId": "plan_premium",
      "planName": "Premium",

```

```
    "billingPeriod": "yearly"
  },

  "createdAt": "2026-08-21T09:00:00Z",
  "paidAt": "2026-08-21T09:04:00Z"
}
}
```

6.32 Gateway Webhooks

Recommended:

```
POST /payments/webhooks/paystack
POST /payments/webhooks/flutterwave
POST /payments/webhooks/transactworld
POST /payments/webhooks/papss
```

Backend must:

1. verify webhook authenticity;
2. locate Payment;
3. verify amount;
4. verify currency;
5. verify provider reference;
6. verify success status;
7. enforce idempotency;
8. update Payment exactly once;
9. emit **PAYMENT_CONFIRMED**.

6.33 Generic Payment Confirmation Event

On confirmation:

```
{
  "event": "PAYMENT_CONFIRMED",

  "payment": {
    "id": "pay_sub_001",
    "paymentReference": "TOFA-PAY-20260821-000200",
    "sourceType": "subscription",
```

```
"sourceId": "subscription_123",  
"purpose": "subscription_purchase",  
"amount": 600000,  
"currency": "NGN"  
}  
}
```

Then:

Subscription Service



receives PAYMENT_CONFIRMED



activates subscription

Checkout:

Order Service



receives PAYMENT_CONFIRMED



processes Checkout into Orders

6.34 Admin Payment Queue

Endpoint

GET /admin/payments

Query

sourceType

purpose

status

paymentMethod

currency

search

page

limit

Response

```
{
  "success": true,
  "data": [
    {
      "paymentId": "pay_sub_002",
      "paymentReference": "TOFA-PAY-20260821-000201",

      "payer": {
        "id": "user_123",
        "name": "Victor Ejiogu",
        "email": "victor@example.com"
      },

      "source": {
        "type": "subscription",
        "id": "subscription_123"
      },

      "purpose": "subscription_purchase",

      "paymentMethod": "direct_bank_transfer",
      "amount": 600000,
      "currency": "NGN",
      "status": "proof_uploaded",

      "transactionReference": "NIP123456789",

      "proofUrl": "https://cdn.tofa.com/payments/proof123.pdf",

      "proofUploadedAt": "2026-08-21T10:30:00Z"
    }
  ],

  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 12,
    "totalPages": 1
  }
}
```

6.35 Admin Starts Review

Endpoint

PATCH /admin/payments/:paymentId/review

Response

```
{
  "success": true,
  "message": "Payment review started successfully.",
  "data": {
    "paymentId": "pay_sub_002",
    "status": "under_review"
  }
}
```

Toast

Payment review started.

6.36 Admin Confirms Payment

Endpoint

PATCH /admin/payments/:paymentId/confirm

Payload

```
{
  "notes": "Payment confirmed in TOFA receiving account."
}
```

Backend Processing

1. validate Admin permission;
2. validate Payment;
3. prevent duplicate confirmation;
4. set status **confirmed**;

5. save `verifiedBy`;
6. save `verifiedAt`;
7. save `paidAt`;
8. create audit log;
9. emit `PAYMENT_CONFIRMED`;
10. owning source module performs its completion action.

Response

```
{
  "success": true,
  "message": "Payment confirmed successfully.",
  "data": {
    "paymentId": "pay_sub_002",
    "status": "confirmed",
    "verifiedAt": "2026-08-21T12:00:00Z"
  }
}
```

Admin Toast

Payment confirmed successfully.

6.37 Generic Payment Confirmation Email

Subject: Payment confirmed — {PaymentReference}

Preview Text: Your TOFA payment has been successfully confirmed.

Hello {PayerName},

Your payment has been successfully confirmed.

Payment Reference: {PaymentReference}

Payment For: {PaymentDescription}

Payment Method: {PaymentMethod}

Amount Paid: {Currency} {Amount}

Payment Date: {PaidAt}

{CompletionMessage}

Best Regards,
Traders of Africa Team

CTA: {CTAName}

Source-specific examples:

Checkout

CompletionMessage:

Your marketplace transaction will now proceed to Order processing.

CTA:

[View Orders](#)

Subscription

CompletionMessage:

Your subscription payment has been confirmed. Your subscription will now be activated.

CTA:

[View Subscription](#)

6.38 Admin Rejects Payment

Endpoint

PATCH /admin/payments/:paymentId/reject

Payload

```
{  
  "reason": "The submitted receipt could not be matched to the expected payment.",  
  "allowResubmission": true  
}
```

Response

```
{  
  "success": true,  
  "message": "Payment proof rejected.",  
  "data": {
```

```
"paymentId": "pay_sub_002",  
"status": "rejected",  
"canResubmitProof": true  
}  
}
```

Toast — Admin

Payment proof rejected.

Toast — Payer

Your payment proof could not be verified. Please review the reason and submit a valid payment proof.

6.39 Payment Rejection Email

Subject: Action required: Payment verification unsuccessful

Preview Text: We could not verify the payment submitted for {PaymentReference}.

Hello {PayerName},

We were unable to verify the payment evidence submitted for your TOFA transaction.

Payment Reference: {PaymentReference}

Payment For: {PaymentDescription}

Payment Method: {PaymentMethod}

Amount Expected: {Currency} {Amount}

Reason:

{RejectionReason}

{ResubmissionInstruction}

If you believe this decision is incorrect, please contact TOFA Support and include your Payment Reference.

Best Regards,
Traders of Africa Team

CTA: Review Payment

6.40 Payment Expiry

Expiry must depend on payment context and method.

Examples:

Gateway

→ short expiry

Direct Bank Transfer

→ longer expiry

TT

→ configurable

LC

→ transaction-specific

Subscription Bank Transfer

→ configurable

Expiry rules must be configuration-driven.

On expiry:

status = expired

The Payment module emits:

PAYMENT_EXPIRED

The owning source decides what happens.

Examples:

Checkout

→ release inventory

Subscription

→ subscription remains inactive/pending

6.41 Payment Expiry Email

Subject: Payment expired — {PaymentReference}

Hello {PayerName},

The payment window for the following TOFA transaction has expired.

Payment Reference: {PaymentReference}

Payment For: {PaymentDescription}

Amount: {Currency} {Amount}

Payment Method: {PaymentMethod}

No confirmed payment was recorded within the allowed period.

Please return to the relevant transaction if you still wish to proceed.

Best Regards,

Traders of Africa Team

CTA: View Transaction

6.42 Cancel Payment

Endpoint

PATCH /payments/:paymentId/cancel

Payload

```
{
  "reason": "I want to use another payment method."
}
```

Allowed when Payment is not confirmed.

Response

```
{
  "success": true,
  "message": "Payment cancelled successfully.",
  "data": {
    "paymentId": "pay_123",
  }
}
```

```
"status": "cancelled"
}
}
```

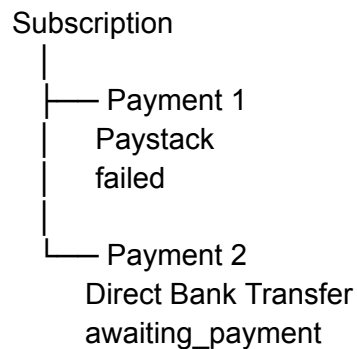
Toast

Payment cancelled successfully.

6.43 Change Payment Method

A new Payment attempt should be created instead of overwriting an existing gateway/manual transaction.

Example:



Endpoint

POST /payments/change-method

Payload

```
{
  "sourceType": "subscription",
  "sourceId": "subscription_123",
  "paymentMethod": "direct_bank_transfer"
}
```

This preserves full transaction history.

6.44 Retry Gateway Payment

Endpoint

POST /payments/:paymentId/retry

Backend must revalidate:

- source still payable;
- amount unchanged;
- currency unchanged;
- provider still supported;
- Payment not confirmed.

Toast

Payment retry initialized.

6.45 PaymentAttempt Structure

Recommended:

PaymentAttempt

Field	Type
id	UUID/String
paymentId	UUID/String
providerId	UUID/String/null
paymentMethod	String
providerReference	String/null
status	String
failureReason	String/null
initiatedAt	Timestamp

completedAt Timestamp/null

This supports retries without losing provider attempt history.

6.46 Underpayment

For manual payments:

Expected = ₦600,000

Received = ₦550,000

Payment must not be confirmed.

Store a discrepancy:

```
{  
  "type": "underpayment",  
  "expectedAmount": 600000,  
  "receivedAmount": 550000,  
  "difference": 50000  
}
```

Keep Payment under review until resolved.

6.47 Overpayment

Example:

Expected = ₦600,000

Received = ₦650,000

Do not automatically allocate the extra amount.

Keep under review and handle the excess according to TOFA refund/credit policy.

6.48 Wrong Currency

The currency received must match the Payment currency unless TOFA explicitly generated a conversion instruction.

Example:

Subscription price = USD 1,000
Buyer independently sends NGN

should not automatically be confirmed.

Currency conversion must be intentional and auditable.

6.49 Letter of Credit

LC is source-independent but will usually apply to larger RFQ/B2B transactions.

A separate LC detail record is recommended.

LetterOfCreditDetail

Field	Type
paymentId	UUID/String
issuingBank	String
advisingBank	String/null
lcReference	String
lcType	String
issueDate	Date
expiryDate	Date
documentUrl	String
status	String
reviewNotes	String/null

Payment still contains:

paymentMethod = letter_of_credit

while LC-specific details remain separated.

6.50 Telegraphic Transfer

TT supports manual proof/document verification.

Possible proof:

MT103
SWIFT confirmation
bank advice

Payment is confirmed only after TOFA verifies the transfer.

6.51 Generic Proof Upload File Rules

Recommended types:

PDF
JPG
JPEG
PNG

Backend must define:

- file-size limit;
- MIME validation;
- private storage;
- malware/file scanning where available;
- permission-based access.

6.52 Payment Security

Backend must:

- never trust frontend amounts;
- never trust frontend currency;

- never trust browser payment-success redirects;
- verify provider webhook signatures;
- validate paid amount;
- validate currency;
- validate provider transaction reference;
- enforce idempotency;
- protect API keys;
- maintain audit logs;
- protect payment proofs;
- restrict manual confirmation permissions.

6.53 Idempotency

Payment confirmation must happen exactly once.

Example:

Provider sends webhook twice

must not produce:

two subscription activations

or:

duplicate Orders

Source completion handlers must also be idempotent.

6.54 Payment Audit Log

Recommended actions:

PAYMENT_CREATED
PAYMENT_INITIALIZED
PAYMENT_RETRY_STARTED
PAYMENT_METHOD_CHANGED
PAYMENT_PROOF_UPLOADED
PAYMENT_REVIEW_STARTED

PAYMENT_CONFIRMED
PAYMENT_REJECTED
PAYMENT_EXPIRED
PAYMENT_CANCELLED

Example:

```
{  
  "action": "PAYMENT_CONFIRMED",  
  "paymentId": "pay_123",  
  "sourceType": "subscription",  
  "sourceId": "subscription_123",  
  "adminId": "admin_123",  
  "createdAt": "2026-08-21T12:00:00Z"  
}
```

6.55 Frontend Implementation Guide

Frontend should not build separate payment systems for:

Checkout
Subscription
Future Services

All should use the same Payment UI components where practical.

Example reusable flow:

PaymentSummary
PaymentMethodSelector
PaymentGatewayButton
PaymentInstructions
PaymentProofUploader
PaymentStatus

The source page passes:

sourceType
sourceId

Payment APIs handle the rest.

6.56 Source-Specific Payment Summary

The Payment response should provide enough information for the frontend to describe what is being purchased.

Checkout:

```
{
  "description": "Marketplace Purchase",
  "metadata": {
    "productsAmount": 300000,
    "logisticsAmount": 15000
  }
}
```

Subscription:

```
{
  "description": "Premium Seller Subscription",
  "metadata": {
    "planName": "Premium",
    "billingPeriod": "Annual"
  }
}
```

This prevents frontend from needing separate payment-detail APIs for basic display.

6.57 Payment Notification Matrix

Event	Toast	Email
Payment initialized	Yes	Manual methods: Yes
Gateway processing	Optional	No
Gateway confirmed	Yes	Yes

Proof uploaded	Yes	Yes
Review started	Admin toast	No
Manual payment confirmed	Yes	Yes
Proof rejected	Yes	Yes
Payment failed	Yes	Optional
Payment expired	Yes	Yes
Payment cancelled	Yes	Usually No
Payment method changed	Yes	Usually No
Source activated/completed	Source module	Source module

6.58 Generic Direct Bank Transfer Email

Subject: Complete your TOFA payment — {PaymentReference}

Preview Text: Use the payment details below and upload your proof after making the transfer.

Hello {PayerName},

A payment has been created for:

{PaymentDescription}

Payment Reference: {PaymentReference}

Amount: {Currency} {Amount}

Bank Details:

Bank Name: {BankName}

Account Name: {AccountName}

Account Number: {AccountNumber}

Where possible, include **{PaymentReference}** in your transfer narration.

After completing the transfer, upload your receipt or transfer confirmation from your TOFA account.

Your payment will remain pending until TOFA verifies receipt.

Payment Deadline: {ExpiresAt}

Best Regards,
Traders of Africa Team

CTA: Upload Payment Proof

6.59 Generic Payment Confirmation Email

Subject: Payment confirmed — {PaymentReference}

Preview Text: Your TOFA payment has been successfully confirmed.

Hello {PayerName},

Your payment has been successfully confirmed.

Payment Reference: {PaymentReference}

Payment For: {PaymentDescription}

Payment Method: {PaymentMethod}

Amount Paid: {Currency} {Amount}

Payment Date: {PaidAt}

{SourceCompletionMessage}

Best Regards,
Traders of Africa Team

CTA: {CTAName}

6.60 Subscription Confirmation Example

When:

sourceType = subscription

use:

SourceCompletionMessage:

Your payment has been confirmed and your subscription is now being activated.

CTA:

[View Subscription](#)

The Subscription module should send a separate **Subscription Activated** email once activation succeeds.

6.61 Checkout Confirmation Example

When:

sourceType = checkout

use:

SourceCompletionMessage:

Your payment has been confirmed and your marketplace Order(s) are now being processed.

CTA:

[View Orders](#)

6.62 Payment Failure Toast

Your payment could not be completed. Please try again or choose another available payment method.

6.63 Payment Method Changed Toast

Payment method updated successfully.

6.64 Payment Proof Upload Toast

Payment proof uploaded successfully. Your payment is awaiting verification.

6.65 Confirmed Payment Toast

Payment confirmed successfully.

6.66 Rejected Payment Toast

Your payment could not be verified. Please review the reason and take the required action.

6.67 Payment Module Integration Guide

Any new module that needs payment should follow this pattern:

1. Create payable source entity
2. Determine:
 - payer
 - amount
 - currency
 - purpose
3. Call Payment module
4. Payment processes transaction
5. Payment emits PAYMENT_CONFIRMED
6. Source module completes its business action

Example future Featured Listing:

Seller requests Featured Product



FeaturedListingRequest created



sourceType = featured_listing



Payment



PAYMENT_CONFIRMED



Product promotion activated

No Payment schema redesign is required.

6.68 Module Summary

The Payment Management module establishes that:

- Payment is a centralized service used across TOFA;
- it is not tied specifically to Cart or Checkout;
- each Payment references a generic `sourceType` and `sourceId`;
- initial payment sources include Checkout and Subscription;
- new source types can be added later without redesigning Payment;
- `purpose` provides additional business context;
- authoritative amount and currency come from the source module;
- frontend cannot determine payment amount;
- payment methods are dynamically selected based on currency and transaction context;
- Paystack, Flutterwave, Transactworld, PAPSS and future gateways use provider adapters;
- PAPSS is part of the dynamic provider-routing architecture;
- Direct Bank Transfer, TT and LC can use proof/admin verification;
- gateway payments use secure provider verification;
- one source can have multiple payment attempts but only one successful payable outcome;
- Payment confirmation emits a generic event;
- Checkout confirmation leads to Order processing;
- Subscription confirmation leads to Subscription activation;
- future services can reuse the same architecture;
- provider eligibility and supported currencies remain backend-configurable;
- all payment confirmation processes must be idempotent and auditable.

MODULE 7. ORDER MANAGEMENT

7. ORDER MANAGEMENT MODULE

7.1 Overview

The Order Management module handles marketplace fulfillment after a Payment has reached the business condition required for Order creation.

For standard payment methods:

`payment.status = confirmed`

For payment methods with special commercial conditions, such as Letter of Credit, the Payment module may emit:

`PAYMENT_READY_FOR_ORDER`

The Order module then creates seller-specific Orders and remains the source of truth for the commercial fulfillment lifecycle.

A single buyer Payment may create several Orders where products belong to different sellers.

One Checkout



One Payment



Payment Ready



├── Seller A Order
├── Seller B Order
└── Seller C Order

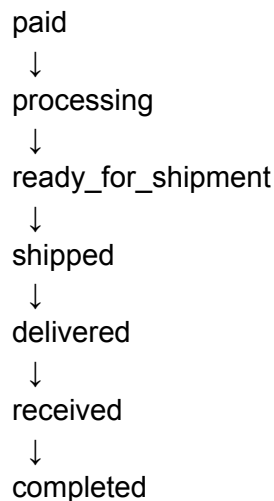
The module is responsible for:

- creating Orders after payment readiness;
- grouping items by seller;
- storing commercial snapshots;
- storing product and variant snapshots;
- preserving RFQ/Quote references;
- preserving Order currency and FX snapshots;

- controlling Order statuses;
- controlling fulfillment actions;
- handling seller-arranged shipping confirmation;
- handling buyer-arranged logistics details and handover;
- supporting integrated logistics events;
- handling receipt confirmation;
- cancellation requests;
- Order completion;
- review eligibility;
- settlement eligibility;
- notifications and emails.

7.2 Order Lifecycle Ownership

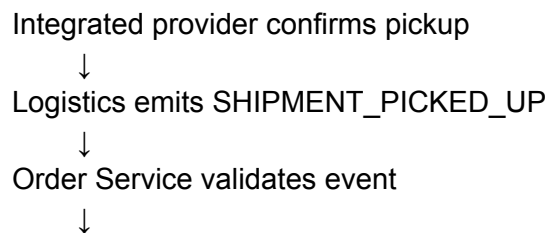
Order Management owns the complete commercial fulfillment lifecycle.



No other module should introduce a second fulfillment lifecycle.

The Logistics module may provide information that causes an Order status transition, but Order Management remains responsible for applying that transition.

Example:



Order = shipped

7.3 Order Source Types

Recommended:

cart
direct_rfq
market_rfq

cart

Created from normal marketplace Cart Checkout.

direct_rfq

Created from an accepted quotation for an RFQ directed to one seller.

market_rfq

Created from the quotation selected by a buyer from a Market RFQ.

7.4 Order Creation Trigger

Order creation should be entirely backend-driven.

Payment Service



PAYMENT_READY_FOR_ORDER



Order Service



Create Order(s)

The frontend must not be responsible for creating Orders after payment.

Order creation must be idempotent.

7.5 One Payment to Multiple Orders

Example Checkout:

Seller A

├── Product 1
└── Product 2

Seller B

└── Product 3

The buyer makes:

One Payment

After payment:

Payment pay_123

├── order_101 → Seller A
└── order_102 → Seller B

Both Orders reference:

paymentId = pay_123

Each Order then has its own:

- Order Reference;
- status;
- seller;
- delivery method;
- shipment;
- tracking;
- cancellation;
- completion.

7.6 Order Currency Principle

The Order must distinguish:

orderCurrency

paymentCurrency

orderCurrency

The currency in which the seller priced the Product or submitted the accepted RFQ quotation.

paymentCurrency

The currency selected by the buyer during Checkout and actually paid to TOFA.

Example:

Seller price = USD 1,000

Buyer selects NGN

orderCurrency = USD

paymentCurrency = NGN

There is no separate TOFA currency.

7.7 Order Database Structure

Order

Field	Type	Required	Description
id	UUID/String	Yes	Internal ID
orderReference	String	Yes	Human-readable Order reference
paymentId	UUID/String	Yes	Payment reference
checkoutId	UUID/String/ null	No	Checkout
buyerId	UUID/String	Yes	Buyer
sellerId	UUID/String	Yes	Seller
sourceType	Enum	Yes	cart/direct_rfq/market_rfq
sourceId	UUID/String/ null	No	RFQ where applicable

quoteId	UUID/String/ null	No	Accepted Quote
quoteVersionId	UUID/String/ null	No	Accepted Quote version
status	Enum	Yes	Order status
orderCurrency	String	Yes	Seller commercial currency
productsSubtotal	Decimal	Yes	Product amount in orderCurrency
logisticsAmount	Decimal/null	No	Logistics paid through TOFA
orderTotal	Decimal	Yes	Products + platform-paid logistics
paymentCurrency	String	Yes	Buyer payment currency
paymentAmount	Decimal	Yes	This Order's payment equivalent
fxApplied	Boolean	Yes	FX used
fxRateSnapshot	Decimal/null	No	Locked FX rate
fxSourceCurrency	String/null	No	FX source
fxTargetCurrency	String/null	No	FX target
fxSourceAmount	Decimal/null	No	Original amount
fxConvertedAmount	Decimal/null	No	Converted amount
fxQuotedAt	Timestamp/ null	No	FX quote time
fxProvider	String/null	No	FX source/provider
deliveryType	Enum	Yes	integrated_logistics/seller_arranged/buyer_arranged/b2b_logistics
deliveryAddressSnapshot	JSON	Yes	Delivery address
buyerNotes	String/null	No	Buyer notes
sellerBuyerNote	String/null	No	Seller note visible to buyer
sellerInternalNote	String/null	No	Seller internal note

receivedAt	Timestamp/ null	No	Buyer receipt confirmation
cancelledAt	Timestamp/ null	No	Cancellation time
cancellationReason	String/null	No	Cancellation reason
completedAt	Timestamp/ null	No	Completion
createdAt	Timestamp	Yes	Created
updatedAt	Timestamp	Yes	Updated

7.8 Logistics Amount Rule

`logisticsAmount` is nullable.

Integrated Logistics

`logisticsAmount` = amount included in TOFA transaction

Seller-Arranged — Cart

`logisticsAmount` = null

because delivery is paid separately outside TOFA.

Buyer-Arranged

`logisticsAmount` = null

Seller-Arranged — RFQ

If the seller quoted logistics and the buyer accepted it:

`logisticsAmount` = quoted amount

The frontend must never interpret:

null

as free delivery.

Display:

Delivery charge handled separately outside TOFA.

7.9 Order Total Rule

If logistics is included:

$\text{orderTotal} =$
 $\text{productsSubtotal} + \text{logisticsAmount}$

If logistics is external:

$\text{orderTotal} =$
 productsSubtotal

Unknown external delivery charges must never be added retrospectively to the Order total.

7.10 FX Rule

FX applies only when:

$\text{orderCurrency} \neq \text{paymentCurrency}$

Example:

Order Total = USD 1,050
Buyer Payment Currency = NGN
FX = 1 USD : NGN 1,650

$\text{paymentAmount} = \text{NGN } 1,732,500$

Store:

```
{  
  "orderCurrency": "USD",  
  "orderTotal": 1050,
```

```
"paymentCurrency": "NGN",
"paymentAmount": 1732500,

"fxApplied": true,

"fxRateSnapshot": 1650,
"fxSourceCurrency": "USD",
"fxTargetCurrency": "NGN",
"fxSourceAmount": 1050,
"fxConvertedAmount": 1732500
}
```

7.11 No FX Rule

If currencies match:

```
{
  "orderCurrency": "USD",
  "orderTotal": 1050,

  "paymentCurrency": "USD",
  "paymentAmount": 1050,

  "fxApplied": false
}
```

FX fields may be null.

7.12 FX Rate Lock

FX must be locked during Checkout before Payment is created.

Buyer selects payment currency



FX quote generated



Buyer sees final amount



Buyer confirms Checkout



FX locked
↓
Payment created
↓
Order later copies FX snapshot

Order must never recalculate historical FX.

7.13 Multi-Seller Multi-Currency Rule

Example:

Seller A → USD
Seller B → NGN
Seller C → GHS

Buyer selects NGN

Each seller Order is converted separately into NGN.

Each Order preserves:

orderCurrency
orderTotal
paymentCurrency
paymentAmount
FX snapshot where applicable

All Orders can still reference the same NGN Payment.

7.14 OrderItem Structure

OrderItem

Field	Type	Required
id	UUID/String	Yes
orderId	UUID/String	Yes
productId	UUID/String	Yes

variantId	UUID/String/null	No
productNameSnapshot	String/JSON	Yes
productImageSnapshot	String/null	No
skuSnapshot	String/null	No
attributesSnapshot	JSON/null	No
unitPrice	Decimal	Yes
discount	Decimal/null	No
finalUnitPrice	Decimal	Yes
quantity	Decimal	Yes
unit	String/null	No
subtotal	Decimal	Yes
createdAt	Timestamp	Yes

All monetary fields above use [orderCurrency](#).

7.15 Product Snapshot Rule

Historical Orders must not depend on current Product data.

Store Product reference plus commercial snapshot.

Changes to:

- Product name;
- image;
- price;
- SKU;
- attributes;

must not modify existing Orders.

7.16 Order Statuses

Recommended:

paid
processing
ready_for_shipment
shipped
delivered
received
completed
cancelled

7.17 Status Definitions

paid

Payment condition has been satisfied.

processing

Seller is preparing the goods.

ready_for_shipment

Seller has completed preparation and goods are ready for delivery/pickup.

shipped

Goods have left the seller's custody.

This means:

- handed to integrated provider;
- handed to seller-arranged provider;
- handed to buyer-arranged provider/representative.

delivered

Goods have reached the destination according to available delivery evidence.

received

Buyer explicitly confirms receipt.

completed

Transaction lifecycle completed.

cancelled

Order cancelled under an approved process.

7.18 Status Permissions

Transition	Actor
Create paid Order	System
paid → processing	Seller
processing → ready_for_shipment	Seller
ready_for_shipment → shipped	Seller or integrated provider event
shipped → delivered	Provider/Seller/Admin under permitted rules
delivered → received	Buyer
received → completed	System/Admin
Cancellation request	Buyer/Seller
Final cancellation	Admin/System

7.19 Order Status History

OrderStatusHistory

Field	Type
id	UUID/String
orderId	UUID/String
fromStatus	String/null
toStatus	String

changedByType	Enum
changedById	UUID/String/null
notes	String/null
createdAt	Timestamp

Every status change must create history.

7.20 Order Delivery Structure

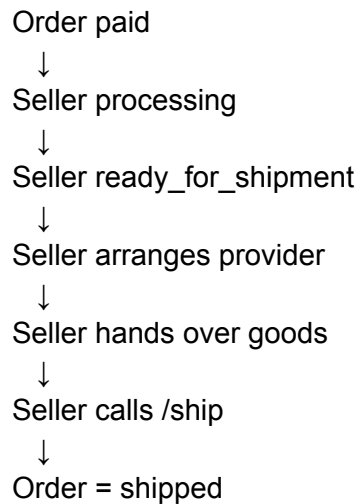
OrderDelivery

Field	Type	Required
id	UUID/String	Yes
orderId	UUID/String	Yes
deliveryType	Enum	Yes
providerId	UUID/String/null	No
providerNameSnapshot	String/null	No
serviceNameSnapshot	String/null	No
logisticsQuoteId	UUID/String/null	No
logisticsAmount	Decimal/null	No
logisticsCurrency	String/null	No
trackingId	String/null	No
trackingUrl	String/null	No
deliveryContact	String/null	No
buyerLogisticsContactName	String/null	No
buyerLogisticsEmail	String/null	No
expectedPickupDate	Date/null	No

handoverTo	String/null	No
handoverReference	String/null	No
pickupAddressSnapshot	JSON	Yes
deliveryAddressSnapshot	JSON	Yes
estimatedDelivery	JSON/null	No
shippedAt	Timestamp/null	No
deliveredAt	Timestamp/null	No
sellerDeliveryNotes	String/null	No
buyerLogisticsNotes	String/null	No
createdAt	Timestamp	Yes
updatedAt	Timestamp	Yes

7.21 Seller-Arranged Delivery Flow

For normal Cart:



The seller enters logistics details when marking shipped.

There is no separate endpoint just to add seller-arranged delivery information.

7.22 Seller-Arranged Ship Endpoint

Endpoint

PATCH /orders/:orderId/ship

Payload

```
{
  "providerName": "ABC Transport",
  "trackingId": "ABC938483",
  "trackingUrl": null,
  "deliveryContact": "+2348012345678",

  "estimatedDelivery": {
    "min": 4,
    "max": 6,
    "unit": "business_days"
  },

  "notes": "Goods handed over to ABC Transport."
}
```

Backend

1. verify seller owns Order;
2. verify Order is `ready_for_shipment`;
3. verify `deliveryType = seller_arranged`;
4. save delivery information;
5. set `shippedAt`;
6. set Order to `shipped`;
7. create status history;
8. notify buyer.

7.23 Buyer-Arranged Delivery Flow

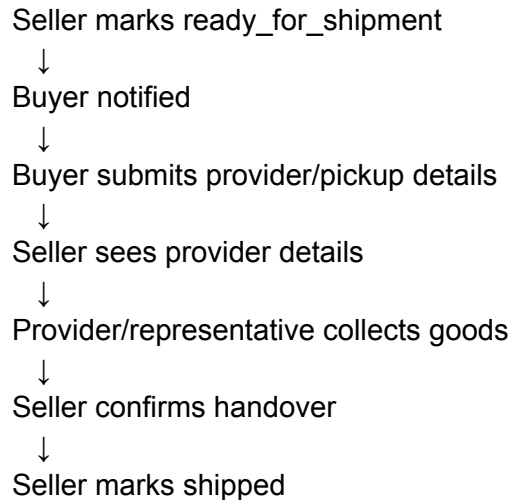
Buyer-arranged delivery uses the same Order lifecycle.

Order paid

↓

Seller processing

↓



The buyer does not directly change Order status.

7.24 Buyer Provides Logistics Details

This endpoint stores coordination information only.

It does **not** change the Order status.

Endpoint

PATCH /orders/:orderId/buyer-logistics

Authentication

Buyer that owns the Order.

Preconditions

deliveryType = buyer_arranged

Recommended when:

status = ready_for_shipment

The system may also permit submission earlier after Order creation, but seller handover still cannot happen before readiness.

Logistics Company Example

```
{
  "providerName": "ABC Logistics",
  "contactName": "Kwame Mensah",
  "phoneNumber": "+233501234567",
  "email": "dispatch@abclogistics.com",
  "trackingReference": "ABC-928281",
  "expectedPickupDate": "2026-08-28",
  "notes": "The driver will call before arrival."
}
```

Individual Pickup Representative Example

```
{
  "providerName": null,
  "contactName": "Chinedu Okafor",
  "phoneNumber": "+2348012345678",
  "email": null,
  "trackingReference": null,
  "expectedPickupDate": "2026-08-28",
  "notes": "Company logistics representative."
}
```

Response

```
{
  "success": true,
  "message": "Pickup details submitted successfully.",
  "data": {
    "orderId": "order_101",
    "deliveryType": "buyer_arranged",
    "providerName": "ABC Logistics",
    "contactName": "Kwame Mensah",
    "phoneNumber": "+233501234567",
    "expectedPickupDate": "2026-08-28"
  }
}
```

Buyer Toast

Pickup details submitted successfully. The seller can now coordinate the handover with your logistics provider.

Seller Toast

The buyer has provided logistics details for this order.

7.25 Buyer Logistics Details Email to Seller

Subject: Buyer logistics details received — {OrderReference}

Hello {SellerName},

The buyer has provided logistics or pickup details for the following Order:

Order Reference: {OrderReference}

Buyer: {BuyerName}

Logistics / Pickup Details:

Provider: {ProviderNameDisplay}

Contact Person: {ContactName}

Phone Number: {PhoneNumber}

Email: {EmailDisplay}

Expected Pickup Date: {ExpectedPickupDate}

Reference: {TrackingReferenceDisplay}

Notes:

{Notes}

Please coordinate the pickup directly with the logistics provider or representative.

Once the goods have been handed over, update the Order as **Shipped**.

Best Regards,

Traders of Africa Team

CTA: View Order

7.26 Buyer-Arranged Ship Confirmation

The seller uses the same existing endpoint:

PATCH /orders/{orderId}/ship

The buyer's provider details already exist on the Order.

The seller only needs to confirm final handover information.

Payload

```
{  
  "handoverTo": "Kwame Mensah",  
  "handoverReference": "ABC-928281",  
  "notes": "Goods handed over to buyer's logistics provider."  
}
```

Backend

1. verify seller;
2. verify Order is `ready_for_shipment`;
3. verify `deliveryType = buyer_arranged`;
4. verify buyer logistics details exist where required;
5. save handover information;
6. set `shippedAt`;
7. set Order to `shipped`;
8. create status history;
9. notify buyer.

Seller Toast

Order handover confirmed and marked as shipped.

Buyer Toast

The seller has confirmed that your logistics provider collected the goods.

7.27 Buyer-Arranged Logistics Details Update

Before handover, the buyer may need to correct provider details.

The same endpoint may be reused:

PATCH `/orders/:orderId/buyer-logistics`

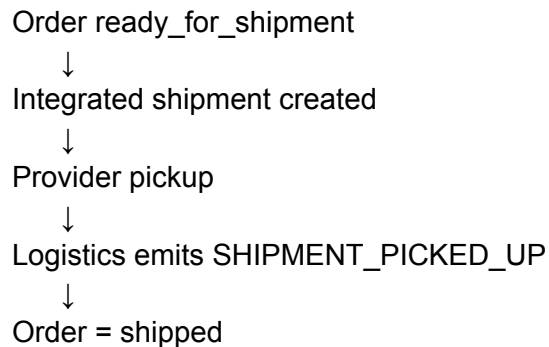
Allowed only before:

status = shipped

Once the seller has confirmed handover, the buyer cannot modify these details directly.

7.28 Integrated Logistics Flow

Integrated delivery differs only in how shipment information is generated.



The seller does not use the external-delivery `/ship` action where provider pickup is automatically confirmed.

7.29 Mark Delivered

For external seller-arranged/buyer-arranged delivery where no provider integration exists:

Endpoint

PATCH `/orders/:orderId/status`

Payload:

```
{
  "status": "delivered"
}
```

Backend must validate who is permitted based on delivery type and platform rules.

Integrated delivery may instead reach `delivered` through provider events.

7.30 Buyer Confirms Receipt

Endpoint

POST /orders/:orderId/confirm-receipt

Payload

```
{  
  "notes": "Goods received in good condition."  
}
```

Response

```
{  
  "success": true,  
  "message": "Order receipt confirmed successfully.",  
  "data": {  
    "orderId": "order_101",  
    "status": "received",  
    "receivedAt": "2026-08-30T17:00:00Z"  
  }  
}
```

7.31 Order Completion

Recommended:

```
received  
+  
no active dispute  
+  
completion conditions  
↓  
completed
```

Completion may be system/Admin controlled.

7.32 Review Eligibility Trigger

When:

status = completed

Order Service emits:

ORDER_COMPLETED

Example payload:

```
{
  "event": "ORDER_COMPLETED",
  "orderId": "order_101",
  "orderReference": "TOFA-ORD-20260825-000101",
  "buyerId": "buyer_123",
  "sellerId": "seller_456",

  "items": [
    {
      "orderItemId": "item_001",
      "productId": "prod_101",
      "variantId": "var_001"
    }
  ],

  "completedAt": "2026-08-30T18:00:00Z"
}
```

Reviews & Rewards listens for this event and creates review eligibility.

Order Management does not award points directly.

7.33 Cancellation Flow

Buyer and seller request cancellation.

Normal cancellation is generally allowed before shipment.

After:

shipped

use Disputes>Returns instead.

Endpoint

POST /orders/:orderId/cancellation-request

Payload:

```
{  
  "reason": "I ordered the wrong quantity.",  
  "notes": "Please cancel before shipment."  
}
```

7.34 Get Orders

Endpoint

GET /orders

Query

type
status
sourceType
search
dateFrom
dateTo
page
limit

type:

incoming
outgoing

7.35 Get Order by ID

Endpoint

GET /orders/:orderId

Response Example

```
{
  "success": true,
  "data": {
    "orderId": "order_101",
    "orderReference": "TOFA-ORD-20260825-000101",

    "buyer": {
      "id": "buyer_123",
      "name": "Victor Ejiogu"
    },

    "seller": {
      "id": "seller_456",
      "storeName": "Agro Supplies Ltd"
    },

    "sourceType": "cart",
    "status": "ready_for_shipment",

    "orderCurrency": "USD",
    "productsSubtotal": 1000,
    "logisticsAmount": null,
    "orderTotal": 1000,

    "paymentCurrency": "NGN",
    "paymentAmount": 1650000,

    "fx": {
      "applied": true,
      "sourceCurrency": "USD",
      "targetCurrency": "NGN",
      "rate": 1650
    },

    "items": [
      {
        "productId": "prod_101",
        "productName": "Premium Cashew Nuts",
        "quantity": 500,
        "unit": "kg",
        "unitPrice": 2,
        "subtotal": 1000
      }
    ]
  }
}
```

```

],

"delivery": {
  "type": "buyer_arranged",

  "buyerLogistics": {
    "providerName": "ABC Logistics",
    "contactName": "Kwame Mensah",
    "phoneNumber": "+233501234567",
    "email": "dispatch@abclogistics.com",
    "trackingReference": "ABC-928281",
    "expectedPickupDate": "2026-08-28",
    "notes": "The driver will call before arrival."
  },

  "handover": null
}
}
}

```

After shipment:

```

{
  "delivery": {
    "type": "buyer_arranged",

    "buyerLogistics": {
      "providerName": "ABC Logistics",
      "contactName": "Kwame Mensah"
    },

    "handover": {
      "handoverTo": "Kwame Mensah",
      "handoverReference": "ABC-928281",
      "shippedAt": "2026-08-28T11:00:00Z",
      "notes": "Goods handed over successfully."
    }
  }
}

```

7.36 Multi-Seller Email Rule

For 3 seller Orders:

1 Payment confirmation

+

3 seller-specific Order confirmation emails

Each Order email is separate because it has:

- different seller;
- different Order Reference;
- different products;
- different delivery method;
- different future tracking/status.

7.37 Buyer Order Confirmation Email

Subject: Order confirmed with {SellerName} — {OrderReference}

Hello {BuyerName},

Your Order with **{SellerName}** has been confirmed successfully.

Order Reference: {OrderReference}

Products

{ProductItems}

Example:

Premium Cashew Nuts

Grade: A

Quantity: 500 kg

Unit Price: USD 2

Subtotal: USD 1,000

Order Summary

Products Total: {OrderCurrency} {ProductsSubtotal}

Logistics: {LogisticsDisplay}

Order Total: {OrderCurrency} {OrderTotal}

{PaymentCurrencySection}

Delivery Method: {DeliveryMethod}

Delivery Address:

{DeliveryAddress}

The seller has been notified and can begin processing the Order.

Best Regards,
Traders of Africa Team

CTA: View Order

For buyer-arranged delivery:

Once the goods are ready, you will be asked to provide your logistics provider or pickup representative details.

7.38 Completed Order Email — Buyer

Subject: Order completed — {OrderReference}

Hello {BuyerName},

Your transaction with {SellerName} has been completed successfully.

Order Reference: {OrderReference}

Order Total: {OrderCurrency} {OrderTotal}

We'd love to hear about your experience.

You can now review the product(s) you purchased and your experience with the seller. Your feedback helps other buyers make informed decisions and supports trust across TOFA Marketplace.

You may also earn **TOFA Points** for submitting an eligible review, subject to the Reviews & Rewards rules.

Best Regards,
Traders of Africa Team

CTA: Review Your Order

7.39 Frontend Implementation Guide

For buyer-arranged delivery:

Before Seller Ready

Display:

Buyer-Arranged Delivery

You will provide your logistics/pickup details once the seller confirms the goods are ready.

Order Ready

Display:

Your Order is Ready

Please provide your logistics provider or pickup representative.

[Provide Logistics Details]

After Details Submitted

Show:

Provider

Contact Person

Phone Number

Expected Pickup Date

Reference

Seller sees the same coordination information.

After Seller Handover

Display:

Goods handed over

Provider

Handover person

Handover reference

Date/time

7.40 Backend Implementation Guide

Order Service owns:

- status transitions;
- `/ship`;
- buyer logistics detail storage;
- seller handover confirmation;
- Order history;
- completion;
- cancellation;
- commercial snapshots.

Logistics Service owns only integrated-provider/B2B functions and supplies provider events to Order Service.

7.41 Internal Events

Recommended:

ORDERS_CREATED
ORDER_PROCESSING
ORDER_READY_FOR_SHIPMENT

BUYER_LOGISTICS_DETAILS_SUBMITTED
BUYER_LOGISTICS_DETAILS_UPDATED

ORDER_SHIPPED
ORDER_DELIVERED
ORDER_RECEIVED
ORDER_COMPLETED

ORDER_CANCELLATION_REQUESTED
ORDER_CANCELLATION_APPROVED
ORDER_CANCELLATION_REJECTED
ORDER_CANCELLED

7.42 Module Summary

The Order module establishes that:

- one fulfillment lifecycle applies to all delivery types;

- seller remains responsible for confirming goods left their custody;
- buyer-arranged logistics does not allow buyer to mark Order shipped;
- buyer provides logistics details for coordination;
- seller confirms actual handover;
- seller-arranged logistics details are supplied when seller marks shipped;
- integrated provider events can update Order status;
- external logistics amounts remain nullable where outside TOFA;
- Order and payment currencies are preserved separately;
- FX is snapshotted;
- multi-seller Orders remain independent;
- completed Orders trigger Reviews & Rewards eligibility.

MODULE 8. LOGISTICS & DELIVERY MANAGEMENT

8. LOGISTICS & DELIVERY MANAGEMENT MODULE

8.1 Overview

The Logistics & Delivery Management module supports delivery execution without creating a separate Order lifecycle.

Order Management remains the source of truth for:

paid
processing
ready_for_shipment
shipped
delivered
received
completed

The Logistics module is responsible for:

- third-party logistics provider configuration;
- integrated logistics quotations;
- integrated shipment creation;
- integrated shipment tracking;
- provider webhooks;
- provider status normalization;
- B2B logistics requests and quotations;
- logistics-provider failures;
- logistics audit information;
- supplying provider events to Order Management.

Seller-arranged and buyer-arranged Order fulfillment actions remain in Order Management.

8.2 Responsibility Separation

Order Management Owns

Order status
Seller processing

Ready for shipment
External shipment confirmation
Buyer logistics details
Seller handover confirmation
Delivered
Buyer receipt
Completion
Cancellation

Logistics Management Owns

Integrated provider configuration
Integrated quote retrieval
Integrated shipment creation
Tracking
Provider webhooks
Shipment status mapping
B2B logistics
Provider failures

No duplicated business action should exist across the two modules.

8.3 Delivery Types

Recommended:

integrated_logistics
seller_arranged
buyer_arranged
b2b_logistics

8.4 Integrated Logistics

TOFA connects to a third-party provider.

TOFA may:

- obtain prices;
- present provider options;
- include logistics in Checkout;
- create shipment;

- obtain tracking;
- receive provider status updates.

TOFA is not the carrier.

8.5 Seller-Arranged Logistics

Seller independently arranges delivery.

For normal Cart:

delivery cost unknown
→ outside TOFA Payment

For RFQ:

seller may quote logistics
→ accepted amount may be paid through TOFA

The physical shipment is still confirmed through:

PATCH /orders/:orderId/ship

as defined in Order Management.

The Logistics module does not provide another seller-arranged shipment endpoint.

8.6 Buyer-Arranged Logistics

Buyer chooses or appoints:

- logistics company;
- freight provider;
- driver;
- pickup representative;
- internal company logistics team.

However, the seller remains responsible for confirming when the goods physically leave their custody.

The flow is:

Seller ready_for_shipment
↓
Buyer provides logistics details
↓
Seller receives details
↓
Buyer provider coordinates pickup
↓
Seller hands over goods
↓
Seller calls Order /ship
↓
Order = shipped

The buyer does not mark the Order as shipped.

8.7 Buyer Logistics Information Ownership

The buyer-provided logistics details are stored as part of `OrderDelivery`.

Order Management owns:

PATCH /orders/:orderId/buyer-logistics

Logistics Management does not duplicate this endpoint.

The Logistics module may consume those details where needed for future integrations, reporting or dispute evidence.

8.8 Buyer Logistics Details Structure

Recommended stored information:

providerName
contactName
phoneNumber
email
trackingReference
expectedPickupDate
notes

These fields are primarily coordination details.

They do not represent confirmed shipment.

Shipment is confirmed only after seller handover.

8.9 Buyer-Arranged Handover

After pickup occurs, seller uses:

PATCH /orders/:orderId/ship

with:

```
{
  "handoverTo": "Kwame Mensah",
  "handoverReference": "ABC-928281",
  "notes": "Goods handed over to buyer's nominated logistics provider."
}
```

This is defined in Order Management.

The Logistics module should only reference this behavior, not recreate it.

8.10 Delivery Responsibility Matrix

Delivery Type	Who Arranges	Who Provides Initial Logistics Details	Who Marks Shipped
Integrated	TOFA-integrated provider	System/provider	Provider event/System
Seller-Arranged	Seller	Seller when shipping	Seller
Buyer-Arranged	Buyer	Buyer	Seller after handover
B2B Logistics	Provider/transaction arrangement	Buyer/System/Provider	Provider/System/Seller depending integration

8.11 Logistics Cost Matrix

Delivery Type	Logistics Through TOFA
Integrated	Yes
Seller-Arranged Cart	No
Seller-Arranged RFQ	Yes if quoted and accepted
Buyer-Arranged	No
B2B	Yes once quoted and accepted

The Logistics module must never append new charges to an already finalized Payment.

8.12 Logistics Provider Structure

LogisticsProvider

Field	Type	Required
id	UUID/String	Yes
code	String	Yes
name	String	Yes
logo	String/null	No
type	Enum	Yes
supportedCountries	Array	Yes
supportedCurrencies	Array	Yes
supportsDomestic	Boolean	Yes
supportsInternational	Boolean	Yes
supportsTracking	Boolean	Yes
supportsWebhook	Boolean	Yes
supportsCancellation	Boolean	Yes
status	Enum	Yes

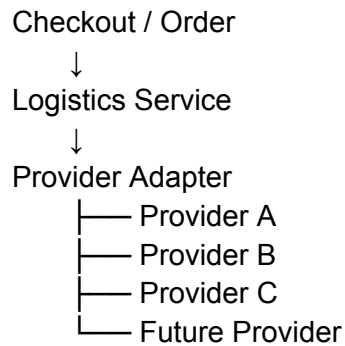
sortOrder	Integer	Yes
createdAt	Timestamp	Yes
updatedAt	Timestamp	Yes

8.13 Provider Statuses

active
inactive

Inactive providers remain visible historically but cannot be used for new quotations.

8.14 Provider Adapter Architecture



Common normalized functions:

getQuote()
createShipment()
getShipment()
trackShipment()
cancelShipment()
handleWebhook()

8.15 Logistics Quote Structure

LogisticsQuote

Field	Type	Required
-------	------	----------

id	UUID/String	Yes
providerId	UUID/String	Yes
sellerId	UUID/String	Yes
buyerId	UUID/String	Yes
sourceType	Enum	Yes
sourceId	UUID/String/null	No
pickupAddressSnapshot	JSON	Yes
deliveryAddressSnapshot	JSON	Yes
amount	Decimal	Yes
currency	String	Yes
serviceName	String/null	No
estimatedDeliveryMin	Integer/null	No
estimatedDeliveryMax	Integer/null	No
estimatedDeliveryUnit	String/null	No
providerQuoteReference	String/null	No
status	Enum	Yes
expiresAt	Timestamp	Yes
createdAt	Timestamp	Yes

8.16 Quote Statuses

active
selected
expired
cancelled

8.17 Quote Expiry

Expired provider quotes cannot be used for Checkout.

Error

```
{  
  "success": false,  
  "code": "LOGISTICS_QUOTE_EXPIRED",  
  "message": "The selected delivery price has expired. Please refresh your delivery options."  
}
```

Toast

The selected delivery price has expired. Please choose a delivery option again.

8.18 Integrated Quote Data Requirements

Provider pricing may require:

pickup address
delivery address
quantity
weight
dimensions
package count
product category
declared value
service

Backend retrieves these values from Product, Variant, Cart, seller and buyer records.

8.19 Product Logistics Data

Recommended optional Product/Variant fields:

weight
weightUnit
length
width
height
dimensionUnit

Variant values may override Product-level values.

8.20 Get Integrated Logistics Quotes

Endpoint

POST /logistics/quotes

Payload

```
{
  "sellerId": "seller_123",
  "deliveryAddressId": "addr_456",
  "items": [
    {
      "productId": "prod_101",
      "variantId": "var_1",
      "quantity": 2
    }
  ]
}
```

When invoked from Checkout, trusted backend Cart data should be preferred.

Response

```
{
  "success": true,
  "data": {
    "sellerId": "seller_123",

    "quotes": [
      {
        "quoteId": "log_quote_001",
        "providerId": "provider_a",
        "providerName": "Logistics Provider A",
        "serviceName": "Standard Delivery",
        "amount": 15000,
        "currency": "NGN",

        "estimatedDelivery": {
          "min": 2,
          "max": 3,

```

```
        "unit": "business_days"
      },
      "expiresAt": "2026-08-25T20:00:00Z"
    }
  ]
}
}
```

8.21 Same Provider Across Sellers

A buyer can use the same integrated provider for multiple sellers where eligible.

This remains:

Seller A → Provider A Shipment 1
Seller B → Provider A Shipment 2
Seller C → Provider A Shipment 3

Each seller group has independent:

- quote;
- price;
- pickup;
- shipment;
- tracking.

8.22 Logistics Currency Rule

Logistics preserves provider quote currency.

Example:

Provider Quote = NGN 50,000

Checkout handles conversion into buyer payment currency where necessary.

Logistics does not independently calculate final buyer FX.

Preserve:

providerQuoteAmount

providerQuoteCurrency

Checkout owns FX locking.

8.23 Integrated Shipment Structure

A separate Shipment entity is required mainly for integrated and B2B delivery.

Shipment

Field	Type	Required
id	UUID/String	Yes
shipmentReference	String	Yes
orderId	UUID/String	Yes
providerId	UUID/String	Yes
providerNameSnapshot	String	Yes
serviceNameSnapshot	String/null	No
externalShipmentId	String/null	No
trackingId	String/null	No
trackingUrl	String/null	No
pickupAddressSnapshot	JSON	Yes
deliveryAddressSnapshot	JSON	Yes
status	Enum	Yes
estimatedPickupAt	Timestamp/null	No
pickedUpAt	Timestamp/null	No
estimatedDeliveryAt	Timestamp/null	No
deliveredAt	Timestamp/null	No
failureReason	String/null	No

createdAt	Timestamp	Yes
updatedAt	Timestamp	Yes

Seller-arranged and buyer-arranged delivery can remain in **OrderDelivery** without a separate Shipment row.

8.24 Shipment Statuses

pending
 shipment_created
 awaiting_pickup
 picked_up
 in_transit
 out_for_delivery
 delivered
 delivery_failed
 cancelled

8.25 Shipment to Order Mapping

Shipment Status	Order Status
pending	ready_for_shipment
shipment_created	ready_for_shipment
awaiting_pickup	ready_for_shipment
picked_up	shipped
in_transit	shipped
out_for_delivery	shipped
delivered	delivered
delivery_failed	shipped with delivery issue

The Logistics module emits events.

Order Service performs Order transitions.

8.26 Create Integrated Shipment

Endpoint

POST /orders/:orderId/delivery/create-shipment

This endpoint remains in Logistics because it communicates with a third-party provider.

Preconditions

deliveryType = integrated_logistics

order.status = ready_for_shipment

Backend

1. verify seller;
2. retrieve Order;
3. retrieve selected provider quote;
4. retrieve OrderItems;
5. retrieve address snapshots;
6. call provider;
7. create Shipment;
8. save provider references;
9. return tracking information.

Response

```
{
  "success": true,
  "message": "Shipment created successfully.",
  "data": {
    "shipmentId": "shipment_123",
    "shipmentReference": "TOFA-SHP-20260825-000123",

    "provider": {
      "id": "provider_a",
      "name": "Logistics Provider A"
    },

    "trackingId": "TRACK-928281",
    "trackingUrl": "https://provider.example/track/TRACK-928281",

    "status": "awaiting_pickup"
  }
}
```

```
}
```

8.27 Integrated Pickup

Provider confirms:

picked_up

Logistics emits:

SHIPMENT_PICKED_UP

Order Service updates:

ready_for_shipment → shipped

This keeps Order status ownership consistent.

8.28 Get Order Delivery

Endpoint

GET /orders/{orderId}/delivery

Integrated Response

```
{
  "success": true,
  "data": {
    "deliveryType": "integrated_logistics",

    "provider": {
      "id": "provider_a",
      "name": "Logistics Provider A"
    },

    "trackingId": "TRACK-928281",
    "trackingUrl": "https://provider.example/track/TRACK-928281",
    "status": "in_transit",
    "pickedUpAt": "2026-08-27T08:00:00Z",
  }
}
```

```
"estimatedDeliveryAt": "2026-08-29T17:00:00Z"
}
}
```

Seller-Arranged Response

```
{
  "success": true,
  "data": {
    "deliveryType": "seller_arranged",

    "providerName": "ABC Transport",
    "trackingId": "ABC938483",
    "trackingUrl": null,
    "deliveryContact": "+2348012345678",

    "estimatedDelivery": {
      "min": 4,
      "max": 6,
      "unit": "business_days"
    },

    "shippedAt": "2026-08-27T10:00:00Z"
  }
}
```

Buyer-Arranged Response — Before Handover

```
{
  "success": true,
  "data": {
    "deliveryType": "buyer_arranged",

    "buyerLogistics": {
      "providerName": "ABC Logistics",
      "contactName": "Kwame Mensah",
      "phoneNumber": "+233501234567",
      "email": "dispatch@abclogistics.com",
      "trackingReference": "ABC-928281",
      "expectedPickupDate": "2026-08-28",
      "notes": "Driver will call seller."
    },
  },
}
```

```
"handover": null
}
}
```

Buyer-Arranged Response — After Handover

```
{
  "success": true,
  "data": {
    "deliveryType": "buyer_arranged",

    "buyerLogistics": {
      "providerName": "ABC Logistics",
      "contactName": "Kwame Mensah",
      "phoneNumber": "+233501234567"
    },

    "handover": {
      "handoverTo": "Kwame Mensah",
      "handoverReference": "ABC-928281",
      "shippedAt": "2026-08-28T11:00:00Z",
      "notes": "Goods collected successfully."
    }
  }
}
```

8.29 Tracking History

Applicable mainly to integrated/B2B providers.

Endpoint

GET /orders/:orderId/delivery/tracking

Response

```
{
  "success": true,
  "data": {
    "trackingId": "TRACK-928281",
    "currentStatus": "in_transit",
```

```
"history": [  
  {  
    "status": "shipment_created",  
    "description": "Shipment created.",  
    "timestamp": "2026-08-26T15:00:00Z"  
  },  
  {  
    "status": "picked_up",  
    "description": "Shipment picked up from seller.",  
    "location": "Lagos",  
    "timestamp": "2026-08-27T08:00:00Z"  
  }  
]  
}  
}
```

8.30 Provider Webhook

Endpoint

POST /logistics/webhooks/:providerCode

Backend:

1. verify provider;
2. verify signature;
3. identify Shipment;
4. normalize provider status;
5. enforce idempotency;
6. update Shipment;
7. create ShipmentStatusHistory;
8. emit relevant event;
9. let Order Service update Order.

8.31 Delivered Provider Event

When:

shipment.status = delivered

Logistics emits:

SHIPMENT_DELIVERED

Order Service then moves:

shipped → delivered

Buyer later uses:

POST /orders/:orderId/confirm-receipt

to move:

delivered → received

8.32 Seller-Arranged Delivered Status

The Logistics module does not create another endpoint.

External seller-arranged delivered status follows the Order Management rules.

8.33 Buyer-Arranged Delivered Status

Buyer-arranged delivery does not have a different lifecycle.

shipped
↓
delivered
↓
received

The method used to establish **delivered** may depend on the operational policy, but Order Management remains responsible for the transition.

The buyer still confirms final receipt.

8.34 B2B Logistics

B2B logistics may require:

cargoType
quantity
unit
weight
volume
containerType
numberOfContainers
incoterm
originPort
destinationPort
customs
insurance
specialHandling

8.35 B2B Logistics Request

Endpoint

POST /logistics/b2b/requests

Payload

```
{  
  "sourceType": "direct_rfq",  
  "sourceId": "rfq_101",  
  
  "cargoType": "Cocoa Beans",  
  "quantity": 10,  
  "unit": "tonnes",  
  
  "pickupAddress": {  
    "city": "Lagos",  
    "country": "Nigeria"  
  },  
  
  "deliveryAddress": {  
    "city": "Accra",  
    "country": "Ghana"  
  },  
  
  "specialInstructions": "Covered transportation required."  
}
```

8.36 B2B Logistics Quote

```
{
  "quoteId": "b2b_quote_123",

  "provider": {
    "id": "provider_freight_1",
    "name": "Freight Partner"
  },

  "amount": 3500,
  "currency": "USD",

  "estimatedTransit": {
    "min": 5,
    "max": 7,
    "unit": "days"
  },

  "validUntil": "2026-08-30T23:59:59Z"
}
```

Buyer must accept before the amount enters Checkout.

8.37 Delivery Failure

Integrated provider may return:

delivery_failed

Do not automatically cancel Order.

Store reason and notify parties.

Buyer Toast

There is an issue with your delivery. Please review the delivery update.

Seller Toast

Delivery could not be completed. Please review the shipment details.

8.38 Tracking Provider Failure

If tracking API fails temporarily:

- retain last known status;
- do not mark shipment failed;
- retry later;
- inform frontend tracking is temporarily unavailable.

8.39 No Integrated Provider Available

If no integrated provider can quote:

seller_arranged
buyer_arranged

remain available where applicable.

8.40 Shipment Address Changes

Changing a saved UserAddress never changes an existing Order.

Shipment uses:

deliveryAddressSnapshot

Any post-Order address change requires a controlled support/provider workflow.

8.41 Cancel Integrated Shipment

Endpoint

POST /orders/:orderId/delivery/cancel

This cancels the provider shipment only.

It does not cancel the Order.

Order cancellation remains in Order Management.

8.42 Admin Providers

Endpoint

GET /admin/logistics/providers

Update

PATCH /admin/logistics/providers/:providerId/status

8.43 Admin Shipments

Endpoint

GET /admin/logistics/shipments

This mainly covers provider-managed integrated/B2B Shipments.

Seller/buyer-arranged delivery remains attached directly to Orders.

8.44 Logistics Notification Ownership

Logistics Sends Notifications For

Integrated shipment created
Provider pickup
Out for delivery
Provider delivery failure
B2B quote available

Order Management Sends Notifications For

Seller-arranged shipped
Buyer-arranged provider details submitted
Buyer-arranged handover/shipped
External delivered
Received
Completed
Cancellation

This prevents duplicate emails.

8.45 Integrated Shipment Email

Subject: Delivery scheduled for {OrderReference}

Hello {BuyerName},

Delivery has been scheduled for your Order.

Order Reference: {OrderReference}

Seller: {SellerName}

Provider: {ProviderName}

Tracking Reference: {TrackingReference}

Estimated Delivery: {EstimatedDelivery}

Best Regards,

Traders of Africa Team

CTA: Track Delivery

8.46 Integrated Pickup Email

Subject: Your order is on the way — {OrderReference}

Hello {BuyerName},

Your Order has been picked up by {ProviderName}.

Order Reference: {OrderReference}

Tracking Reference: {TrackingReference}

Estimated Delivery: {EstimatedDelivery}

Best Regards,

Traders of Africa Team

CTA: Track Delivery

8.47 Out for Delivery Email

Subject: Your order is out for delivery — {OrderReference}

Hello {BuyerName},

Your Order is currently out for delivery.

Order Reference: {OrderReference}

Provider: {ProviderName}

Tracking Reference: {TrackingReference}

Please ensure someone is available to receive the delivery where required.

Best Regards,

Traders of Africa Team

CTA: Track Delivery

8.48 B2B Quote Email

Subject: Logistics quotation available for {Reference}

Hello {BuyerName},

A B2B logistics quotation is available for your transaction.

Reference: {Reference}

Provider: {ProviderName}

Amount: {Currency} {Amount}

Estimated Transit: {EstimatedTransit}

Valid Until: {ValidUntil}

Please review the quotation before it expires.

Best Regards,

Traders of Africa Team

CTA: Review Logistics Quote

8.49 Frontend Implementation Guide

Buyer-Arranged

Before seller is ready:

Buyer-Arranged Delivery

You will provide your logistics details once the seller confirms that the goods are ready.

When Order becomes ready:

Goods Ready for Pickup

[Provide Logistics Details]

After buyer submits details:

Provider

Contact Person

Phone

Email

Pickup Date

Reference

Seller sees the same information.

After seller confirms handover:

Collected by your provider

Handover Person

Reference

Date

No duplicate logistics status controls should appear.

Seller-Arranged

Seller enters logistics details only through the Order **ship** action.

Integrated

Frontend may display:

Provider

Tracking

Shipment Timeline

Estimated Delivery

8.50 Backend Implementation Guide

Logistics Service owns:

- provider configuration;
- quotes;
- provider APIs;
- integrated shipments;
- tracking;
- webhooks;
- provider statuses;
- B2B requests/quotes.

Order Service owns:

- buyer logistics details;
- seller external handover;
- `/ship`;
- Order statuses;
- receipt;
- completion.

8.51 Internal Logistics Events

Recommended:

LOGISTICS_QUOTE_CREATED
LOGISTICS_QUOTE_SELECTED
LOGISTICS_QUOTE_EXPIRED

SHIPMENT_CREATED
SHIPMENT_PICKED_UP
SHIPMENT_IN_TRANSIT
SHIPMENT_OUT_FOR_DELIVERY
SHIPMENT_DELIVERED
SHIPMENT_DELIVERY_FAILED
SHIPMENT_CANCELLED

B2B_LOGISTICS_REQUEST_CREATED
B2B_LOGISTICS_QUOTE_CREATED
B2B_LOGISTICS_QUOTE_ACCEPTED

Do not duplicate:

BUYER_LOGISTICS_DETAILS_SUBMITTED

ORDER_SHIPPED
ORDER_DELIVERED

because those belong to Order Management.

8.52 Module Summary

The Logistics module establishes that:

- there is one Order lifecycle;
- Order Management owns fulfillment status;
- buyer-arranged logistics includes a buyer-provided coordination step;
- buyer does not mark goods shipped;
- seller confirms goods left seller custody;
- seller-arranged delivery data is entered at shipment time through Order;
- no duplicate shipment endpoint exists for external delivery;
- integrated shipment creation remains a Logistics responsibility;
- provider pickup/delivery events can trigger Order transitions;
- integrated quote and tracking logic remain provider-agnostic;
- B2B logistics is supported without changing the Order lifecycle;
- Logistics and Order Management complement each other without overlapping ownership.

MODULE 9. DIRECT RFQ MANAGEMENT

9. DIRECT RFQ MANAGEMENT MODULE

9.1 Overview

The Direct RFQ Management module handles a Request for Quotation sent by a buyer to one specific seller.

A Direct RFQ is different from a Market RFQ because only the selected seller can respond.

The purpose of Direct RFQ is to allow a buyer to negotiate commercial terms before proceeding to Checkout.

The module supports:

- creating an RFQ from a Product;
- selecting a specific seller;
- defining required quantity;
- defining delivery location;
- selecting a preferred delivery method;
- seller quotation;
- quotation validity;
- negotiation;
- quotation versioning;
- buyer acceptance;
- buyer rejection;
- RFQ cancellation;
- conversion of an accepted Quote into Checkout;
- Message Center linkage;
- notifications and emails;
- preservation of accepted commercial terms.

The Direct RFQ module does not:

- process Payment;
- create Orders directly;
- manage shipment fulfillment;
- award reviews or points.

Those responsibilities remain in their existing modules.

9.2 Direct RFQ Core Principle

A Direct RFQ belongs to:

One Buyer



One Seller



One RFQ



One Active Commercial Negotiation

Only the selected seller can submit quotations.

Other sellers cannot see or respond to the Direct RFQ.

9.3 Direct RFQ Relationship

The relationship is:

Product



Direct RFQ



Quotation



Negotiation



Accepted Quote



Checkout



Payment



Order

The Direct RFQ module stops when a valid accepted quotation has been passed into Checkout.

9.4 Direct RFQ Source

A Direct RFQ may originate from:

Product Detail Page

Seller Store

For the initial implementation, the primary source should be a Product.

9.5 Direct RFQ Statuses

Recommended RFQ statuses:

open
viewed
quoted
negotiating
accepted
rejected
cancelled
expired

9.6 Direct RFQ Status Definitions

open

Buyer created the RFQ successfully.

Seller has not yet opened it.

viewed

Seller opened the RFQ.

quoted

Seller has submitted a quotation.

negotiating

Buyer or seller has proposed revised commercial terms.

accepted

Buyer accepted the current valid quotation.

Once accepted, commercial terms are locked for Checkout.

rejected

Buyer rejected the seller's quotation and does not currently wish to continue.

A seller may submit an updated quotation later if the RFQ remains commercially open according to platform rules.

cancelled

Buyer cancelled the RFQ.

No further quotation or negotiation should be allowed.

expired

The RFQ or accepted quotation is no longer valid according to configured expiry rules.

9.7 Why **draft Is Not Required**

The buyer creates the Direct RFQ only when they submit it.

If the frontend needs to preserve an incomplete form, that should be handled through:

frontend local state

or

temporary form persistence

rather than introducing an RFQ **draft** state unless TOFA later explicitly wants server-side RFQ drafts.

Therefore the first backend RFQ status is:

open

9.8 Direct RFQ Database Structure

DirectRFQ

Field	Type	Required	Description
id	UUID/String	Yes	RFQ ID
rfqReference	String	Yes	Human-readable reference
buyerId	UUID/String	Yes	Buyer
sellerId	UUID/String	Yes	Selected seller
productId	UUID/String	Yes	Product being requested
variantId	UUID/String/ null	No	Variant if RFQ is variant-specific
status	Enum	Yes	RFQ status
quantity	Decimal	Yes	Requested quantity
unit	String	Yes	Quantity unit
description	String	Yes	Buyer requirement
expectedDelivery Date	Date/null	No	Requested delivery date
deliveryAddressId	UUID/String/ null	No	Saved buyer address reference
deliveryAddressS napshot	JSON	Yes	Delivery destination snapshot
deliveryType	Enum	Yes	integrated_logistics/seller_arranged/buyer_arranged/b2b_logistics
currencyPreference	String/null	No	Optional preferred commercial currency
buyerNotes	String/null	No	Additional notes
currentQuoteId	UUID/String/ null	No	Current active/latest Quote
acceptedQuoteId	UUID/String/ null	No	Accepted Quote

acceptedQuoteVersionId	UUID/String/null	No	Accepted version
viewedAt	Timestamp/null	No	First seller view
acceptedAt	Timestamp/null	No	Acceptance time
rejectedAt	Timestamp/null	No	Rejection time
cancelledAt	Timestamp/null	No	Cancellation
expiresAt	Timestamp/null	No	Optional RFQ expiry
createdAt	Timestamp	Yes	Created
updatedAt	Timestamp	Yes	Updated

9.9 Direct RFQ Reference

Example:

TOFA-RFQ-20260825-000101

The backend generates this reference.

It should be used in:

- UI;
- emails;
- Message Center;
- Admin dashboard;
- audit logs.

9.10 Product Snapshot

The RFQ should preserve the Product context at the time of creation.

Recommended:

DirectRFQProductSnapshot

```
{
  "productId": "prod_101",
  "productName": "Premium Cashew Nuts",
  "productImage": "https://cdn.tofa.com/products/cashew.jpg",
  "sellerId": "seller_123",
  "sellerStoreName": "Agro Supplies Ltd",

  "variant": {
    "variantId": "var_1",
    "sku": "CAS-A-10",
    "attributes": {
      "Grade": "A",
      "Weight": "10kg"
    }
  }
}
```

This ensures future Product edits do not change RFQ history.

9.11 Delivery Address in Direct RFQ

The buyer must provide a delivery destination because delivery terms may affect the seller's quotation.

The frontend should use the existing saved-address behavior already defined in Checkout/User modules:

Default saved address
or
Another saved address
or
Add new address

The RFQ stores:

deliveryAddressId
+
deliveryAddressSnapshot

The buyer should not need to type an existing address again.

9.12 Delivery Method Selection

Buyer chooses the preferred delivery method while creating the RFQ.

Supported:

integrated_logistics
seller_arranged
buyer_arranged
b2b_logistics

The seller's quotation behavior depends on this selection.

9.13 Delivery Type Rules

seller_arranged

Seller must provide the logistics amount when submitting the quotation.

The seller is responsible for obtaining the applicable delivery cost before quoting.

Example:

Products = USD 10,000
Seller Logistics = USD 700

Quote Total = USD 10,700

buyer_arranged

Seller does not provide logistics pricing.

The Quote covers only Product commercial terms.

integrated_logistics

Seller provides Product pricing.

Integrated logistics is calculated through the Logistics module after the product Quote is accepted or at the appropriate Checkout stage.

The Direct RFQ module must not call a second seller logistics workflow for this.

b2b_logistics

Seller provides Product commercial terms.

The Logistics module handles the B2B logistics quotation.

Checkout cannot be finalized until required B2B logistics pricing is known and accepted.

9.14 Create Direct RFQ

Endpoint

POST /rfqs/direct

Authentication

Authenticated buyer required.

Payload

```
{  
  "productId": "prod_101",  
  "variantId": "var_1",  
  
  "quantity": 500,  
  "unit": "kg",  
  
  "description": "Export-grade cashew nuts with moisture content below 10%.",  
  
  "expectedDeliveryDate": "2026-09-30",  
  
  "deliveryAddressId": "addr_456",  
  
  "deliveryType": "seller_arranged",  
  
  "currencyPreference": "USD",  
  
  "buyerNotes": "Export packaging is required."  
}
```

currencyPreference is optional.

The seller may quote in an allowed commercial currency according to TOFA business rules.

9.15 Create Direct RFQ Backend Validation

Backend must:

1. verify buyer;
2. verify Product exists;
3. verify Product is eligible for RFQ;
4. retrieve Product seller;
5. prevent buyer from requesting their own Product;
6. verify variant belongs to Product if supplied;
7. verify quantity > 0;
8. validate unit;
9. validate delivery address belongs to buyer;
10. snapshot delivery address;
11. validate deliveryType;
12. snapshot Product information;
13. generate RFQ Reference;
14. create RFQ;
15. set status **open**;
16. notify seller.

9.16 Create Direct RFQ Response

```
{
  "success": true,
  "message": "Request for quotation submitted successfully.",
  "data": {
    "rfqId": "rfq_101",
    "rfqReference": "TOFA-RFQ-20260825-000101",
    "status": "open",

    "seller": {
      "id": "seller_123",
      "storeName": "Agro Supplies Ltd"
    },

    "product": {
      "id": "prod_101",
      "name": "Premium Cashew Nuts"
    },

    "quantity": 500,
    "unit": "kg",
    "deliveryType": "seller_arranged",
```

```
"createdAt": "2026-08-25T10:00:00Z"  
}  
}
```

Buyer Toast

Request for quotation submitted successfully.

Seller Toast

You have received a new quotation request.

9.17 New Direct RFQ Email — Seller

Subject: New quotation request — {RFQReference}

Hello {SellerName},

You have received a new Request for Quotation on TOFA Marketplace.

RFQ Reference: {RFQReference}

Buyer: {BuyerName}

Product:

{ProductName}

{VariantDetails}

Requested Quantity: {Quantity} {Unit}

Requested Delivery Date: {ExpectedDeliveryDateDisplay}

Delivery Method: {DeliveryMethod}

Delivery Location:

{DeliveryAddress}

Buyer Requirement:

{Description}

Additional Notes:

{BuyerNotesDisplay}

Please review the request and submit your quotation.

Best Regards,
Traders of Africa Team

CTA: Review RFQ

9.18 Seller Views RFQ

The first time the seller opens the RFQ:

open
→ viewed

This should normally happen automatically through the GET endpoint.

Endpoint

GET /rfqs/direct/:rfqId

If authenticated seller is the RFQ seller and:

status = open

backend may set:

status = viewed
viewedAt = currentTime

The buyer may receive an in-app notification.

Email is not necessary merely because the seller opened the RFQ.

9.19 Get Direct RFQ

Endpoint

GET /rfqs/direct/:rfqId

Access

Allowed for:

- buyer;
- selected seller;
- authorized Admin.

Response

```
{
  "success": true,
  "data": {
    "rfqId": "rfq_101",
    "rfqReference": "TOFA-RFQ-20260825-000101",

    "status": "quoted",

    "buyer": {
      "id": "buyer_123",
      "name": "Victor Ejiogu"
    },

    "seller": {
      "id": "seller_456",
      "storeName": "Agro Supplies Ltd"
    },

    "product": {
      "id": "prod_101",
      "name": "Premium Cashew Nuts",
      "image": "https://cdn.tofa.com/products/cashew.jpg",

      "variant": {
        "variantId": "var_1",
        "sku": "CAS-A",
        "attributes": {
          "Grade": "A"
        }
      }
    },

    "request": {
      "quantity": 500,
      "unit": "kg",
```

```
"description": "Export-grade cashew nuts with moisture content below 10%.",

"expectedDeliveryDate": "2026-09-30",

"deliveryType": "seller_arranged",

"deliveryAddress": {
  "city": "Accra",
  "country": "Ghana"
},

"currentQuote": {
  "quoteId": "quote_901",
  "versionId": "quote_version_1",
  "version": 1,

  "quantity": 500,
  "unit": "kg",

  "pricePerUnit": 20,
  "productsTotal": 10000,

  "currency": "USD",

  "delivery": {
    "type": "seller_arranged",
    "logisticsAmount": 700
  },

  "quoteTotal": 10700,

  "estimatedDeliveryDate": "2026-09-28",

  "validUntil": "2026-08-30T23:59:59Z",

  "message": "Export packaging included."
},

"createdAt": "2026-08-25T10:00:00Z"
}
```


9.20 Direct RFQ Quote Structure

Quotation should be separated from the RFQ itself.

RFQQuote

Field	Type	Required
id	UUID/String	Yes
rfqId	UUID/String	Yes
sellerId	UUID/String	Yes
currentVersionId	UUID/String	Yes
status	Enum	Yes
createdAt	Timestamp	Yes
updatedAt	Timestamp	Yes

Recommended Quote statuses:

active
accepted
rejected
expired
superseded

9.21 Quote Version Structure

Every commercial revision should create a new version.

RFQQuoteVersion

Field	Type	Required
id	UUID/String	Yes
quoteId	UUID/String	Yes
version	Integer	Yes
createdByType	Enum	Yes

createdById	UUID/String	Yes
quantity	Decimal	Yes
unit	String	Yes
pricePerUnit	Decimal	Yes
productsTotal	Decimal	Yes
currency	String	Yes
deliveryType	Enum	Yes
logisticsAmount	Decimal/null	No
quoteTotal	Decimal	Yes
estimatedDeliveryDate	Date/null	No
message	String/null	No
validUntil	Timestamp	Yes
createdAt	Timestamp	Yes

9.22 Why Quote Versioning Is Required

Quotation negotiation should not overwrite historical terms.

Example:

Version 1
USD 20/kg
500kg

Buyer counter-offer

Version 2
USD 18/kg
600kg

Seller counter-offer

Version 3
USD 19/kg

600kg

If Version 3 is accepted, Order/Checkout must preserve:

quoteVersionId = Version 3

Versions 1 and 2 remain available for audit.

9.23 Seller Submits First Quotation

Endpoint

POST /rfqs/direct/:rfqId/quotes

Authentication

Selected seller only.

Seller-Arranged Payload

```
{
  "quantity": 500,
  "unit": "kg",

  "pricePerUnit": 20,
  "currency": "USD",

  "delivery": {
    "type": "seller_arranged",
    "logisticsAmount": 700
  },

  "estimatedDeliveryDate": "2026-09-28",

  "validUntil": "2026-08-30T23:59:59Z",

  "message": "Export packaging is included."
}
```

Backend calculates:

productsTotal =
quantity × pricePerUnit

quoteTotal =
productsTotal + logisticsAmount

9.24 Buyer-Arranged Quote Payload

```
{  
  "quantity": 500,  
  "unit": "kg",  
  
  "pricePerUnit": 20,  
  "currency": "USD",  
  
  "delivery": {  
    "type": "buyer_arranged",  
    "logisticsAmount": null  
  },  
  
  "estimatedDeliveryDate": "2026-09-28",  
  
  "validUntil": "2026-08-30T23:59:59Z",  
  
  "message": "Goods will be available for collection from our Lagos warehouse."  
}
```

9.25 Integrated Logistics Quote Payload

Seller quotes Product terms only.

```
{  
  "quantity": 500,  
  "unit": "kg",  
  
  "pricePerUnit": 20,  
  "currency": "USD",  
  
  "delivery": {  
    "type": "integrated_logistics",  
    "logisticsAmount": null  
  },  
}
```

```
"estimatedDeliveryDate": "2026-09-28",  
  
"validUntil": "2026-08-30T23:59:59Z"  
}
```

Integrated logistics pricing is obtained from the existing Logistics module.

The seller must not manually enter an integrated logistics amount.

9.26 B2B Logistics Quote Payload

Seller again quotes Product commercial terms only.

```
{  
  "quantity": 10,  
  "unit": "tonnes",  
  
  "pricePerUnit": 1000,  
  "currency": "USD",  
  
  "delivery": {  
    "type": "b2b_logistics",  
    "logisticsAmount": null  
  },  
  
  "validUntil": "2026-08-30T23:59:59Z"  
}
```

B2B logistics amount is obtained through the Logistics module.

9.27 Seller Quote Backend Validation

Backend must:

1. verify RFQ;
2. verify authenticated seller owns RFQ;
3. verify RFQ is not accepted/cancelled/expired;
4. validate quantity;
5. validate unit;
6. validate currency;
7. verify deliveryType matches buyer RFQ selection;

8. require logisticsAmount only for seller-arranged delivery;
9. prohibit seller logisticsAmount for buyer-arranged/integrated/B2B;
10. validate `validUntil`;
11. calculate Product total;
12. calculate Quote total;
13. create RFQQuote if first Quote;
14. create QuoteVersion;
15. update `currentQuoteId`;
16. update RFQ status to `quoted`;
17. notify buyer.

9.28 Quote Response

```
{
  "success": true,
  "message": "Quotation submitted successfully.",
  "data": {
    "rfqId": "rfq_101",
    "status": "quoted",

    "quote": {
      "quoteId": "quote_901",
      "versionId": "quote_version_1",
      "version": 1,

      "pricePerUnit": 20,
      "quantity": 500,
      "unit": "kg",

      "productsTotal": 10000,
      "logisticsAmount": 700,
      "quoteTotal": 10700,

      "currency": "USD",

      "validUntil": "2026-08-30T23:59:59Z"
    }
  }
}
```

Seller Toast

Quotation submitted successfully.

Buyer Toast

A quotation has been received for your RFQ.

9.29 Quotation Received Email — Buyer

Subject: Quotation received — {RFQReference}

Hello {BuyerName},

{SellerName} has submitted a quotation for your Direct RFQ.

RFQ Reference: {RFQReference}

Product: {ProductName}

Quantity: {Quantity} {Unit}

Price Per Unit: {Currency} {PricePerUnit}

Products Total: {Currency} {ProductsTotal}

Logistics: {LogisticsDisplay}

Quote Total: {Currency} {QuoteTotal}

Estimated Delivery: {EstimatedDeliveryDisplay}

Quote Valid Until: {ValidUntil}

Seller Message:

{SellerMessage}

Please review the quotation before it expires.

You can accept the quotation, reject it, or propose revised terms.

Best Regards,

Traders of Africa Team

CTA: Review Quotation

9.30 Negotiation Principle

Negotiation is handled by creating new Quote versions.

The system should not maintain a second separate negotiation price object that conflicts with the Quote.

Conceptually:

Current Quote Version



Buyer/Seller proposes change



New Quote Version



Current Quote Version updated

Every version represents the full commercial state.

9.31 Submit Counter-Offer

Use one endpoint for either buyer or seller where permitted.

Endpoint

POST /rfqs/direct/:rfqId/quotes/:quoteId/counter-offer

Buyer Example

```
{
  "quantity": 600,
  "unit": "kg",

  "pricePerUnit": 18,
  "currency": "USD",

  "delivery": {
    "type": "seller_arranged",
    "logisticsAmount": 750
  },

  "expectedDeliveryDate": "2026-09-30",

  "validUntil": "2026-08-31T23:59:59Z",

  "message": "Can you accept USD 18/kg for 600kg?"
}
```


For seller-arranged delivery, if the **buyer** proposes logistics amount changes, the seller still needs to agree by responding with another Quote version.

The buyer cannot unilaterally determine the seller's logistics charge.

9.32 Counter-Offer Backend Rules

Backend must:

- verify actor is buyer or selected seller;
- verify Quote/RFQ active;
- verify current version not expired where applicable;
- create a new immutable version;
- set `createdByType`;
- set RFQ status `negotiating`;
- update current version;
- notify the opposite party.

The counter-offer must contain the complete proposed commercial terms, not only changed fields.

This prevents ambiguity about the current agreed proposal.

9.33 Counter-Offer Response

```
{
  "success": true,
  "message": "Counter-offer submitted successfully.",
  "data": {
    "rfqId": "rfq_101",
    "status": "negotiating",

    "currentQuote": {
      "quoteId": "quote_901",
      "versionId": "quote_version_2",
      "version": 2,

      "createdBy": "buyer",

      "quantity": 600,
      "unit": "kg",
      "pricePerUnit": 18,
```

```
"currency": "USD",

"productsTotal": 10800,
"logisticsAmount": 750,
"quoteTotal": 11550,

"validUntil": "2026-08-31T23:59:59Z"
}
}
}
```

Toast

Counter-offer submitted successfully.

9.34 Counter-Offer Email

Subject: New counter-offer — {RFQReference}

Hello {RecipientName},

A new counter-offer has been submitted for your RFQ.

RFQ Reference: {RFQReference}

Product: {ProductName}

Proposed Terms:

Quantity: {Quantity} {Unit}

Price Per Unit: {Currency} {PricePerUnit}

Products Total: {Currency} {ProductsTotal}

Logistics: {LogisticsDisplay}

Quote Total: {Currency} {QuoteTotal}

Expected Delivery: {ExpectedDeliveryDisplay}

Valid Until: {ValidUntil}

Message:

{Message}

Please review the updated terms.

Best Regards,

Traders of Africa Team

CTA: Review Counter-Offer

9.35 Get Quote Details

Endpoint

GET /rfqs/direct/:rfqId/quotes/:quoteId

Response

```
{
  "success": true,
  "data": {
    "rfqId": "rfq_101",
    "rfqReference": "TOFA-RFQ-20260825-000101",

    "quoteId": "quote_901",

    "currentVersion": {
      "versionId": "quote_version_3",
      "version": 3,
      "createdBy": "seller",

      "quantity": 600,
      "unit": "kg",
      "pricePerUnit": 19,
      "currency": "USD",

      "productsTotal": 11400,
      "logisticsAmount": 750,
      "quoteTotal": 12150,

      "estimatedDeliveryDate": "2026-09-30",

      "validUntil": "2026-09-01T23:59:59Z",

      "message": "USD 19/kg is our final offer."
    },

    "history": [
      {
        "version": 1,
        "createdBy": "seller",
        "pricePerUnit": 20,
```

```

    "quantity": 500,
    "quoteTotal": 10700,
    "createdAt": "2026-08-25T12:00:00Z"
  },
  {
    "version": 2,
    "createdBy": "buyer",
    "pricePerUnit": 18,
    "quantity": 600,
    "quoteTotal": 11550,
    "createdAt": "2026-08-25T13:00:00Z"
  },
  {
    "version": 3,
    "createdBy": "seller",
    "pricePerUnit": 19,
    "quantity": 600,
    "quoteTotal": 12150,
    "createdAt": "2026-08-25T14:00:00Z"
  }
]
}
}

```

9.36 Quote Validity

Every seller-side Quote version intended for buyer acceptance must have:

`validUntil`

The buyer cannot accept after:

`currentTime > validUntil`

Error

```

{
  "success": false,
  "code": "QUOTE_EXPIRED",
  "message": "This quotation has expired. Please request updated terms from the seller."
}

```

Toast

This quotation has expired. Please request an updated quotation from the seller.

9.37 Expired Quote Update

An expired Quote should not be edited in place.

Seller submits a new version with:

new price
new logistics where applicable
new validity

Previous versions remain immutable.

9.38 Accept Direct RFQ Quote

The buyer accepts the current valid seller-generated Quote version.

Endpoint

POST /rfqs/direct/:rfqId/quotes/:quoteId/accept

Payload

```
{  
  "quoteVersionId": "quote_version_3"  
}
```

The frontend does not send price or totals.

Backend obtains accepted terms from the Quote version.

9.39 Accept Quote Backend Processing

Backend must:

1. verify buyer owns RFQ;
2. verify RFQ is active;
3. verify Quote belongs to RFQ;

4. verify version is current;
5. verify version was offered by seller;
6. verify version is not expired;
7. verify no other Quote has already been accepted;
8. set Quote status **accepted**;
9. set RFQ status **accepted**;
10. store acceptedQuoteId;
11. store acceptedQuoteVersionId;
12. store acceptedAt;
13. lock commercial terms;
14. trigger Checkout preparation;
15. notify seller.

The Direct RFQ module does not create Payment or Order.

9.40 Accept Quote Response

If seller-arranged or buyer-arranged requires no additional platform logistics pricing:

```
{
  "success": true,
  "message": "Quotation accepted successfully.",
  "data": {
    "rfqId": "rfq_101",
    "status": "accepted",

    "acceptedQuote": {
      "quoteId": "quote_901",
      "quoteVersionId": "quote_version_3"
    },

    "nextStep": "checkout"
  }
}
```

Buyer Toast

Quotation accepted successfully. Please continue to checkout.

Seller Toast

The buyer has accepted your quotation.

9.41 Accepted Quote with Integrated Logistics

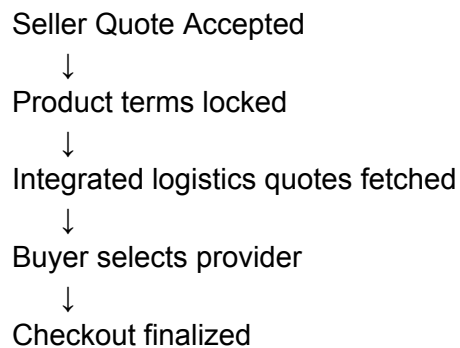
If:

deliveryType = integrated_logistics

acceptance locks Product commercial terms first.

Then Checkout/Logistics obtains integrated logistics options.

Flow:



Direct RFQ must not create a separate logistics quotation flow.

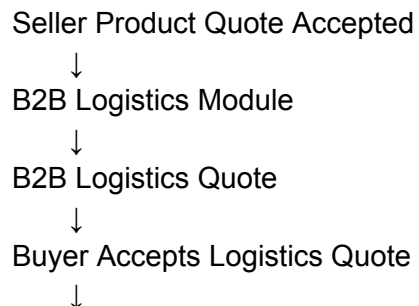
It reuses the Logistics module.

9.42 Accepted Quote with B2B Logistics

If:

deliveryType = b2b_logistics

then:



Checkout finalized

Payment cannot begin until every platform-payable amount is known.

9.43 Quote Accepted Email — Seller

Subject: Your quotation has been accepted — {RFQReference}

Hello {SellerName},

{BuyerName} has accepted your quotation for the following Direct RFQ:

RFQ Reference: {RFQReference}

Product: {ProductName}

Accepted Terms:

Quantity: {Quantity} {Unit}

Price Per Unit: {Currency} {PricePerUnit}

Products Total: {Currency} {ProductsTotal}

Logistics: {LogisticsDisplay}

Accepted Quote Total: {Currency} {QuoteTotal}

The buyer will now proceed through the applicable Checkout and Payment process.

You will receive an Order notification once the transaction becomes eligible for fulfillment.

Best Regards,

Traders of Africa Team

CTA: View RFQ

This wording is important because accepting the Quote does **not** mean an Order already exists.

9.44 Reject Quote

Endpoint

POST /rfqs/direct/{rfqId}/quotes/{quoteId}/reject

Payload

```
{  
  "reason": "The current price is above our budget."  
}
```



```
}
```

Backend

1. verify buyer;
2. verify Quote;
3. set Quote status **rejected**;
4. set RFQ status **rejected**;
5. store rejection reason/time;
6. notify seller.

Response

```
{  
  "success": true,  
  "message": "Quotation rejected.",  
  "data": {  
    "rfqId": "rfq_101",  
    "status": "rejected"  
  }  
}
```

Buyer Toast

Quotation rejected successfully.

9.45 Quote Rejected Email — Seller

Subject: Quotation update — {RFQReference}

Hello {SellerName},

The buyer has declined your current quotation for:

RFQ Reference: {RFQReference}

Product: {ProductName}

Reason:

{RejectionReason}

Where the RFQ remains eligible for further discussion, you may submit revised terms.

Best Regards,
Traders of Africa Team

CTA: View RFQ

9.46 Cancel Direct RFQ

Buyer may cancel before Quote acceptance.

Endpoint

POST /rfqs/direct/:rfqId/cancel

Payload

```
{  
  "reason": "Requirement is no longer needed."  
}
```

Not Allowed When

status = accepted

because an accepted RFQ has moved into the Checkout/payment transaction flow.

If the buyer wants to stop after acceptance, the applicable Checkout/Payment/Order cancellation rules should be used depending on transaction stage.

Response

```
{  
  "success": true,  
  "message": "RFQ cancelled successfully.",  
  "data": {  
    "rfqId": "rfq_101",  
    "status": "cancelled"  
  }  
}
```

9.47 RFQ Cancelled Email — Seller

Subject: RFQ cancelled — {RFQReference}

Hello {SellerName},

The buyer has cancelled the following Direct RFQ:

RFQ Reference: {RFQReference}

Product: {ProductName}

Reason:

{CancellationReason}

No further action is required for this RFQ.

Best Regards,
Traders of Africa Team

CTA: View RFQ

9.48 Get User Direct RFQs

Endpoint

GET /rfqs/direct

Query

type

status

search

dateFrom

dateTo

page

limit

type:

outgoing

incoming

outgoing

Buyer-created Direct RFQs.

incoming

RFQs received by authenticated seller.

9.49 Outgoing RFQ Response

```
{
  "success": true,
  "data": [
    {
      "rfqId": "rfq_101",
      "rfqReference": "TOFA-RFQ-20260825-000101",

      "seller": {
        "id": "seller_456",
        "storeName": "Agro Supplies Ltd"
      },

      "product": {
        "id": "prod_101",
        "name": "Premium Cashew Nuts",
        "image": "https://cdn.tofa.com/products/cashew.jpg"
      },

      "quantity": 500,
      "unit": "kg",
      "deliveryType": "seller_arranged",
      "status": "quoted",

      "currentQuote": {
        "currency": "USD",
        "quoteTotal": 10700,
        "validUntil": "2026-08-30T23:59:59Z"
      },

      "createdAt": "2026-08-25T10:00:00Z"
    }
  ],

  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 12,
    "totalPages": 2
  }
}
```

```
}  
}
```

9.50 Incoming RFQ Response

```
{  
  "success": true,  
  "data": [  
    {  
      "rfqId": "rfq_101",  
      "rfqReference": "TOFA-RFQ-20260825-000101",  
  
      "buyer": {  
        "id": "buyer_123",  
        "name": "Victor Ejiogu"  
      },  
  
      "product": {  
        "id": "prod_101",  
        "name": "Premium Cashew Nuts"  
      },  
  
      "quantity": 500,  
      "unit": "kg",  
  
      "deliveryType": "seller_arranged",  
      "status": "viewed",  
  
      "createdAt": "2026-08-25T10:00:00Z"  
    }  
  ]  
}
```

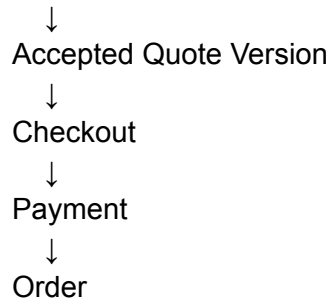
9.51 Checkout Relationship

Direct RFQ uses the existing Checkout module.

An accepted quotation does not create a separate RFQ-specific Payment architecture.

Use:

Direct RFQ



Checkout receives:

sourceType = direct_rfq
sourceId = rfqId
quoteId
quoteVersionId

9.52 Pricing Relationship with Checkout

For Direct RFQ:

Checkout MUST NOT use current Product price.

It uses:

accepted Quote version

The current Product may have changed since RFQ negotiation started.

Accepted commercial terms remain authoritative.

9.53 FX Relationship

The Direct RFQ Quote stores:

commercial currency

This becomes the Order's future:

orderCurrency

The buyer may select another:

paymentCurrency

during Checkout.

FX is handled by the existing Checkout rules.

Direct RFQ does not calculate buyer FX.

Example:

Quote = USD 10,700

Buyer chooses NGN payment



Checkout obtains FX quote



Buyer sees NGN total



Payment in NGN

Order later preserves:

orderCurrency = USD

paymentCurrency = NGN

FX snapshot

9.54 Logistics Relationship

Direct RFQ uses existing Logistics rules.

seller_arranged

Seller supplies logistics amount in Quote.

buyer_arranged

No logistics amount in Quote.

After Order becomes ready, buyer provides logistics details using the existing Order endpoint:

PATCH /orders/:orderId/buyer-logistics

integrated_logistics

Logistics module obtains provider quotes.

b2b_logistics

B2B Logistics module/request flow obtains the logistics quotation.

There must not be RFQ-specific duplicate logistics endpoints.

9.55 Message Center Relationship

Each Direct RFQ should be linkable to a Message Center conversation.

Recommended relationship:

conversationContextType = direct_rfq
contextId = rfqId

The Message Center can support discussion between buyer and seller.

However, formal commercial terms must remain in Quote versions.

A price written only in chat is not an accepted commercial Quote.

Example:

Chat:

"Can you do USD 18?"

Not authoritative.

Quote Version:

USD 18

Quantity 600

Valid Until...

The Quote Version is authoritative.

9.56 Direct RFQ Notification Matrix

Event	Buyer Toast	Seller Toast	Email
RFQ created	Yes	Yes	Seller
RFQ viewed	Optional	No	No
Quote received	Yes	Yes	Buyer
Counter-offer	Yes	Yes	Opposite party
Quote accepted	Yes	Yes	Seller
Quote rejected	Yes	Yes	Seller
Quote expired	Yes	Optional	Optional
RFQ cancelled	Yes	Yes	Seller
Checkout ready	Yes	No	Existing Checkout flow

9.57 Admin Get Direct RFQs

Endpoint

GET /admin/rfqs/direct

Query

status
buyerId
sellerId
productId
deliveryType
currency
search
dateFrom
dateTo
page
limit

Response

```
{
  "success": true,
  "data": [
    {
      "rfqId": "rfq_101",
      "rfqReference": "TOFA-RFQ-20260825-000101",

      "buyer": {
        "id": "buyer_123",
        "name": "Victor Ejiogu"
      },

      "seller": {
        "id": "seller_456",
        "storeName": "Agro Supplies Ltd"
      },

      "product": {
        "id": "prod_101",
        "name": "Premium Cashew Nuts"
      },

      "status": "negotiating",
      "deliveryType": "seller_arranged",

      "currentQuote": {
        "currency": "USD",
        "quoteTotal": 12150,
        "version": 3
      },

      "createdAt": "2026-08-25T10:00:00Z"
    }
  ],

  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 40,
    "totalPages": 2
  }
}
```

9.58 Admin Get Direct RFQ Details

Endpoint

GET /admin/rfqs/direct/:rfqId

Admin may view:

- buyer;
- seller;
- Product snapshot;
- RFQ requirements;
- current Quote;
- all Quote versions;
- delivery method;
- Message Center reference;
- acceptance details;
- Checkout reference where applicable;
- Payment reference after Payment creation;
- Order reference after Order creation;
- audit history.

Admin should not casually edit Quote commercial terms.

9.59 Frontend Implementation Guide

Buyer RFQ Details

Show:

RFQ Reference

Seller

Product

Variant

Requested Quantity

Delivery Address

Delivery Method

Requested Delivery Date

Buyer Requirement

Current Quote

Negotiation History

Message Seller

Available Actions

Available buyer actions depend on status:

open/viewed

→ Wait / Message Seller / Cancel

quoted

→ Accept / Counter / Reject

negotiating

→ Accept seller offer / Counter

accepted

→ Continue to Checkout

cancelled/expired

→ No commercial action

Seller RFQ Details

Show:

Buyer

Product

Quantity

Destination

Delivery Method

Requirements

Quote History

Message Buyer

Seller actions:

open/viewed

→ Submit Quote

negotiating

→ Submit Counter-Offer

accepted

→ Wait for Order

cancelled

→ No Action

9.60 Backend Implementation Guide

Direct RFQ Service owns:

- RFQ creation;
- selected seller relationship;
- RFQ statuses;
- Quote creation;
- Quote versioning;
- negotiation;
- acceptance;
- rejection;
- RFQ cancellation;
- commercial term locking.

It does not own:

- FX calculation;
- Payment;
- Order creation;
- integrated logistics provider APIs;
- buyer-arranged handover;
- review points.

Those use existing modules.

9.61 Internal Events

Recommended:

DIRECT_RFQ_CREATED

DIRECT_RFQ_VIEWED

DIRECT_RFQ_QUOTE_CREATED

DIRECT_RFQ_COUNTER_OFFER_CREATED

DIRECT_RFQ_QUOTE_ACCEPTED

DIRECT_RFQ_QUOTE_REJECTED

DIRECT_RFQ_QUOTE_EXPIRED

DIRECT_RFQ_CANCELLED

When Quote is accepted:

DIRECT_RFQ_QUOTE_ACCEPTED



Checkout module



prepare Direct RFQ Checkout

9.62 Idempotency Rules

Acceptance must be idempotent.

If the same request is sent twice:

Accept quote

Accept quote again

the system must not create two Checkouts or two accepted versions.

The same accepted Quote version remains authoritative.

9.63 Audit Rules

Important actions should be preserved:

RFQ created

Seller viewed

Quote submitted

Counter-offer submitted

Quote accepted

Quote rejected

Quote expired

RFQ cancelled

Quote versions must never be deleted merely because a later version exists.

9.64 Module Summary

The Direct RFQ module establishes that:

- a Direct RFQ targets exactly one seller;
- only that seller can respond;
- RFQ starts at **open**, not draft;
- Product and delivery information are snapshotted;
- buyer selects delivery method during RFQ creation;
- seller-arranged delivery pricing is supplied by seller in the Quote;
- buyer-arranged delivery contains no logistics amount;
- integrated and B2B logistics reuse the existing Logistics module;
- negotiation uses immutable Quote versions;
- current Quote represents the latest complete commercial proposal;
- expired Quotes cannot be accepted;
- buyer acceptance locks the exact Quote version;
- accepted Quote does not immediately create an Order;
- accepted Quote proceeds to the existing Checkout module;
- Checkout uses accepted Quote pricing, not current Product pricing;
- Checkout handles buyer Payment currency and FX;
- Payment and Order reuse their existing modules;
- Message Center may support discussion but does not replace formal Quote terms;
- no duplicate logistics, payment, or Order endpoint is introduced.

MODULE 10. MARKET RFQ MANAGEMENT

10. MARKET RFQ MANAGEMENT MODULE

10.1 Overview

The Market RFQ Management module handles a Request for Quotation that is not directed to one specific seller.

Instead, eligible sellers on TOFA Marketplace can discover and respond to the RFQ.

The Market RFQ flow is intentionally similar to the Direct RFQ flow, with one major difference:

Direct RFQ

→ one selected seller can respond

Market RFQ

→ multiple eligible sellers can respond

The buyer can compare quotations and select one seller's quotation to proceed.

The module supports:

- creating a Market RFQ;
- specifying Product or product requirement;
- specifying quantity;
- specifying delivery location;
- selecting delivery method;
- making the RFQ available to eligible sellers;
- multiple seller quotations;
- seller-specific negotiation;
- independent Quote versioning;
- buyer comparison of quotations;
- buyer acceptance of one quotation;
- automatic closure of competing quotations after acceptance;
- buyer rejection of individual quotations;
- RFQ cancellation;
- quotation expiry;
- Checkout creation from the selected quotation;
- Message Center relationship;
- notifications and emails.

The Market RFQ module does not process:

- Payment;
- Order creation;
- shipment execution;
- FX conversion;
- review rewards.

Those responsibilities remain in their existing modules.

10.2 Core Market RFQ Principle

A Market RFQ has:

One Buyer



One Market RFQ



Multiple Eligible Sellers



Multiple Seller Quotations



Buyer Selects One

Each seller has a separate commercial negotiation with the buyer.

Therefore:

Seller A Quote

Seller B Quote

Seller C Quote

must never share one quotation history.

Each seller's Quote and negotiation history is independent.

10.3 Market RFQ Relationship

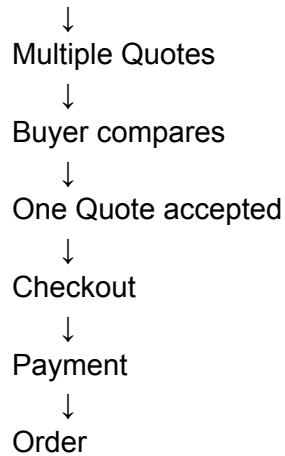
Buyer Requirement



Market RFQ



Eligible Sellers



Once one Quote is accepted, the Market RFQ is awarded to that seller and the remaining competing quotations are closed.

10.4 Market RFQ Source

A Market RFQ may be created:

- from a Product/category context;
- from the buyer dashboard;
- from a general “Request a Quote” action;
- from a requirement where the buyer does not yet know the seller.

Unlike Direct RFQ, **productId** does not always need to be mandatory.

This is particularly important for B2B sourcing.

Example:

Buyer wants:

500 tonnes of Grade A cocoa beans

Buyer does not care which seller supplies it.

The RFQ can therefore target:

Category

+

Product Requirement

rather than one existing Product.

10.5 Market RFQ Statuses

Recommended:

open
quoted
negotiating
awarded
cancelled
expired

Individual seller quotations maintain their own Quote statuses.

10.6 Market RFQ Status Definitions

open

Market RFQ has been published and eligible sellers may respond.

quoted

At least one seller has submitted a quotation.

The RFQ remains available to additional eligible sellers unless its response period has ended.

negotiating

Buyer is actively negotiating with at least one seller.

Other eligible sellers may still submit or maintain quotations where RFQ rules permit.

awarded

Buyer accepted one seller's valid quotation.

This is the Market RFQ equivalent of Direct RFQ **accepted**.

Only one seller can be awarded for the initial implementation.

cancelled

Buyer cancelled the RFQ before award.

expired

The Market RFQ response window ended without an accepted quotation, or the RFQ otherwise reached its configured expiry.

10.7 Why **viewed** Is Not a Market RFQ Status

For Direct RFQ, **viewed** is useful because there is one recipient seller.

For Market RFQ, multiple sellers may view the same request.

Changing the entire RFQ status to:

viewed

would not provide useful information.

Instead, seller views can be stored separately.

Example:

MarketRFQSellerView

Field	Type
id	UUID/String
rfqId	UUID/String
sellerId	UUID/String
firstViewedAt	Timestamp
lastViewedAt	Timestamp

This provides analytics without complicating the Market RFQ lifecycle.

10.8 Market RFQ Database Structure

MarketRFQ

Field	Type	Required	Description
id	UUID/String	Yes	RFQ ID
rfqReference	String	Yes	Human-readable RFQ reference
buyerId	UUID/String	Yes	Buyer
productId	UUID/String/null	No	Existing Product where applicable
variantId	UUID/String/null	No	Variant context where applicable
categoryIds	Array	Yes	Relevant categories
requirementTitle	String/JSON	Yes	Requirement title
description	String/JSON	Yes	Buyer requirement
quantity	Decimal	Yes	Requested quantity
unit	String	Yes	Quantity unit
expectedDeliveryDate	Date/null	No	Requested delivery date
deliveryAddressId	UUID/String/null	No	Saved address
deliveryAddressSnapshot	JSON	Yes	Delivery destination
deliveryType	Enum	Yes	integrated/seller/buyer/B2B
currencyPreference	String/null	No	Preferred Quote currency
buyerNotes	String/null	No	Additional notes
status	Enum	Yes	RFQ lifecycle
quotesCount	Integer	Yes	Number of seller Quotes
awardedSellerId	UUID/String/null	No	Winning seller
acceptedQuoteId	UUID/String/null	No	Winning Quote
acceptedQuoteVersionId	UUID/String/null	No	Winning version
submissionDeadline	Timestamp	Yes	Seller response deadline
awardedAt	Timestamp/null	No	Award time

cancelledAt	Timestamp/null	No	Cancellation
cancellationReason	String/null	No	Cancellation reason
createdAt	Timestamp	Yes	Created
updatedAt	Timestamp	Yes	Updated

10.9 Requirement Title and Description Translation

Market RFQs may be visible to sellers in multiple countries.

Therefore the existing internationalization architecture should apply.

Example:

```
{
  "requirementTitle": {
    "en": "Export Grade Cocoa Beans",
    "fr": "Fèves de cacao de qualité export"
  },

  "description": {
    "en": "We require Grade A cocoa beans...",
    "fr": "Nous recherchons des fèves..."
  }
}
```

The system's default display language remains English.

The full translation/storage rules remain owned by the Internationalization module.

The Market RFQ module should not introduce separate translation behavior.

10.10 Seller Eligibility

Not every seller should automatically receive every Market RFQ.

Recommended seller eligibility may consider:

- verified seller status;
- active account;
- Product/category match;

- seller country;
- delivery capability;
- supported business sector;
- Marketplace compliance;
- seller subscription level where future RFQ benefits allow;
- Admin-configured RFQ distribution rules.

Subscription may influence visibility or priority, but should not allow an unqualified seller to bypass eligibility rules.

10.11 Market RFQ Visibility

Recommended:

Eligible sellers can discover the RFQ in:
Seller Dashboard → Market RFQs

TOFA may also notify matching sellers.

A seller should not see confidential buyer information beyond what is required to quote.

For example, before Quote acceptance, the platform may expose:

Buyer Company Name
Country
Delivery City/Country
Requirement
Quantity

while withholding unnecessary personal/contact information.

Message Center can provide controlled communication.

10.12 Delivery Method Rules

Market RFQ reuses the same delivery rules as Direct RFQ.

Supported:

seller_arranged
buyer_arranged
integrated_logistics
b2b_logistics

seller_arranged

Each seller independently provides their own logistics amount.

This means quotations may differ in both:

Product Price
+
Logistics Price

buyer_arranged

Seller quotes Product terms only.

integrated_logistics

Seller quotes Product terms only.

Integrated logistics comes from the Logistics module.

b2b_logistics

Seller quotes Product terms.

B2B Logistics module provides applicable freight quote.

10.13 Create Market RFQ

Endpoint

POST /rfqs/market

Authentication

Authenticated buyer required.

Example Using Existing Product Context

```
{  
  "productId": "prod_101",  
  "variantId": null,
```

```
"categoryIds": [  
  "cat_agriculture",  
  "cat_cashew"  
],  
  
"requirementTitle": "Premium Cashew Nuts",  
  
"description": "Export-grade cashew nuts with moisture content below 10%.",  
  
"quantity": 500,  
"unit": "kg",  
  
"expectedDeliveryDate": "2026-09-30",  
  
"deliveryAddressId": "addr_456",  
  
"deliveryType": "seller_arranged",  
  
"currencyPreference": "USD",  
  
"submissionDeadline": "2026-09-01T23:59:59Z",  
  
"buyerNotes": "Export packaging required."  
}
```

10.14 Create Market RFQ Without Existing Product

Example:

```
{  
  "productId": null,  
  "variantId": null,  
  
  "categoryIds": [  
    "cat_agriculture",  
    "cat_cocoa"  
  ],  
  
  "requirementTitle": "Grade A Cocoa Beans",  
  
  "description": "We require export-quality Grade A cocoa beans for delivery to Accra.",  
}
```

```
"quantity": 20,  
"unit": "tonnes",  
  
"expectedDeliveryDate": "2026-10-15",  
  
"deliveryAddressId": "addr_456",  
  
"deliveryType": "b2b_logistics",  
  
"currencyPreference": "USD",  
  
"submissionDeadline": "2026-09-05T23:59:59Z"  
}
```

10.15 Create Market RFQ Backend Processing

Backend must:

1. verify buyer;
2. validate categoryIds;
3. validate Product/Variant if provided;
4. validate quantity;
5. validate unit;
6. validate delivery address ownership;
7. snapshot delivery address;
8. validate delivery type;
9. validate submission deadline;
10. generate RFQ Reference;
11. create Market RFQ;
12. set status **open**;
13. determine eligible sellers;
14. create seller visibility/index records where required;
15. notify appropriate sellers.

10.16 Create Market RFQ Response

```
{  
  "success": true,  
  "message": "Market RFQ published successfully.",  
  
  "data": {  
    "rfqId": "market_rfq_101",  
    "rfqReference": "TOFA-MRFQ-20260825-000101",  
  }  
}
```

```
"requirementTitle": "Grade A Cocoa Beans",

"quantity": 20,
"unit": "tonnes",

"deliveryType": "b2b_logistics",

"status": "open",

"submissionDeadline": "2026-09-05T23:59:59Z",

"createdAt": "2026-08-25T20:00:00Z"
}
}
```

Buyer Toast

Market RFQ published successfully.

10.17 Eligible Seller Notification

Not every seller necessarily needs an email for every matching RFQ because this may generate excessive email volume.

Recommended:

- in-app notification for all eligible matches;
- email where seller notification preferences allow;
- future daily/periodic RFQ digest for high-volume environments.

Seller Toast

A new Market RFQ matching your business is available.

Seller Email

Subject: New Market RFQ available — {RFQReference}

Hello {SellerName},

A new sourcing requirement matching your business is available on TOFA Marketplace.

RFQ Reference: {RFQReference}
Requirement: {RequirementTitle}
Quantity: {Quantity} {Unit}
Delivery Location: {DeliveryLocation}
Delivery Method: {DeliveryMethod}
Quotation Deadline: {SubmissionDeadline}

Buyer Requirement:

{Description}

If you can supply the requested goods, please review the RFQ and submit your quotation before the deadline.

Best Regards,
Traders of Africa Team

CTA: View RFQ

10.18 Get Available Market RFQs for Seller

Endpoint

GET /rfqs/market/available

Authentication

Verified seller required.

Query

categoryId
deliveryCountry
deliveryType
search
dateFrom
dateTo
page
limit

Response

```
{  
  "success": true,
```

```
"data": [  
  {  
    "rfqId": "market_rfq_101",  
    "rfqReference": "TOFA-MRFQ-20260825-000101",  
  
    "requirementTitle": "Grade A Cocoa Beans",  
  
    "categories": [  
      {  
        "id": "cat_cocoa",  
        "name": "Cocoa"  
      }  
    ],  
  
    "quantity": 20,  
    "unit": "tonnes",  
  
    "delivery": {  
      "city": "Accra",  
      "country": "Ghana",  
      "type": "b2b_logistics"  
    },  
  
    "currencyPreference": "USD",  
  
    "quotesCount": 4,  
  
    "submissionDeadline": "2026-09-05T23:59:59Z",  
  
    "createdAt": "2026-08-25T20:00:00Z"  
  }  
],  
  
"pagination": {  
  "page": 1,  
  "limit": 10,  
  "total": 35,  
  "totalPages": 4  
}
```

10.19 Seller View Tracking

When an eligible seller opens a Market RFQ, backend may record:

sellerId
rfqId
firstViewedAt
lastViewedAt

This does not change the Market RFQ status.

10.20 Get Market RFQ by ID

Endpoint

GET /rfqs/market/:rfqId

Buyer Response

Buyer may see all seller quotations.

```
{
  "success": true,

  "data": {
    "rfqId": "market_rfq_101",
    "rfqReference": "TOFA-MRFQ-20260825-000101",

    "status": "quoted",

    "requirement": {
      "title": "Grade A Cocoa Beans",
      "description": "We require export-quality Grade A cocoa beans...",
      "quantity": 20,
      "unit": "tonnes",
      "expectedDeliveryDate": "2026-10-15"
    },

    "delivery": {
      "type": "seller_arranged",
      "address": {
        "city": "Accra",
```

```
    "country": "Ghana"
  }
},
```

```
"quotesCount": 3,
```

```
"quotes": [
  {
    "quoteId": "quote_A",
    "seller": {
      "id": "seller_A",
      "storeName": "Agro Exports Ltd",
      "rating": 4.8,
      "totalReviews": 120
    },
```

```
    "currentVersion": {
      "versionId": "quote_A_v1",
      "quantity": 20,
      "unit": "tonnes",
      "pricePerUnit": 2000,
      "productsTotal": 40000,
      "logisticsAmount": 3500,
      "quoteTotal": 43500,
      "currency": "USD",
      "validUntil": "2026-09-03T23:59:59Z"
    }
  },
```

```
  {
    "quoteId": "quote_B",
    "seller": {
      "id": "seller_B",
      "storeName": "West Africa Cocoa Ltd",
      "rating": 4.5,
      "totalReviews": 78
    },
```

```
    "currentVersion": {
      "versionId": "quote_B_v2",
      "quantity": 20,
      "unit": "tonnes",
      "pricePerUnit": 1900,
      "productsTotal": 38000,
```



```

    "logisticsAmount": 4200,
    "quoteTotal": 42200,
    "currency": "USD",
    "validUntil": "2026-09-02T23:59:59Z"
  }
},
  "submissionDeadline": "2026-09-05T23:59:59Z",

  "createdAt": "2026-08-25T20:00:00Z"
}
}

```

10.21 Seller View of Market RFQ

A seller must not see competing seller commercial quotations.

Seller response should show:

- RFQ requirement;
- buyer information permitted by platform;
- seller's own Quote if one exists;
- seller's own negotiation history.

It must not return:

Seller A price
 Seller B price
 Seller C price

to competing sellers.

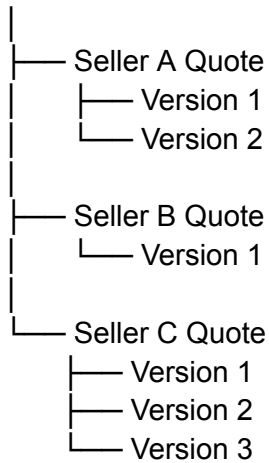
This protects fair competition.

10.22 Market RFQ Quote Structure

Each seller has their own Quote.

Relationship:

Market RFQ



The existing Quote versioning principle from Direct RFQ remains unchanged.

10.23 Market RFQ Quote

MarketRFQQuote

Field	Type	Required
id	UUID/String	Yes
rfqId	UUID/String	Yes
sellerId	UUID/String	Yes
currentVersionId	UUID/String	Yes
status	Enum	Yes
createdAt	Timestamp	Yes
updatedAt	Timestamp	Yes

Recommended Quote statuses:

active
accepted
rejected
expired
closed

closed is useful when another seller wins the Market RFQ.

10.24 One Quote Per Seller

A seller should have one Quote thread for a Market RFQ.

They do not create:

Quote A
Quote B
Quote C

for the same RFQ.

Instead:

One Seller Quote
↓
Multiple Quote Versions

This matches the existing Direct RFQ negotiation architecture.

Recommended database constraint:

unique:
rfqId + sellerId

10.25 Seller Submits Market RFQ Quote

Endpoint

POST /rfqs/market/:rfqId/quotes

Authentication

Eligible verified seller.

Seller-Arranged Example

```
{  
  "quantity": 20,  
  "unit": "tonnes",  
  
  "pricePerUnit": 2000,
```

```
"currency": "USD",

"delivery": {
  "type": "seller_arranged",
  "logisticsAmount": 3500
},

"estimatedDeliveryDate": "2026-10-10",

"validUntil": "2026-09-03T23:59:59Z",

"message": "Export packaging and transport to Accra included."
}
```

10.26 Buyer-Arranged Seller Quote

```
{
  "quantity": 20,
  "unit": "tonnes",

  "pricePerUnit": 2000,
  "currency": "USD",

  "delivery": {
    "type": "buyer_arranged",
    "logisticsAmount": null
  },

  "estimatedDeliveryDate": "2026-10-10",

  "validUntil": "2026-09-03T23:59:59Z",

  "message": "Goods available for pickup from Lagos."
}
```

10.27 Integrated/B2B Seller Quote

The seller must not submit TOFA-integrated logistics pricing.

Example:

```
{
```

```
"quantity": 20,  
"unit": "tonnes",  
  
"pricePerUnit": 2000,  
"currency": "USD",  
  
"delivery": {  
  "type": "b2b_logistics",  
  "logisticsAmount": null  
},  
  
"validUntil": "2026-09-03T23:59:59Z"  
}
```

Logistics pricing is handled by the existing Logistics module.

10.28 Seller Quote Validation

Backend must:

1. verify seller;
2. verify seller eligibility;
3. verify Market RFQ is open/quoted/negotiating;
4. verify submissionDeadline has not passed;
5. verify seller has not already created a separate Quote;
6. validate quantity/unit;
7. validate currency;
8. enforce deliveryType from RFQ;
9. require seller logistics only for seller-arranged;
10. prohibit seller-entered integrated/B2B logistics;
11. validate Quote expiry;
12. calculate Product totals;
13. calculate Quote total;
14. create Quote;
15. create QuoteVersion;
16. increment quotesCount;
17. update Market RFQ status to **quoted**;
18. notify buyer.

10.29 Seller Quote Response

```
{  
  "success": true,
```

```
"message": "Quotation submitted successfully.",

"data": {
  "rfqId": "market_rfq_101",

  "quote": {
    "quoteId": "quote_A",
    "versionId": "quote_A_v1",
    "version": 1,

    "pricePerUnit": 2000,
    "quantity": 20,
    "unit": "tonnes",

    "productsTotal": 40000,
    "logisticsAmount": 3500,
    "quoteTotal": 43500,

    "currency": "USD",

    "validUntil": "2026-09-03T23:59:59Z"
  }
}
```

Seller Toast

Quotation submitted successfully.

Buyer Toast

A new seller quotation has been received for your Market RFQ.

10.30 New Market Quote Email — Buyer

Subject: New quotation received — {RFQReference}

Hello {BuyerName},

A seller has submitted a quotation for your Market RFQ.

RFQ Reference: {RFQReference}
Seller: {SellerName}
Requirement: {RequirementTitle}

Quotation:

Quantity: {Quantity} {Unit}
Price Per Unit: {Currency} {PricePerUnit}
Products Total: {Currency} {ProductsTotal}
Logistics: {LogisticsDisplay}
Quote Total: {Currency} {QuoteTotal}
Estimated Delivery: {EstimatedDelivery}
Valid Until: {ValidUntil}

Seller Message:

{SellerMessage}

You can compare this quotation with other responses from your RFQ dashboard.

Best Regards,
Traders of Africa Team

CTA: Compare Quotations

10.31 Buyer Quote Comparison

Buyer should be able to compare seller Quotes in one view.

Recommended comparison data:

Seller
Rating
Verified Status
Subscription/Featured indicator where applicable
Price Per Unit
Products Total
Logistics
Quote Total
Delivery Estimate
Quote Validity
Seller Country

The backend should provide normalized comparison fields.

The frontend should not calculate winning or “best” Quote automatically unless TOFA explicitly introduces recommendation logic.

10.32 Get Quotes for Market RFQ

Endpoint

GET /rfqs/market/:rfqId/quotes

Authentication

Buyer that owns RFQ.

Query

status
currency
sortBy
page
limit

Possible **sortBy**:

lowest_total
highest_rating
earliest_delivery
latest

Response

```
{
  "success": true,

  "data": [
    {
      "quoteId": "quote_B",

      "seller": {
        "id": "seller_B",
        "storeName": "West Africa Cocoa Ltd",
        "isVerified": true,
        "rating": 4.5,
```



```

    "totalReviews": 78
  },

  "currentVersion": {
    "versionId": "quote_B_v2",
    "version": 2,
    "quantity": 20,
    "unit": "tonnes",
    "pricePerUnit": 1900,
    "productsTotal": 38000,
    "logisticsAmount": 4200,
    "quoteTotal": 42200,
    "currency": "USD",
    "estimatedDeliveryDate": "2026-10-12",
    "validUntil": "2026-09-02T23:59:59Z"
  }
}
],

"pagination": {
  "page": 1,
  "limit": 10,
  "total": 5,
  "totalPages": 1
}
}

```

10.33 Negotiation Principle

Negotiation remains seller-specific.

Example:

Buyer ↔ Seller A Quote

Buyer ↔ Seller B Quote

These are two independent negotiations.

Buyer negotiating with Seller A must not modify Seller B's Quote.

10.34 Counter-Offer Endpoint

Reuse the same conceptual pattern as Direct RFQ.

Endpoint

POST /rfqs/market/:rfqId/quotes/:quoteId/counter-offer

Both buyer and the Quote-owning seller may use it when permitted.

Example

```
{
  "quantity": 25,
  "unit": "tonnes",

  "pricePerUnit": 1850,
  "currency": "USD",

  "delivery": {
    "type": "seller_arranged",
    "logisticsAmount": 4500
  },

  "estimatedDeliveryDate": "2026-10-12",

  "validUntil": "2026-09-03T23:59:59Z",

  "message": "Can you supply 25 tonnes at this rate?"
}
```

Each counter-offer creates a new Quote version.

10.35 Negotiation Privacy

Only:

Buyer
+
Quote-owning Seller
+
Authorized Admin

may see the negotiation history for that seller Quote.

Competing sellers cannot see it.

10.36 Buyer Rejects Individual Quote

Rejecting one seller's Quote does not reject the Market RFQ.

Endpoint

POST /rfqs/market/:rfqId/quotes/:quoteId/reject

Payload

```
{  
  "reason": "The delivery time is too long."  
}
```

Result

Quote status = rejected

Market RFQ remains:
open / quoted / negotiating

Other seller Quotes remain active.

10.37 Quote Rejection Response

```
{  
  "success": true,  
  "message": "Quotation rejected successfully.",  
  
  "data": {  
    "rfqId": "market_rfq_101",  
    "quoteId": "quote_A",  
    "quoteStatus": "rejected",  
    "rfqStatus": "quoted"  
  }  
}
```

Buyer Toast

Quotation rejected successfully.

Seller Toast

The buyer has declined your quotation.

10.38 Quote Rejected Email — Seller

Subject: Quotation update — {RFQReference}

Hello {SellerName},

The buyer has declined your quotation for the following Market RFQ:

RFQ Reference: {RFQReference}

Requirement: {RequirementTitle}

Reason:

{RejectionReason}

The Market RFQ may remain open to other quotations or further actions depending on its current status.

Best Regards,
Traders of Africa Team

CTA: View RFQ

10.39 Accept / Award Seller Quote

The buyer may accept only one seller Quote in the initial implementation.

Endpoint

POST /rfqs/market/:rfqId/quotes/:quoteId/accept

Payload

```
{  
  "quoteVersionId": "quote_B_v2"  
}
```

Frontend does not send commercial amounts.

10.40 Market Quote Acceptance Validation

Backend must:

1. verify buyer owns Market RFQ;
2. verify RFQ is not already awarded/cancelled/expired;
3. verify Quote belongs to RFQ;
4. verify Quote belongs to seller;
5. verify requested version is current;
6. verify latest acceptable version was seller-generated;
7. verify Quote has not expired;
8. verify submission/award rules;
9. atomically lock the Market RFQ;
10. mark winning Quote **accepted**;
11. set RFQ **awarded**;
12. set awardedSellerId;
13. set acceptedQuoteId;
14. set acceptedQuoteVersionId;
15. set awardedAt;
16. close competing active Quotes;
17. trigger Checkout preparation;
18. notify winning seller;
19. notify unsuccessful sellers where appropriate.

The operation must be atomic so two sellers cannot be awarded simultaneously.

10.41 Accepted Market Quote Response

```
{
  "success": true,
  "message": "Quotation accepted successfully.",

  "data": {
    "rfqId": "market_rfq_101",
    "rfqStatus": "awarded",

    "awardedSeller": {
      "id": "seller_B",
      "storeName": "West Africa Cocoa Ltd"
    },

    "acceptedQuote": {
```

```
    "quoteId": "quote_B",
    "quoteVersionId": "quote_B_v2"
  },

  "nextStep": "checkout"
}
```

Buyer Toast

Quotation accepted successfully. Please continue to checkout.

Winning Seller Toast

Your quotation has been accepted.

10.42 Winning Seller Email

Subject: Your quotation has been accepted — {RFQReference}

Hello {SellerName},

The buyer has selected your quotation for the following Market RFQ:

RFQ Reference: {RFQReference}

Requirement: {RequirementTitle}

Accepted Terms:

Quantity: {Quantity} {Unit}

Price Per Unit: {Currency} {PricePerUnit}

Products Total: {Currency} {ProductsTotal}

Logistics: {LogisticsDisplay}

Quote Total: {Currency} {QuoteTotal}

The buyer will now proceed through the applicable Checkout and Payment process.

An Order will be created only after the transaction reaches the required Payment condition.

Best Regards,
Traders of Africa Team

CTA: View RFQ

10.43 Competing Quote Closure

When one Quote is accepted:

Winning Quote

→ accepted

Other active Quotes

→ closed

Do not mark all competing Quotes:

rejected

because the buyer did not necessarily reject them individually.

closed more accurately means:

Another seller was selected for this RFQ.

10.44 Unsuccessful Seller Email

Where notification is appropriate:

Subject: Market RFQ closed — {RFQReference}

Hello {SellerName},

The following Market RFQ has now been awarded to another supplier:

RFQ Reference: {RFQReference}

Requirement: {RequirementTitle}

Your quotation is now closed and no further action is required.

Thank you for participating in the sourcing request.

Best Regards,

Traders of Africa Team

CTA: View Other Market RFQs

10.45 Integrated Logistics After Quote Award

If:

deliveryType = integrated_logistics

the winning seller's Product terms are locked first.

Then:

Winning Quote



Integrated Logistics module



Buyer selects provider



Checkout finalized

No competing seller Quote is reopened if the buyer later changes integrated logistics provider.

10.46 B2B Logistics After Quote Award

If:

deliveryType = b2b_logistics

then:

Seller Quote Awarded



B2B Logistics Request/Quote



Buyer accepts logistics



Checkout

The Market RFQ remains awarded to the seller while B2B logistics is being finalized.

10.47 Buyer-Arranged Delivery After Award

No logistics amount is included in seller Quote.

After Payment and Order creation:

Order becomes ready_for_shipment



Buyer provides logistics details

using the existing Order endpoint:

PATCH /orders/:orderId/buyer-logistics

No Market RFQ-specific delivery endpoint is created.

10.48 FX Relationship

Market RFQ sellers quote in a commercial currency.

The winning Quote currency becomes the future:

orderCurrency

Buyer chooses:

paymentCurrency

during Checkout.

Existing Checkout FX rules apply.

Market RFQ must not calculate its own buyer payment FX.

Example:

Winning Quote = USD 42,200

Buyer chooses NGN



Checkout FX



Payment in NGN



Order preserves:

orderCurrency = USD

paymentCurrency = NGN

10.49 Different Sellers Quoting Different Currencies

TOFA may permit different sellers to quote in different allowed currencies.

Example:

Seller A = USD 40,000

Seller B = EUR 37,000

Seller C = GHS equivalent

Buyer comparison becomes difficult if raw totals are shown without normalization.

Recommended comparison behavior:

- preserve every Quote in its original commercial currency;
- optionally show a **comparison equivalent** in the buyer's preferred display currency;
- comparison conversion is informational only;
- actual FX is locked only when the buyer selects a Quote and proceeds to Checkout.

Example:

Seller A

USD 40,000

≈ NGN X

Seller B

EUR 37,000

≈ NGN Y

The comparison FX rate must be labeled:

Estimated equivalent

It is not the final Checkout rate.

10.50 Quote Comparison FX Must Not Be Commercially Binding

The Market RFQ comparison page may use current FX for convenience.

But:

comparisonFx

!=
checkoutFx

The binding FX rate remains the rate locked by Checkout.

This distinction must be clear to frontend and backend.

10.51 Market RFQ Submission Deadline

Every Market RFQ should have:

submissionDeadline

After the deadline:

- new sellers cannot submit first Quotes;
- existing Quote negotiations may either close or continue according to platform configuration;
- buyer can still choose among valid received quotations while those quotations remain valid.

Recommended initial rule:

The submission deadline stops new sellers from entering the RFQ. Existing valid quotations remain selectable until their own `validUntil` expires or the RFQ is awarded/cancelled.

10.52 Expire Market RFQ

Market RFQ may become `expired` if:

- response period ended;
- all quotations expired;
- no valid quotation remains;
- buyer did not award within configured award period.

Exact expiry timing should be configurable.

10.53 Cancel Market RFQ

Buyer may cancel before award.

Endpoint

POST /rfqs/market/:rfqId/cancel

Payload

```
{  
  "reason": "Requirement is no longer needed."  
}
```

Not Allowed

After:

status = awarded

the transaction has moved into Checkout.

Later cancellation uses existing Checkout/Payment/Order rules depending on stage.

10.54 Cancellation Response

```
{  
  "success": true,  
  "message": "Market RFQ cancelled successfully",  
  
  "data": {  
    "rfqId": "market_rfq_101",  
    "status": "cancelled"  
  }  
}
```

All active seller Quotes should become:

closed

with closure reason:

rfq_cancelled

10.55 RFQ Cancellation Email — Sellers

Subject: Market RFQ cancelled — {RFQReference}

Hello {SellerName},

The buyer has cancelled the following Market RFQ:

RFQ Reference: {RFQReference}

Requirement: {RequirementTitle}

Reason:

{CancellationReason}

Any active quotation you submitted for this RFQ has now been closed.

Best Regards,
Traders of Africa Team

CTA: View Other Market RFQs

10.56 Buyer Get Market RFQs

Endpoint

GET /rfqs/market

Query

status
search
dateFrom
dateTo
page
limit

This returns RFQs created by the authenticated buyer.

Response

```
{  
  "success": true,
```

```

"data": [
  {
    "rfqId": "market_rfq_101",
    "rfqReference": "TOFA-MRFQ-20260825-000101",

    "requirementTitle": "Grade A Cocoa Beans",

    "quantity": 20,
    "unit": "tonnes",

    "deliveryType": "seller_arranged",

    "status": "quoted",

    "quotesCount": 5,

    "submissionDeadline": "2026-09-05T23:59:59Z",

    "createdAt": "2026-08-25T20:00:00Z"
  }
],

"pagination": {
  "page": 1,
  "limit": 10,
  "total": 7,
  "totalPages": 1
}
}

```

10.57 Seller Get Own Market RFQ Responses

Seller may need a separate view showing RFQs they responded to.

Endpoint

GET /rfqs/market/my-responses

Query

quoteStatus
rfqStatus
search

page
limit

Response

```
{
  "success": true,

  "data": [
    {
      "rfqId": "market_rfq_101",
      "rfqReference": "TOFA-MRFQ-20260825-000101",

      "requirementTitle": "Grade A Cocoa Beans",

      "rfqStatus": "quoted",

      "quote": {
        "quoteId": "quote_A",
        "status": "active",
        "version": 1,
        "currency": "USD",
        "quoteTotal": 43500,
        "validUntil": "2026-09-03T23:59:59Z"
      }
    }
  ]
}
```

10.58 Message Center Relationship

Each seller negotiation should have its own context.

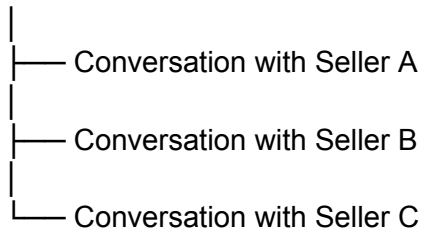
Recommended:

```
conversationContextType = market_rfq_quote
contextId = quoteId
```

This is preferable to one shared RFQ conversation because sellers must not see one another.

Example:

Market RFQ



Formal Quote terms remain authoritative.

Chat messages do not replace Quote versions.

10.59 Admin Get Market RFQs

Endpoint

GET /admin/rfqs/market

Query

status
buyerId
categoryId
deliveryType
awardedSellerId
search
dateFrom
dateTo
page
limit

Response

```
{
  "success": true,

  "data": [
    {
      "rfqId": "market_rfq_101",
      "rfqReference": "TOFA-MRFQ-20260825-000101",

      "buyer": {
```



```

      "id": "buyer_123",
      "name": "Victor Ejiogu"
    },

    "requirementTitle": "Grade A Cocoa Beans",

    "status": "quoted",

    "deliveryType": "seller_arranged",

    "quotesCount": 5,

    "awardedSeller": null,

    "submissionDeadline": "2026-09-05T23:59:59Z",

    "createdAt": "2026-08-25T20:00:00Z"
  }
],

  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 30,
    "totalPages": 2
  }
}

```

10.60 Admin Get Market RFQ Details

Endpoint

GET /admin/rfqs/market/:rfqId

Admin may see:

- buyer;
- requirement;
- seller eligibility;
- sellers that viewed;
- all seller Quotes;
- all Quote versions;

- negotiation histories;
- selected seller;
- Checkout reference;
- Payment reference after payment creation;
- Order reference after Order creation;
- audit trail.

Admin visibility does not mean Admin should normally change seller commercial Quote terms.

10.61 Frontend Buyer Market RFQ Page

Recommended sections:

RFQ Information

Requirement

Quantity

Delivery

Submission Deadline

Quotes Received

Seller A

Seller B

Seller C

Compare Quotations

Message Seller

Negotiation History

Accept Quote

Reject Quote

Counter-Offer

Once awarded:

Selected Supplier

Accepted Quote

Continue to Checkout

10.62 Frontend Seller Market RFQ Page

Show:

Buyer Company information permitted
Requirement
Quantity
Delivery Location
Delivery Method
Submission Deadline
Preferred Currency
Description

If seller has no Quote:

[Submit Quotation]

If seller has Quote:

Your Current Quote
Your Negotiation History
Message Buyer

Do not show competing seller Quotes.

10.63 Market RFQ Notification Matrix

Event	Buyer Toast	Seller Toast	Email
Market RFQ created	Yes	Matching sellers	Optional sellers
Seller Quote submitted	Yes	Yes	Buyer
Counter-offer submitted	Yes	Yes	Other party
Individual Quote rejected	Yes	Yes	Seller
Quote accepted/awarded	Yes	Yes	Winning seller
RFQ awarded to another seller	No	Yes	Optional unsuccessful seller

Market RFQ cancelled	Yes	Yes	Sellers with Quotes
Quote expired	Yes	Yes	Optional
Submission deadline reached	Optional	No	Optional

10.64 Internal Events

Recommended:

MARKET_RFQ_CREATED

MARKET_RFQ_SELLER_VIEWED

MARKET_RFQ_QUOTE_CREATED

MARKET_RFQ_COUNTER_OFFER_CREATED

MARKET_RFQ_QUOTE_REJECTED

MARKET_RFQ_QUOTE_EXPIRED

MARKET_RFQ_AWARDED

MARKET_RFQ_QUOTE_CLOSED

MARKET_RFQ_CANCELLED

MARKET_RFQ_EXPIRED

When awarded:

MARKET_RFQ_AWARDED

↓

Checkout Module

↓

Prepare Market RFQ Checkout

10.65 Checkout Relationship

The winning Quote uses the existing Checkout architecture.

Pass:

sourceType = market_rfq

sourceId = rfqId

quoteId

quoteVersionId

Checkout obtains commercial amounts from the accepted Quote version.

It must not use:

- current Product price;
- another seller's Quote;
- comparison FX value.

10.66 Payment Relationship

Market RFQ does not create Payment directly.

Flow remains:

Awarded Quote



Checkout



Payment

The generic Payment module receives:

sourceType = checkout

or the finalized Checkout reference according to the existing Payment architecture.

No Market RFQ-specific Payment endpoint is needed.

10.67 Order Relationship

Market RFQ does not create Order directly.

After Payment becomes ready:

Payment



Order Service

The resulting Order stores:

sourceType = market_rfq
sourceId = rfqId
quoteId
quoteVersionId
sellerId = awarded seller

10.68 Logistics Relationship

Market RFQ reuses existing Logistics behavior.

seller_arranged

Sellers provide a logistics amount in their own Quote.

buyer_arranged

No logistics amount in Quote.

Buyer later provides provider/pickup details through:

PATCH /orders/:orderId/buyer-logistics

after Order reaches the appropriate state.

integrated_logistics

Provider pricing comes from Logistics.

b2b_logistics

B2B logistics request and Quote come from Logistics.

No duplicate RFQ logistics APIs should be introduced.

10.69 Quote Acceptance Atomicity

Market RFQ award is especially sensitive to race conditions.

If buyer submits two accept requests nearly simultaneously:

Accept Seller A

Accept Seller B

backend must guarantee only one succeeds.

Recommended database transaction:

Lock Market RFQ



Check status != awarded



Accept winning Quote



Set awarded



Close competitors



Commit

10.70 Idempotency

Repeated acceptance of the same winning Quote should return the already-awarded result rather than creating duplicate Checkout records.

Repeated seller Quote submission must not create multiple Quote roots for the same seller.

10.71 Audit Rules

Preserve:

- RFQ creation;
- seller visibility;
- seller views;
- each seller Quote;
- all Quote versions;
- negotiation actions;
- rejected Quotes;
- winning Quote;
- competing Quote closure;
- RFQ cancellation;
- RFQ expiry.

Commercial history should not be deleted when RFQ closes.

10.72 Module Summary

The Market RFQ module establishes that:

- Market RFQ is open to multiple eligible sellers;
- Market RFQ can exist without an existing Product;
- sellers are matched based on marketplace eligibility rules;
- seller views are tracked separately rather than using a global **viewed** status;
- each seller has one Quote thread with immutable versions;
- each seller negotiation is private and independent;
- competing sellers cannot see one another's commercial terms;
- buyer can compare seller quotations;
- seller-arranged logistics pricing is provided independently by each seller;
- integrated and B2B logistics reuse the Logistics module;
- buyer-arranged logistics reuses the existing Order buyer-logistics flow;
- only one seller Quote can be accepted in the initial implementation;
- award must be atomic;
- the winning Quote becomes **accepted**;
- competing active Quotes become **closed**, not falsely **rejected**;
- awarded Quote proceeds to existing Checkout;
- FX for actual payment is handled only by Checkout;
- Quote comparison FX is informational and not commercially binding;
- Payment and Order reuse existing generic modules;
- Message Center conversations remain seller-specific;
- no duplicate Checkout, Payment, Order, Logistics, or Message Center lifecycle is created.

MODULE 11. SUBSCRIPTION MANAGEMENT

11. SUBSCRIPTION MANAGEMENT MODULE

11.1 Overview

The Subscription Management Module controls the plans, limits, benefits, restrictions, transaction charges and premium capabilities available to users on TOFA Marketplace.

Subscription Plans must be dynamic.

Admin should be able to create, update, activate and deactivate Subscription Plans without requiring backend changes whenever TOFA wants to introduce a new Plan or modify an existing one.

Examples may include:

- Basic
- Growth
- Premium
- Enterprise

These names are examples only.

The system must not hardcode business logic based on Plan names.

A Subscription Plan may control:

- Subscription price;
- billing period;
- number of Products a Seller can upload;
- number of images allowed per Product;
- number of Product variants;
- ability to view Buyer information;
- ability to view Seller information;
- Product Search priority;
- Market RFQ access;
- Market RFQ response limits;
- Market RFQ priority;
- featured Product eligibility;

- number of featured Products;
- featured Store eligibility;
- Store priority;
- Analytics access;
- Analytics level;
- report export access;
- support level;
- TOFA transaction fee percentage;
- future marketplace benefits.

The Subscription module owns the configuration of these benefits.

Other modules consume the resulting entitlements.

11.2 Core Subscription Principle

Business functionality must not depend directly on a Subscription Plan name.

Do not implement:

- `if plan.name == "Premium"`
- `transactionFee = 2%`

Instead:

- User
- ↓
- Active Subscription
- ↓
- Subscription Plan
- ↓
- Effective Entitlements
- ↓
- Marketplace Modules

Example effective entitlements:

- `max_products = 100`
- `max_images_per_product = 10`

- transaction_fee_percentage = 3
- product_priority = 20
- analytics_level = advanced

The relevant marketplace module reads the entitlement it requires.

11.3 Free and Paid Plans

TOFA may have:

- Free Plans
- Paid Plans

The free/basic Plan must still exist as a normal Subscription Plan.

Example:

- Basic
-
- Price = 0
- Transaction Fee = 5%
- Product Limit = 10
- Analytics = Basic

A free Subscription does not mean TOFA earns nothing from the User.

TOFA may still earn transaction fees when the User completes eligible marketplace transactions.

11.4 Subscription Plan Structure

SubscriptionPlan

Field	Type	Required	Description
id	UUID/String	Yes	Plan ID
name	String	Yes	Plan name
description	String	No	Plan description
audience	Enum	Yes	Users eligible for Plan
status	Enum	Yes	Plan availability
isFree	Boolean	Yes	Whether Plan has no Subscription price
isDefault	Boolean	Yes	Whether automatically assigned
displayOrder	Integer	No	Frontend display order
createdBy	UUID/String	Yes	Admin that created Plan

createdAt	Timestamp	Yes	Created date
-----------	-----------	-----	--------------

updatedAt	Timestamp	Yes	Last update
-----------	-----------	-----	-------------

Recommended audience:

- seller
- buyer
- all

Initial TOFA Subscription benefits may primarily target Sellers.

The structure remains flexible enough to support Buyer Plans later.

11.5 Subscription Plan Status

Recommended:

- draft
- active
- inactive
- archived

draft

Plan is being configured and cannot be subscribed to.

active

Plan is available for eligible Users.

inactive

New Users cannot subscribe to the Plan.

Existing Subscription handling follows TOFA's Subscription policy.

archived

Plan is retained for historical records but cannot be selected.

A Plan should not be hard-deleted if historical Subscriptions or Payments reference it.

11.6 Subscription Plan Pricing

Subscription prices should support multiple currencies.

SubscriptionPlanPrice

Field	Type	Required
id	UUID/String	Yes
planId	UUID/String	Yes
currency	String	Yes
amount	Decimal	Yes
billingPeriod	Enum	Yes

status	Enum	Yes
--------	------	-----

createdAt	Timestamp	Yes
-----------	-----------	-----

updatedAt	Timestamp	Yes
-----------	-----------	-----

Recommended billing periods:

- free
- monthly
- quarterly
- yearly

The structure may support additional periods later.

11.7 Multi-Currency Subscription Pricing

Example:

```
{
  "planId": "plan_growth",
  "prices": [
    {
      "currency": "NGN",
      "amount": 50000,
      "billingPeriod": "monthly"
    },
    {
      "currency": "USD",
      "amount": 35,
      "billingPeriod": "monthly"
    }
  ]
}
```


- {
- "currency": "GHS",
- "amount": 450,
- "billingPeriod": "monthly"
- }
-]
- }

Admin controls supported prices.

The Payment module determines which Payment methods are available for the selected currency.

11.8 Subscription Entitlements

Benefits and restrictions should be stored as configurable entitlements.

Recommended initial entitlement codes:

- max_products
-
- max_images_per_product
-
- max_variants_per_product
-
- view_buyer_info
-
- view_seller_info
-
- product_priority
-
- can_view_market_rfqs
-
- max_market_rfq_responses
-
- market_rfq_priority
-
- featured_product_eligibility
-
- max_featured_products

-
- featured_store_eligibility
-
- store_priority
-
- analytics_access
-
- analytics_level
-
- report_export
-
- support_level
-
- transaction_fee_percentage

Additional entitlements can be introduced later.

11.9 Entitlement Definition Structure

EntitlementDefinition

Field	Type	Required
id	UUID/String	Yes
code	String	Yes
name	String	Yes

descripti on	String	No
valueTy pe	Enum	Yes
defaultV alue	Any/null	No
category	String	No
status	Enum	Yes
created At	Timesta mp	Yes
updated At	Timesta mp	Yes

Recommended `valueType`:

- boolean
- integer
- decimal
- string
- enum

11.10 Example Entitlement Definition

- {
- "code": "transaction_fee_percentage",
-
- "name": "Transaction Fee Percentage",
-
- "description": "Percentage charged by TOFA on eligible marketplace transactions completed by users on this plan.",
-
- "valueType": "decimal",
-
- "defaultValue": 5,
-
- "category": "transaction_fees",
-
- "status": "active"
- }

11.11 Plan Entitlement Structure

SubscriptionPlanEntitlement

Field	Type	Required
id	UUID/String	Yes
planId	UUID/String	Yes
entitlementCode	String	Yes

value	Any	Yes
createdAt	Timestamp	Yes
updatedAt	Timestamp	Yes

Example:

- {
- "planId": "plan_basic",
- "entitlementCode": "transaction_fee_percentage",
- "value": 5
- }

11.12 Example Basic Plan

- {
- "name": "Basic",
-
- "description": "Basic marketplace access.",
-
- "audience": "seller",
-
- "isFree": true,
-
- "isDefault": true,
-
- "prices": [
- {
- "currency": "NGN",
- "amount": 0,
- "billingPeriod": "free"

- }
-],
-
- "entitlements": {
- "max_products": 10,
-
- "max_images_per_product": 4,
-
- "max_variants_per_product": 20,
-
- "view_buyer_info": false,
-
- "view_seller_info": true,
-
- "product_priority": 0,
-
- "can_view_market_rfqs": true,
-
- "max_market_rfq_responses": 5,
-
- "market_rfq_priority": 0,
-
- "featured_product_eligibility": false,
-
- "max_featured_products": 0,
-
- "featured_store_eligibility": false,
-
- "store_priority": 0,
-
- "analytics_access": true,
-
- "analytics_level": "basic",
-
- "report_export": false,
-
- "support_level": "standard",
-
- "transaction_fee_percentage": 5
- }
- }

11.13 Example Growth Plan

- {
- "name": "Growth",
-
- "description": "Expanded marketplace access for growing sellers.",
-
- "audience": "seller",
-
- "isFree": false,
-
- "isDefault": false,
-
- "prices": [- {
- "currency": "NGN",
- "amount": 50000,
- "billingPeriod": "monthly"
- }
-],
-
- "entitlements": {
- "max_products": 100,
-
- "max_images_per_product": 10,
-
- "max_variants_per_product": 100,
-
- "view_buyer_info": true,
-
- "view_seller_info": true,
-
- "product_priority": 20,
-
- "can_view_market_rfqs": true,
-
- "max_market_rfq_responses": 50,
-
- "market_rfq_priority": 20,
-
- "featured_product_eligibility": true,
-
- "max_featured_products": 5,

-
- "featured_store_eligibility": false,
-
- "store_priority": 20,
-
- "analytics_access": true,
-
- "analytics_level": "advanced",
-
- "report_export": true,
-
- "support_level": "priority",
-
- "transaction_fee_percentage": 3
- }
- }

11.14 Example Premium Plan

- {
- "name": "Premium",
-
- "description": "Premium marketplace access and enhanced visibility.",
-
- "audience": "seller",
-
- "isFree": false,
-
- "isDefault": false,
-
- "prices": [
- {
- "currency": "NGN",
- "amount": 150000,
- "billingPeriod": "monthly"
- }
-],
-
- "entitlements": {

- "max_products": 500,
-
- "max_images_per_product": 15,
-
- "max_variants_per_product": 500,
-
- "view_buyer_info": true,
-
- "view_seller_info": true,
-
- "product_priority": 50,
-
- "can_view_market_rfqs": true,
-
- "max_market_rfq_responses": 200,
-
- "market_rfq_priority": 50,
-
- "featured_product_eligibility": true,
-
- "max_featured_products": 10,
-
- "featured_store_eligibility": true,
-
- "store_priority": 50,
-
- "analytics_access": true,
-
- "analytics_level": "advanced",
-
- "report_export": true,
-
- "support_level": "priority",
-
- "transaction_fee_percentage": 2
- }
- }

The values above are examples.

Admin controls the actual Plans and their benefits.

11.15 Transaction Fee Entitlement

Every Seller Subscription Plan should define:

- transaction_fee_percentage

This represents the percentage TOFA charges on the Product/commercial value of an eligible marketplace transaction.

Example:

- Basic
- 5%
-
- Growth
- 3%
-
- Premium
- 2%

The percentage is configured by Admin.

11.16 Transaction Fee Validation

Recommended validation:

- transaction_fee_percentage >= 0
-
- transaction_fee_percentage <= 100

TOFA may apply a stricter business validation if desired.

Decimal percentages should be supported.

Example:

- 2
 - 2.5
 - 3.25
 - 5
-

11.17 Transaction Fee Calculation

Recommended formula:

- transactionFeeAmount
- =
- feeBaseAmount
- ×
- (transactionFeePercentage / 100)

Example:

- Product Amount = USD 10,000
 -
 - Transaction Fee = 5%
 -
 - TOFA Fee = USD 500
-

11.18 Transaction Fee Base

The recommended fee base is:

- Product / Commercial Amount

Logistics should not automatically form part of the TOFA transaction fee base.

Example:

- Product Amount = USD 10,000
-
- Logistics = USD 1,500
-
- Buyer Total = USD 11,500
-
- Transaction Fee Percentage = 5%

TOFA fee:

- USD 500

not:

- USD 575

unless TOFA later explicitly decides to charge commission on Logistics.

11.19 Transaction Fee Across Order Sources

The same principle applies to:

- Cart Orders
-
- Direct RFQ Orders
-
- Market RFQ Orders

The source of the Order should not determine the Subscription fee logic.

Example:

- Market RFQ Quote Product Value = USD 50,000
-
- Logistics = USD 4,000
-
- Seller Transaction Fee = 3%

Buyer payment:

- USD 54,000

TOFA transaction fee:

- USD 1,500

assuming Logistics is excluded from the fee base.

11.20 Multi-Seller Checkout Fee

A multi-Seller Checkout may result in several Seller Orders.

The Buyer still makes one Checkout Payment where the existing Checkout flow permits it.

TOFA's transaction fee must be calculated independently for each Seller.

Example:

- Seller A
-
- Product Amount = NGN 2,000,000
-
- Plan Transaction Fee = 5%
-
- TOFA Fee = NGN 100,000

Seller B:

- Product Amount = NGN 3,000,000
-
- Plan Transaction Fee = 2%
-
- TOFA Fee = NGN 60,000

TOFA must not apply one Seller's Subscription fee to the entire multi-Seller Checkout.

11.21 Transaction Fee Financial Snapshot

The applicable fee must be stored when the Order's commercial financials are locked.

Recommended financial fields:

- subscriptionId
-
- subscriptionPlanId
-
- transactionFeePercentage
-
- feeBaseAmount
-
- transactionFeeAmount
-
- sellerNetProductAmount

Example:

- {
- "productSubtotal": 10000,
-
- "logisticsAmount": 1500,
-
- "totalAmount": 11500,
-
- "transactionFeePercentage": 5,

-
- "feeBaseAmount": 10000,
-
- "transactionFeeAmount": 500,
-
- "sellerNetProductAmount": 9500,
-
- "currency": "USD"
- }

Where Logistics is payable to the Seller, its settlement treatment should remain separately identifiable rather than being included in `sellerNetProductAmount`.

11.22 Subscription Snapshot Rule

An existing transaction must not change because the Seller later changes Subscription.

Example:

- Order created:
-
- Basic Plan
- Transaction Fee = 5%

Seller later upgrades:

- Premium
- Transaction Fee = 2%

The existing Order remains:

- 5%

New eligible transactions use:

- 2%

This prevents historical financial values from changing.

11.23 Admin Fee Change Rule

Suppose Admin changes:

- Basic
-
- 5% → 4%

Transactions whose financials were already locked at 5% remain at 5%.

New transactions use the effective 4% rate.

Analytics must also use the stored transaction fee rather than recalculating historical transactions.

11.24 Create Subscription Plan

Endpoint

- POST /admin/subscription-plans

Payload

- {
- "name": "Growth",
-
- "description": "Expanded marketplace access for growing sellers.",
-
- "audience": "seller",

-
- "isFree": false,
-
- "isDefault": false,
-
- "displayOrder": 2,
-
- "prices": [
- {
- "currency": "NGN",
- "amount": 50000,
- "billingPeriod": "monthly"
- }
-],
-
- "entitlements": {
- "max_products": 100,
- "max_images_per_product": 10,
- "view_buyer_info": true,
- "view_seller_info": true,
- "product_priority": 20,
- "can_view_market_rfqs": true,
- "max_market_rfq_responses": 50,
- "market_rfq_priority": 20,
- "featured_product_eligibility": true,
- "max_featured_products": 5,
- "featured_store_eligibility": false,
- "store_priority": 20,
- "analytics_access": true,
- "analytics_level": "advanced",
- "report_export": true,
- "support_level": "priority",
- "transaction_fee_percentage": 3
- }
- }

11.25 Create Plan Response

- {

- "success": true,
-
- "message": "Subscription plan created successfully.",
-
- "data": {
- "id": "plan_growth",
- "name": "Growth",
- "status": "draft"
- }
- }

Admin Toast

- Subscription plan created successfully.
-

11.26 Update Subscription Plan

Endpoint

- PATCH /admin/subscription-plans/:planId

Admin may update:

- name;
- description;
- prices;
- billing periods;
- entitlements;
- transaction fee percentage;
- display order;
- other configurable benefits.

Updates to financial benefits must not modify already-locked historical transactions.

11.27 Activate Subscription Plan

Endpoint

- PATCH /admin/subscription-plans/:planId/status

Payload

- {
- "status": "active"
- }

Response

- {
- "success": true,
- "message": "Subscription plan activated successfully."
- }

Toast

- Subscription plan activated successfully.
-

11.28 Deactivate Subscription Plan

Endpoint

- PATCH /admin/subscription-plans/:planId/status

Payload

- {
- "status": "inactive"
- }

Deactivation prevents new Subscription selection.

The backend must preserve existing Subscription records and historical references.

11.29 Get Subscription Plans

Public/User Endpoint

- GET /subscription-plans

Recommended response:

- {
- "success": true,
-
- "data": [
- {
- "id": "plan_basic",
-
- "name": "Basic",
-
- "description": "Basic marketplace access.",
-
- "isFree": true,
-
- "prices": [
- {
- "currency": "NGN",
- "amount": 0,
- "billingPeriod": "free"
- }
-],
-
- "benefits": {

- "maxProducts": 10,
- "maxImagesPerProduct": 4,
- "marketRfqResponses": 5,
- "analyticsLevel": "basic",
- "transactionFeePercentage": 5
- }
- }
-]
- }

Only active Plans appropriate for the User should be returned.

11.30 Get Admin Subscription Plans

Endpoint

- GET /admin/subscription-plans

Admin may filter by:

- status
- audience
- isFree

Admin response should include complete Plan configuration.

11.31 Get Subscription Plan

Endpoint

- GET /subscription-plans/:planId

Returns the Plan's public details, price and relevant benefits.

11.32 User Subscription Structure

UserSubscription

Field	Type	Required
id	UUID/String	Yes
userId	UUID/String	Yes
planId	UUID/String	Yes
status	Enum	Yes
billingPeriod	Enum	Yes
currency	String	Yes
pricePaid	Decimal	Yes
startedAt	Timestamp	Yes

expiresAt	Timestamp/null	No
autoRenew	Boolean	Yes
paymentId	UUID/String/null	No
createdAt	Timestamp	Yes
updatedAt	Timestamp	Yes

Recommended statuses:

- pending
- active
- expired
- cancelled

11.33 Free Plan Assignment

Where TOFA has a default free Plan:

- Seller becomes eligible for marketplace selling
- ↓
- Default Free Plan
- ↓
- UserSubscription created
- ↓

- status = active

No Payment is required.

The Plan's transaction fee still applies to eligible transactions.

11.34 Subscribe to Paid Plan

Endpoint

- POST /subscriptions

Payload

- {
- "planId": "plan_growth",
- "priceId": "price_growth_ngn_monthly",
- "paymentMethod": "paystack"
- }

The Payment Module handles the actual Subscription Payment.

Subscription should not create a separate Payment architecture.

11.35 Subscription Payment Flow

- User selects Plan
- ↓
- Selects Plan Price/Currency
- ↓
- Available Payment Methods determined by Payment Module
- ↓
- Payment created

- ↓
- Payment successful/verified
- ↓
- Subscription activated

For manual Payment methods:

- Payment Proof Uploaded
- ↓
- Admin verifies Payment
- ↓
- Payment becomes successful
- ↓
- Subscription activated

Uploading proof alone must not activate a paid Subscription.

11.36 Subscription Payment Purpose

The Payment module should receive:

- paymentFor = subscription

with the Subscription/Plan reference.

This keeps Payment dynamic enough to support:

- order
 - subscription
 - future payment purposes
-

11.37 Subscription Activation

Once Payment is confirmed:

- SUBSCRIPTION_ACTIVATED

is emitted.

The User's effective entitlements become those of the new Plan.

New transactions subsequently use the new:

- transaction_fee_percentage
-

11.38 Current Subscription

Endpoint

- GET /subscriptions/current

Response

- {
- "success": true,
-
- "data": {
- "subscriptionId": "sub_101",
-
- "plan": {
- "id": "plan_growth",
- "name": "Growth"
- },
-
- "status": "active",
-

- "startedAt": "2026-08-01T10:00:00Z",
-
- "expiresAt": "2026-09-01T10:00:00Z",
-
- "entitlements": {
- "max_products": 100,
- "max_images_per_product": 10,
- "product_priority": 20,
- "market_rfq_priority": 20,
- "analytics_access": true,
- "analytics_level": "advanced",
- "report_export": true,
- "transaction_fee_percentage": 3
- }
- }
- }

11.39 Effective Entitlement Service

Backend should provide an internal service similar to:

- `getEffectiveEntitlement(userId, entitlementCode)`

Example:

- `getEffectiveEntitlement(`
- `sellerId,`
- `"transaction_fee_percentage"`
- `)`

returns:

- 3

Other modules should use this service rather than querying Plan names.

11.40 Product Limit Enforcement

Before Product creation:

- Current qualifying Product count
- ↓
- max_products
- ↓
- Allow / Reject

If limit reached:

- {
- "success": false,
- "code": "PRODUCT_LIMIT_REACHED",
- "message": "You have reached the product limit available on your current subscription."
- }

Toast

- You have reached your product limit. Upgrade your plan to add more products.

11.41 Product Image Limit

Product Management reads:

- max_images_per_product

Example:

- Basic = 4

- Growth = 10
- Premium = 15

The Product module enforces the limit.

Subscription only defines it.

11.42 Variant Limit

Product Management reads:

- max_variants_per_product

This applies only where the Product is configured as a variable Product according to the Product Management module.

11.43 Market RFQ Access

Market RFQ module reads:

- can_view_market_rfqs
-
- max_market_rfq_responses
-
- market_rfq_priority

Subscription does not determine RFQ eligibility by itself.

A Seller must still satisfy Market RFQ eligibility rules.

11.44 Product Priority

Search reads:

- product_priority

This provides a controlled ranking boost.

It does not:

- make inactive Products searchable;
 - override Search relevance;
 - guarantee position one.
-

11.45 Featured Product Eligibility

Search/Discovery reads:

- featured_product_eligibility
-
- max_featured_products

Eligibility does not mean every Product is automatically featured.

The Product must still be actively featured according to the Search & Discovery module.

11.46 Store Priority

Seller discovery may consume:

- store_priority
-
- featured_store_eligibility

These are ranking/visibility benefits.

They do not override Seller verification.

11.47 Analytics Entitlements

Analytics consumes:

- analytics_access
-
- analytics_level
-
- report_export

Example:

- Basic
- analytics_level = basic
-
- Growth
- analytics_level = advanced
-
- Premium
- analytics_level = advanced

Additional levels may be introduced without changing Analytics architecture.

11.48 Subscription Upgrade

A User may upgrade from one Plan to another.

Recommended flow:

- Current Plan
- ↓
- Select New Plan
- ↓
- Payment where required
- ↓
- Payment successful
- ↓
- New Subscription activated
- ↓
- New entitlements become effective

New transactions use the new transaction fee.

Existing locked transactions remain unchanged.

11.49 Subscription Downgrade

Downgrades should not delete existing User data.

Example:

- Premium max_products = 500
-
- Basic max_products = 10
-
- Seller currently has 40 Products

Downgrading should not automatically delete 30 Products.

Recommended:

- preserve existing Products;
- prevent creation of additional Products while over the limit;
- optionally require Seller to reduce usage before certain actions;
- never silently delete marketplace records.

Search continues to follow Product status.

11.50 Subscription Expiry

When a paid Subscription expires:

- Paid Subscription
- ↓
- expired
- ↓
- Default Free Plan becomes effective

where TOFA uses a default free Plan.

New transactions then use the free Plan's transaction fee.

Existing transactions retain their snapshot fee.

11.51 Subscription Renewal

Endpoint

- POST /subscriptions/:subscriptionId/renew

Renewal creates the appropriate Subscription Payment.

Subscription becomes renewed/extended only after successful Payment.

11.52 Cancel Auto-Renewal

Endpoint

- PATCH /subscriptions/:subscriptionId/auto-renew

Payload

- {
- "autoRenew": false
- }

Response

- {
- "success": true,
- "message": "Automatic renewal has been turned off."
- }

Toast

- Automatic renewal has been turned off.

Turning off auto-renewal does not immediately terminate the active paid period.

11.53 Subscription History

Endpoint

- GET /subscriptions/history

Response

- {
- "success": true,
-
- "data": [
- {

- "subscriptionId": "sub_101",
-
- "plan": {
- "id": "plan_growth",
- "name": "Growth"
- },
-
- "status": "active",
-
- "pricePaid": 50000,
- "currency": "NGN",
-
- "startedAt": "2026-08-01T10:00:00Z",
- "expiresAt": "2026-09-01T10:00:00Z"
- }
-]
- }

11.54 Subscription Activated Email

Subject

- Your TOFA Marketplace Subscription Is Active

Body

- Hello {FirstName},
-
- Your {PlanName} subscription on TOFA Marketplace has been activated successfully.
-
- Plan: {PlanName}
- Amount Paid: {Currency} {Amount}
- Billing Period: {BillingPeriod}
- Start Date: {StartDate}
- Next Renewal/Expiry Date: {ExpiryDate}
-
- Your plan benefits are now available on your account.

-
- Your applicable TOFA marketplace transaction fee under this plan is {TransactionFeePercentage}% of the eligible product/commercial transaction value.
-
- You can view your subscription details from your TOFA Marketplace account.
-
- Best Regards,
- Traders of Africa Team

11.55 Free Plan Activated Email

Where TOFA wants an activation email for the default free Plan:

Subject

- Your TOFA Marketplace Plan Is Active

Body

- Hello {FirstName},
 -
 - Your {PlanName} plan on TOFA Marketplace is now active.
 -
 - You can now access the marketplace features available under your plan.
 -
 - Your applicable TOFA marketplace transaction fee is {TransactionFeePercentage}% of eligible product/commercial transaction value.
 -
 - You can view your plan benefits and upgrade options from your account at any time.
 -
 - Best Regards,
 - Traders of Africa Team
-

11.56 Subscription Upgrade Email

Subject

- Your TOFA Marketplace Plan Has Been Upgraded

Body

- Hello {FirstName},
-
- Your TOFA Marketplace subscription has been successfully upgraded to {PlanName}.
-
- Your new plan benefits are now active.
-
- Your transaction fee for new eligible marketplace transactions is now {TransactionFeePercentage}%.
-
- Transactions whose financial terms were already locked before this upgrade will retain the transaction fee that applied at the time they were created.
-
- You can view your updated benefits from your account.
-
- Best Regards,
- Traders of Africa Team

11.57 Subscription Expiry Email

Subject

- Your TOFA Marketplace Subscription Has Expired

Body

- Hello {FirstName},

-
- Your {PlanName} subscription has expired.
-
- Your account has now moved to {FallbackPlanName}, where applicable.
-
- The benefits and transaction fee applicable to new marketplace transactions will now follow your current active plan.
-
- Existing transactions are not affected by this change.
-
- You can renew or select another plan from your TOFA Marketplace account.
-
- Best Regards,
- Traders of Africa Team

11.58 Subscription Expiry Reminder

Recommended reminders:

- 7 days before expiry
-
- 1 day before expiry

Subject

- Your TOFA Marketplace Subscription Expires Soon

Body

- Hello {FirstName},
-
- Your {PlanName} subscription will expire on {ExpiryDate}.
-
- Renew your subscription to continue enjoying your current plan benefits and transaction fee.
-

- You can manage your subscription from your TOFA Marketplace account.
 -
 - Best Regards,
 - Traders of Africa Team
-

11.59 Subscription Toasts

Activation

- Your subscription has been activated successfully.

Upgrade

- Your subscription has been upgraded successfully.

Renewal

- Your subscription has been renewed successfully.

Payment Pending Verification

- Your payment proof has been submitted. Your subscription will be activated after payment confirmation.

Payment Failed

- We couldn't complete your subscription payment. Please try again.
-

11.60 Admin Plan Builder

Recommended sections:

- Basic Information
-
- Plan Name
- Description
- Audience
- Default Plan
- Display Order
-
-
- Pricing
-
- Currency
- Price
- Billing Period
-
-
- Product Limits
-
- Maximum Products
- Maximum Images Per Product
- Maximum Variants Per Product
-
-
- Marketplace Access
-
- View Buyer Information
- View Seller Information
-
-
- Marketplace Visibility
-
- Product Priority
- Store Priority
- Featured Product Eligibility
- Maximum Featured Products
- Featured Store Eligibility
-
-
- Market RFQ

-
- Can View Market RFQs
- Maximum Market RFQ Responses
- Market RFQ Priority
-
-
- Analytics
-
- Analytics Access
- Analytics Level
- Report Export
-
-
- Transaction Fees
-
- TOFA Transaction Fee (%)
-
-
- Support
-
- Support Level

Future entitlement definitions may automatically appear in this builder.

11.61 Admin Plan Update Warning

When Admin changes benefits such as:

- transaction_fee_percentage

the UI should clearly state:

- Changes will apply to future eligible transactions. Existing transactions with locked financial terms will not be recalculated.
-

11.62 Subscription and Order Relationship

Order Management consumes the Seller's effective:

- transaction_fee_percentage

when transaction financials are locked.

Order Management then stores:

- transactionFeePercentage
- transactionFeeAmount
- feeBaseAmount
- subscriptionId
- subscriptionPlanId

Subscription Management does not calculate settlement after the Order has been created.

11.63 Subscription and Payment Relationship

Subscription Payment uses the existing Payment Module.

The Subscription module must not introduce separate gateway logic.

Payment Module continues to determine available methods based on:

- currency
- paymentFor
- payment configuration

including applicable options such as:

- Paystack
- Flutterwave
- Transactworld
- PAPSS
- Bank Transfer

where configured and eligible.

11.64 Subscription and Analytics Relationship

Analytics consumes:

- analytics_access
- analytics_level
- report_export

to determine available Seller reporting capabilities.

Analytics also consumes the actual stored:

- transactionFeeAmount

from marketplace transactions to calculate TOFA transaction revenue.

Analytics must not calculate revenue using the Seller's current Subscription fee.

11.65 Subscription and Search Relationship

Search consumes:

- product_priority
- store_priority
- market_rfq_priority
- featured_product_eligibility
- featured_store_eligibility

Subscription does not directly reorder Search results.

Search remains responsible for ranking.

11.66 Subscription Admin Permissions

Recommended:

- subscription_plans.view
-
- subscription_plans.create
-
- subscription_plans.update
-
- subscription_plans.activate
-
- subscription_plans.deactivate
-
- subscriptions.view
-
- subscriptions.manage

Use the existing Admin permission architecture.

Do not create separate Subscription Admin roles.

11.67 Subscription Audit Events

Recommended:

- SUBSCRIPTION_PLAN_CREATED
-
- SUBSCRIPTION_PLAN_UPDATED
-
- SUBSCRIPTION_PLAN_ACTIVATED
-
- SUBSCRIPTION_PLAN_DEACTIVATED
-
- SUBSCRIPTION_ACTIVATED
-
- SUBSCRIPTION_UPGRADED
-
- SUBSCRIPTION_RENEWED
-
- SUBSCRIPTION_EXPIRED
-
- SUBSCRIPTION_CANCELLED
-
- SUBSCRIPTION_AUTO_RENEW_UPDATED

Where a transaction fee is changed on a Plan, the Plan update Audit record should preserve:

- previousValue
- newValue
- adminId
- timestamp

11.68 Backend Implementation Guide

Backend should:

- keep Subscription Plans dynamic;
- store Plan prices separately;
- store configurable entitlements;
- expose an effective entitlement service;
- assign the default Plan where applicable;
- integrate paid Plans with the Payment module;
- activate paid Subscriptions only after Payment confirmation;

- enforce Subscription limits through relevant business modules;
- provide transaction fee percentage to Order Management;
- preserve historical Subscription records;
- never recalculate locked transaction fees after Plan changes.

Backend should not:

- hardcode Basic = 5%;
 - hardcode Growth = 3%;
 - hardcode Premium = 2%;
 - determine Search ranking;
 - calculate Product ratings;
 - calculate historical transaction revenue;
 - create separate Payment logic.
-

11.69 Frontend Implementation Guide

Frontend should:

- display available Plans;
- display Plan prices;
- display benefits;
- display transaction fee percentage;
- allow eligible Users to select a Plan;
- allow currency/billing period selection;
- show Subscription status;
- show usage and limits;
- show renewal/expiry information;
- show upgrade options;
- display locked features;
- show transaction fee clearly when comparing Plans.

Frontend should not:

- calculate authoritative transaction fees;
 - determine effective entitlements;
 - activate paid Subscriptions before backend confirmation;
 - hardcode benefits based on Plan names.
-

11.70 Module Summary

The Subscription Management Module establishes that:

- Subscription Plans are dynamically managed by Admin;
- free and paid Plans use the same Plan architecture;
- the free/basic Plan is configurable;
- Plan names do not determine marketplace behaviour;
- benefits are controlled through entitlements;
- Admin can configure Product limits;
- Admin can configure Product image limits;
- Admin can configure variant limits;
- Admin can configure Buyer/Seller information visibility;
- Admin can configure Product priority;
- Admin can configure Market RFQ access and priority;
- Admin can configure featured Product and Store benefits;
- Admin can configure Analytics access;
- Admin can configure report export;
- Admin can configure support level;
- every applicable Plan can define a `transaction_fee_percentage`;
- transaction fee percentages may differ by Plan;
- percentages such as 5%, 3% and 2% are configuration values rather than hardcoded Plan rules;
- transaction fees apply to eligible Product/commercial transaction value;
- Logistics is excluded from the transaction fee base by default;
- Cart, Direct RFQ and Market RFQ Orders use the same transaction fee principle;
- multi-Seller Checkout calculates TOFA's transaction fee separately for each Seller;
- the applicable transaction fee is snapshotted when transaction financials are locked;
- later upgrades, downgrades or Plan changes do not alter existing transaction fees;
- Admin changes to transaction fee percentages apply to future transactions;
- paid Subscription activation depends on successful/verified Payment;
- Subscription Payments use the existing dynamic Payment module;
- Subscription expiry may return the User to the configured default free Plan;
- downgrades do not silently delete Products;
- other marketplace modules consume effective entitlements rather than Plan names;
- Analytics uses stored transaction fees to calculate TOFA transaction revenue.

MODULE 12. REVIEWS, RATINGS & REWARDS

12. REVIEWS, RATINGS & REWARDS MODULE

12.1 Overview

The Reviews, Ratings & Rewards module manages verified buyer reviews after a completed marketplace transaction and awards TOFA Points where the review qualifies.

This module builds directly on the existing Order Management flow.

The Order module already emits:

ORDER_COMPLETED

when an Order reaches:

completed

The Reviews & Rewards module listens for this event and determines what the buyer is eligible to review.

The module is responsible for:

- creating review eligibility after completed Orders;
- allowing buyers to review purchased Products;
- allowing buyers to review the Seller;
- preventing reviews from users who did not complete a valid transaction;
- preventing duplicate reviews;
- calculating Product ratings;
- calculating Seller ratings;
- storing review history;
- supporting optional review images;
- supporting review moderation;
- awarding TOFA Points for eligible reviews;
- preventing repeated/reward abuse;
- maintaining user points balance;
- exposing review summaries to Product and Seller modules;
- notifying buyers and sellers when appropriate.

The Reviews module does not:

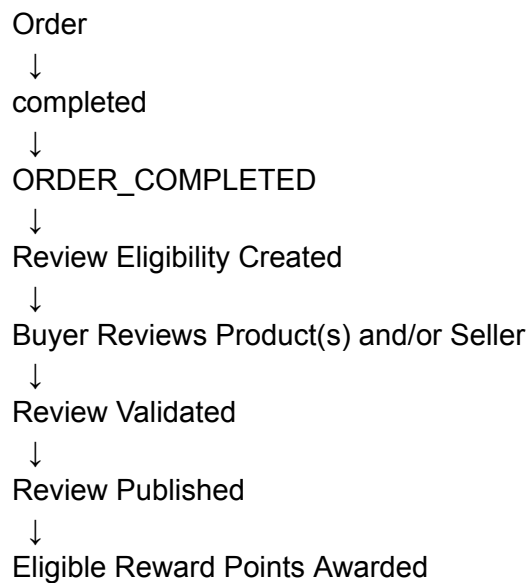
- determine Order completion;
- process seller payout;
- modify Product commercial information;
- modify Order history;
- create Payment;
- determine subscription entitlements.

Those responsibilities remain in their existing modules.

12.2 Core Review Principle

Standard Marketplace reviews must be based on verified transactions.

The expected flow is:



A buyer should not be able to review a Product or Seller simply because they can access the Product page.

They must have completed an eligible transaction.

12.3 What Can Be Reviewed

For a completed seller Order, the buyer may be eligible to review:

Each purchased Product

+

The Seller

Example:

Order 101

Seller:
Agro Supplies Ltd

Items:
Cashew Nuts
Dried Ginger
Cocoa Powder

The buyer may submit:

1 Cashew Nuts Review
1 Dried Ginger Review
1 Cocoa Powder Review
1 Seller Review

This is more useful than one generic Order review because Product ratings and Seller ratings serve different purposes.

12.4 Product Review vs Seller Review

Product Review

Represents the buyer's experience with a specific purchased Product.

Possible areas include:

- Product quality;
- accuracy of description;
- value;
- packaging;
- condition.

Seller Review

Represents the buyer's experience with the Seller.

Possible areas include:

- communication;
- reliability;
- fulfillment;
- professionalism;
- accuracy of supplied goods;
- overall transaction experience.

The frontend may present both within one review flow, but backend should store them separately.

12.5 Review Eligibility

Review eligibility is created from:

ORDER_COMPLETED

The Reviews module should create eligibility records for:

- every eligible OrderItem;
- the Order Seller.

Example:

Order completed

Item 1 → Product Review Eligible

Item 2 → Product Review Eligible

Seller → Seller Review Eligible

12.6 ReviewEligibility Structure

ReviewEligibility

Field	Type	Required	Description
id	UUID/String	Yes	Eligibility ID
orderId	UUID/String	Yes	Completed Order
orderItemId	UUID/String/null	No	Product item where applicable

buyerId	UUID/String	Yes	Buyer
sellerId	UUID/String	Yes	Seller
productId	UUID/String/null	No	Product where applicable
reviewType	Enum	Yes	product/seller
status	Enum	Yes	eligible/submitted/expired/revoked
reviewId	UUID/String/null	No	Created review
eligibleAt	Timestamp	Yes	Eligibility creation
expiresAt	Timestamp/null	No	Optional review deadline
submittedAt	Timestamp/null	No	Review submission
createdAt	Timestamp	Yes	Created
updatedAt	Timestamp	Yes	Updated

12.7 Review Eligibility Statuses

Recommended:

eligible
submitted
expired
revoked

eligible

Buyer can submit the review.

submitted

Buyer already submitted the allowed review.

expired

Review period has passed where TOFA enforces a review deadline.

revoked

Eligibility was manually/system revoked.

Example reasons:

- Order reversed;
- fraud;
- transaction invalidated;
- duplicate transaction record.

12.8 Review Window

TOFA may permit reviews for a configurable period after completion.

Example:

30 days

60 days

90 days

Recommended:

Make this backend-configurable.

Example setting:

`review_submission_window_days = 60`

If TOFA does not want review expiration initially:

`expiresAt = null`

The architecture should still support it later.

12.9 ProductReview Structure

ProductReview

Field	Type	Required	Description
id	UUID/String	Yes	Review ID
eligibilityId	UUID/String	Yes	Eligibility reference

orderId	UUID/String	Yes	Completed Order
orderItemId	UUID/String	Yes	Purchased item
buyerId	UUID/String	Yes	Reviewer
sellerId	UUID/String	Yes	Seller
productId	UUID/String	Yes	Product
rating	Integer	Yes	1–5
title	String/null	No	Review title
comment	String	Yes	Written review
images	Array	No	Review images
status	Enum	Yes	Review moderation status
isVerifiedPurchase	Boolean	Yes	Must be true for standard flow
pointsAwarded	Integer	Yes	Points granted
helpfulCount	Integer	Yes	Helpful votes
notHelpfulCount	Integer	Yes	Not-helpful votes
moderationReason	String/null	No	Moderation information
createdAt	Timestamp	Yes	Created
updatedAt	Timestamp	Yes	Updated

12.10 SellerReview Structure

SellerReview

Field	Type	Required
id	UUID/String	Yes
eligibilityId	UUID/String	Yes
orderId	UUID/String	Yes

buyerId	UUID/String	Yes
sellerId	UUID/String	Yes
overallRating	Integer	Yes
communicationRating	Integer/null	No
fulfillmentRating	Integer/null	No
reliabilityRating	Integer/null	No
comment	String	Yes
status	Enum	Yes
isVerifiedTransaction	Boolean	Yes
pointsAwarded	Integer	Yes
helpfulCount	Integer	Yes
notHelpfulCount	Integer	Yes
moderationReason	String/null	No
createdAt	Timestamp	Yes
updatedAt	Timestamp	Yes

12.11 Review Rating Scale

Use:

- 1
- 2
- 3
- 4
- 5

Meaning:

- 1 = Very Poor
- 2 = Poor
- 3 = Fair
- 4 = Good

5 = Excellent

Frontend may display stars.

Backend should store an integer.

12.12 Review Statuses

Recommended:

published
pending_moderation
hidden
rejected
deleted

published

Review is publicly visible.

pending_moderation

Review is temporarily awaiting moderation.

hidden

Review remains stored but is not publicly visible.

rejected

Review failed moderation and will not be published.

deleted

Buyer/Admin deleted the review according to permitted rules, but the record should remain auditable.

12.13 Default Moderation Approach

For the initial implementation, TOFA can allow normal reviews to publish immediately unless automated/manual rules flag them.

Recommended:

Normal Review

→ published

Flagged Review

→ pending_moderation

Potential flags:

- abusive language;
- spam;
- personal contact information;
- suspicious repeated content;
- fraudulent review patterns;
- unrelated content.

12.14 Get Review Eligibility

Endpoint

GET /reviews/eligibility

Authentication

Authenticated buyer.

Query

status
orderId
page
limit

Response

```
{
  "success": true,
  "data": [
    {
      "orderId": "order_101",
      "orderReference": "TOFA-ORD-20260825-000101",
```

```

"seller": {
  "id": "seller_456",
  "storeName": "Agro Supplies Ltd"
},

"productReviews": [
  {
    "eligibilityId": "elig_product_1",
    "orderItemId": "item_001",
    "productId": "prod_101",
    "productName": "Premium Cashew Nuts",
    "productImage": "https://cdn.tofa.com/products/cashew.jpg",
    "status": "eligible"
  },
  {
    "eligibilityId": "elig_product_2",
    "orderItemId": "item_002",
    "productId": "prod_202",
    "productName": "Dried Ginger",
    "productImage": "https://cdn.tofa.com/products/ginger.jpg",
    "status": "submitted"
  }
],

"sellerReview": {
  "eligibilityId": "elig_seller_1",
  "status": "eligible"
},

"completedAt": "2026-08-25T17:30:00Z"
}
]
}

```

12.15 Get Review Eligibility by Order

Endpoint

GET /orders/:orderId/review-eligibility

Response

```
{
```

```
"success": true,
"data": {
  "orderId": "order_101",
  "canReview": true,

  "products": [
    {
      "eligibilityId": "elig_001",
      "productId": "prod_101",
      "productName": "Premium Cashew Nuts",
      "status": "eligible"
    }
  ],

  "seller": {
    "eligibilityId": "elig_seller_001",
    "sellerId": "seller_456",
    "storeName": "Agro Supplies Ltd",
    "status": "eligible"
  }
}
```

12.16 Submit Product Review

Endpoint

POST /reviews/products

Payload

```
{
  "eligibilityId": "elig_product_1",
  "rating": 5,
  "title": "Excellent quality",
  "comment": "The cashew nuts matched the description and arrived in good condition.",
  "images": [
    "https://cdn.tofa.com/reviews/review1.jpg"
  ]
}
```

Images are optional.

12.17 Product Review Backend Validation

Backend must:

1. verify buyer;
2. retrieve ReviewEligibility;
3. verify eligibility belongs to buyer;
4. verify `reviewType = product`;
5. verify status `eligible`;
6. verify associated Order is completed;
7. verify Product/OrderItem relationship;
8. verify rating between 1 and 5;
9. validate comment;
10. validate image limits;
11. detect moderation flags;
12. create ProductReview;
13. mark eligibility `submitted`;
14. recalculate Product review summary;
15. calculate reward eligibility;
16. award points if qualified;
17. notify seller where applicable.

12.18 Product Review Response

```
{
  "success": true,
  "message": "Product review submitted successfully.",

  "data": {
    "reviewId": "review_product_123",
    "status": "published",
    "rating": 5,
    "pointsAwarded": 10,

    "reward": {
      "awarded": true,
      "points": 10,
      "newTotalPoints": 180
    }
  }
}
```

Buyer Toast

Review submitted successfully. You earned 10 TOFA Points.

If no points:

Review submitted successfully.

12.19 Submit Seller Review

Endpoint

POST /reviews/sellers

Payload

```
{
  "eligibilityId": "elig_seller_1",

  "overallRating": 5,
  "communicationRating": 4,
  "fulfillmentRating": 5,
  "reliabilityRating": 5,

  "comment": "The seller communicated clearly and fulfilled the order as agreed."
}
```

Only **overallRating** and **comment** need to be mandatory initially.

12.20 Seller Review Response

```
{
  "success": true,
  "message": "Seller review submitted successfully.",

  "data": {
    "reviewId": "seller_review_123",
    "status": "published",

    "overallRating": 5,
```

```
"pointsAwarded": 10,  
  
"reward": {  
  "awarded": true,  
  "points": 10,  
  "newTotalPoints": 190  
}  
}  
}
```

12.21 Review Reward Principle

Points should reward meaningful verified reviews without encouraging spam.

Points must not simply be granted because a review endpoint was called.

A review should qualify only if:

- buyer had valid eligibility;
- Order was completed;
- this eligibility has not already earned points;
- review satisfies minimum requirements;
- review is not rejected for abuse/fraud.

12.22 Reward Configuration

Reward values should be configurable by Admin rather than hardcoded.

Recommended settings:

```
product_review_points  
seller_review_points  
review_with_image_bonus  
minimum_review_characters  
max_review_reward_per_order
```

Example:

```
{  
  "product_review_points": 10,  
  "seller_review_points": 10,  
  "review_with_image_bonus": 5,
```

```
"minimum_review_characters": 30,  
"max_review_reward_per_order": 50  
}
```

The actual values above are examples only.

12.23 Why Reward Caps Are Recommended

Consider an Order containing:

20 Product items

If every review pays unrestricted points, one large Order could create an unexpectedly large reward.

A configurable:

`max_review_reward_per_order`

helps control exposure.

Example:

20 Product Reviews

Potential reward = 200

Order reward cap = 50

Actual maximum reward = 50

12.24 Points Ledger

Do not rely only on:

`User.totalPoints`

The User table may keep `totalPoints` for quick reads, but all point changes should have ledger entries.

PointsTransaction

Field	Type	Required
id	UUID/String	Yes
userId	UUID/String	Yes
type	Enum	Yes
points	Integer	Yes
sourceType	Enum	Yes
sourceId	UUID/String	Yes
description	String	Yes
balanceAfter	Integer	Yes
createdAt	Timestamp	Yes

12.25 Points Transaction Types

Recommended:

earned
redeemed
adjusted
expired
reversed

12.26 Points Source Types

Initial:

product_review
seller_review
admin_adjustment
referral

Future:

purchase campaign promotion loyalty_bonus

This keeps the points engine reusable.

12.27 Award Points Flow

Review Submitted



Review Validation



Reward Rules Checked



PointsTransaction Created



User.totalPoints Updated

Both actions should occur in one database transaction.

The system must not:

update totalPoints

without creating a PointsTransaction.

12.28 Reward Idempotency

One ReviewEligibility must only earn a particular review reward once.

Recommended unique protection:

sourceType + sourceId + rewardType

If the submission request is retried:

POST review

POST review again due network retry

the user must not receive double points.

12.29 Editing a Review and Points

Editing a published review should not award points again.

Example:

Original review

→ 10 points

Edit comment

→ 0 additional points

If review image bonus is introduced later, adding an image after initial submission should only earn a bonus if TOFA explicitly allows it.

Recommended initial implementation:

All reward calculation happens at first valid submission only.

12.30 Review Deletion and Points

If a buyer voluntarily deletes a legitimate review after receiving points, TOFA must define whether the reward is reversed.

Recommended initial rule:

Buyer deletes own review

→ points remain

unless abuse is detected.

However:

Review removed for fraud/manipulation

→ points may be reversed

This avoids punishing ordinary users who later remove a review while still allowing fraud controls.

12.31 Reverse Review Reward

Internal/Admin action:

Review determined fraudulent

↓

Points reversal

Create a new ledger entry:

```
{
  "type": "reversed",
  "points": -10,
  "sourceType": "product_review",
  "sourceId": "review_product_123",
  "description": "Review reward reversed following moderation."
}
```

Never delete the original earned transaction.

12.32 Get User Points

Endpoint

GET /rewards/points

Response

```
{
  "success": true,
  "data": {
    "totalPoints": 190,

    "summary": {
      "earned": 220,
      "redeemed": 30,
      "expired": 0,
      "adjusted": 0
    }
  }
}
```

12.33 Get Points History

Endpoint

GET /rewards/points/history

Query

type
sourceType
dateFrom
dateTo
page
limit

Response

```
{
  "success": true,

  "data": [
    {
      "id": "points_tx_123",
      "type": "earned",
      "points": 10,

      "sourceType": "product_review",
      "sourceId": "review_product_123",

      "description": "Points earned for reviewing Premium Cashew Nuts.",

      "balanceAfter": 190,

      "createdAt": "2026-08-25T18:00:00Z"
    }
  ],

  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 12,
    "totalPages": 1
  }
}
```

12.34 Get Product Reviews

Endpoint

GET /products/:productId/reviews

Query

rating
withImages
sortBy
page
limit

Possible **sortBy**:

latest
highest_rating
lowest_rating
most_helpful

Response

```
{
  "success": true,

  "summary": {
    "averageRating": 4.6,
    "totalReviews": 120,

    "distribution": {
      "5": 80,
      "4": 24,
      "3": 10,
      "2": 4,
      "1": 2
    }
  },

  "data": [
    {
      "reviewId": "review_101",

      "buyer": {
        "displayName": "Victor E.",
        "profileImage": null
      },
```

```
"rating": 5,  
"title": "Excellent quality",  
"comment": "The product matched the description and arrived in good condition.",  
  
"images": [  
  "https://cdn.tofa.com/reviews/review1.jpg"  
],  
  
"isVerifiedPurchase": true,  
  
"helpfulCount": 12,  
"notHelpfulCount": 1,  
  
"createdAt": "2026-08-25T18:00:00Z"  
}  
]  
}
```

12.35 Product Rating Aggregation

The Product module already uses:

averageRating
totalReviews

The Reviews module should be the authoritative source for these values.

After review changes:

ProductReview created/hidden/rejected

↓

Reviews Service recalculates summary

↓

Product summary updated/cached

Only:

published

reviews should normally count toward public rating.

12.36 Seller Rating Aggregation

Seller/User table already contains:

totalReviewCount
totalAverageReviews

These values should be maintained from SellerReview.

Recommended naming consistency if possible:

totalSellerReviews
sellerAverageRating

However, if existing fields have already been implemented, the Reviews service can maintain:

totalReviewCount
totalAverageReviews

to avoid breaking existing modules.

Only published Seller reviews count.

12.37 Get Seller Reviews

Endpoint

GET /sellers/:sellerId/reviews

Query

rating
sortBy
page
limit

Response

```
{  
  "success": true,
```



```

"summary": {
  "averageRating": 4.7,
  "totalReviews": 85,

  "categoryRatings": {
    "communication": 4.8,
    "fulfillment": 4.6,
    "reliability": 4.7
  }
},

"data": [
  {
    "reviewId": "seller_review_123",

    "buyer": {
      "displayName": "Victor E."
    },

    "overallRating": 5,
    "communicationRating": 5,
    "fulfillmentRating": 4,
    "reliabilityRating": 5,

    "comment": "Reliable supplier and good communication.",

    "isVerifiedTransaction": true,

    "createdAt": "2026-08-25T18:10:00Z"
  }
]
}

```

12.38 Helpful / Not Helpful

Users may indicate whether a Product or Seller review was helpful.

A user should only have one current vote per review.

Recommended values:

```

helpful
not_helpful

```

ReviewVote Structure

Field	Type
id	UUID/String
reviewType	Enum
reviewId	UUID/String
userId	UUID/String
vote	Enum
createdAt	Timestamp
updatedAt	Timestamp

Unique:

reviewType + reviewId + userId

12.39 Vote on Product Review

Endpoint

PUT /reviews/products/:reviewId/vote

Payload

```
{  
  "vote": "helpful"  
}
```

Response

```
{  
  "success": true,  
  "message": "Review feedback updated successfully.",  
  "data": {  
    "helpfulCount": 13,  
    "notHelpfulCount": 1,  
  }  
}
```

```
"userVote": "helpful"
}
}
```

Toast

Thanks for your feedback.

12.40 Remove Review Vote

Endpoint

DELETE /reviews/products/:reviewId/vote

Equivalent Seller Review endpoints may use:

PUT /reviews/sellers/:reviewId/vote

DELETE /reviews/sellers/:reviewId/vote

12.41 Review Image Rules

Product review images should be optional.

Recommended configurable limit:

max_review_images

Example:

5

Supported file types may include:

JPG

JPEG

PNG

WEBP

Backend must validate:

- MIME type;
- file size;
- image count;
- ownership;
- upload security.

Images should not be accepted merely because frontend supplies a URL.

12.42 Edit Product Review

Endpoint

PATCH /reviews/products/:reviewId

Payload

```
{  
  "rating": 4,  
  "title": "Good product",  
  "comment": "The product was good overall, but packaging could be improved."  
}
```

Rules

Buyer must own review.

Points are not awarded again.

Review may return to moderation if changed content triggers moderation rules.

12.43 Edit Seller Review

Endpoint

PATCH /reviews/sellers/:reviewId

Payload

```
{  
  "overallRating": 4,  
  "communicationRating": 5,  
  "fulfillmentRating": 4,  
}
```

```
"reliabilityRating": 4,  
"comment": "Good experience overall."  
}
```

12.44 Delete Own Review

Product Review

DELETE /reviews/products/:reviewId

Seller Review

DELETE /reviews/sellers/:reviewId

Recommended behavior:

status = deleted

rather than physically deleting the database row.

Eligibility should not automatically become available again.

Otherwise users could:

- review
- earn points
- delete review
- review again
- earn more points

Therefore:

ReviewEligibility.status remains submitted

unless Admin explicitly resets it.

12.45 Seller Response to Review

A useful seller feature is allowing the Seller to publicly respond once to a review.

Recommended:

ReviewResponse

Field	Type
id	UUID/String
reviewType	Enum
reviewId	UUID/String
sellerId	UUID/String
comment	String
status	Enum
createdAt	Timestamp
updatedAt	Timestamp

The seller response should not affect rating.

12.46 Add Seller Response

Endpoint

POST /reviews/products/:reviewId/response

or for Seller review:

POST /reviews/sellers/:reviewId/response

Payload

```
{  
  "comment": "Thank you for your feedback. We appreciate your business."  
}
```

Response

```
{  
  "success": true,  
}
```

```
"message": "Response submitted successfully."
}
```

12.47 Edit Seller Response

Endpoint

PATCH /reviews/responses/:responseId

Seller must own the response.

12.48 Review Seller Notification

When a Product/Seller review becomes published, seller may receive an in-app notification.

For low ratings, email may be useful.

Recommended:

1–2 stars

→ seller email

3–5 stars

→ in-app notification only

This avoids excessive email traffic.

12.49 Low Rating Email — Seller

Subject: New customer review received — {OrderReference}

Hello {SellerName},

A buyer has submitted a review following a completed transaction with your store.

Order Reference: {OrderReference}

Review Type: {ReviewType}

Rating: {Rating}/5

Review:

{ReviewComment}

Please review the feedback from your Seller Dashboard.

You may respond professionally to the review where the response option is available.

Best Regards,
Traders of Africa Team

CTA: View Review

12.50 Review Submitted Email — Buyer

An email is optional because the buyer just performed the action.

However, because points are awarded, a confirmation email may be useful.

Subject: Thanks for your review — you earned {PointsAwarded} TOFA Points

Hello {BuyerName},

Thank you for sharing your feedback on TOFA Marketplace.

Review: {ReviewTargetName}

Rating: {Rating}/5

You earned:

{PointsAwarded} TOFA Points

Your current points balance is:

{TotalPoints} TOFA Points

Your feedback helps other buyers make informed purchasing decisions and supports trusted sellers across the marketplace.

Best Regards,
Traders of Africa Team

CTA: View Your Points

If:

PointsAwarded = 0

use a standard review confirmation email without the reward wording.

12.51 Review Reminder

If buyer has not reviewed after Order completion, TOFA may send one reminder.

Recommended:

3–7 days after completion

This should be configurable.

Email

Subject: How was your order with {SellerName}?

Hello {BuyerName},

We hope your recent TOFA Marketplace transaction went well.

Order Reference: {OrderReference}

You can now review the product(s) you purchased and your experience with {SellerName}.

Eligible reviews may earn TOFA Points under the current Rewards rules.

Your feedback helps other buyers and supports quality sellers on the Marketplace.

Best Regards,
Traders of Africa Team

CTA: Review Your Order

Avoid sending repeated reminders after the buyer has already completed all eligible reviews.

12.52 Review Reminder Event

Recommended:

REVIEW_ELIGIBILITY_CREATED



Reminder scheduler



Check eligibility still open



Send one reminder

If submitted:

do not send

12.53 Get My Reviews

Endpoint

GET /reviews/me

Query

type
status
page
limit

Response

```
{
  "success": true,

  "data": [
    {
      "reviewType": "product",
      "reviewId": "review_101",

      "target": {
        "id": "prod_101",
        "name": "Premium Cashew Nuts"
      },

      "rating": 5,
      "comment": "Excellent product.",

      "pointsAwarded": 10,

      "createdAt": "2026-08-25T18:00:00Z"
    }
  ]
}
```

12.54 Admin Review List

Endpoint

GET /admin/reviews

Query

reviewType
status
rating
buyerId
sellerId
productId
search
dateFrom
dateTo
page
limit

Response

```
{
  "success": true,

  "data": [
    {
      "reviewType": "product",
      "reviewId": "review_101",

      "buyer": {
        "id": "buyer_123",
        "name": "Victor Ejiogu"
      },

      "seller": {
        "id": "seller_456",
        "storeName": "Agro Supplies Ltd"
      },

      "product": {
        "id": "prod_101",
        "name": "Premium Cashew Nuts"
      }
    }
  ]
}
```

```
    },  
    "rating": 1,  
    "status": "pending_moderation",  
    "pointsAwarded": 10,  
    "createdAt": "2026-08-25T18:00:00Z"  
  }  
],  
  
  "pagination": {  
    "page": 1,  
    "limit": 20,  
    "total": 40,  
    "totalPages": 2  
  }  
}
```

12.55 Admin Review Details

Endpoint

GET /admin/reviews/:reviewType/:reviewId

Admin may see:

- review;
- buyer;
- Order;
- OrderItem;
- seller;
- Product;
- eligibility;
- points transaction;
- moderation flags;
- seller response;
- audit history.

12.56 Admin Moderate Review

Endpoint

PATCH /admin/reviews/:reviewType/:reviewId/status

Payload — Publish

```
{  
  "status": "published",  
  "reason": "Review passed moderation."  
}
```

Payload — Hide

```
{  
  "status": "hidden",  
  "reason": "Review contains inappropriate personal information."  
}
```

Payload — Reject

```
{  
  "status": "rejected",  
  "reason": "Review violates marketplace review policy."  
}
```

12.57 Moderation and Rating Recalculation

When a review changes from:

published
→ hidden/rejected

the Reviews service must recalculate:

Product averageRating
Product totalReviews

or

Seller averageRating
Seller totalReviews

A hidden/rejected review should not affect public ratings.

12.58 Admin Points Configuration

Review reward rules should be manageable through Admin configuration.

Endpoint

GET /admin/rewards/settings

Response

```
{
  "success": true,
  "data": {
    "productReviewPoints": 10,
    "sellerReviewPoints": 10,
    "reviewWithImageBonus": 5,
    "minimumReviewCharacters": 30,
    "maxReviewRewardPerOrder": 50,
    "reviewSubmissionWindowDays": 60,
    "reviewReminderDays": 5
  }
}
```

12.59 Update Reward Settings

Endpoint

PATCH /admin/rewards/settings

Payload

```
{
  "productReviewPoints": 10,
  "sellerReviewPoints": 10,
  "reviewWithImageBonus": 5,
  "minimumReviewCharacters": 30,
  "maxReviewRewardPerOrder": 50
}
```

Response

```
{
  "success": true,
  "message": "Reward settings updated successfully."
}
```

Admin Toast

Reward settings updated successfully.

12.60 Admin Adjust User Points

Authorized Admin may need to correct a balance.

Endpoint

POST /admin/rewards/points/adjust

Payload

```
{
  "userId": "user_123",
  "points": 50,
  "reason": "Manual reward adjustment following support review."
}
```

For deduction:

```
{
  "userId": "user_123",
  "points": -20,
  "reason": "Correction of duplicate reward."
}
```

Every adjustment must create a PointsTransaction.

12.61 User.totalPoints Relationship

The User module already includes:

`totalPoints`

The Points Ledger is the authoritative transaction history.

`User.totalPoints` may be maintained as a cached/current balance for faster reads.

Relationship:

PointsTransactions



Current calculated balance



`User.totalPoints`

The two must stay synchronized.

12.62 Points Cannot Become Invalid Silently

If an action would reduce balance below zero, backend must apply configured rules.

Recommended initial rule:

`totalPoints` cannot be negative

If Admin reversal exceeds available balance:

- reject;
- or allow a separate controlled negative balance policy later.

Do not silently clamp without an audit record.

12.63 Future Points Redemption

This module initially focuses on earning Points from Reviews.

The Points Ledger is intentionally generic so TOFA can later introduce:

- subscription discounts;
- featured Product credits;
- promotional benefits;

- marketplace vouchers;
- campaign rewards.

Redemption rules should be a separate future Rewards Redemption feature rather than being mixed into the first Review implementation.

12.64 Product Review Relationship

Product Management reads:

averageRating
totalReviews

from Reviews-derived summary data.

Product does not create or edit ratings itself.

12.65 Seller Relationship

Seller profile reads its public rating from Seller Reviews.

Subscription must not directly increase rating.

A Premium seller can receive ranking priority, but:

subscription priority
!=
review rating

The two should remain separate ranking signals.

12.66 Order Relationship

Order Management only provides review eligibility trigger.

ORDER_COMPLETED
↓
Reviews Module

Order should not:

- create Review rows;
- award points;
- calculate ratings.

12.67 Message Center Relationship

Reviews should not automatically become Message Center messages.

However, a seller may receive a notification and navigate to the review.

If buyer and seller wish to discuss an issue before completion, that conversation belongs in Message Center/Disputes rather than Review.

12.68 Dispute Relationship

A buyer with an unresolved material issue should generally use Dispute functionality before confirming receipt/completion.

However, completing a transaction does not prevent a buyer from leaving a critical review.

Admin should not remove legitimate negative reviews merely because a seller disagrees.

Moderation should address policy violations, fraud, abuse or irrelevant content, not unfavorable opinions.

12.69 Duplicate Review Rules

Recommended uniqueness:

Product

one Product Review
per OrderItem eligibility

Seller

one Seller Review
per completed Order

If buyer purchases the same Product in another future Order, they may review it again because it represents another verified transaction.

This means a buyer can have multiple verified reviews of the same Product over time if they make multiple legitimate purchases.

12.70 Review Privacy

Public review identity should avoid exposing unnecessary personal information.

Recommended display:

First Name + Last Initial

Example:

Victor E.

Seller/Admin may have access to the related buyer identity where business rules permit.

Do not expose buyer:

- phone;
- email;
- delivery address;

inside public reviews.

12.71 Review Notification Matrix

Event	Buyer Toast	Seller Toast	Email
Review eligibility created	Optional	No	Completion email handles it
Product review submitted	Yes	Yes	Buyer optional
Seller review submitted	Yes	Yes	Buyer optional
Points awarded	Yes	No	Buyer
Low rating published	No	Yes	Seller
Review reminder	No	No	Buyer
Review hidden/rejected	Yes	No	Buyer where appropriate

Seller responds

Yes

No

Buyer optional

12.72 Seller Response Notification

When Seller responds:

Buyer Toast

The seller has responded to your review.

Optional email:

Subject: Seller response to your review

Hello {BuyerName},

{SellerName} has responded to your review.

Review: {ReviewTargetName}

Seller Response:

{SellerResponse}

You can view the response from the marketplace.

Best Regards,
Traders of Africa Team

CTA: View Review

12.73 Frontend Review Flow

Recommended after clicking:

Review Your Order

Show one review experience.

Example:

Order TOFA-ORD-...

Product 1
★★★★★
Title
Comment
Images

Product 2
★★★★★
Title
Comment
Images

Seller Experience
Overall Rating
Communication
Fulfillment
Reliability
Comment

[Submit Reviews]

Frontend may submit each review independently or use a batch endpoint.

12.74 Batch Review Submission

A batch endpoint can improve usability without changing Review storage.

Endpoint

POST /reviews/order/:orderId

Payload

```
{
  "productReviews": [
    {
      "eligibilityId": "elig_product_1",
      "rating": 5,
      "title": "Excellent quality",
      "comment": "The product matched the description.",
      "images": []
    },
    {
```

```

    "eligibilityId": "elig_product_2",
    "rating": 4,
    "title": "Good product",
    "comment": "Good overall quality.",
    "images": []
  }
],

  "sellerReview": {
    "eligibilityId": "elig_seller_1",
    "overallRating": 5,
    "communicationRating": 5,
    "fulfillmentRating": 4,
    "reliabilityRating": 5,
    "comment": "Good experience with the seller."
  }
}

```

Backend still validates and stores each Review separately.

12.75 Batch Review Response

```

{
  "success": true,
  "message": "Reviews submitted successfully.",

  "data": {
    "submittedReviews": 3,
    "totalPointsAwarded": 30,
    "newTotalPoints": 210
  }
}

```

Toast

Reviews submitted successfully. You earned 30 TOFA Points.

This batch endpoint is recommended for frontend convenience, while individual endpoints remain useful for editing and standalone review submission.

12.76 Backend Implementation Guide

Reviews Service owns:

- review eligibility;
- Product Reviews;
- Seller Reviews;
- moderation state;
- rating aggregation;
- review rewards;
- Points Ledger;
- review reminder eligibility.

Order Service owns only:

ORDER_COMPLETED

Product Service consumes rating summaries.

Seller Service consumes Seller rating summaries.

Admin configuration controls reward rules.

12.77 Frontend Implementation Guide

Frontend should never calculate:

points earned
rating averages
review eligibility

itself.

Frontend should receive:

eligibility
reward result
rating summary

from backend.

Do not assume:

every review = 10 points

because Admin-configured reward rules may change.

12.78 Internal Events

Recommended:

REVIEW_ELIGIBILITY_CREATED

PRODUCT_REVIEW_SUBMITTED

PRODUCT_REVIEW_PUBLISHED

PRODUCT_REVIEW_HIDDEN

PRODUCT_REVIEW_REJECTED

SELLER_REVIEW_SUBMITTED

SELLER_REVIEW_PUBLISHED

SELLER_REVIEW_HIDDEN

SELLER_REVIEW_REJECTED

REVIEW_RESPONSE_CREATED

REVIEW_POINTS_AWARDED

REVIEW_POINTS_REVERSED

REVIEW_REMINDER_DUE

12.79 Review Reward Idempotency

If:

PRODUCT_REVIEW_SUBMITTED

is processed twice, reward must still occur once.

The reward handler must check existing PointsTransaction source before inserting another one.

12.80 Admin Audit Events

Recommended:

REVIEW_STATUS_CHANGED

REVIEW_HIDDEN

REVIEW_REJECTED
REVIEW_RESTORED

REWARD_SETTINGS_UPDATED

POINTS_ADMIN_ADJUSTED
POINTS_REVERSED

12.81 Module Summary

The Reviews, Ratings & Rewards module establishes that:

- reviews are tied to completed verified transactions;
- `ORDER_COMPLETED` creates review eligibility;
- Product and Seller Reviews are separate;
- each purchased OrderItem can generate one Product Review eligibility;
- each completed Order can generate one Seller Review eligibility;
- duplicate reviews are prevented;
- ratings use a 1–5 scale;
- Product ratings are maintained by the Reviews module;
- Seller ratings are maintained by the Reviews module;
- public review identities do not expose sensitive buyer information;
- reviews may be edited but do not earn points again;
- deleting a review does not reopen eligibility;
- sellers may respond to reviews;
- review moderation does not rewrite Order history;
- legitimate negative reviews should not be removed simply because they are unfavorable;
- TOFA Points are awarded only when configurable reward rules are satisfied;
- reward amounts are Admin-configurable;
- review rewards are recorded through a Points Ledger;
- `User.totalPoints` represents the current cached points balance;
- points awards and reversals are auditable;
- reward processing is idempotent;
- review reminders can be sent after Order completion;
- a batch Order review endpoint can provide a simple frontend experience;
- the Rewards structure can later support other point sources and redemption without redesigning Review logic.

MODULE 13. MESSAGE CENTER

13. MESSAGE CENTER MODULE

13.1 Overview

The Message Center module provides direct communication between marketplace users.

A user may be:

- primarily a buyer;
- primarily a seller;
- a seller who also buys from other sellers.

For this reason, Message Center should not permanently identify participants as:

buyerId

sellerId

Instead, Conversations should simply exist between marketplace users.

Example:

User A

↔

User B

Their buyer/seller role depends on the transaction they are performing outside the Message Center.

The Message Center allows users to:

- send text Messages;
- send emojis;
- send images;
- send documents/files;
- reply to Messages;
- see read/unread Messages;
- receive Messages in real time;
- search Conversations;
- view previous Conversations;
- initiate enquiries with sellers;
- navigate from a Conversation to a seller's Products;

- start a Direct RFQ from a seller's Product.

The Message Center should remain simple and should not become responsible for other marketplace workflows.

It does not:

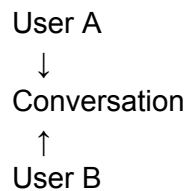
- create quotation terms;
- accept RFQs;
- update Orders;
- confirm Payments;
- upload official Payment proof;
- update logistics details;
- process disputes;
- process Reviews.

Where required, the frontend can provide buttons that take the user into the relevant module.

13.2 Core Message Center Principle

A Conversation represents communication between two marketplace users.

Recommended relationship:



There is no:

contextType
contextId

for the initial implementation.

A user who sells on TOFA may still initiate a Conversation with another seller because they may be purchasing from that seller.

Example:

Victor Agro
Seller on TOFA

Victor Agro wants to purchase equipment from:
Industrial Machines Ltd

Victor Agro acts as Buyer
Industrial Machines Ltd acts as Seller

Message Center simply sees:

User Victor Agro

↔

User Industrial Machines Ltd

13.3 Conversation Participant Principle

Do not store participants directly as:

buyerId
sellerId

Recommended architecture:

Conversation

↓

ConversationParticipant

This makes the system flexible.

Although the initial implementation supports two participants per Conversation, this structure also avoids redesigning the database if future group conversations are introduced.

13.4 Conversation Structure

Conversation

Field	Type	Required	Description
id	UUID/String	Yes	Conversation ID

conversationReference	String	Yes	Human-readable reference
status	Enum	Yes	active/blocked
lastMessageId	UUID/String/null	No	Latest Message
lastMessageAt	Timestamp/null	No	Latest Message time
createdAt	Timestamp	Yes	Created
updatedAt	Timestamp	Yes	Updated

13.5 ConversationParticipant Structure

ConversationParticipant

Field	Type	Required	Description
id	UUID/String	Yes	Participant record
conversationId	UUID/String	Yes	Conversation
userId	UUID/String	Yes	User
unreadCount	Integer	Yes	Number of unread Messages
lastReadMessageId	UUID/String/null	No	Last Message read
joinedAt	Timestamp	Yes	Participation date
createdAt	Timestamp	Yes	Created
updatedAt	Timestamp	Yes	Updated

For the initial implementation:

Conversation

→ exactly 2 participants

13.6 Conversation Statuses

Recommended:

active
blocked

active

Users can communicate normally.

blocked

Conversation has been restricted by Admin/System.

Historical Messages should remain available according to TOFA policy.

A user does not need to manually close a normal Conversation.

13.7 Conversation Uniqueness

A pair of users should normally have one active Conversation.

Example:

User 123

+

User 456

→ Conversation conv_001

If User 123 later wants to discuss another Product from User 456, they continue using the same Conversation.

Backend should prevent duplicate Conversations between the same two participants.

Conceptually:

sorted(userIdA, userIdB)

→ unique participant pair

The implementation may maintain an internal unique conversation key.

Example:

participantPairKey = min(userIdA,userIdB) + ":" + max(userIdA,userIdB)

This field does not need to be returned to frontend.

13.8 Conversation Reference

Example:

TOFA-MSG-20260827-000123

This may be useful for:

- support;
- Admin;
- reporting;
- moderation.

It does not need to be prominently displayed in the normal Message Center UI.

13.9 Who Can Start a Conversation

A marketplace user may initiate a Conversation with another eligible marketplace user.

Most commonly:

Buyer
→ Seller

However:

Seller
→ Another Seller

must also be permitted because the first Seller may be buying.

The backend should determine eligibility based on:

- authenticated user;
- target user exists;
- target user is active;
- user is not messaging themselves;
- messaging access is permitted.

13.10 Start Conversation

Endpoint

POST /messages/conversations

Payload

```
{  
  "recipientUserId": "user_456"  
}
```

Do not use:

sellerId

because the recipient may simply be another marketplace user.

13.11 Start Conversation Backend Processing

Backend must:

1. verify authenticated user;
2. retrieve recipient;
3. verify recipient exists;
4. verify recipient is active;
5. prevent messaging self;
6. validate messaging access;
7. check Subscription limits where applicable;
8. search for existing Conversation between both users;
9. return existing Conversation if found;
10. otherwise create Conversation;
11. create both ConversationParticipant records;
12. return Conversation.

13.12 Start Conversation Response

```
{  
  "success": true,  
  "message": "Conversation ready.",  
  
  "data": {
```

```
"conversationId": "conv_123",
"conversationReference": "TOFA-MSG-20260827-000123",

"participant": {
  "id": "user_456",
  "displayName": "Agro Supplies Ltd",
  "profileImage": "https://cdn.tofa.com/store/logo.jpg"
},

"status": "active",

"createdAt": "2026-08-27T09:00:00Z"
}
}
```

If Conversation already exists, return the existing Conversation.

Do not create another one.

13.13 Product Page Message Entry

On a Product page, frontend may show:

[Message Seller]

Flow:

Buyer clicks Message Seller



Frontend gets Product seller userId



POST /messages/conversations



Existing/new Conversation opens

The Product itself is not stored as the Conversation context.

Frontend may optionally prefill:

Hi, I'm interested in your Premium Cashew Nuts.

The user should be able to edit before sending.

13.14 Seller Buying from Another Seller

If a Seller sees another seller's Product:

Seller A
views
Seller B Product

Seller A can click:

[Message Seller]

The request remains:

```
{  
  "recipientUserId": "seller_B_user_id"  
}
```

Seller A's account does not need to change from Seller to Buyer.

The marketplace transaction itself determines who acts as buyer.

13.15 RFQ Shortcut in Message Center

The Message Center may include a frontend action:

[Request Quote]

This should not create an RFQ inside Message Center.

Recommended flow:

User clicks Request Quote
↓
Frontend opens recipient's Store/Products
↓
User selects Product
↓
Existing Direct RFQ flow opens

The Direct RFQ module remains responsible for:

- Product;
- quantity;
- unit;
- delivery address;
- delivery method;
- description;
- seller quotation;
- negotiation;
- acceptance.

13.16 View Seller Products Shortcut

Where the other participant is a verified Seller, chat header may show:

[View Store]

[Request Quote]

If the other participant is not currently a Seller, these buttons do not need to appear.

Frontend may retrieve Seller information from existing Seller/User APIs.

Message Center does not store the seller's Product list.

13.17 Message Structure

Message

Field	Type	Required	Description
id	UUID/String	Yes	Message ID
conversationId	UUID/String	Yes	Conversation
senderId	UUID/String	Yes	Sender
clientMessageId	String	Yes	Frontend idempotency reference
messageType	Enum	Yes	text/image/file/mixed

content	String/null	No	Text and emojis
replyToMessageId	UUID/String/null	No	Replied Message
status	Enum	Yes	sent/delivered/read/deleted
sentAt	Timestamp	Yes	Sent
deliveredAt	Timestamp/null	No	Delivered
readAt	Timestamp/null	No	Read
editedAt	Timestamp/null	No	Edited
deletedAt	Timestamp/null	No	Deleted
createdAt	Timestamp	Yes	Created
updatedAt	Timestamp	Yes	Updated

13.18 Message Types

Recommended:

text
image
file
mixed

text

Text and emojis.

Example:

Hello, is this Product still available? 😊

image

One or more images.

file

One or more documents/files.

mixed

Text plus image/file attachment.

Example:

Please see the specification attached.

+ PDF

13.19 Emoji Support

Emojis should be stored as normal Unicode text inside:

content

No separate emoji table is necessary.

Example:

```
{  
  "messageType": "text",  
  "content": "Thank you 😊 We'll review this and get back to you."  
}
```

Backend/database encoding must support full Unicode.

13.20 Message Attachment Structure

MessageAttachment

Field	Type	Required	Description
id	UUID/String	Yes	Attachment ID
messageId	UUID/String	Yes	Parent Message
fileName	String	Yes	Original/display filename
fileUrl	String	Yes	Stored file

attachmentType	Enum	Yes	image/file
mimeType	String	Yes	MIME type
fileSize	Integer	Yes	Bytes
createdAt	Timestamp	Yes	Created

13.21 Supported Images

Recommended:

jpg
jpeg
png
webp

Frontend should display image preview directly in chat.

Example:

[Image Preview]

Clicking image may open a larger viewer.

13.22 Supported Files

Recommended initial formats:

pdf
doc
docx
xls
xlsx
csv
txt

The allowed types should be Admin-configurable.

Frontend should display:

 Product Specification.pdf
2.4 MB

[Open / Download]

depending on platform UI capabilities.

13.23 File Upload Principle

Attachments should first be uploaded through the platform's secure upload service.

Recommended flow:

Select File



Upload



Receive fileId



Send Message

Do not allow frontend to provide arbitrary URLs as trusted attachments.

13.24 Send Text Message

Endpoint

POST /messages/conversations/:conversationId/messages

Payload

```
{
  "clientMessageId": "client-msg-001",
  "messageType": "text",
  "content": "Hello, is this Product still available? 😊",
  "replyToMessageId": null,
  "attachments": []
}
```

13.25 Send Image Message

Payload

```
{
  "clientMessageId": "client-msg-002",
  "messageType": "image",
  "content": null,

  "attachments": [
    {
      "fileId": "upload_image_123"
    }
  ]
}
```

13.26 Send Text with Image

```
{
  "clientMessageId": "client-msg-003",
  "messageType": "mixed",
  "content": "Is this the exact colour available?",

  "attachments": [
    {
      "fileId": "upload_image_456"
    }
  ]
}
```

13.27 Send File Message

```
{
  "clientMessageId": "client-msg-004",
  "messageType": "file",
  "content": "Please review the attached specification.",

  "attachments": [
    {
      "fileId": "upload_file_123"
    }
  ]
}
```

13.28 Send Message Backend Validation

Backend must:

1. verify authenticated user;
2. retrieve Conversation;
3. verify user is a participant;
4. verify Conversation is active;
5. validate `clientId`;
6. validate Message type;
7. validate text length;
8. validate attachments;
9. validate allowed attachment types;
10. validate attachment ownership;
11. apply spam/rate limits;
12. create Message;
13. create attachments;
14. update Conversation lastMessage;
15. increment other participant's unread count;
16. emit real-time Message event;
17. return Message.

13.29 Send Message Response

```
{
  "success": true,
  "message": "Message sent successfully.",

  "data": {
    "id": "msg_123",
    "conversationId": "conv_123",

    "sender": {
      "id": "user_123",
      "displayName": "Victor E."
    },

    "messageType": "text",
    "content": "Hello, is this Product still available? 😊",

    "attachments": [],

    "status": "sent",
```

```
"sentAt": "2026-08-27T09:30:00Z"  
}  
}
```

13.30 Message Idempotency

`clientId` prevents duplicate Messages caused by network retries.

Recommended unique constraint:

senderId + clientId

Example:

Frontend sends Message

Network times out

Frontend retries

Backend sees same clientId

→ returns existing Message

→ does not create duplicate

13.31 Get Conversations

Endpoint

GET /messages/conversations

Query

search

unreadOnly

page

limit

Example:

GET /messages/conversations?unreadOnly=true&page=1&limit=20

13.32 Get Conversations Response

```
{
  "success": true,

  "data": [
    {
      "conversationId": "conv_123",

      "participant": {
        "id": "user_456",
        "displayName": "Agro Supplies Ltd",
        "profileImage": "https://cdn.tofa.com/store/logo.jpg",
        "isSeller": true
      },

      "lastMessage": {
        "id": "msg_980",
        "senderId": "user_456",
        "messageType": "text",
        "content": "Yes, we currently have it in stock.",
        "sentAt": "2026-08-27T09:20:00Z"
      },

      "unreadCount": 2,

      "lastMessageAt": "2026-08-27T09:20:00Z"
    },
  ],

  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 12,
    "totalPages": 1
  }
}
```

13.33 Conversation List Ordering

Conversations should be ordered by:

lastMessageAt DESC

The most recently active Conversation appears first.

If Conversation has no Message yet, use:

`createdAt`

as fallback.

13.34 Conversation Search

Search should initially match the other participant's:

- first name;
- last name;
- company name;
- store name.

Example:

Search: Agro

returns Conversations with:

Agro Supplies Ltd
Victor Agro Stores

No need to search all Message contents initially.

13.35 Get Messages

Endpoint

GET `/messages/conversations/:conversationId/messages`

Recommended cursor-based pagination:

`cursor`
`limit`

Example:

GET /messages/conversations/conv_123/messages?limit=30

13.36 Get Messages Response

```
{
  "success": true,

  "data": [
    {
      "id": "msg_101",

      "sender": {
        "id": "user_123",
        "displayName": "Victor E."
      },

      "messageType": "text",

      "content": "Hello, is this Product still available?",

      "attachments": [],

      "replyTo": null,

      "status": "read",

      "sentAt": "2026-08-26T16:10:00Z",
      "readAt": "2026-08-26T16:12:00Z"
    },

    {
      "id": "msg_102",

      "sender": {
        "id": "user_456",
        "displayName": "Agro Supplies Ltd"
      },

      "messageType": "mixed",

      "content": "Yes. This is the current packaging.",
    }
  ]
}
```

```

"attachments": [
  {
    "id": "attachment_101",
    "type": "image",
    "fileName": "packaging.jpg",
    "fileUrl": "https://cdn.tofa.com/messages/packaging.jpg",
    "mimeType": "image/jpeg",
    "fileSize": 480000
  }
],

"replyTo": {
  "messageId": "msg_101",
  "contentPreview": "Hello, is this Product still available?"
},

"status": "read",

"sentAt": "2026-08-26T16:15:00Z"
},

"pagination": {
  "nextCursor": "msg_100",
  "hasMore": true
}
}

```

13.37 Message Grouping by Day

Messages should be grouped visually by date, similar to common messaging applications.

The backend does not need to physically store Message groups.

Backend returns:

sentAt

for every Message.

Frontend groups them.

13.38 Today Group

If Message date equals the user's current local date, display:

Today

Example:

Today

10:05 AM

Hello, are these available?

10:07 AM

Yes, they are.

13.39 Yesterday Group

If Message date is exactly the previous local calendar day:

Yesterday

Example:

Yesterday

4:30 PM

Can you send the Product specification?

4:35 PM

Sure.

13.40 Older Message Groups

For older Messages, display the actual date.

Example:

Monday, 24 August 2026

or:

24 August 2026

Recommended frontend rule:

Today

Yesterday

Monday, 24 August 2026

Sunday, 23 August 2026

...

13.41 Important Timezone Rule

Grouping must use the user's local timezone, not UTC directly.

Backend timestamps remain:

UTC / ISO timestamp

Frontend converts to the user's timezone before deciding:

Today

Yesterday

Date

This prevents a Message sent late at night from being grouped under the wrong day.

13.42 Example Grouped Response Presentation

Backend response remains flat.

Frontend turns:

```
[
  {
    "id": "msg_1",
    "sentAt": "2026-08-27T07:00:00Z"
  },
  {
    "id": "msg_2",
```

```
"sentAt": "2026-08-27T08:00:00Z"
},
{
  "id": "msg_3",
  "sentAt": "2026-08-26T13:00:00Z"
}
]
```

into:

Yesterday

Message 3

Today

Message 1

Message 2

The dividing line shown above is illustrative UI behavior only; no document or backend field is required.

13.43 Time Display

Each Message should display local time.

Example:

10:45 AM

or according to the user's locale:

10:45

The Internationalization module can later control date/time formatting.

13.44 Reply to Message

A user can reply to a specific Message.

Payload

```
{  
  "clientMessageId": "client-msg-005",  
  "messageType": "text",  
  "content": "Yes, that specification works for us.",  
  "replyToMessageId": "msg_102",  
  "attachments": []  
}
```

Backend must verify the replied Message belongs to the same Conversation.

13.45 Reply Display

Frontend may display:

Replying to:

"Can you deliver 500kg?"

Yes, we can.

Only a short preview should be shown.

13.46 Message Statuses

Recommended:

sent
delivered
read
deleted

sent

Message successfully stored by backend.

delivered

Recipient's connected client received/synchronized the Message.

read

Recipient opened/read the Message.

deleted

Message was soft-deleted.

13.47 Mark Conversation as Read

Endpoint

PATCH /messages/conversations/:conversationId/read

Payload

```
{
  "lastReadMessageId": "msg_102"
}
```

Backend:

1. verifies participant;
2. updates `lastReadMessageId`;
3. marks applicable Messages read;
4. resets/recalculates participant unread count;
5. emits read event.

13.48 Mark Read Response

```
{
  "success": true,

  "data": {
    "conversationId": "conv_123",
    "unreadCount": 0,
    "lastReadMessageId": "msg_102"
  }
}
```

13.49 Unread Message Count

Endpoint

GET /messages/unread-count

Response

```
{
  "success": true,

  "data": {
    "totalUnread": 8
  }
}
```

This may be used for:

Messages  8

in navigation/header.

13.50 Real-Time Messaging

Recommended:

WebSocket / Socket.IO

REST remains the persistence source of truth.

Flow:

User sends Message



Backend validates



Message stored



Real-time event emitted



Recipient sees Message

13.51 Recommended Real-Time Events

message:new
message:delivered
message:read
message:edited
message:deleted

conversation:updated

typing:start
typing:stop

13.52 Typing Indicator

Users may see:

Agro Supplies Ltd is typing...

Typing events should not be stored.

Example socket payload:

```
{  
  "conversationId": "conv_123",  
  "userId": "user_456"  
}
```

13.53 Edit Message

Endpoint

PATCH /messages/:messageId

Payload

```
{  
  "content": "Can you deliver 500kg before Friday?"  
}
```

Rules

- sender must own Message;
- Message must not be deleted;
- Message must be text or mixed text;
- edit must occur within configured edit window.

Recommended initial:

15 minutes

13.54 Edit Message Response

```
{
  "success": true,
  "message": "Message updated successfully.",

  "data": {
    "messageId": "msg_123",
    "content": "Can you deliver 500kg before Friday?",
    "editedAt": "2026-08-27T09:45:00Z"
  }
}
```

Frontend should display:

Edited

13.55 Delete Message

Endpoint

DELETE /messages/:messageId

Recommended:

soft delete

The record remains for audit purposes.

Frontend displays:

This message was deleted.

13.56 Delete Message Response

```
{  
  "success": true,  
  "message": "Message deleted successfully."  
}
```

13.57 Delete Rules

Only Message sender may delete their Message within permitted policy.

Deletion should not delete:

- related attachments from historical audit immediately;
- Conversation;
- other Messages.

A deleted Message cannot be used to undo another module's transaction action.

13.58 Image Preview

Image Messages should display directly in the chat.

Recommended behavior:

Thumbnail/Preview



Click



Full image viewer

For multiple images:

2–4 image grid

with additional count where necessary.

Example:

[Image] [Image]
[Image] +2

This is frontend behavior only.

13.59 File Preview

For supported files, frontend should show:

File icon
File Name
File Size
File Type

Example:

 Product-Specification.pdf
2.4 MB

Where browser preview is supported, frontend may open it.

13.60 Important File Rule

Files shared in Message Center are chat attachments only.

Examples:

Quotation PDF
Payment receipt
Invoice
Delivery document

do not automatically become formal transaction records.

If a formal flow exists elsewhere, that flow remains required.

Example:

Buyer sends Payment proof in chat

Payment status does not change.

Buyer must still use the Payment proof endpoint defined in the Payment module.

13.61 Request Quote Shortcut

Recommended chat header:

Agro Supplies Ltd

[View Store] [Request Quote]

If **Request Quote** is selected:

Seller active Products



Select Product



Direct RFQ form

The RFQ remains completely outside Message Center.

13.62 Request Quote from Product Mention

A future frontend enhancement may allow users to share a Product card inside Conversation.

Example:

Premium Cashew Nuts

USD 20/kg

[View Product]

[Request Quote]

However, this should be treated as a display/share feature only.

Message Center should not duplicate Product data storage.

For the first version, this can remain optional.

13.63 Conversation Header

Recommended header information:

Participant Name / Store Name

Profile Image

Seller Verification where applicable

[View Store]

[Request Quote]

If participant does not have a Seller profile:

View Store

Request Quote

should not appear.

13.64 Participant Details

Conversation API may return lightweight participant details.

Example:

```
{
  "id": "user_456",
  "displayName": "Agro Supplies Ltd",
  "profileImage": "https://cdn.tofa.com/store/logo.jpg",

  "seller": {
    "isSeller": true,
    "isVerified": true,
    "storeName": "Agro Supplies Ltd"
  }
}
```

Do not duplicate full Seller profile inside Message Center response.

13.65 User Identity Display

Where the user has a verified Seller business:

Prefer:

storeName

as the primary display name in marketplace messaging.

Where the user does not have a Seller store:

Use:

firstName + lastName

according to privacy/display rules.

13.66 Subscription Relationship

Subscription module already supports potential entitlements such as:

message_center_access

max_new_conversations_per_month

attachment_access

Message Center may enforce these.

However, Subscription must not make existing important Conversations inaccessible.

Recommended:

Subscription limits

→ creation of new Conversations / premium messaging benefits

Existing Conversation history remains available.

13.67 New Conversation Limit

If:

max_new_conversations_per_month = 20

backend checks the number of Conversations initiated by the user during the current Subscription period.

If reached:

```
{  
  "success": false,  
  "code": "MESSAGE_CONVERSATION_LIMIT_REACHED",  
  "message": "You have reached the number of new conversations allowed on your current plan."  
}
```

Toast

You have reached your new conversation limit. Upgrade your plan to start more conversations.

13.68 Attachment Access

If:

attachment_access = false

the user may still send normal text Messages.

Attempting attachment:

```
{  
  "success": false,  
  "code": "MESSAGE_ATTACHMENT_NOT_ALLOWED",  
  "message": "File and image attachments are not available on your current plan."  
}
```

This entitlement is optional and can be configured by Admin through Subscription.

13.69 Message Center Access

If:

message_center_access = false

TOFA may prevent starting new Conversations.

Existing Conversation behavior should follow TOFA's chosen business rules.

Recommended:

Existing Conversations

→ readable

New Conversations

→ blocked

If TOFA needs transaction-related communication later, exemptions can be introduced without changing the basic Message Center structure.

13.70 Message Notifications

New user Messages should generate:

Real-time notification

+

In-app unread count

Email should not be sent immediately for every Message.

This would create too many emails.

Recommended:

Recipient offline

+

Message remains unread for configured duration

→ send one email notification

13.71 Unread Message Email

Subject: You have a new message on TOFA Marketplace

Hello {UserName},

You have received a new message from {SenderName} on TOFA Marketplace.

Message Preview:

{MessagePreview}

Sign in to TOFA Marketplace to view the full Conversation and respond.

For your security, formal quotations, Payments and other marketplace transaction actions should always be completed through the appropriate TOFA Marketplace workflow.

Best Regards,
Traders of Africa Team

CTA: View Message

13.72 Multiple Unread Messages

Do not send:

1 email
per
1 Message

Recommended:

Conversation receives multiple Messages



Recipient remains unread



Send one notification email

Example:

Subject: You have new messages from {SenderName}

Hello {UserName},

You have {UnreadMessageCount} unread messages from {SenderName} on TOFA Marketplace.

Latest Message:

{LatestMessagePreview}

Sign in to view the Conversation and respond.

Best Regards,
Traders of Africa Team

CTA: View Messages

13.73 Notifications Matrix

Event	Real-Time	In-App	Email
New Message	Yes	Yes	If unread rule applies
New Image	Yes	Yes	If unread rule applies
New File	Yes	Yes	If unread rule applies
Message Read	Yes	No	No
Message Edited	Yes	No	No
Message Deleted	Yes	No	No
Conversation Blocked	Yes	Yes	Optional
Message Reported	No	Reporter toast	No

13.74 Report Message

Users should be able to report inappropriate or suspicious Messages.

Endpoint

POST /messages/:messageId/report

Payload

```
{
  "reason": "fraud_attempt",
  "details": "The user asked me to make payment to a personal account outside TOFA."
}
```

Recommended reasons:

spam

abusive_content
fraud_attempt
payment_scam
inappropriate_content
off_platform_solicitation
other

13.75 Report Message Response

```
{  
  "success": true,  
  "message": "Message reported successfully. Our team will review it."  
}
```

Toast

Message reported successfully.

13.76 MessageReport Structure

MessageReport

Field	Type	Required
id	UUID/String	Yes
messageId	UUID/String	Yes
conversationId	UUID/String	Yes
reportedBy	UUID/String	Yes
reportedUserId	UUID/String	Yes
reason	Enum	Yes
details	String/null	No
status	Enum	Yes
reviewedBy	UUID/String/null	No
reviewNotes	String/null	No

createdAt	Timestamp	Yes
reviewedAt	Timestamp/null	No

13.77 Report Statuses

Recommended:

pending
reviewing
resolved
dismissed

13.78 Admin Get Conversations

Endpoint

GET /admin/messages/conversations

Query

status
userId
reportedOnly
search
dateFrom
dateTo
page
limit

Response

```
{
  "success": true,

  "data": [
    {
      "conversationId": "conv_123",
      "conversationReference": "TOFA-MSG-20260827-000123",

      "participants": [
```

```

    {
      "id": "user_123",
      "displayName": "Victor E."
    },
    {
      "id": "user_456",
      "displayName": "Agro Supplies Ltd"
    }
  ],

  "status": "active",

  "reportsCount": 0,

  "lastMessageAt": "2026-08-27T09:30:00Z"
}
],

"pagination": {
  "page": 1,
  "limit": 20,
  "total": 150,
  "totalPages": 8
}
}

```

13.79 Admin Get Conversation by ID

Endpoint

GET /admin/messages/conversations/:conversationId

Authorized Admin may see:

- participants;
- Messages;
- attachments;
- reports;
- moderation status;
- Conversation history.

Admin access to private Conversations should require the appropriate existing Admin permission.

13.80 Admin Get Reports

Endpoint

GET /admin/messages/reports

Query

status
reason
reportedUserId
reportedBy
dateFrom
dateTo
page
limit

13.81 Admin Review Report

Endpoint

PATCH /admin/messages/reports/:reportId

Payload

```
{  
  "status": "resolved",  
  "action": "warning_issued",  
  "notes": "User warned regarding off-platform payment solicitation."  
}
```

Possible actions:

no_action
warning_issued
message_hidden
conversation_blocked
user_restricted
escalated

If User account restriction is required, reuse the existing User/Admin module.

Do not create another account-status mechanism inside Message Center.

13.82 Admin Block Conversation

Endpoint

PATCH /admin/messages/conversations/:conversationId/status

Payload

```
{
  "status": "blocked",
  "reason": "Conversation restricted following fraud investigation."
}
```

Response

```
{
  "success": true,
  "message": "Conversation blocked successfully."
}
```

Blocking a Conversation does not automatically:

- disable either User;
- cancel an RFQ;
- cancel an Order;
- cancel Payment.

Those modules remain independent.

13.83 Message Center Settings

Recommended Admin-configurable settings:

maxMessageCharacters
maxAttachmentsPerMessage
maxAttachmentSizeMb
allowedAttachmentTypes
messageEditWindowMinutes
unreadEmailDelayMinutes
messageReportingEnabled

13.84 Get Admin Message Settings

Endpoint

GET /admin/messages/settings

Response

```
{
  "success": true,

  "data": {
    "maxMessageCharacters": 5000,
    "maxAttachmentsPerMessage": 5,
    "maxAttachmentSizeMb": 20,

    "allowedAttachmentTypes": [
      "jpg",
      "jpeg",
      "png",
      "webp",
      "pdf",
      "doc",
      "docx",
      "xls",
      "xlsx",
      "csv",
      "txt"
    ],

    "messageEditWindowMinutes": 15,
    "unreadEmailDelayMinutes": 30,
    "messageReportingEnabled": true
  }
}
```

13.85 Update Message Settings

Endpoint

PATCH /admin/messages/settings

Payload

```
{
  "maxMessageCharacters": 5000,
  "maxAttachmentsPerMessage": 5,
  "maxAttachmentSizeMb": 20,
  "messageEditWindowMinutes": 15,
  "unreadEmailDelayMinutes": 30
}
```

Response

```
{
  "success": true,
  "message": "Message Center settings updated successfully."
}
```

Admin Toast

Message Center settings updated successfully.

13.86 Message Privacy

Only the two Conversation participants may access normal Conversation Messages.

Authorized Admin may access Messages only where:

- support;
- moderation;
- fraud investigation;
- compliance;

requires it.

Backend must enforce access.

Knowing:

conversationId

must never allow another user to retrieve the Conversation.

13.87 Public Contact Information

Users may communicate inside TOFA without requiring the platform to expose personal:

- phone numbers;
- emails;
- addresses.

Where Subscription permits additional business information, the existing Subscription and privacy rules still apply.

Message Center itself should not automatically expose private contact information.

13.88 Message Retention

Messages should not automatically disappear because:

- Product becomes inactive;
- seller changes Subscription;
- User becomes buyer/seller;
- an RFQ is completed;
- an Order is completed.

Conversations may be needed for:

- support;
- audit;
- disputes;
- fraud investigation.

Retention policy should be defined separately according to TOFA's data policy.

13.89 User Account Status Relationship

If User becomes:

disabled

existing User-module restrictions apply.

Message Center should not invent another User status.

If User is:

deleted

historical Conversation data may remain for transaction/audit purposes, but the User should not be able to send new Messages.

13.90 Internationalization

The Message Center interface uses the User's existing:

selectedLanguage

Default:

English

This affects interface labels such as:

Messages

Today

Yesterday

Type a message

Send

Reply

Delete

User-written Message content should be stored exactly as written.

It should not automatically be replaced with another language.

13.91 Future Translation Support

If TOFA later adds automatic translation:

Original Message

+

Translated Display Version

should be maintained.

Example:

French Message



English translation shown to English user

[View Original]

The Internationalization module should define that architecture.

It is not required for the initial Message Center implementation.

13.92 Frontend Message Center Layout

Recommended desktop layout:

Conversation List

	Active Conversation

	Participant Header
	Messages
	Message Input
	Attachments
	Emojis

13.93 Conversation List UI

Each item should show:

Participant Image

Participant Name / Store Name

Last Message Preview

Message Time

Unread Badge

Example:

[Logo] Agro Supplies Ltd
Yes, the Product is available.
10:32 AM 2

13.94 Chat Header UI

Where participant has a Seller profile:

[Logo] Agro Supplies Ltd
Verified Seller

[View Store] [Request Quote]

Where participant is not a Seller:

[Avatar] Victor E.

No seller-related action buttons are needed.

13.95 Message Composer

Recommended:

[+] Type a message... 😊 [Send]

+ may open:

Photo
File

Emoji icon opens emoji picker.

13.96 Message Bubble Alignment

Recommended:

Current User
→ right side

Other User
→ left side

Example:

Are these available? ✓✓

Yes, we have 500 units available.

This is frontend presentation only.

13.97 Date Grouping UI

Messages should be grouped as:

Yesterday

<Message>

<Message>

Today

<Message>

<Message>

Older:

Monday, 24 August 2026

<Message>

The labels should remain visible between date groups.

13.98 Date Grouping Logic

Frontend helper conceptually:

if messageDate == today:

"Today"

else if messageDate == yesterday:
 "Yesterday"

else:
 formatted date

All comparisons use the user's local timezone.

13.99 Empty Conversation State

If Conversation has no Messages:

Start a conversation with {ParticipantName}.

Ask about products, availability, specifications or other enquiries.

Where recipient is Seller:

[View Store]
[Request Quote]

may also be shown.

13.100 No Conversations State

If User has never chatted:

No conversations yet.

When you message another marketplace user, your conversations will appear here.

Optional CTA:

[Explore Marketplace]

13.101 Failed Message UI

If sending fails:

Message not sent.
[Retry]

Frontend should reuse the same:

`clientMessageId`

on retry.

This prevents duplicate Messages.

13.102 Real-Time Reconnection

After network reconnection:

Frontend should:

1. restore authenticated socket;
2. fetch unread count;
3. fetch Messages after last received Message;
4. reconcile pending Messages using `clientMessageId`;
5. update Conversation list.

13.103 Backend Implementation Guide

Message Center backend owns:

- Conversations;
- ConversationParticipants;
- Messages;
- attachments;
- unread counts;
- read receipts;
- real-time Message delivery;
- replies;
- edit/delete;
- message reports;
- conversation moderation;
- Message Center platform settings.

It does not own:

- Product data;
- RFQ commercial terms;
- Order actions;
- Payment;
- Logistics;
- Reviews;
- Seller verification;
- Subscription definitions.

Those modules are referenced where necessary.

13.104 Frontend Implementation Guide

Frontend should keep Message Center simple.

The main functions are:

Conversation List

Chat

Text

Emoji

Images

Files

Reply

Read Status

Search

Request Quote Shortcut

View Store Shortcut

Do not add RFQ forms, Payment forms, Order status actions or logistics forms directly into Message Center.

Buttons should navigate to the existing workflows.

13.105 Subscription Implementation Guide

Subscription may influence:

Can start new Conversation?

How many new Conversations?

Can send attachments?

Message Center should call the existing Entitlement Service.

Do not use:

if plan.name == "Premium"

Use:

message_center_access
max_new_conversations_per_month
attachment_access

as already defined in Subscription Management.

13.106 Internal Events

Recommended:

CONVERSATION_CREATED
CONVERSATION_BLOCKED

MESSAGE_SENT
MESSAGE_DELIVERED
MESSAGE_READ
MESSAGE_EDITED
MESSAGE_DELETED

MESSAGE_REPORTED
MESSAGE_REPORT_RESOLVED

There is no need for Message Center to consume Order/RFQ events in the initial simplified implementation.

13.107 Admin Permissions

Use the existing Admin permission architecture.

Recommended permissions:

messages.view
messages.view_reports
messages.moderate

messages.block_conversation
messages.manage_settings

Super Admin/Admin creation remains in the existing User/Admin module.

13.108 Security Rules

Backend must:

- authenticate every Message request;
- verify Conversation membership;
- prevent IDOR access;
- validate recipient;
- prevent self-messaging;
- validate file ownership;
- validate file MIME type;
- validate file size;
- sanitize text rendering appropriately;
- rate-limit spam;
- protect WebSocket authentication;
- authorize socket room subscriptions;
- soft-delete Messages;
- preserve moderation/audit information.

13.109 Module Relationships

Product Management

Product Page
→ Message Seller

Frontend obtains Seller User ID from Product.

No Product reference is required in Conversation.

Direct RFQ

Message Center
→ Request Quote button
→ Seller Products
→ Direct RFQ Module

RFQ remains independent.

Market RFQ

No Message Center-specific Market RFQ flow is required initially.

If buyer and seller already have a Conversation, they can communicate normally.

Formal Market RFQ negotiations remain inside Quote versions.

Order

Buyer and seller can continue using their normal Conversation.

Order status remains entirely in Order Management.

Payment

Payment proof and Payment status remain in Payment.

Logistics

Formal buyer logistics details remain in Order Management.

Subscription

Subscription may control messaging access and limits.

Reviews

Reviews remain independent of Message Center.

13.110 Module Summary

The Message Center module establishes that:

- Message Center is a simple user-to-user chat system;
- there is no `contextType` or `contextId`;
- users are not permanently labeled buyer/seller inside Message Center;
- a Seller may buy from another Seller;
- Conversations therefore use generic marketplace User participants;
- `ConversationParticipant` is used instead of `buyerId/sellerId`;
- one user pair normally has one Conversation;
- duplicate Conversations are prevented;

- users can send text;
- users can send emojis;
- users can send images;
- users can send documents/files;
- users can reply to Messages;
- users can edit eligible Messages;
- users can soft-delete eligible Messages;
- Messages support sent/delivered/read states;
- unread counts are supported;
- real-time messaging is supported;
- Message sending is idempotent using `clientMessageId`;
- Messages are grouped visually by day;
- current-day Messages display under `Today`;
- previous-day Messages display under `Yesterday`;
- older Messages display under their formatted dates;
- day grouping uses the user's local timezone;
- Product pages can open the Seller Conversation;
- Conversation header can provide `View Store`;
- Conversation header can provide `Request Quote`;
- Request Quote opens the existing Direct RFQ flow;
- RFQ commercial terms are never stored as chat authority;
- Payment actions remain in Payment;
- Order actions remain in Order Management;
- logistics details remain in their existing structured flows;
- Subscription may dynamically control messaging benefits;
- Admin moderation uses existing Admin permissions;
- Message Center remains intentionally lightweight and familiar, similar to standard messaging applications.

MODULE 14. NOTIFICATIONS MANAGEMENT

14. NOTIFICATIONS MANAGEMENT MODULE

14.1 Overview

The Notifications Management module provides a centralized system for delivering in-app notifications and managing notification preferences across TOFA Marketplace.

The module supports notifications generated from existing marketplace activities such as:

- Authentication
- Seller verification
- Product approval and rejection
- Direct RFQ activity
- Market RFQ activity
- Payments
- Orders
- Logistics
- Subscription
- Reviews and Points
- Message Center
- Admin announcements

The Notifications module does not own the business action that created the notification.

Example:

Order Management

→ ORDER_SHIPPED

→ Notification Service

→ Buyer receives notification

The Order module remains responsible for changing the Order status.

Similarly:

Payment Module

→ PAYMENT_CONFIRMED

→ Notification Service

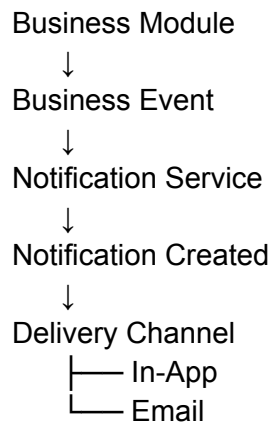
→ User receives notification

The Payment module remains responsible for the Payment status.

The Notification module is responsible only for delivering and storing the alert.

14.2 Core Notification Principle

Recommended flow:



Future channels may include:

Push Notification
SMS
WhatsApp

These are not required for the initial implementation.

14.3 Notification Channels

Initial supported channels:

in_app
email

in_app

Notification appears inside the TOFA Marketplace application.

Examples:

Your product has been approved.

A seller has responded to your RFQ.

Your order has been shipped.

email

Email notification sent to the User's registered email address.

Email delivery must follow the email rules already defined in the individual business modules.

The Notification module should not create new or duplicate email templates for business events that already have an email template.

14.4 Important Email Ownership Rule

Each business module remains responsible for defining the email content related to its business activity.

Examples:

Product Approved

→ Product Management email template

Quote Received

→ Direct RFQ / Market RFQ email template

Order Shipped

→ Order Management email template

Subscription Activated

→ Subscription Management email template

The Notification module may handle sending the email, but it should use the template supplied by the relevant module.

This prevents duplicate or contradictory email templates.

14.5 Notification Structure

Notification

Field	Type	Required	Description
id	UUID/String	Yes	Notification ID
userId	UUID/String	Yes	Recipient
type	String	Yes	Notification event/type
category	Enum	Yes	General notification category
title	String	Yes	Notification title
message	String	Yes	Short notification body
actionType	Enum/null	No	Type of frontend action
actionId	UUID/String/null	No	Related entity ID
actionUrl	String/null	No	Frontend route where applicable
imageUrl	String/null	No	Optional image/icon
isRead	Boolean	Yes	Read status
readAt	Timestamp/null	No	Read time
createdAt	Timestamp	Yes	Created
updatedAt	Timestamp	Yes	Updated

14.6 Notification Categories

Recommended:

account
seller
product
rfq
payment
order
logistics
subscription
review
reward

message
system

These categories are mainly useful for:

- filtering
- grouping
- frontend icons
- notification preferences

They should not replace the specific **type**.

14.7 Notification Type

type identifies the exact event.

Examples:

EMAIL_VERIFIED
SELLER_APPLICATION_APPROVED
SELLER_APPLICATION_REJECTED

PRODUCT_APPROVED
PRODUCT_REJECTED

DIRECT_RFQ_CREATED
DIRECT_RFQ_QUOTE_RECEIVED
DIRECT_RFQ_COUNTER_OFFER_RECEIVED
DIRECT_RFQ_QUOTE_ACCEPTED
DIRECT_RFQ_QUOTE_REJECTED

MARKET_RFQ_QUOTE_RECEIVED
MARKET_RFQ_COUNTER_OFFER_RECEIVED
MARKET_RFQ_AWARDED
MARKET_RFQ_CLOSED

PAYMENT_CONFIRMED
PAYMENT_FAILED
PAYMENT_PROOF_REJECTED

ORDER_CREATED
ORDER_PROCESSING
ORDER_READY_FOR_SHIPMENT

BUYER_LOGISTICS_DETAILS_SUBMITTED
ORDER_SHIPPED
ORDER_DELIVERED
ORDER_RECEIVED
ORDER_COMPLETED

SHIPMENT_PICKED_UP
SHIPMENT_OUT_FOR_DELIVERY
SHIPMENT_DELIVERY_FAILED

SUBSCRIPTION_ACTIVATED
SUBSCRIPTION_EXPIRING
SUBSCRIPTION_EXPIRED

PRODUCT_REVIEW_RECEIVED
SELLER_REVIEW_RECEIVED
REVIEW_POINTS_AWARDED

NEW_MESSAGE

SYSTEM_ANNOUNCEMENT

The Notification Service should use a consistent naming convention.

Recommended:

UPPER_SNAKE_CASE

14.8 Action Type

A Notification may optionally direct the user somewhere.

Recommended values:

none
product
seller
direct_rfq
market_rfq
quote
payment
order

subscription
review
message
external

Example:

```
{  
  "actionType": "order",  
  "actionId": "order_101",  
  "actionUrl": "/orders/order_101"  
}
```

The frontend may use:

actionType + actionId

or simply:

actionUrl

depending on routing architecture.

14.9 Example Notification

```
{  
  "id": "notification_123",  
  "type": "ORDER_SHIPPED",  
  "category": "order",  
  
  "title": "Your order has been shipped",  
  
  "message": "Agro Supplies Ltd has shipped Order TOFA-ORD-20260827-000123.",  
  
  "actionType": "order",  
  "actionId": "order_123",  
  "actionUrl": "/orders/order_123",  
  
  "isRead": false,  
  
  "createdAt": "2026-08-27T10:00:00Z"
```

```
}
```

14.10 Notification Creation

Notifications should primarily be created by backend business events.

Example:

```
ORDER_SHIPPED
  ↓
Notification Service
  ↓
Create Notification for Buyer
```

Frontend should not create marketplace business notifications.

14.11 Internal Create Notification Service

This does not need to be a public frontend endpoint.

Recommended internal service:

```
NotificationService.createNotification()
```

Example internal payload:

```
{
  "userId": "user_123",
  "type": "ORDER_SHIPPED",
  "category": "order",

  "title": "Your order has been shipped",

  "message": "Agro Supplies Ltd has shipped your order.",

  "actionType": "order",
  "actionId": "order_101",
  "actionUrl": "/orders/order_101"
}
```

14.12 Get Notifications

Endpoint

GET /notifications

Query

category

isRead

page

limit

Example:

GET /notifications?isRead=false&page=1&limit=20

14.13 Get Notifications Response

```
{
  "success": true,

  "data": [
    {
      "id": "notification_123",

      "type": "ORDER_SHIPPED",
      "category": "order",

      "title": "Your order has been shipped",

      "message": "Agro Supplies Ltd has shipped your order.",

      "actionType": "order",
      "actionId": "order_101",
      "actionUrl": "/orders/order_101",

      "isRead": false,
      "readAt": null,

      "createdAt": "2026-08-27T10:00:00Z"
    },
  ],
}
```

```

{
  "id": "notification_124",

  "type": "DIRECT_RFQ_QUOTE_RECEIVED",
  "category": "rfq",

  "title": "Quotation received",

  "message": "You have received a quotation for Premium Cashew Nuts.",

  "actionType": "direct_rfq",
  "actionId": "rfq_101",
  "actionUrl": "/rfqs/direct/rfq_101",

  "isRead": true,
  "readAt": "2026-08-27T09:20:00Z",

  "createdAt": "2026-08-27T09:00:00Z"
},

{
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 30,
    "totalPages": 2
  }
}

```

14.14 Notification Ordering

Notifications should be ordered by:

createdAt DESC

Newest notifications appear first.

14.15 Unread Notification Count

Endpoint

GET /notifications/unread-count

Response

```
{
  "success": true,

  "data": {
    "unreadCount": 7
  }
}
```

Frontend may display:



14.16 Mark Notification as Read

Endpoint

PATCH /notifications/:notificationId/read

No payload required.

Response

```
{
  "success": true,

  "message": "Notification marked as read.",

  "data": {
    "notificationId": "notification_123",
    "isRead": true,
    "readAt": "2026-08-27T11:00:00Z"
  }
}
```

14.17 Mark All Notifications as Read

Endpoint

PATCH /notifications/read-all

Response

```
{
  "success": true,

  "message": "All notifications marked as read.",

  "data": {
    "updatedCount": 7
  }
}
```

Toast

All notifications marked as read.

14.18 Notification Click Behaviour

When a user clicks a Notification:

Frontend should:

1. mark it as read;
2. navigate to `actionUrl` where available.

Example:

Quotation received



Click



PATCH notification read



Navigate to Direct RFQ

If there is no action:

`actionType = none`

actionUrl = null

the Notification simply opens/displays its information.

14.19 Delete Notification

Users may optionally remove Notification items from their personal Notification list.

Endpoint

DELETE /notifications/:notificationId

Recommended:

soft delete

rather than physical deletion where audit history is important.

Response

```
{
  "success": true,
  "message": "Notification removed successfully."
}
```

This action deletes/hides only the Notification.

It does not change the underlying:

- Order
- RFQ
- Payment
- Product
- Subscription

14.20 Notification Preferences

Users should be able to choose which optional Notification channels they want.

However, some important operational/security emails should remain mandatory.

Recommended preference categories:

product
rfq
payment
order
logistics
subscription
review
reward
message
marketing

14.21 NotificationPreference Structure

NotificationPreference

Field	Type	Required
id	UUID/String	Yes
userId	UUID/String	Yes
category	Enum	Yes
inAppEnabled	Boolean	Yes
emailEnabled	Boolean	Yes
createdAt	Timestamp	Yes
updatedAt	Timestamp	Yes

Recommended unique constraint:

userId + category

14.22 Default Notification Preferences

Recommended initial defaults:

Account / Security

inApp = true
email = true

mandatory

Payment

inApp = true

email = true

mandatory for critical events

Order

inApp = true

email = true

RFQ

inApp = true

email = true

Logistics

inApp = true

email = true for important milestones

Subscription

inApp = true

email = true

Reviews / Rewards

inApp = true

email = true or optional according to event

Messages

inApp = true

email = unread/offline rule

Marketing

inApp = optional

email = user preference

14.23 Mandatory Notifications

Users should not be able to disable notifications that are required for:

- security;
- verification;
- Payment;
- significant Order changes;
- account restriction;
- legal/compliance notices.

Example:

Password changed

Payment confirmed

Payment proof rejected

Account disabled

These may always be sent by email regardless of optional user preferences.

14.24 Get Notification Preferences

Endpoint

GET /notifications/preferences

Response

```
{
  "success": true,

  "data": [
    {
      "category": "rfq",
      "inAppEnabled": true,
      "emailEnabled": true,
      "isMandatory": false
    },

    {
```

```
    "category": "order",
    "inAppEnabled": true,
    "emailEnabled": true,
    "isMandatory": false
  },

  {
    "category": "payment",
    "inAppEnabled": true,
    "emailEnabled": true,
    "isMandatory": true
  },

  {
    "category": "marketing",
    "inAppEnabled": false,
    "emailEnabled": false,
    "isMandatory": false
  }
]
```

14.25 Update Notification Preferences

Endpoint

PATCH /notifications/preferences

Payload

```
{
  "preferences": [
    {
      "category": "rfq",
      "emailEnabled": true,
      "inAppEnabled": true
    },

    {
      "category": "marketing",
      "emailEnabled": false,
      "inAppEnabled": false
    }
  ]
}
```

```
]
}
```

14.26 Update Preference Response

```
{
  "success": true,
  "message": "Notification preferences updated successfully."
}
```

Toast

Notification preferences updated successfully.

14.27 Mandatory Preference Validation

If frontend tries:

```
{
  "category": "payment",
  "emailEnabled": false
}
```

where Payment notifications are mandatory, backend should either ignore the restricted value or reject it.

Recommended response:

```
{
  "success": false,
  "code": "MANDATORY_NOTIFICATION_CHANNEL",
  "message": "Email notifications for critical payment activity cannot be disabled."
}
```

14.28 In-App Notification Grouping

Frontend may group Notifications by date similarly to Message Center.

Recommended:

Today

Quotation received
Order shipped

Yesterday

Payment confirmed

24 August 2026

Subscription activated

Backend continues returning:

createdAt

Frontend performs date grouping using the user's local timezone.

14.29 Notification Grouping by Category

Frontend may optionally provide filters:

All
Orders
RFQs
Payments
Messages
Subscriptions

This is based on:

category

No additional backend structure is required.

14.30 Real-Time In-App Notifications

Recommended:

WebSocket / Socket.IO

When a Notification is created:

Notification stored



notification:new



Connected recipient UI updates

REST remains the source of truth.

14.31 Recommended Real-Time Events

notification:new

notification:read

notification:read_all

notification:deleted

14.32 Authentication Notifications

Examples include:

EMAIL_VERIFIED

PASSWORD_CHANGED

ACCOUNT_DISABLED

ACCOUNT_REACTIVATED

Example in-app:

Your email has been verified successfully.

Critical authentication emails remain owned by Authentication/User modules.

14.33 Seller Verification Notifications

Examples:

SELLER_APPLICATION_APPROVED
SELLER_APPLICATION_REJECTED
SELLER_VERIFICATION_UPDATED

Example:

Your seller application has been approved.

Action:

seller dashboard/profile

14.34 Product Notifications

Examples:

PRODUCT_APPROVED
PRODUCT_REJECTED
PRODUCT_INACTIVE

Example:

Your product "Premium Cashew Nuts" has been approved and is now active.

Action:

product

Product Management remains responsible for Product status.

14.35 Direct RFQ Notifications

Reuse events already established in Direct RFQ.

Examples:

DIRECT_RFQ_CREATED
DIRECT_RFQ_QUOTE_CREATED
DIRECT_RFQ_COUNTER_OFFER_CREATED

DIRECT_RFQ_QUOTE_ACCEPTED
DIRECT_RFQ_QUOTE_REJECTED
DIRECT_RFQ_CANCELLED

Example seller Notification:

New quotation request

You received a Direct RFQ for Premium Cashew Nuts.

Example buyer Notification:

Quotation received

Agro Supplies Ltd submitted a quotation for your RFQ.

14.36 Market RFQ Notifications

Examples:

MARKET_RFQ_QUOTE_CREATED
MARKET_RFQ_COUNTER_OFFER_CREATED
MARKET_RFQ_AWARDED
MARKET_RFQ_QUOTE_CLOSED
MARKET_RFQ_CANCELLED

Example:

New quotation received

A seller has responded to your Market RFQ for Grade A Cocoa Beans.

14.37 Payment Notifications

Examples:

PAYMENT_CREATED
PAYMENT_PROOF_UPLOADED
PAYMENT_CONFIRMED

PAYMENT_FAILED
PAYMENT_EXPIRED
PAYMENT_CANCELLED

Example:

Payment confirmed

Your payment of NGN 1,650,000 has been confirmed.

Payment module remains authoritative.

14.38 Order Notifications

Reuse existing Order events:

ORDERS_CREATED
ORDER_PROCESSING
ORDER_READY_FOR_SHIPMENT
BUYER_LOGISTICS_DETAILS_SUBMITTED
ORDER_SHIPPED
ORDER_DELIVERED
ORDER_RECEIVED
ORDER_COMPLETED
ORDER_CANCELLED

Example:

Order ready for shipment

Your order with Agro Supplies Ltd is ready for delivery.

14.39 Buyer-Arranged Logistics Notification

When:

BUYER_LOGISTICS_DETAILS_SUBMITTED

notify seller:

Buyer logistics details received

The buyer has provided pickup details for Order TOFA-ORD-20260827-000101.

Action:

order

This notification does not duplicate the Order's business logic.

14.40 Logistics Notifications

Integrated Logistics events may include:

SHIPMENT_CREATED
SHIPMENT_PICKED_UP
SHIPMENT_OUT_FOR_DELIVERY
SHIPMENT_DELIVERED
SHIPMENT_DELIVERY_FAILED

Example:

Your order is out for delivery.

These events originate from the Logistics module.

14.41 Subscription Notifications

Examples:

SUBSCRIPTION_ACTIVATED
SUBSCRIPTION_CHANGED
SUBSCRIPTION_EXPIRING
SUBSCRIPTION_EXPIRED
SUBSCRIPTION_RENEWED

Example:

Subscription activated

Your Growth Seller subscription is now active.

14.42 Review Notifications

Examples:

PRODUCT_REVIEW_PUBLISHED
SELLER_REVIEW_PUBLISHED
REVIEW_RESPONSE_CREATED

Example Seller Notification:

New review received

A buyer left a review for Premium Cashew Nuts.

14.43 Rewards Notifications

Examples:

REVIEW_POINTS_AWARDED
POINTS_ADMIN_ADJUSTED
POINTS_REVERSED

Example:

You earned 10 TOFA Points

Thanks for reviewing Premium Cashew Nuts.

14.44 Message Center Notification

Message Center already manages:

unread Message counts
real-time chat

The Notification module should avoid duplicating every Message as a full Notification record if this causes unnecessary volume.

Recommended:

Message Center

→ manages Message unread count

Notifications

→ optional "New message" alert

For initial implementation, it is acceptable to create a simple Notification for a new Message, but do not create duplicate email behavior.

Example:

New message from Agro Supplies Ltd

Action:

message conversation

14.45 Message Email Rule

Message Center already defines:

unread/offline email

Notification module must not send another email for the same new Message.

Recommended:

Message Center owns Message email rules.

Notification Service may create in-app notification only.

14.46 Notification Deduplication

The system should prevent duplicate Notifications for the same business event.

Recommended optional fields:

eventId
deduplicationKey

Example:

ORDER_SHIPPED:order_101

If the same backend event is processed twice:

create one Notification

not two.

14.47 Recommended Updated Notification Structure

Add:

Field	Type	Required
eventId	String/null	No
deduplicationKey	String/null	No

Example:

```
{  
  "type": "ORDER_SHIPPED",  
  "deduplicationKey": "ORDER_SHIPPED:order_101"  
}
```

14.48 Notification Idempotency

Example:

Order Service emits ORDER_SHIPPED

Network/Event retry

ORDER_SHIPPED emitted again

Notification Service should detect:

ORDER_SHIPPED:order_101

already exists for that recipient.

No duplicate Notification should be created.

14.49 Notification Expiry

Some Notifications may become old but do not necessarily need to be deleted.

Recommended:

- maintain Notification history;
- paginate older records;
- optionally archive very old Notifications according to retention policy.

No automatic deletion is required initially.

14.50 Admin System Announcements

Admin may need to notify marketplace users about:

- maintenance;
- important platform changes;
- service interruption;
- new features;
- Marketplace notices.

This is different from automated business notifications.

14.51 SystemAnnouncement Structure

SystemAnnouncement

Field	Type	Required
id	UUID/String	Yes
title	String/JSON	Yes
message	String/JSON	Yes
audience	Enum	Yes

actionUrl	String/null	No
sendInApp	Boolean	Yes
sendEmail	Boolean	Yes
status	Enum	Yes
scheduledAt	Timestamp/null	No
sentAt	Timestamp/null	No
createdBy	UUID/String	Yes
createdAt	Timestamp	Yes
updatedAt	Timestamp	Yes

14.52 Announcement Audience

Recommended:

all
buyers
sellers
admins
specific_users

Future audience filtering may include:

country
subscription_plan
seller_category

14.53 Announcement Status

Recommended:

draft
scheduled
sent
cancelled

14.54 Admin Create Announcement

Endpoint

POST /admin/notifications/announcements

Payload

```
{
  "title": {
    "en": "Scheduled Marketplace Maintenance"
  },

  "message": {
    "en": "TOFA Marketplace will undergo scheduled maintenance on Saturday."
  },

  "audience": "all",

  "sendInApp": true,
  "sendEmail": true,

  "actionUrl": null,

  "scheduledAt": "2026-08-29T22:00:00Z"
}
```

14.55 Create Announcement Response

```
{
  "success": true,
  "message": "Announcement created successfully.",

  "data": {
    "announcementId": "announcement_123",
    "status": "scheduled"
  }
}
```

Admin Toast

Announcement created successfully.

14.56 Specific User Announcement

Example:

```
{
  "title": {
    "en": "Account Update"
  },

  "message": {
    "en": "Please review your seller verification information."
  },

  "audience": "specific_users",

  "userIds": [
    "user_123",
    "user_456"
  ],

  "sendInApp": true,
  "sendEmail": false
}
```

14.57 Admin Get Announcements

Endpoint

GET /admin/notifications/announcements

Query

status
audience
dateFrom
dateTo
search
page
limit

Response

```
{
  "success": true,

  "data": [
    {
      "announcementId": "announcement_123",

      "title": {
        "en": "Scheduled Marketplace Maintenance"
      },

      "audience": "all",
      "status": "scheduled",

      "sendInApp": true,
      "sendEmail": true,

      "scheduledAt": "2026-08-29T22:00:00Z",

      "createdAt": "2026-08-27T10:00:00Z"
    }
  ],

  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 5,
    "totalPages": 1
  }
}
```

14.58 Cancel Scheduled Announcement

Endpoint

POST /admin/notifications/announcements/:announcementId/cancel

Response

```
{
  "success": true,
```

```
"message": "Scheduled announcement cancelled successfully."
}
```

A sent Announcement cannot be unsent.

14.59 Admin Get Notification Statistics

Optional but useful.

Endpoint

GET /admin/notifications/stats

Response

```
{
  "success": true,

  "data": {
    "totalNotifications": 125000,
    "unreadNotifications": 9200,

    "today": {
      "created": 2100,
      "read": 1650
    },

    "byCategory": {
      "order": 45000,
      "rfq": 30000,
      "payment": 15000,
      "message": 20000,
      "other": 15000
    }
  }
}
```

14.60 Admin Notification Permissions

Use the existing Admin permission system.

Possible permissions:

notifications.view
notifications.manage_announcements
notifications.view_stats
notifications.manage_settings

Do not create new Admin roles inside Notification Management.

14.61 Internationalization

The Notification module should follow the existing user:

`selectedLanguage`

Default:

English

In-app Notification content should be generated/displayed in the user's preferred language where translation exists.

Example user:

`selectedLanguage = fr`

Notification:

Votre commande a été expédiée.

rather than:

Your order has been shipped.

The exact translation architecture remains governed by the Internationalization module.

14.62 Notification Translation Storage

Where templates are stored, use multilingual values.

Example:

```
{
  "title": {
    "en": "Your order has been shipped",
    "fr": "Votre commande a été expédiée"
  },

  "message": {
    "en": "{SellerName} has shipped your order.",
    "fr": "{SellerName} a expédié votre commande."
  }
}
```

The Notification Service selects the appropriate version using the user's `selectedLanguage`.

Fallback:

selected language unavailable
→ English

14.63 Notification Preferences and Internationalization

Preferences are not language-specific.

Example:

`emailEnabled = false`

applies regardless of whether User language is:

English
French
Arabic
Portuguese

14.64 Frontend Notification UI

Recommended header:



with unread badge.

Click opens:

Notifications

Example:

Notifications

Today

Your order has been shipped
Agro Supplies Ltd has shipped your order.
10:45 AM

Quotation received
You received a quotation for Premium Cashew Nuts.
9:20 AM

Yesterday

Payment confirmed
Your payment has been confirmed.
4:15 PM

14.65 Read/Unread UI

Unread Notification:

highlighted background
unread dot

Read Notification:

normal background

Frontend presentation may vary.

14.66 Empty Notifications State

No notifications yet.

Important marketplace updates will appear here.

14.67 Notification Settings UI

Recommended:

Notification Preferences

RFQs

In-App ☒ [On]

Email ☒ [On]

Orders

In-App ☒ [On]

Email ☒ [On]

Subscriptions

In-App ☒ [On]

Email ☒ [On]

Messages

In-App ☒ [On]

Email ☒ [On]

Marketing

In-App ☐ [Off]

Email ☐ [Off]

Mandatory controls should appear disabled with appropriate explanation.

Example:

Payment Notifications

Email ☒ [On - Required]

14.68 Notification Security

Backend must:

- ensure Notification belongs to authenticated User;
- prevent one User reading another User's Notification;
- validate actions;
- protect Admin announcement permissions;
- prevent spoofed business Notifications from frontend;
- ensure critical Notification events originate from trusted backend modules.

Frontend must never send:

Create PAYMENT_CONFIRMED notification

as a business event.

14.69 Backend Implementation Guide

Notification Service owns:

- Notification persistence;
- unread counts;
- read state;
- user Notification preferences;
- in-app delivery;
- real-time Notification events;
- email-channel orchestration;
- Notification deduplication;
- Admin announcements.

Business modules own:

- business event;
- recipient determination where business-specific;
- business email template/content;
- underlying entity state.

Example:

Order Service

determines:

Buyer needs notification

Notification Service
determines:
How to store/send it

14.70 Frontend Implementation Guide

Frontend should:

- fetch Notification list;
- fetch unread count;
- receive real-time notifications;
- mark Notifications read;
- navigate using `actionUrl`;
- group Notifications by date;
- allow category filters;
- allow preference management.

Frontend must not determine whether a business event occurred.

Example:

Frontend must not decide:
Order appears shipped
→ generate notification

Backend Order event does that.

14.71 Internal Events Consumed

Notification Service may consume existing events including:

SELLER_APPLICATION_APPROVED
SELLER_APPLICATION_REJECTED

PRODUCT_APPROVED
PRODUCT_REJECTED

DIRECT_RFQ_CREATED
DIRECT_RFQ_QUOTE_CREATED
DIRECT_RFQ_COUNTER_OFFER_CREATED
DIRECT_RFQ_QUOTE_ACCEPTED
DIRECT_RFQ_QUOTE_REJECTED

DIRECT_RFQ_CANCELLED

MARKET_RFQ_QUOTE_CREATED
MARKET_RFQ_COUNTER_OFFER_CREATED
MARKET_RFQ_AWARDED
MARKET_RFQ_QUOTE_CLOSED
MARKET_RFQ_CANCELLED

PAYMENT_CONFIRMED
PAYMENT_FAILED
PAYMENT_EXPIRED

ORDERS_CREATED
ORDER_PROCESSING
ORDER_READY_FOR_SHIPMENT
BUYER_LOGISTICS_DETAILS_SUBMITTED
ORDER_SHIPPED
ORDER_DELIVERED
ORDER_RECEIVED
ORDER_COMPLETED

SHIPMENT_CREATED
SHIPMENT_PICKED_UP
SHIPMENT_OUT_FOR_DELIVERY
SHIPMENT_DELIVERY_FAILED

SUBSCRIPTION_ACTIVATED
SUBSCRIPTION_CHANGED
SUBSCRIPTION_EXPIRED

PRODUCT_REVIEW_PUBLISHED
SELLER_REVIEW_PUBLISHED
REVIEW_POINTS_AWARDED

MESSAGE_SENT

The exact events already owned by previous modules should be reused instead of creating new parallel events.

14.72 Internal Notification Events

Recommended events owned by Notification module:

NOTIFICATION_CREATED
NOTIFICATION_READ
NOTIFICATIONS_READ_ALL

ANNOUNCEMENT_CREATED
ANNOUNCEMENT_SCHEDULED
ANNOUNCEMENT_SENT
ANNOUNCEMENT_CANCELLED

NOTIFICATION_PREFERENCES_UPDATED

14.73 Notification Deduplication Guide

Recommended:

deduplicationKey =
eventType + recipientId + entityId

Example:

ORDER_SHIPPED:user_123:order_101

If processed again:

existing notification found
→ do not duplicate

For Messages, deduplication may use:

NEW_MESSAGE:user_123:message_456

14.74 Email Failure Rule

An email failure must not undo the business transaction.

Example:

Order shipped successfully



Notification stored

↓
Email provider fails

The Order remains:

shipped

Backend may retry email separately.

Notification delivery failure must not roll back:

- Payment;
- Order;
- RFQ;
- Subscription;
- Product status.

14.75 In-App Delivery Failure Rule

If User is offline:

Notification remains in database

When User returns:

GET /notifications

retrieves it.

Real-time delivery is an enhancement, not the persistence source of truth.

14.76 Notification and Message Center Relationship

Message Center owns:

Conversation
Messages
Message unread counts
Message email delay

Notification module may provide:

general in-app alert:

"New message from Agro Supplies Ltd"

but should not replace Message Center unread counts.

Recommended frontend:

Bell:

General notifications

Messages:

Message unread count

This avoids mixing chat unread state with general marketplace notifications.

14.77 Notification and Email Documentation Relationship

A separate Email Notification Templates document can contain the complete email copy for all business events.

This Notification module should define:

when

recipient

channel

preference rules

while the Email Templates document defines the final detailed email content.

Existing module-specific email templates remain the source material for that consolidated document.

14.78 Module Summary

The Notifications Management module establishes that:

- Notifications are centralized;
- underlying business actions remain owned by their respective modules;
- initial delivery channels are in-app and email;
- business modules retain ownership of their email content;

- Notification Service handles persistence and channel delivery;
- each Notification has a specific type and broader category;
- Notifications can link users back to related marketplace entities;
- Users can view Notification history;
- Users can mark one or all Notifications as read;
- unread Notification count is supported;
- Notifications are displayed newest first;
- frontend may group Notifications by Today, Yesterday and older dates;
- user local timezone is used for date grouping;
- user preferences can control optional channels;
- critical security/Payment notifications may be mandatory;
- Message Center unread logic remains separate from Notification unread logic;
- duplicate Notifications are prevented through idempotency/deduplication keys;
- email delivery failure does not roll back a business transaction;
- real-time Notification delivery is supported but database persistence remains authoritative;
- Admin can create system announcements;
- announcements may target all Users, Buyers, Sellers or specific Users;
- Admin access uses the existing permission system;
- Notification content follows the user's selected language where translations exist;
- English remains the fallback language;
- Notification Service reuses events already defined across Product, RFQ, Payment, Order, Logistics, Subscription, Reviews and Message Center modules rather than creating conflicting business flows.

MODULE 15.

INTERNATIONALIZATION & LOCALIZATION

15. INTERNATIONALIZATION & LOCALIZATION MANAGEMENT MODULE

15.1 Overview

The Internationalization & Localization Management module controls language preferences, multilingual marketplace content, translation storage, language fallback, locale-aware API responses, and frontend language behaviour across TOFA Marketplace.

The default marketplace language is:

English

A user may change their preferred language.

Once changed:

User selects language



Frontend updates immediately



Backend saves selectedLanguage



Future authenticated requests



Backend returns supported content
in the user's preferred language

The selected language must therefore not exist only in browser local storage.

It should also be stored against the User in the database.

The existing User field:

selectedLanguage

remains the source of truth for an authenticated user's preferred language.

The module applies to:

- User-facing marketplace content;
- Products;
- Categories;
- Seller/business descriptions;
- Direct RFQs;
- Market RFQs;
- Notifications;
- Subscription Plans;
- Admin announcements;
- static frontend interface text;
- email templates where multilingual versions exist;
- future marketplace content requiring translation.

The module does not change the underlying business logic of any existing module.

15.2 Core Language Principle

TOFA Marketplace should distinguish between:

Interface Language

and:

Marketplace Content Language

Interface Language

Frontend labels such as:

Add to Cart
Request Quote
Checkout
Messages
Orders
Notifications

are translated using frontend translation resources.

Marketplace Content Language

Dynamic backend content such as:

Product Name
Product Description
Category Name
Company Bio
Subscription Description
Notification Text
RFQ Requirement

is stored with multilingual values and returned according to the user's preferred language.

15.3 Initial Supported Languages

Recommended initial core marketplace languages:

en = English
fr = French
ar = Arabic
sw = Swahili
pt = Portuguese

English remains:

defaultLanguage = en

The architecture must remain dynamic so that TOFA can add more languages later without redesigning Product, Category, RFQ or Notification tables.

Possible future languages may include other regional languages where business requirements justify them.

15.4 Language Structure

Language

Field	Type	Required	Description
id	UUID/String	Yes	Language record
code	String	Yes	ISO-style language code

name	String	Yes	English language name
nativeName	String	Yes	Native language name
direction	Enum	Yes	ltr/rtl
isDefault	Boolean	Yes	Platform default
status	Enum	Yes	active/inactive
sortOrder	Integer	Yes	Display order
createdAt	Timestamp	Yes	Created
updatedAt	Timestamp	Yes	Updated

Example:

```
{
  "code": "fr",
  "name": "French",
  "nativeName": "Français",
  "direction": "ltr",
  "isDefault": false,
  "status": "active"
}
```

Arabic:

```
{
  "code": "ar",
  "name": "Arabic",
  "nativeName": "العربية",
  "direction": "rtl",
  "isDefault": false,
  "status": "active"
}
```

15.5 Language Statuses

Recommended:

active
inactive

active

Language may be selected by users and returned by the platform.

inactive

Language can no longer be selected by new users.

Historical translated content may remain stored.

Deactivating a language must not delete its translations.

15.6 User Language Field

The existing User structure should retain:

selectedLanguage

Recommended storage:

en
fr
ar
sw
pt

rather than:

English
French
Arabic

This makes language handling consistent across systems.

Example:

```
{  
  "selectedLanguage": "fr"
```

```
}
```

15.7 Default User Language

For a newly created User:

`selectedLanguage = en`

unless the platform explicitly detects/asks for another language during onboarding.

Recommended initial rule:

New account

→ English

The user can change it later.

15.8 Update Preferred Language

Endpoint

PATCH /users/language

Payload

```
{  
  "language": "fr"  
}
```

Backend Processing

Backend must:

1. verify authenticated User;
2. verify language exists;
3. verify language is active;
4. update `User.selectedLanguage`;
5. return updated language information.

15.9 Update Language Response

```
{
  "success": true,
  "message": "Language preference updated successfully.",

  "data": {
    "selectedLanguage": "fr",

    "language": {
      "code": "fr",
      "name": "French",
      "nativeName": "Français",
      "direction": "ltr"
    }
  }
}
```

Toast

The toast should immediately use the newly selected language.

English example:

Language preference updated successfully.

French version should be shown after switching to French.

15.10 Get Supported Languages

Endpoint

GET /languages

Authentication should not be required.

Response

```
{
  "success": true,

  "data": [
```



```

{
  "code": "en",
  "name": "English",
  "nativeName": "English",
  "direction": "ltr",
  "isDefault": true
},

{
  "code": "fr",
  "name": "French",
  "nativeName": "Français",
  "direction": "ltr",
  "isDefault": false
},

{
  "code": "ar",
  "name": "Arabic",
  "nativeName": "العربية",
  "direction": "rtl",
  "isDefault": false
},

{
  "code": "sw",
  "name": "Swahili",
  "nativeName": "Kiswahili",
  "direction": "ltr",
  "isDefault": false
},

{
  "code": "pt",
  "name": "Portuguese",
  "nativeName": "Português",
  "direction": "ltr",
  "isDefault": false
}
]
}

```

Only active languages should be returned.

15.11 Language Resolution Priority

For authenticated requests, backend should resolve language using the following order:

1. Explicit request language where supported
2. User.selectedLanguage
3. Platform default language
4. English fallback

Recommended explicit request language:

Accept-Language: fr

or:

?lang=fr

For normal authenticated marketplace usage:

User.selectedLanguage

should usually determine the returned language.

15.12 Guest Language Behaviour

Unauthenticated users do not yet have:

User.selectedLanguage

Recommended flow:

Guest



Frontend selected language



localStorage/cookie



Accept-Language header

If no language preference exists:

English

is used.

When the guest later creates an account, the selected frontend language may be sent during signup or synchronized after signup.

The existing authentication flow does not need to be redesigned for this.

15.13 Multilingual Backend Storage Principle

Dynamic translatable fields should be stored in multiple languages.

Example Product:

```
{
  "productName": {
    "en": "Premium Cashew Nuts",
    "fr": "Noix de cajou premium",
    "ar": "كاجو ممتاز",
    "sw": "Koroshu Bora",
    "pt": "Castanhas de Caju Premium"
  }
}
```

The same structure may apply to:

productDescription
companyBio
categoryName
categoryDescription
subscriptionPlanDescription
announcementTitle
announcementMessage
RFQ requirement content

15.14 Translation JSON Structure

Recommended generic structure:

```
{
  "en": "English value",
  "fr": "French value",
  "ar": "Arabic value",
  "sw": "Swahili value",
  "pt": "Portuguese value"
}
```

Do not store translated values as unrelated columns such as:

```
productNameEnglish
productNameFrench
productNameArabic
```

The JSON/translation structure is easier to extend when more languages are added.

15.15 Original Content Language

When a seller creates content, the backend should know which language the seller originally entered.

Recommended field:

```
sourceLanguage
```

Example:

```
{
  "productName": "Premium Cashew Nuts",
  "productDescription": "Export-grade Nigerian cashew nuts.",
  "sourceLanguage": "en"
}
```

The backend then creates/stores translated versions.

15.16 Product Creation Translation Flow

This modifies only the language-handling portion of the existing Product Management module.

The Product creation business endpoint remains:

POST /seller/products

The seller does not need to manually enter five translations.

Example seller payload:

```
{
  "productName": "Premium Cashew Nuts",
  "productDescription": "Export-grade Nigerian cashew nuts.",
  "sourceLanguage": "en"
}
```

Backend:

```
graph TD
    A[Seller creates Product] --> B[Store original English]
    B --> C[Translation Service]
    C --> D[Generate configured translations]
    D --> E[Store multilingual Product fields]
```

Final stored structure:

```
{
  "productName": {
    "en": "Premium Cashew Nuts",
    "fr": "Noix de cajou premium",
    "ar": "كاجو ممتاز",
    "sw": "Koroshu Bora",
    "pt": "Castanhas de Caju Premium"
  }
}
```

15.17 Important Product Flow Rule

Internationalization must not change the existing Product workflow.

Example:

Seller creates Product



Product status = draft



Translation generated



Product still follows existing Product moderation/status rules

Translation completion does not automatically activate a Product.

Product Management remains responsible for Product status.

15.18 Product Edit Translation Flow

When a seller changes a translatable field:

Product name changed

or

Product description changed

backend should regenerate translations for that field.

Example:

Original:

Premium Cashew Nuts

Updated:

Premium Organic Cashew Nuts

Backend:

Update source value



Mark other translations stale



Regenerate translations



Store new multilingual values

Existing Product rules still apply regarding Product status after editing.

15.19 Translation Status

Where asynchronous or automated translation is used, it is useful to track translation status.

Recommended:

pending
completed
failed

pending

Translation is waiting to be processed.

completed

Configured translations are available.

failed

One or more translations could not be generated.

A translation failure should not delete the original content.

15.20 Translation Metadata Structure

Recommended optional structure:

TranslationMetadata

Field	Type	Required
id	UUID/String	Yes
entityType	String	Yes
entityId	UUID/String	Yes
fieldName	String	Yes
sourceLanguage	String	Yes
status	Enum	Yes

lastTranslatedAt	Timestamp/null	No
failedLanguages	Array	No
createdAt	Timestamp	Yes
updatedAt	Timestamp	Yes

This is useful if translation is handled by an external translation service.

15.21 Translation Failure Rule

Suppose:

English = available

French = available

Arabic = translation failed

Arabic User should not receive:

null

Recommended fallback:

Arabic missing

↓

English

The user should always receive meaningful content where the original English/source content exists.

15.22 Fallback Rule

Recommended universal fallback:

Requested Language

↓

If available → return it

↓

If unavailable → English

↓
If English unavailable → sourceLanguage

Example:

selectedLanguage = sw

Swahili translation unavailable

→ English value returned

15.23 Normal User API Response Principle

Although multilingual values are stored in the database, normal marketplace APIs should preferably return content already localized for the user.

Example stored:

```
{
  "productName": {
    "en": "Premium Cashew Nuts",
    "fr": "Noix de cajou premium"
  }
}
```

French user:

GET /products/prod_101

receives:

```
{
  "productName": "Noix de cajou premium"
}
```

English user receives:

```
{
  "productName": "Premium Cashew Nuts"
}
```

This keeps frontend usage simple.

15.24 Why Normal APIs Should Return Localized Strings

The frontend should not repeatedly perform:

```
product.productName[selectedLanguage]
```

for every field across every module.

Instead:

Backend resolves selected language

↓

Frontend receives display-ready content

This gives consistent fallback behaviour and reduces duplicated frontend logic.

15.25 Admin/Seller Edit Response Principle

Admin and Seller management/edit screens may require all translations.

For those endpoints, backend may return:

```
{
  "productName": {
    "en": "Premium Cashew Nuts",
    "fr": "Noix de cajou premium",
    "ar": "كاجو ممتاز",
    "sw": "Koroshu Bora",
    "pt": "Castanhas de Caju Premium"
  }
}
```

Recommended query:

```
includeTranslations=true
```

Example:

GET /seller/products/prod_101?includeTranslations=true

This should be limited to users who genuinely need multilingual editing/inspection.

15.26 Product Listing Example

French User:

Endpoint

GET /products

Response

```
{
  "success": true,

  "data": [
    {
      "id": "prod_101",

      "productName": "Noix de cajou premium",

      "categories": [
        {
          "id": "cat_agriculture",
          "name": "Agriculture"
        }
      ],

      "seller": {
        "id": "seller_456",
        "storeName": "Agro Supplies Ltd"
      },

      "currency": "USD",
      "price": 20,
      "averageRating": 4.6,
      "totalReviews": 120,

      "mainImage": "https://cdn.tofa.com/products/cashew.jpg",

      "countryOfOrigin": "Nigeria"
    }
  ]
}
```

```
    }  
  ],  
  
  "language": "fr"  
}
```

15.27 Product Details Example

Endpoint

GET /products/:productId

For French User:

```
{  
  "success": true,  
  
  "data": {  
    "id": "prod_101",  
  
    "productName": "Noix de cajou premium",  
  
    "description": "Noix de cajou nigérianes de qualité export.",  
  
    "categories": [  
      {  
        "id": "cat_agriculture",  
        "name": "Agriculture"  
      }  
    ],  
  
    "countryOfOrigin": "Nigeria",  
  
    "seller": {  
      "id": "seller_456",  
      "storeName": "Agro Supplies Ltd"  
    }  
  },  
  
  "language": "fr"  
}
```

Variants such as:

SKU
quantity
price
stock

do not require translation.

Variant option labels may require translation where they are displayed as language-specific descriptors.

15.28 Category Translation

The existing Category module remains responsible for Category creation.

Admin may enter Category name in English or the configured source language.

Stored example:

```
{  
  "name": {  
    "en": "Construction Materials",  
    "fr": "Matériaux de construction",  
    "ar": "مواد البناء",  
    "sw": "Vifaa vya Ujenzi",  
    "pt": "Materiais de Construção"  
  }  
}
```

Normal User API returns only the resolved language.

Admin Category API may return all translations.

15.29 Seller Business Translation

Translatable Seller fields may include:

companyBio
storeDescription

Do not translate structured information such as:

companyName
registrationNumber
phoneNumber
email
address identifiers

unless there is a specific business need.

Company legal names should generally remain exactly as registered.

15.30 RFQ Translation Principle

RFQ content written by a buyer may need to be understood by sellers using another language.

Fields potentially translated:

requirementTitle
description
buyerNotes

Structured commercial data should not be translated:

quantity
unit code
currency
price
dates
deliveryType

Direct RFQ and Market RFQ business flows remain unchanged.

15.31 Direct RFQ Translation Example

Buyer writes in French:

Nous avons besoin de 500 kg de noix de cajou de qualité export.

Store:

```
{  
  "description": {
```

```
"fr": "Nous avons besoin de 500 kg de noix de cajou de qualité export.",  
"en": "We require 500 kg of export-quality cashew nuts.",  
"ar": "...",  
"sw": "...",  
"pt": "..."  
},  
  
"sourceLanguage": "fr"  
}
```

Seller with:

selectedLanguage = en

receives the English version.

15.32 RFQ Commercial Terms Must Not Be Altered by Translation

Translation applies to explanatory text only.

It must never modify:

500kg
USD 20
USD 10,000
30 September 2026
seller_arranged

These are structured fields.

The backend must not attempt to infer or rewrite commercial amounts through translation.

15.33 Quote Message Translation

Seller Quote fields such as:

message
notes

may be translated.

Commercial fields remain structured.

Example:

pricePerUnit
currency
quantity
logisticsAmount
quoteTotal
validUntil

are not translation content.

15.34 Message Center Translation

The existing simplified Message Center stores user Messages exactly as written.

Initial implementation:

Messages are NOT automatically translated.

Interface labels follow the user's selected language.

Future automatic chat translation may be added later.

If added:

Original Message
+
Generated translation

must both be preserved.

The original Message must never be overwritten.

15.35 Notification Translation

The existing Notifications module already requires Notifications to follow:

User.selectedLanguage

This module defines the underlying rule.

Example notification template:

```
{
  "title": {
    "en": "Your order has been shipped",
    "fr": "Votre commande a été expédiée",
    "ar": "تم شحن طلبك",
    "sw": "Agizo lako limesafirishwa",
    "pt": "O seu pedido foi enviado"
  }
}
```

Notification Service chooses the User's language.

15.36 Email Translation

Where an email template has multiple supported languages:

Recipient.selectedLanguage



Email Template Resolver



Send preferred language

If translation is unavailable:

English email

is used.

Existing business modules still own when an email is triggered.

This module only defines how the language is selected.

15.37 Subscription Plan Translation

Subscription Plan fields such as:

name
description
badgeText
benefitDisplayName

may support translations.

The entitlement codes themselves remain language-independent.

Example:

max_products
product_priority
view_buyer_info

should never change by language.

Frontend labels can translate these codes into the user's preferred language.

15.38 Review Translation

User Review content should initially remain in the original language submitted.

Do not automatically rewrite the buyer's review.

The interface:

Reviews
Helpful
Reply
Verified Purchase

uses the user's preferred language.

Automatic Review translation may be introduced later using the same:

original + translated

principle.

15.39 Order Content Translation

Order commercial snapshots should preserve the Product's relevant display values at Order creation.

The commercial Order itself must not depend on future translation changes.

Recommended OrderItem snapshot may preserve:

productNameSnapshot

in the transaction's commercial/source language.

For display, the frontend/backend may additionally resolve the Product's translated display name where appropriate.

Historical commercial meaning must remain intact.

15.40 Country and Currency Localization

Country and currency values should use stable codes internally.

Example:

NG
GH
KE

NGN
USD
GHS
KES

Frontend may display translated labels.

Example:

NG
→ Nigeria

FR UI:
→ Nigéria

Currency codes remain:

USD
NGN
GHS

and should not be translated.

15.41 Units

Product and RFQ quantity units should preferably use stable codes.

Example:

kg
tonne
piece
carton
litre

Frontend may translate human-readable labels.

Example:

piece

English:
Pieces

French:
Pièces

The stored unit identifier remains unchanged.

15.42 Date Localization

Backend should continue returning dates as ISO timestamps.

Example:

2026-08-27T10:00:00Z

Frontend handles user-friendly date display.

English:

27 August 2026

French:

27 août 2026

Arabic and other supported locales should follow their appropriate locale display.

Business timestamps in the database must not be stored as localized text.

15.43 Number Formatting

Backend monetary amounts remain numbers.

Example:

```
{  
  "amount": 1250000.50,  
  "currency": "NGN"  
}
```

Frontend applies locale-aware formatting.

Do not store:

~~"N~~1,250,000.50"

as the authoritative amount.

15.44 RTL Support

Arabic requires:

direction = rtl

When User selects Arabic:

Frontend should change document/layout direction appropriately.

Example:

```
dir="rtl"
```

This applies to:

- navigation;
- forms;
- labels;
- cards;
- text alignment;
- notification UI;
- dashboard layouts where appropriate.

Product images and numeric business information should not be mirrored unnecessarily.

15.45 Frontend Language Initialization

On authenticated app load:

Application starts



GET current User



selectedLanguage



Initialize frontend i18n

Example:

```
{  
  "selectedLanguage": "fr"  
}
```

Frontend initializes:

fr

15.46 Guest Frontend Initialization

For guest:

Check stored guest language



If found

→ use it

Otherwise

→ English

Do not automatically force browser-detected language if TOFA does not support it.

Example:

Browser = German

Supported languages do not include German

→ English

15.47 Language Switcher

Recommended frontend:

Language

English

Français

العربية

Kiswahili

Português

When authenticated User selects a language:

PATCH /users/language



update frontend language



refetch/revalidate localized dynamic content

15.48 Immediate Language Change Behaviour

The User should not need to:

logout
login again

after changing language.

Recommended:

User selects French



Save preference



Frontend labels switch



Queries containing translated content refetch



French backend content appears

15.49 Frontend Cache Rule

If frontend uses caching tools such as TanStack Query, language should form part of localized query caching.

Conceptually:

["products", filters, selectedLanguage]

instead of only:

["products", filters]

Otherwise:

English Product response

may remain incorrectly cached after switching to French.

15.50 Backend Cache Rule

Backend/CDN caches for localized resources must also vary by language.

Example cache identity:

/products?page=1 + language=fr

must not return an English cached response.

Where **Accept-Language** is used, caching infrastructure must account for it.

15.51 Translation Service Architecture

Recommended:

Entity Created/Updated



Translation Service



Translation Provider



Normalize Result



Store Translations

The Product, Category, RFQ and other modules should not independently integrate with different translation providers.

Use one Translation Service abstraction.

15.52 Translation Service Responsibilities

Translation Service owns:

- source-language handling;
- supported target languages;
- translation provider communication;
- retry logic;
- translation status;
- fallback;
- storage normalization.

It does not own:

- Product status;

- Category status;
- RFQ status;
- Order status;
- Notification business event.

15.53 Translation Provider Independence

Other modules should call:

TranslationService.translate()

rather than a specific third-party provider directly.

This allows TOFA to change translation providers later without redesigning marketplace modules.

15.54 Translation Request Example

Internal:

```
{
  "sourceLanguage": "en",

  "targetLanguages": [
    "fr",
    "ar",
    "sw",
    "pt"
  ],

  "fields": {
    "productName": "Premium Cashew Nuts",
    "productDescription": "Export-grade Nigerian cashew nuts."
  }
}
```

15.55 Translation Result Example

```
{
  "sourceLanguage": "en",

  "translations": {
```

```

"fr": {
  "productName": "Noix de cajou premium",
  "productDescription": "Noix de cajou nigérianes de qualité export."
},

"ar": {
  "productName": "كاجو ممتاز",
  "productDescription": "كاجو نيجيري بجودة تصديرية."
},

"sw": {
  "productName": "Korosho Bora",
  "productDescription": "Korosho za Nigeria za ubora wa kuuza nje."
},

"pt": {
  "productName": "Castanhas de Caju Premium",
  "productDescription": "Castanhas de caju nigerianas de qualidade para exportação."
}
}

```

15.56 Translation Should Be Field-Specific

Do not send unnecessary structured fields to translation.

Translate:

productName
productDescription
companyBio
requirementTitle
description
notes

Do not translate:

SKU
barcode
price
currency
stock

quantity
email
phoneNumber
trackingId
Order Reference
RFQ Reference
Payment Reference

15.57 Admin Language Management

Authorized Admin should be able to manage available platform languages.

Endpoint

GET /admin/languages

Response

```
{
  "success": true,

  "data": [
    {
      "id": "lang_en",
      "code": "en",
      "name": "English",
      "nativeName": "English",
      "direction": "ltr",
      "isDefault": true,
      "status": "active"
    },

    {
      "id": "lang_fr",
      "code": "fr",
      "name": "French",
      "nativeName": "Français",
      "direction": "ltr",
      "isDefault": false,
      "status": "active"
    }
  ]
}
```

15.58 Admin Add Language

Endpoint

POST /admin/languages

Payload

```
{  
  "code": "es",  
  "name": "Spanish",  
  "nativeName": "Español",  
  "direction": "ltr"  
}
```

Adding a language should not automatically assume every historical Product has been translated.

A translation backfill process may be required.

15.59 Add Language Response

```
{  
  "success": true,  
  "message": "Language added successfully.",  
  
  "data": {  
    "id": "lang_es",  
    "code": "es",  
    "status": "inactive"  
  }  
}
```

Recommended:

New language
→ inactive initially

until translation readiness has been reviewed.

15.60 Update Language

Endpoint

PATCH /admin/languages/:languageId

Payload

```
{
  "name": "French",
  "nativeName": "Français",
  "sortOrder": 2
}
```

15.61 Update Language Status

Endpoint

PATCH /admin/languages/:languageId/status

Payload

```
{
  "status": "active"
}
```

Before activation, backend/Admin should confirm:

- frontend interface resources exist;
- major system templates exist;
- translation service supports the language;
- fallback remains available.

15.62 Default Language Rule

There must always be exactly one platform default language.

Initially:

English

Admin should not be able to deactivate the current default language without first assigning another default.

15.63 Change Default Language

If TOFA ever decides to change the default:

Endpoint

PATCH /admin/languages/:languageId/default

Payload

```
{  
  "isDefault": true  
}
```

Backend removes default status from the previous language.

For the current marketplace specification:

English remains default.

15.64 Translation Backfill

When TOFA adds a new language:

Spanish added

existing multilingual entities may not contain Spanish.

Recommended:

New Language Created



Translation Backfill Job



Existing Products

Categories

Subscription Plans

System Templates

↓
Spanish translations generated

The language should preferably not become publicly active until important platform content is ready.

15.65 Admin Translation Monitoring

Useful Admin endpoint:

Endpoint

GET /admin/translations/status

Query

entityType
language
status
page
limit

Response

```
{
  "success": true,

  "data": {
    "summary": {
      "completed": 12400,
      "pending": 230,
      "failed": 18
    },

    "failedItems": [
      {
        "entityType": "product",
        "entityId": "prod_101",
        "fieldName": "productDescription",
        "language": "ar",
        "status": "failed"
      }
    ]
  }
}
```



```
]
}
}
```

15.66 Retry Translation

Endpoint

POST /admin/translations/:translationId/retry

Response

```
{
  "success": true,
  "message": "Translation queued for retry."
}
```

The retry should not modify the underlying Product/RFQ/Category status.

15.67 Manual Translation Override

Admin may need to correct an automated translation.

Endpoint

PATCH /admin/translations/:translationId

Payload

```
{
  "value": "Noix de cajou premium"
}
```

Manual correction should be auditable.

Recommended metadata:

translationSource = manual

instead of:

machine

15.68 Translation Source

Recommended:

original
machine
manual

original

Content submitted by User/Admin.

machine

Generated through Translation Service.

manual

Corrected or entered manually by authorized Admin/user workflow.

15.69 Protecting Manual Translations

If a translation has been manually corrected, automatic translation should not overwrite it without an explicit rule.

Example:

French Product translation
translationSource = manual

Seller edits English description.

Backend may:

- mark French translation as potentially stale;
- notify/queue review;
- avoid silently replacing manual French content.

For initial implementation, TOFA may choose to regenerate all translations, but manually corrected translations should ideally be protected.

15.70 Search Relationship

Search must work across supported languages.

Example French query:

noix de cajou

should be capable of matching a Product whose original language was English but whose French translation exists.

Search/Ranking module should therefore index supported translated fields.

The Internationalization module provides translated data.

Search module remains responsible for indexing and ranking.

15.71 Search Result Language

Search results should still return:

User.selectedLanguage

Example:

Search query = cashew
User language = French

Result name can still display:

Noix de cajou premium

where that is the localized Product name.

15.72 URLs and Slugs

Stable URLs should preferably not depend entirely on translated names.

Recommended:

/products/prod_101

or:

/products/premium-cashew-nuts-prod_101

Changing language should not create a different Product identity.

Localized SEO URLs may be considered later.

15.73 Admin Announcement Relationship

The existing Notification module already supports multilingual announcement content.

Example:

```
{
  "title": {
    "en": "Scheduled Maintenance",
    "fr": "Maintenance programmée"
  }
}
```

Notification Service selects the recipient's language.

No separate announcement translation flow should be created.

15.74 Subscription Relationship

Subscription entitlements remain stable codes.

Example:

```
max_products
max_images_per_product
view_buyer_info
product_priority
```

These are never translated in backend logic.

Only their display labels/descriptions are localized.

This avoids logic such as:

"Maximum Products"

being used as an entitlement identifier.

15.75 Message Center Relationship

Message Center UI uses selected language.

Examples:

Today
Yesterday
Type a message
Reply
Delete
Report

The actual user Message remains exactly as written.

No automatic Message translation is required initially.

15.76 Notification Relationship

Notifications use:

`selectedLanguage`

at creation/rendering time.

Notification type remains stable:

`ORDER_SHIPPED`

regardless of language.

Only:

title
message

are translated.

15.77 Product Relationship

Product's translatable fields are stored in supported languages.

Product commercial/technical fields remain language-neutral.

Do not create separate Product records per language.

One Product:

prod_101

contains multilingual display content.

15.78 Category Relationship

One Category ID exists regardless of language.

Example:

cat_construction

English display:

Construction Materials

French display:

Matériaux de construction

Both refer to the same Category.

15.79 RFQ Relationship

One RFQ exists regardless of language.

Example:

rfq_101

Buyer may submit it in French.

Seller may view translated English.

Both are acting on:

same RFQ

same quantity

same commercial values

same Quote versions

There must never be separate RFQ records per translation.

15.80 Frontend Implementation Guide

Frontend owns:

- static UI translations;
- date formatting;
- number formatting;
- currency display formatting;
- RTL layout;
- language switcher;
- guest language persistence;
- localized query cache invalidation.

Frontend should not own:

- dynamic Product translation;
- Category translation;
- RFQ translation storage;
- backend content fallback rules.

15.81 Backend Implementation Guide

Backend owns:

- User.selectedLanguage;
- supported Language records;

- dynamic content translations;
- translation service;
- fallback resolution;
- locale-aware API responses;
- translation metadata;
- translation retry;
- translation backfill.

Feature modules should call centralized localization helpers.

Recommended internal methods:

`getUserLanguage(userId)`

`resolveLanguage(request, user)`

`localizeField(field, language)`

`translateEntity(entityType, entityId)`

`getSupportedLanguages()`

15.82 Localization Helper Example

Stored:

```
{  
  "en": "Premium Cashew Nuts",  
  "fr": "Noix de cajou premium"  
}
```

Call:

`localizeField(productName, "fr")`

returns:

Noix de cajou premium

Call:


```
localizeField(productName, "sw")
```

if Swahili is unavailable returns:

Premium Cashew Nuts

through fallback.

15.83 API Response Language Metadata

Localized APIs may optionally include:

language

Example:

```
{  
  "success": true,  
  "data": {},  
  "language": "fr"  
}
```

This can help frontend debugging and cache management.

It is not necessary to repeat this inside every object.

15.84 Admin Permissions

Use the existing Admin permission architecture.

Recommended permissions:

languages.view
languages.manage

translations.view
translations.manage
translations.retry

Do not create another Admin role system.

15.85 Security Rules

Backend must:

- validate supported language codes;
- sanitize translatable user content;
- prevent arbitrary translation keys;
- restrict Admin translation overrides;
- preserve original content;
- prevent translation processing from changing financial/commercial fields;
- audit manual translation changes;
- prevent inactive languages being selected.

15.86 Translation Failure Must Not Break Business Actions

Example:

Seller creates Product



Product saved successfully



French translation succeeds

Arabic translation fails

The entire Product creation should not be rolled back solely because one machine translation failed.

Instead:

Original content saved

Translation status records failure

Fallback available

Retry translation

The same principle applies to:

- Category;
- RFQ;
- Subscription Plan;
- Announcement.

15.87 Translation and Financial Safety

Translation systems must never calculate or change:

price
discount
quantity
tax
logisticsAmount
orderTotal
paymentAmount
FX rate
subscription price

These values remain controlled by their existing modules.

15.88 Translation and Product Variants

Variant attributes may contain display labels requiring localization.

Example:

Color
Size
Material

Frontend labels may show:

French:

Couleur
Taille
Matière

But attribute identity should use stable internal codes where possible.

Example:

```
{  
  "code": "color",  
  "value": "blue"  
}
```

Display:

English:

Color: Blue

French:

Couleur: Bleu

SKU generation continues using the Product Management rules and must not change because of language selection.

15.89 SKU Language Independence

Example:

VS-M-B-C

must remain:

VS-M-B-C

regardless of whether User language is:

English

French

Arabic

Swahili

Portuguese

SKU is an inventory identifier, not translated content.

15.90 Audit Events

Recommended:

USER_LANGUAGE_CHANGED

LANGUAGE_CREATED

LANGUAGE_UPDATED

LANGUAGE_ACTIVATED
LANGUAGE_DEACTIVATED
DEFAULT_LANGUAGE_CHANGED

TRANSLATION_CREATED
TRANSLATION_COMPLETED
TRANSLATION_FAILED
TRANSLATION_RETRIED
TRANSLATION_MANUALLY_UPDATED

TRANSLATION_BACKFILL_STARTED
TRANSLATION_BACKFILL_COMPLETED

15.91 Internal Events

Recommended:

USER_LANGUAGE_UPDATED

ENTITY_TRANSLATION_REQUESTED
ENTITY_TRANSLATION_COMPLETED
ENTITY_TRANSLATION_FAILED

LANGUAGE_ACTIVATED
LANGUAGE_DEACTIVATED

Business modules may emit:

ENTITY_TRANSLATION_REQUESTED

after translatable content is created or modified.

Translation Service handles the rest.

15.92 Example End-to-End Product Flow

Seller selectedLanguage = English

Seller creates:

"Premium Cashew Nuts"



POST /seller/products



Product saved



Translation Service produces:

English

French

Arabic

Swahili

Portuguese



Product continues through normal
Product Management workflow



French Buyer opens Product



Backend sees:
selectedLanguage = fr



GET /products/:id



Buyer receives:
"Noix de cajou premium"

15.93 Example End-to-End Language Change

User currently:

selectedLanguage = en



User selects Français



PATCH /users/language

```
{  
  "language": "fr"  
}
```



Database:
selectedLanguage = fr



Frontend switches UI to French



Localized API queries revalidated



Products
Categories
Notifications
Subscription content
RFQ content

returned/displayed in French



User logs out



Logs in tomorrow



Current User response:
selectedLanguage = fr



Marketplace opens in French

The preference therefore persists across sessions and devices.

15.94 Example End-to-End RFQ Flow

French-speaking Buyer
creates Market RFQ in French



RFQ stored with:
sourceLanguage = fr



Translations generated



English-speaking Seller
selectedLanguage = en



Seller opens Market RFQ



Backend returns English requirement text



Seller responds with Quote in English



Quote notes translated



French Buyer sees French translation



Commercial fields remain exactly the same:

quantity

price

currency

delivery

validity

15.95 Module Summary

The Internationalization & Localization Management module establishes that:

- English is the default TOFA Marketplace language;
- initial core supported languages are English, French, Arabic, Swahili and Portuguese;
- supported languages are dynamically managed;
- authenticated User language is stored in `selectedLanguage`;
- language preference persists across sessions and devices;
- guest language may be maintained locally until authentication;
- changing language immediately updates frontend language;
- changing language causes localized backend content to be refetched;
- backend stores multilingual dynamic content;
- sellers do not need to manually translate Products into every supported language;
- centralized Translation Service handles configured translations;
- source language is preserved;
- translations use fallback rules;
- requested language is preferred;
- English is the primary fallback;
- normal marketplace APIs should return localized display strings;
- Seller/Admin editing endpoints may return all translations where required;
- Products remain one Product record regardless of language;
- Categories remain one Category record regardless of language;
- RFQs remain one RFQ regardless of language;
- translations never create duplicate marketplace entities;
- structured commercial fields are never translated;

- prices, currencies, quantities, FX rates and totals remain untouched by translation;
- Product SKU remains language-independent;
- Message Center user content remains in the original language for the initial implementation;
- Notifications use the User's preferred language;
- email language follows the recipient's selected language where translated templates exist;
- frontend owns static interface translation, local date/number formatting and RTL rendering;
- backend owns dynamic translations, language resolution and fallback;
- Arabic uses right-to-left layout;
- translation failures do not roll back successful marketplace business actions;
- translation provider logic is centralized instead of being duplicated across modules;
- Search can later index multilingual content from this module;
- Subscription entitlement codes remain language-neutral;
- manual Admin translation corrections can be supported and audited;
- inactive languages remain stored historically but cannot be newly selected;
- all existing Product, RFQ, Order, Payment, Subscription, Notification and Message Center business flows remain unchanged.

MODULE 16. SEARCH, DISCOVERY & RANKING

16. SEARCH, DISCOVERY & RANKING MODULE

16.1 Overview

The Search, Discovery & Ranking module controls how Products, Sellers and marketplace opportunities are discovered and ordered across TOFA Marketplace.

The module supports:

- Product search;
- Product discovery;
- Category browsing;
- Seller discovery;
- filtering;
- sorting;
- pagination;
- multilingual search;
- subscription-based Product priority;
- subscription-based Seller/store priority;
- Market RFQ opportunity priority;
- featured Products;
- featured Sellers;
- search suggestions;
- recently searched terms;
- popular searches;
- search relevance;
- availability filtering;
- seller verification signals;
- review/rating signals;
- Product activity status;
- future personalization.

The Search module does not own:

- Product creation;
- Product approval;
- Seller verification;
- Subscription Plans;
- Reviews;

- RFQ eligibility;
- Product inventory;
- Category creation.

Those modules remain responsible for their own data.

The Search module consumes their data and determines how eligible records are surfaced.

16.2 Core Search Principle

Search should separate:

Eligibility

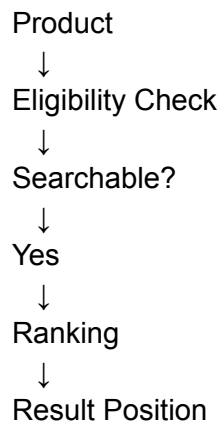
from:

Ranking

A Product must first be eligible to appear.

Then Search determines where it appears.

Recommended:



A Subscription benefit must never make an ineligible Product visible.

Example:

Product status = inactive

Seller subscription = Premium

Result:

Product does NOT appear

Subscription priority is a ranking signal, not a permission override.

16.3 Searchable Product Rule

A Product should appear in normal marketplace search only when:

status = active

and:

seller is eligible to sell

and:

Product is not deleted

and:

Product passes applicable marketplace visibility rules

The Product Management module remains authoritative for Product status.

16.4 Existing Product Status Relationship

The existing Product module uses:

draft

active

inactive

archived

deleted

Search behavior:

draft

Not searchable.

active

Searchable.

inactive

Not shown in normal buyer discovery.

archived

Not searchable.

deleted

Never shown in normal Search.

16.5 Inventory Relationship

An active Product may still have:

inventoryStatus = out_of_stock

TOFA must decide whether out-of-stock Products remain discoverable.

Recommended initial behavior:

active + in_stock

→ normal Search

active + out_of_stock

→ searchable but ranked lower / clearly marked unavailable

This is better than completely hiding the Product because buyers may:

- request an RFQ;
- contact the Seller;
- return later when stock is replenished.

Frontend should clearly display:

Out of Stock

Where TOFA wants stricter behavior later, this can become configurable.

16.6 Searchable Seller Rule

A Seller may appear in Seller discovery when:

seller verification = approved/verified

and:

User status = active

and:

Seller/business profile is eligible for public marketplace visibility

Subscription may affect Seller priority but not verification eligibility.

16.7 Searchable Market RFQ Rule

Market RFQ discovery must continue using the eligibility rules defined in the Market RFQ module.

Search may rank eligible Market RFQs for Sellers.

It does not decide whether the Seller is legally/business eligible to respond.

Flow:

Market RFQ



Market RFQ eligibility



Eligible Seller



Search/Discovery Ranking



RFQ position

16.8 Search Entities

Initial Search entities:

products
sellers
market_rfqs
categories

These may use separate endpoints rather than one universal Search endpoint.

This allows each entity to maintain appropriate filters and response structures.

16.9 Product Search Endpoint

Endpoint

GET /products

The existing Product listing endpoint can also serve as the Product Search endpoint.

There is no need to introduce a duplicate:

GET /search/products

unless infrastructure later requires it.

The existing:

GET /products

should support Search parameters.

16.10 Product Search Query Parameters

Recommended:

search
categoryIds
sellerId
countryOfOrigin

minPrice
maxPrice
currency
minRating
inventoryStatus
hasDiscount
sortBy
page
limit

Example:

GET
/products?search=cashew&categoryIds=cat_agriculture&minRating=4&sortBy=relevance&page=1&limit=20

16.11 Product Search Response

```
{
  "success": true,

  "data": [
    {
      "id": "prod_101",

      "productName": "Premium Cashew Nuts",

      "categories": [
        {
          "id": "cat_agriculture",
          "name": "Agriculture"
        },
        {
          "id": "cat_nuts",
          "name": "Nuts"
        }
      ],

      "seller": {
        "id": "seller_456",
        "storeName": "Agro Supplies Ltd",
        "isVerified": true,
      }
    }
  ]
}
```

```

    "rating": 4.7,
    "totalReviews": 85
  },

  "countryOfOrigin": "Nigeria",

  "currency": "USD",

  "price": 20,

  "discount": 5,

  "inventoryStatus": "in_stock",

  "averageRating": 4.6,
  "totalReviews": 120,

  "mainImage": "https://cdn.tofa.com/products/cashew.jpg",

  "isFeatured": true
},

"pagination": {
  "page": 1,
  "limit": 20,
  "total": 145,
  "totalPages": 8
},

"language": "en"
}

```

Localized Product fields follow the existing Internationalization module.

16.12 Search Text Matching

Product Search should consider:

```

productName
productDescription
category names

```

seller/store name
SKU where appropriate
countryOfOrigin
relevant searchable Product attributes

The exact indexed fields may depend on the Search engine used.

Search should give higher relevance to stronger matches.

Recommended conceptual priority:

Exact Product Name Match
↓
Product Name Partial Match
↓
Category Match
↓
Description Match
↓
Seller/Store Match

16.13 Multilingual Search

The Internationalization module already stores multilingual dynamic content.

Search should index translated fields.

Example Product:

```
{  
  "productName": {  
    "en": "Premium Cashew Nuts",  
    "fr": "Noix de cajou premium"  
  }  
}
```

A French search:

noix de cajou

should match the same Product.

An English search:

cashew nuts

should also match it.

There is still only one Product:

prod_101

16.14 Search Language Principle

Search input may be in a language different from:

User.selectedLanguage

Search should preferably match across supported translated content.

However, results should display in:

User.selectedLanguage

where translation exists.

Example:

User language = French

Search query = cashew

Result may display:

Noix de cajou premium

because that is the user's preferred display language.

16.15 Product Filters

Recommended Product filters:

Categories

categoryIds[]

A Product may belong to multiple Categories as already defined in Product Management.

Country of Origin

countryOfOrigin

Price Range

minPrice
maxPrice

Currency

currency

Rating

minRating

Inventory

inventoryStatus

Discount

hasDiscount

Seller

sellerId

Future filters may include:

- minimum order quantity;
- supply capacity;
- lead time;
- seller country;

- verification level.

16.16 Price Filter and Multi-Currency Rule

Products may be listed in different currencies.

Filtering by raw price becomes misleading if:

USD Product
NGN Product
GHS Product

appear together.

Recommended initial rule:

If the buyer selects a currency filter:

currency = USD

price filtering applies to Products in USD.

If no currency filter is selected, **minPrice/maxPrice** should either:

1. be unavailable; or
2. use a normalized comparison currency through the existing FX infrastructure.

Recommended initial implementation:

Price Range
→ require selected/display currency

The Search module should not invent a separate FX service.

16.17 Display Currency and Search

If TOFA already allows users to browse using a preferred/display/payment currency later, Search may use current FX to display equivalent Product prices.

This value is:

display estimate

not a Checkout FX lock.

The existing Checkout module remains responsible for binding FX.

16.18 Product Sort Options

Recommended:

relevance
newest
price_low_to_high
price_high_to_low
highest_rating
most_reviewed

Optional future:

most_popular
best_selling

16.19 Default Product Sort

When a search term exists:

sortBy = relevance

should be default.

When browsing without search:

Recommended:

ranking score

that combines marketplace relevance, subscription priority and quality signals.

16.20 Search Ranking Principle

Ranking should use multiple signals.

Conceptually:

Final Ranking Score

=

Search Relevance

+

Product Quality Signal

+

Seller Quality Signal

+

Subscription Priority

+

Freshness Signal

+

Availability Signal

+

Featured Signal

The exact formula should remain backend-controlled.

Frontend should never calculate final ranking.

16.21 Subscription Product Priority

Subscription Management already provides:

product_priority

Example:

Basic = 0

Growth = 20

Premium = 50

Search consumes this value.

Flow:

Seller Subscription



Effective Entitlements



product_priority



Search Ranking

Do not write:

```
if planName == Premium  
move to top
```

Use the entitlement.

16.22 Product Priority Does Not Guarantee Position One

Example:

Premium Product

Product Name = Industrial Generator

Buyer Search = Cashew Nuts

The Generator should not appear above highly relevant Cashew Products just because its Seller has a Premium plan.

Subscription ranking works within relevance.

Recommended principle:

Relevance first

Subscription boost within eligible/relevant results

16.23 Featured Product Eligibility

Subscription already defines:

featured_product_eligibility

max_featured_products

This means a Seller may be allowed to feature Products.

However:

eligible to feature

is different from:

currently featured

The system needs a separate Product promotion record.

16.24 FeaturedProduct Structure

FeaturedProduct

Field	Type	Required
id	UUID/String	Yes
productId	UUID/String	Yes
sellerId	UUID/String	Yes
status	Enum	Yes
startAt	Timestamp	Yes
endAt	Timestamp/null	No
createdAt	Timestamp	Yes
updatedAt	Timestamp	Yes

Recommended statuses:

scheduled
active
expired
cancelled

16.25 Feature Product Endpoint

Endpoint

POST /seller/products/:productId/feature

Payload

```
{  
  "durationDays": 7  
}
```

If featured duration is controlled entirely by Subscription, payload may instead be empty and backend applies configured duration.

For initial implementation, TOFA may simply allow featuring while Subscription remains active.

16.26 Feature Product Backend Processing

Backend must:

1. verify Seller owns Product;
2. verify Product is active;
3. retrieve effective Subscription entitlements;
4. verify `featured_product_eligibility = true`;
5. verify active featured Product count is below `max_featured_products`;
6. create FeaturedProduct record;
7. return result.

16.27 Feature Product Response

```
{  
  "success": true,  
  "message": "Product featured successfully",  
  
  "data": {  
    "productId": "prod_101",  
    "featured": true,  
    "startAt": "2026-08-27T10:00:00Z",  
    "endAt": "2026-09-03T10:00:00Z"  
  }  
}
```

Toast

Product featured successfully.

16.28 Featured Product Limit Error

```
{  
  "success": false,  
  "code": "FEATURED_PRODUCT_LIMIT_REACHED",  
  "message": "You have reached the number of featured products allowed on your current plan."  
}
```

Toast

You have reached your featured product limit.

16.29 Featured Product Ranking

Featured status may provide an additional ranking signal.

It should not override basic relevance.

Example:

Search = Cocoa

Featured Cocoa Product

→ gets boost

Featured Generator

→ does not become relevant to Cocoa search

16.30 Featured Product Display

Frontend should clearly identify promoted Products where required.

Example:

Featured

or:

Sponsored

depending on TOFA's commercial terminology.

Do not make paid ranking indistinguishable where marketplace transparency requires disclosure.

16.31 Seller Discovery Endpoint

Endpoint

GET /sellers

Query

search
country
categoryId
minRating
isVerified
sortBy
page
limit

16.32 Seller Discovery Response

```
{
  "success": true,

  "data": [
    {
      "id": "seller_456",

      "storeName": "Agro Supplies Ltd",

      "companyLogo": "https://cdn.tofa.com/store/logo.jpg",

      "companyBio": "Trusted supplier of agricultural commodities.",

      "country": "Nigeria",

      "isVerified": true,
```

```
"averageRating": 4.7,  
"totalReviews": 85,  
  
"activeProductsCount": 42,  
  
"isFeatured": true  
}  
],  
  
"pagination": {  
  "page": 1,  
  "limit": 20,  
  "total": 80,  
  "totalPages": 4  
}  
}
```

companyBio follows Internationalization rules.

16.33 Seller Search Matching

Seller Search may match:

storeName
companyName
companyBio
categories
Product categories
country

Legal company information remains displayed according to Seller/User rules.

16.34 Seller Ranking Signals

Recommended:

search relevance
seller rating
review volume
verified status
active Product quality
subscription store priority

featured status

Subscription provides:

store_priority

featured_store_eligibility

as already defined.

16.35 Featured Store Structure

FeaturedStore

Field	Type	Required
id	UUID/String	Yes
sellerId	UUID/String	Yes
status	Enum	Yes
startAt	Timestamp	Yes
endAt	Timestamp/null	No
createdAt	Timestamp	Yes
updatedAt	Timestamp	Yes

16.36 Feature Store

If TOFA allows Sellers to activate this directly:

Endpoint

POST /seller/store/feature

Backend checks:

featured_store_eligibility

before activation.

Alternatively, this may be automatically activated by Plan.

The entitlement configuration should determine the behaviour.

16.37 Category Browsing

The existing Category module remains authoritative.

Search may retrieve Products by:

categoryIds

Example:

GET /products?categoryIds=cat_construction

Because Products may belong to multiple Categories:

Product appears if it belongs to the selected Category

For multiple selected Categories, TOFA should define whether filtering uses:

ANY

or:

ALL

Recommended for marketplace usability:

ANY selected category

unless the frontend explicitly provides a stricter filter.

16.38 Category Search

Endpoint

Existing Category endpoint may support:

GET /categories?search=construction

No separate Search Category service is required.

16.39 Search Suggestions

The system may provide suggestions while the user types.

Endpoint

GET /search/suggestions

Query

q
type
limit

Example:

GET /search/suggestions?q=cas&type=products&limit=8

16.40 Search Suggestion Response

```
{
  "success": true,
  "data": [
    {
      "type": "product",
      "id": "prod_101",
      "label": "Premium Cashew Nuts",
      "image": "https://cdn.tofa.com/products/cashew.jpg"
    },
    {
      "type": "category",
      "id": "cat_cashew",
      "label": "Cashew"
    }
  ],
}
```

```
{
  "type": "seller",
  "id": "seller_123",
  "label": "Cashew Exporters Ltd"
}
]
```

Results follow the user's preferred language.

16.41 Suggestion Rules

Suggestions should:

- use active/eligible records;
- respect Product visibility;
- respect Seller visibility;
- be fast;
- return only limited fields;
- never expose inactive/deleted data.

16.42 Search History

Authenticated users may have recent Search terms stored.

SearchHistory

Field	Type	Required
id	UUID/String	Yes
userId	UUID/String	Yes
query	String	Yes
createdAt	Timestamp	Yes

Recommended:

Store only a reasonable number such as:

last 20–50 searches

This may be configurable.

16.43 Get Recent Searches

Endpoint

GET /search/history

Response

```
{
  "success": true,

  "data": [
    {
      "id": "search_101",
      "query": "cashew nuts",
      "createdAt": "2026-08-27T10:00:00Z"
    },

    {
      "id": "search_100",
      "query": "construction materials",
      "createdAt": "2026-08-26T15:00:00Z"
    }
  ]
}
```

16.44 Delete One Search History Item

Endpoint

DELETE /search/history/:searchHistoryId

Response

```
{
  "success": true,
  "message": "Search removed successfully."
}
```

16.45 Clear Search History

Endpoint

DELETE /search/history

Response

```
{  
  "success": true,  
  "message": "Search history cleared successfully."  
}
```

Toast

Search history cleared.

16.46 Guest Search History

Guest Search history may remain:

localStorage

and does not need backend persistence.

16.47 Popular Searches

TOFA may aggregate frequently searched terms.

Recommended:

Endpoint

GET /search/popular

Response

```
{  
  "success": true,  
  
  "data": [  

```

```

{
  "query": "cashew nuts"
},
{
  "query": "cement"
},
{
  "query": "cocoa beans"
}
]
}

```

Popular Search aggregation should not expose individual user identities.

16.48 Search Analytics

Search service may collect anonymous/aggregated events such as:

```

SEARCH_PERFORMED
SEARCH_RESULT_CLICKED
SEARCH_NO_RESULTS

```

This information may later feed Analytics.

Search analytics should not change ranking automatically unless TOFA explicitly introduces behavioural ranking.

16.49 Search No Results

When no Product matches:

```

{
  "success": true,
  "data": [],

  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 0,
    "totalPages": 0
  }
}

```

Frontend empty state:

No products found.

Try adjusting your search or filters.

Optional actions:

[Clear Filters]

[Create Market RFQ]

The **Create Market RFQ** button can be useful because a buyer who cannot find a Product may instead publish a sourcing requirement.

This opens the existing Market RFQ flow.

16.50 Market RFQ Discovery

The Market RFQ module already defines:

GET /rfqs/market/available

Search should not create a duplicate endpoint.

That endpoint should support ranking and filtering.

Recommended parameters remain:

categoryId
deliveryCountry
deliveryType
search
dateFrom
dateTo
page
limit

Optional:

sortBy

16.51 Market RFQ Ranking

Subscription already defines:

market_rfq_priority

This entitlement can influence how eligible Market RFQs are ordered for a Seller.

Example:

Basic = 0

Growth = 20

Premium = 50

However, Subscription does not make a Seller eligible for an unrelated RFQ.

Eligibility remains in Market RFQ.

16.52 Market RFQ Priority Interpretation

market_rfq_priority should be treated as a benefit that improves access/discovery.

Possible implementations include:

- earlier placement in Seller RFQ feed;
- highlighted eligible RFQs;
- priority recommendation feed;
- earlier notification where business policy allows.

It should not change:

- RFQ requirements;
- Buyer terms;
- Seller eligibility;
- submission deadline.

16.53 Market RFQ Response Priority

Subscription may also include:

rfq_response_priority

This can influence how buyer-facing quote lists are presented only if TOFA deliberately wants that behaviour.

However, buyer quote comparison should remain commercially fair.

Recommended initial use:

Do not use subscription to reorder buyer quote comparisons by default.

Buyers should primarily compare by:

price
seller rating
delivery
verification

Subscription priority is more appropriate for Seller opportunity discovery than for manipulating a buyer's received Quote comparison.

16.54 Product Ranking Signals

Recommended Product ranking signals:

searchTextRelevance
categoryRelevance
productStatus
inventoryAvailability
productRating
reviewVolume
sellerRating
sellerVerified
productPriority
featuredProduct
freshness

Optional future:

salesPerformance
conversionRate

buyerPreference
geographicalRelevance

16.55 Suggested Ranking Weight Principle

Exact weights should not be exposed as fixed frontend rules.

Conceptually:

Relevance = strongest signal

Quality = meaningful signal

Subscription/Featured = controlled boost

Freshness = moderate signal

This avoids a marketplace where paid ranking completely destroys search quality.

16.56 Example Ranking

Search:

cashew nuts

Product A

Exact Cashew Match

Rating 4.8

Verified Seller

Basic Plan

Product B

Partial Cashew Match

Rating 4.0

Premium Plan

Product A may still rank above Product B.

Subscription is not guaranteed dominance.

16.57 Rating Relationship

Product rating comes from:

Reviews Module

Seller rating comes from:

Seller Reviews

Search should use only public/published rating summaries.

It must not calculate its own review averages.

16.58 Seller Verification Relationship

Seller verification comes from:

Seller / Business Verification Module

Search may use:

isVerified

as:

- a filter;
- a display badge;
- a ranking quality signal.

Search must not set verification.

16.59 Subscription Relationship

Search consumes effective entitlements such as:

product_priority

featured_product_eligibility

max_featured_products
store_priority
featured_store_eligibility
market_rfq_priority

Search does not own Plan definitions.

The existing Entitlement Service remains authoritative.

16.60 Product Management Relationship

Product Management provides:

status
inventoryStatus
price
discount
categories
seller
origin
ratings summary references

Search consumes these.

Search does not modify Product status or inventory.

16.61 Category Relationship

Category module provides:

Category IDs
Category hierarchy where applicable
Translated Category names
Category active state

Search must not return Products through inactive/deleted Categories in ways that violate Category/Product rules.

16.62 Internationalization Relationship

Search indexes multilingual translatable fields from Internationalization.

Search response displays localized content according to:

User.selectedLanguage

with:

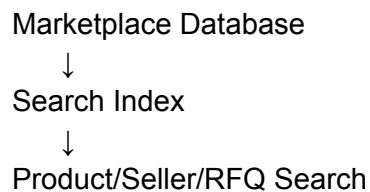
English fallback

Search does not create translations.

16.63 Search Index Architecture

For small initial datasets, database search may be sufficient.

For scale, recommended architecture:



Search index may later use a dedicated Search engine.

The application should abstract Search access sufficiently so underlying technology can change.

16.64 Search Index Product Document

Recommended indexed representation:

```
{
  "productId": "prod_101",

  "names": {
    "en": "Premium Cashew Nuts",
    "fr": "Noix de cajou premium",
    "ar": "كاجو ممتاز"
  },

  "descriptions": {
    "en": "Export-grade cashew nuts..."
  }
}
```

```
},  
  
"categoryIds": [  
  "cat_agriculture",  
  "cat_nuts"  
],  
  
"sellerId": "seller_456",  
"sellerStoreName": "Agro Supplies Ltd",  
  
"countryOfOrigin": "Nigeria",  
  
"currency": "USD",  
"price": 20,  
  
"status": "active",  
"inventoryStatus": "in_stock",  
  
"averageRating": 4.6,  
"totalReviews": 120,  
  
"sellerRating": 4.7,  
"sellerVerified": true,  
  
"productPriority": 20,  
  
"featured": true,  
  
"createdAt": "2026-08-20T10:00:00Z"  
}
```

This is Search infrastructure, not a second Product database source of truth.

16.65 Search Index Sync

Search index should update when relevant business data changes.

Examples:

```
PRODUCT_CREATED  
PRODUCT_UPDATED  
PRODUCT_STATUS_CHANGED  
PRODUCT_DELETED
```

PRODUCT_INVENTORY_CHANGED

SELLER_VERIFICATION_UPDATED

PRODUCT_REVIEW_PUBLISHED

PRODUCT_REVIEW_HIDDEN

SELLER_REVIEW_PUBLISHED

SUBSCRIPTION_ACTIVATED

SUBSCRIPTION_CHANGED

SUBSCRIPTION_EXPIRED

FEATURED_PRODUCT_CHANGED

Search listens and updates indexed representation.

16.66 Search Index Failure Rule

If Search indexing temporarily fails:

Product database update remains successful

Search indexing should retry.

Do not roll back:

- Product edit;
- Subscription activation;
- Review publication;

just because Search indexing failed.

Search may temporarily show stale results until synchronization succeeds.

16.67 Search Index Event Idempotency

Repeated:

PRODUCT_UPDATED

events should not create duplicate Search records.

Search record key should use:

productId

and update the existing indexed Product.

16.68 Seller Index

Seller Search index may include:

sellerId
storeName
companyName
companyBio translations
country
categories
rating
totalReviews
verification
storePriority
featuredStore
activeProductsCount

16.69 Market RFQ Index

Where a Search index is used for Market RFQ discovery, it may contain:

rfqId
requirementTitle translations
description translations
categoryIds
deliveryCountry
deliveryType
submissionDeadline
status

Seller eligibility should still be checked through Market RFQ business logic.

16.70 Search Suggestions and Multilingual Content

Suggestions should display in User language.

Example:

User language = fr

Types:

"cas"

Possible suggestion:

Noix de cajou premium

even if original Product was entered in English.

16.71 Search Result Click Tracking

Optional endpoint:

POST /search/events

Payload

```
{  
  "eventType": "result_click",  
  "searchQuery": "cashew nuts",  
  "entityType": "product",  
  "entityId": "prod_101",  
  "position": 3  
}
```

This may support Analytics later.

The event must not be used as a transactional action.

16.72 Recently Viewed Products

Useful discovery feature:

ProductView

Field	Type
id	UUID/String
userId	UUID/String
productId	UUID/String
viewedAt	Timestamp

Guest recently viewed Products may remain client-side.

16.73 Get Recently Viewed Products

Endpoint

GET /products/recently-viewed

Response

```
{
  "success": true,

  "data": [
    {
      "id": "prod_101",
      "productName": "Premium Cashew Nuts",
      "currency": "USD",
      "price": 20,
      "mainImage": "https://cdn.tofa.com/products/cashew.jpg"
    }
  ]
}
```

Only currently viewable Products should be returned.

If a Product becomes deleted:

do not return it

even if it exists in view history.

16.74 Product View Tracking

Endpoint

POST /products/:productId/view

Payload not required.

Recommended backend:

- validate Product;
- record/aggregate view;
- avoid excessive duplicate writes;
- support analytics.

The Product GET endpoint may alternatively record views internally.

Do not require frontend to count Product views itself.

16.75 Recommended Products

This may be introduced later.

Potential signals:

recently viewed
search history
categories viewed
previous purchases
country
popular Products

Initial implementation does not need advanced personalization.

Recommended initial:

Related Products
Popular Products
Recently Viewed

rather than complex AI recommendation logic.

16.76 Related Products

Endpoint

GET /products/:productId/related

Query

limit

Example:

GET /products/prod_101/related?limit=8

16.77 Related Products Logic

Recommended:

same Categories

+

active Products

+

not same Product

+

relevance

+

ranking signals

May optionally prioritize different Sellers to increase marketplace discovery.

16.78 Related Products Response

```
{
  "success": true,

  "data": [
    {
      "id": "prod_202",
      "productName": "Raw Cashew Nuts",
      "currency": "USD",
      "price": 18,
```

```
    "mainImage": "https://cdn.tofa.com/products/cashew2.jpg",  
    "averageRating": 4.4  
  }  
]  
}
```

16.79 Trending / Popular Products

Optional:

Endpoint

GET /products/popular

Possible ranking may include:

recent views
purchases
RFQ activity
review quality
availability

Do not define popularity only by Subscription.

Paid priority and popularity should remain separate concepts.

16.80 Search and Deleted Product Rule

If Product status becomes:

deleted

Search index should remove/hide it.

Existing:

- Orders;
- RFQs;
- historical Reviews;

remain untouched through their own snapshots/references.

Search hiding does not delete history.

16.81 Search and Archived Product Rule

Archived Products should not appear in normal Search.

Seller can still access them through Seller Product Management.

16.82 Search and Inactive Product Rule

Inactive Products do not appear publicly.

If Product becomes active again:

Search index updates

→ Product becomes discoverable

16.83 Search and Subscription Expiry

Suppose Seller has:

Premium

product_priority = 50

then Subscription expires and Seller falls back to:

Basic

product_priority = 0

Search should update ranking signal.

Existing Products remain active subject to Product/Subscription limit rules already defined.

Search does not automatically deactivate Products because Subscription expires.

16.84 Search and Over-Limit Seller

Subscription module already states that if Seller downgrades and exceeds Product limit:

existing Products are not automatically deleted

Search should continue following Product status.

If Products remain **active**, Search may continue to show them unless Subscription/Product policy explicitly inactivates them.

Do not introduce a different limit-enforcement rule inside Search.

16.85 Featured Product and Subscription Expiry

If featured eligibility comes from Subscription and the Subscription expires:

Recommended:

featured Product promotion ends

or is marked:

expired

Search then removes featured boost.

Product itself remains active.

16.86 Search Admin Configuration

TOFA may need some ranking/configuration values adjustable by Admin.

Recommended configuration:

featuredBoost
subscriptionPriorityMaxWeight
ratingWeight
freshnessWeight
availabilityWeight
outOfStockPenalty
searchHistoryLimit
popularSearchWindowDays

However, Admin should not need to manually build arbitrary Search algorithms.

16.87 Get Search Settings

Endpoint

GET /admin/search/settings

Response

```
{
  "success": true,

  "data": {
    "outOfStockProductsVisible": true,

    "searchHistoryLimit": 30,

    "featuredBoostEnabled": true,

    "ranking": {
      "subscriptionPriorityEnabled": true,
      "ratingEnabled": true,
      "freshnessEnabled": true,
      "availabilityEnabled": true
    }
  }
}
```

Exact internal ranking weights may remain hidden from general Admin if TOFA prefers.

16.88 Update Search Settings

Endpoint

PATCH /admin/search/settings

Payload

```
{
  "outOfStockProductsVisible": true,
  "searchHistoryLimit": 30,
  "featuredBoostEnabled": true,
```



```
"ranking": {
  "subscriptionPriorityEnabled": true,
  "ratingEnabled": true,
  "freshnessEnabled": true,
  "availabilityEnabled": true
}
```

Response

```
{
  "success": true,
  "message": "Search settings updated successfully."
}
```

Admin Toast

Search settings updated successfully.

16.89 Admin Search Index Health

Useful for operations:

Endpoint

GET /admin/search/index/status

Response

```
{
  "success": true,

  "data": {
    "productsIndexed": 12500,
    "sellersIndexed": 2300,
    "marketRfqsIndexed": 480,

    "pendingUpdates": 15,
    "failedUpdates": 2,

    "lastSuccessfulSyncAt": "2026-08-27T10:30:00Z"
  }
}
```

```
}
```

16.90 Reindex Entity

Authorized Admin/system may trigger a reindex.

Endpoint

POST /admin/search/reindex

Payload

```
{  
  "entityType": "product",  
  "entityId": "prod_101"  
}
```

or:

```
{  
  "entityType": "products"  
}
```

for a full Product reindex, if permitted.

16.91 Reindex Response

```
{  
  "success": true,  
  "message": "Search reindex requested successfully."  
}
```

This does not change the Product itself.

16.92 Search Result Security

Search must never expose:

- deleted Product data;
- private Seller information;

- restricted buyer information;
- private RFQ negotiations;
- seller-specific Market RFQ Quotes;
- payment information;
- personal addresses.

Search indexes should contain only information appropriate for the discovery use case.

16.93 Seller Information Visibility

Subscription defines:

view_seller_info
view_buyer_info

Public Search should still show basic marketplace Seller information required to sell Products.

Premium information visibility is handled by Seller/Subscription rules.

Search must not use Subscription to expose private contact data.

16.94 Search API Authorization

Public Product Search may be accessible without authentication.

Authenticated Search can additionally support:

- Search history;
- recently viewed;
- personalized language;
- future personalization.

Seller Market RFQ Search requires Seller authentication and Market RFQ eligibility.

16.95 Search Guest Language

Guest Search uses:

Accept-Language

or frontend-selected guest language.

Fallback:

English

as defined in Internationalization.

16.96 Search Performance

Search APIs should use pagination.

Recommended defaults:

page = 1
limit = 20

Recommended maximum:

limit <= 100

to avoid unbounded result responses.

16.97 Pagination Consistency

Product and Seller listings should follow the same pagination structure already used across existing modules:

```
{  
  "pagination": {  
    "page": 1,  
    "limit": 20,  
    "total": 145,  
    "totalPages": 8  
  }  
}
```

16.98 Search Response Consistency

All Search/Discovery endpoints should use:

```
{
```

```
"success": true,  
"data": [],  
"pagination": {}  
}
```

where pagination applies.

This follows the broader marketplace API response style.

16.99 Search Frontend Product Page

Recommended Search result Product card:

[Image]

Premium Cashew Nuts

Agro Supplies Ltd ✓

USD 20

★ 4.6 (120)

Nigeria

Featured

Do not show Subscription plan name unless TOFA wants it publicly visible.

Use a neutral:

Featured

badge where applicable.

16.100 Search Filters Frontend

Recommended:

Search

Category
Country of Origin
Price
Rating
Availability
Discount

Sort By

Mobile may use:

[Filter] [Sort]

16.101 Seller Discovery Frontend

Recommended Seller card:

[Logo]

Agro Supplies Ltd
Verified Seller

Nigeria

★ 4.7 (85 reviews)

42 Products

[View Store]

16.102 Market RFQ Discovery Frontend

Seller dashboard may show:

Available Market RFQs

Grade A Cocoa Beans
20 tonnes
Accra, Ghana
Seller-Arranged Delivery
Deadline: 5 September

[View RFQ]

Subscription priority may influence ordering but should not change RFQ terms.

16.103 No Search Result to Market RFQ Flow

A useful marketplace flow is:

Buyer searches



No suitable Product



Create a Market RFQ

Frontend empty state may show:

Can't find what you're looking for?

Create a Market RFQ and let eligible sellers respond.

[Create Market RFQ]

This uses the existing Market RFQ module.

No separate Search RFQ endpoint is required.

16.104 Search Event Structure

Optional:

SearchEvent

Field	Type
id	UUID/String
userId	UUID/String/null
sessionId	String/null
query	String/null

entityType	Enum
eventType	Enum
entityId	UUID/String/null
resultPosition	Integer/null
filters	JSON/null
createdAt	Timestamp

Recommended **eventType**:

search
result_click
no_result

This may feed Analytics.

16.105 Search Data Privacy

Search history belongs to the User.

Do not expose it to other marketplace Users.

Admin access to aggregated Search analytics should not unnecessarily expose identifiable search histories.

16.106 Search Frontend Implementation Guide

Frontend should:

- send Search query and filters;
- render backend-ranked results;
- manage filter UI;
- manage sorting selection;
- show localized content;
- show featured labels;
- maintain guest search history locally where applicable;
- send Search analytics events where enabled.

Frontend should not:

- calculate Product rank;
- add Subscription boost;
- determine Seller verification;
- calculate review averages;
- convert binding FX;
- decide RFQ eligibility.

16.107 Backend Implementation Guide

Search Service owns:

- Search query execution;
- ranking;
- filtering;
- Search indexing;
- suggestions;
- Search history;
- recently viewed records where implemented;
- related Product discovery;
- popular Search aggregation;
- Search analytics events;
- ranking configuration.

It consumes:

Product Management

Seller Verification

Categories

Subscription Entitlements

Reviews

Internationalization

Market RFQ

without taking ownership of those modules.

16.108 Internal Events Consumed

Recommended:

PRODUCT_CREATED

PRODUCT_UPDATED

PRODUCT_STATUS_CHANGED

PRODUCT_INVENTORY_CHANGED

PRODUCT_DELETED

SELLER_APPLICATION_APPROVED
SELLER_VERIFICATION_UPDATED
USER_STATUS_UPDATED

PRODUCT_REVIEW_PUBLISHED
PRODUCT_REVIEW_HIDDEN
SELLER_REVIEW_PUBLISHED
SELLER_REVIEW_HIDDEN

SUBSCRIPTION_ACTIVATED
SUBSCRIPTION_CHANGED
SUBSCRIPTION_EXPIRED

MARKET_RFQ_CREATED
MARKET_RFQ_AWARDED
MARKET_RFQ_CANCELLED
MARKET_RFQ_EXPIRED

ENTITY_TRANSLATION_COMPLETED

Reuse existing event names where already defined rather than creating parallel business events.

16.109 Search-Owned Internal Events

Recommended:

SEARCH_PERFORMED
SEARCH_NO_RESULTS
SEARCH_RESULT_CLICKED

SEARCH_INDEX_UPDATED
SEARCH_INDEX_FAILED

PRODUCT_FEATURED
PRODUCT_FEATURE_EXPIRED

STORE_FEATURED
STORE_FEATURE_EXPIRED

16.110 Notification Relationship

Search generally should not send emails.

Potential in-app events such as:

featured Product expired

may notify Seller if TOFA wants.

Notification Service should handle delivery.

No separate Search email architecture is required.

16.111 Product Feature Expiry Notification

Optional Seller in-app Notification:

Featured product period ended

Premium Cashew Nuts is no longer featured.

Email is optional and probably unnecessary initially.

16.112 Admin Permissions

Use existing Admin permission architecture.

Recommended:

search.view_settings
search.manage_settings
search.view_index
search.reindex

Do not create Search-specific Admin roles.

16.113 Search Audit Events

Recommended:

SEARCH_SETTINGS_UPDATED

SEARCH_REINDEX_REQUESTED

PRODUCT_FEATURE_ACTIVATED
PRODUCT_FEATURE_CANCELLED

STORE_FEATURE_ACTIVATED
STORE_FEATURE_CANCELLED

16.114 Important Consistency Rules

Product Status

Search follows Product Management.

It does not create new Product visibility statuses.

Subscription

Search consumes entitlements.

It does not hardcode Plan names.

Reviews

Search consumes public rating summaries.

It does not calculate ratings.

Internationalization

Search indexes translations.

It does not translate content itself.

Market RFQ

Search ranks only already-eligible opportunities.

It does not override Seller eligibility.

FX

Search display conversion, if added, is non-binding.

Checkout remains responsible for locked FX.

Categories

Search uses existing Category IDs.

It does not create new Category records.

16.115 Module Summary

The Search, Discovery & Ranking module establishes that:

- Search eligibility and Search ranking are separate;
- only eligible Products appear in marketplace discovery;
- active Product status remains controlled by Product Management;
- inactive, archived and deleted Products do not appear in normal Search;
- out-of-stock Products may remain discoverable but should be clearly identified and can be ranked lower;
- Search supports Products, Sellers, Categories and Market RFQ discovery;
- the existing `GET /products` endpoint is reused rather than creating a duplicate Product Search endpoint;
- Product Search supports query, filters, sorting and pagination;
- Products may be filtered by multiple Categories;
- multilingual Product and Category content is searchable;
- results display in the User's preferred language;
- English remains the fallback;
- Search ranking combines relevance, quality and controlled marketplace priority signals;
- Subscription `product_priority` is a ranking signal rather than an eligibility override;
- Premium Subscription does not guarantee the first result position;
- Search relevance remains the strongest factor;
- featured Product eligibility comes from Subscription;
- currently featured Products are stored separately from Product eligibility;
- featured status may provide a controlled ranking boost;
- featured Store eligibility and store priority can similarly influence Seller discovery;
- buyer-facing Market RFQ Quote comparison should not be unfairly reordered based on Seller Subscription;
- Market RFQ opportunity ranking for Sellers may use `market_rfq_priority`;
- Seller eligibility for Market RFQ remains owned by the Market RFQ module;
- Product and Seller ratings come from Reviews;
- Seller verification comes from Seller Verification;
- Categories come from the Category module;
- translations come from Internationalization;

- Subscription entitlements come from the Entitlement Service;
- Search does not create or modify those records;
- Search history may be stored for authenticated Users;
- guest Search history may remain local;
- Search suggestions are supported;
- recently viewed Products may be supported;
- related Products may be supported;
- popular Products and searches may be added without changing Product ownership;
- no-result Search can direct Buyers to the existing Market RFQ flow;
- Search indexing can be introduced independently of the primary Marketplace database;
- Search index is not the business source of truth;
- indexing failures do not roll back Product, Review or Subscription business actions;
- indexing is event-driven and idempotent;
- Search APIs remain paginated;
- ranking remains backend-controlled;
- frontend renders backend-ranked results and must not calculate its own Subscription boosts or ranking.

MODULE 17. ANALYTICS & REPORTING

17. ANALYTICS & REPORTING MODULE

17.1 Overview

The Analytics & Reporting Module provides Sellers, Buyers and authorized Admin Users with structured insights into activities and performance across TOFA Marketplace.

The module consolidates information generated by existing modules including:

- Users;
- Seller / Business Verification;
- Products;
- Categories;
- Cart and Checkout;
- Payments;
- Orders;
- Delivery;
- Direct RFQs;
- Market RFQs;
- Reviews;
- Subscriptions;
- Message Center;
- Notifications;
- Search and Discovery.

Analytics does not become the source of truth for these records.

Example:

Orders Module

→ owns Orders

Payment Module

→ owns Payments

Product Module

→ owns Products

RFQ Module

→ owns RFQs

Subscription Module

→ owns Plans and Entitlements

Analytics Module

→ reads and aggregates their data

The module supports:

Seller Analytics

Buyer Analytics

Admin / Marketplace Analytics

17.2 Core Analytics Principle

Analytics must distinguish between:

Transactional Data

and:

Analytical Data

Example:

Order Product Value = USD 15,000

Transaction Fee = 3%

TOFA Transaction Fee = USD 450

The Order/financial record owns those values.

Analytics may aggregate:

Total Sales = USD 150,000

TOFA Transaction Revenue = USD 4,500

Completed Orders = 12

Analytics must not modify the underlying transactions.

17.3 Analytics Audiences

Seller

Seller Analytics may cover:

- Product performance;
- Product views;
- Orders;
- Sales;
- RFQs;
- Quotes;
- Reviews;
- customer insights;
- Search performance;
- Subscription usage.

Buyer

Buyer Analytics may cover:

- Orders;
- spending;
- RFQs;
- suppliers;
- sourcing activity;
- purchasing categories.

Admin

Admin Analytics may cover:

- Users;

- Sellers;
 - Buyers;
 - Products;
 - Orders;
 - GMV;
 - TOFA revenue;
 - Payments;
 - RFQs;
 - Subscriptions;
 - Categories;
 - Countries;
 - Reviews;
 - Search activity;
 - marketplace performance.
-

17.4 Seller Analytics Subscription Access

The Subscription module provides:

analytics_access

analytics_level

report_export

Analytics consumes these entitlements.

Do not implement:

if planName == "Premium"

Instead:

Seller



Effective Subscription



Analytics Entitlements



Available Analytics Features

17.5 Recommended Analytics Levels

Possible levels:

basic
advanced
enterprise

These are entitlement values rather than Subscription Plan names.

Basic

May include:

- total Product views;
- total Orders;
- completed Orders;
- total Sales;
- total RFQs;
- average rating;
- basic recent performance.

Advanced

May include:

- custom date ranges;
- Product-level analytics;
- Sales trends;
- Order trends;
- Quote conversion;
- RFQ performance;
- customer geography;
- category performance;
- Search performance;
- downloadable reports.

Enterprise

May include Advanced plus:

- extended historical reporting;
- deeper customer insights;
- expanded data export;
- advanced marketplace reporting;
- future scheduled reports.

Admin controls which Plans receive each Analytics entitlement.

17.6 Analytics Currency Principle

Different currencies must not be blindly combined.

Do not calculate:

USD 10,000
+
NGN 5,000,000
+
GHS 20,000

as one number.

Every financial metric must retain:

amount
currency

unless an explicit reporting conversion is performed.

17.7 Buyer Currency

Buyer Analytics uses the actual currency the Buyer paid in.

Example:

Buyer Checkout Currency = NGN
Buyer Payment = NGN 15,000,000

Buyer spending records:

NGN 15,000,000

even where the Seller's Product was originally listed in USD.

17.8 Seller Sales Currency

Seller Sales should use the commercial/settlement currency applicable to the Seller Order.

Example:

Product Currency = USD

Seller Commercial Product Amount = USD 10,000

Seller Sales Analytics records:

USD 10,000

If a Seller transacts in multiple currencies, Analytics reports them separately.

Example:

```
{
  "salesByCurrency": [
    {
      "currency": "USD",
      "amount": 45000
    },
    {
      "currency": "NGN",
      "amount": 12500000
    }
  ]
}
```

17.9 Marketplace Reporting Currency

Admin may require a single normalized currency for marketplace-wide management reporting.

TOFA may configure:

Marketplace Reporting Currency

Example:

USD

This is for Analytics only.

It is not:

- Buyer Payment Currency;
- Seller settlement currency;
- Product currency;
- Quote currency.

17.10 Historical FX Rule

Historical financial reporting must use the transaction-associated FX/reporting conversion information where applicable.

Do not use today's FX rate to recalculate old transactions.

Example:

Order Date:

1 USD = NGN 1,520

Later:

Current Rate:

1 USD = NGN 1,610

Historical reporting should not change merely because the current FX rate changed.

17.11 Seller Analytics Overview

Endpoint

GET /seller/analytics/overview

Query

dateFrom

dateTo

Response

```
{
  "success": true,
  "data": {
    "products": {
      "total": 45,
      "active": 38,
      "inactive": 3,
      "draft": 2,
      "archived": 2
    },
    "productViews": {
      "total": 12480,
      "percentageChange": 20.93
    },
    "orders": {
      "total": 120,
      "completed": 88,
      "cancelled": 7,
```



```
    "other": 25
  },

  "salesByCurrency": [
    {
      "currency": "USD",
      "amount": 125000
    },
    {
      "currency": "NGN",
      "amount": 8500000
    }
  ],

  "rfqs": {
    "directReceived": 45,
    "marketResponded": 30,
    "quotesAccepted": 18
  },

  "reviews": {
    "averageRating": 4.7,
    "totalReviews": 230
  },

  "subscription": {
    "analyticsLevel": "advanced",
    "transactionFeePercentage": 3
  }
}
```

17.12 Percentage Change

Recommended:

Percentage Change

=

(Current Period - Previous Period)

÷ Previous Period

× 100

If previous period is zero:

```
{  
  "percentageChange": null,  
  "comparisonAvailable": false  
}
```

Frontend:

No previous period data

17.13 Seller Sales Analytics

Endpoint

GET /seller/analytics/sales

Query

dateFrom
dateTo
groupBy
currency

Supported grouping:

day
week
month

Response

```
{  
  "success": true,  
  
  "data": {  
    "currency": "USD",
```

```
"summary": {
  "grossProductSales": 125000,

  "completedProductSales": 110000,

  "transactionFees": 3300,

  "netProductAmount": 106700,

  "refundedProductAmount": 5000,

  "totalOrders": 120,

  "completedOrders": 88,

  "averageOrderValue": 1250
},

"trend": [
  {
    "period": "2026-08-01",
    "productSales": 3500,
    "orders": 4
  }
]
}
```

17.14 Seller Sales Definitions

Gross Product Sales

Product/commercial value attributable to Seller Orders before applicable refunds.

Completed Product Sales

Product value from successfully completed Orders.

Transaction Fees

Actual TOFA transaction fees stored against qualifying Seller Orders.

Net Product Amount

Recommended:

Completed Product Sales

-

TOFA Transaction Fees

-

Applicable Product Refunds/Adjustments

Logistics settlement should remain separately identifiable where applicable.

Average Order Value

Recommended:

Completed Product Sales

÷

Completed Orders

17.15 TOFA Transaction Fee Analytics

TOFA transaction fees are determined by the Seller's Subscription at the time the transaction financials are locked.

Example:

Basic = 5%

Growth = 3%

Premium = 2%

These are examples of configured Plan values.

Analytics must not determine the percentage from the Plan name.

17.16 Stored Fee Rule

Order/financial records should already contain:

transactionFeePercentage

feeBaseAmount

transactionFeeAmount

subscriptionId

subscriptionPlanId

Analytics uses:

transactionFeeAmount

as the authoritative TOFA transaction revenue for that transaction.

Do not calculate:

Current Product Value × Seller's Current Subscription Fee

inside Analytics.

17.17 Historical Subscription Fee Rule

Example:

Seller completed Order A while on:

Basic

5%

Later upgrades to:

Premium

2%

Analytics must continue reporting Order A's fee as:

5%

New transactions use:

2%

Historical TOFA revenue therefore remains stable.

17.18 Logistics and Transaction Fee

Under the current recommended business rule:

Transaction Fee Base = Product/Commercial Amount

not:

Product Amount + Logistics

Example:

Product = USD 10,000

Logistics = USD 1,500

Buyer Pays = USD 11,500

TOFA Fee = 5%

Analytics reports:

Product Value = USD 10,000

Logistics Value = USD 1,500

TOFA Transaction Revenue = USD 500

assuming Logistics is excluded from the fee base.

17.19 Multi-Seller Transaction Analytics

For multi-Seller Checkout, Analytics must aggregate each Seller Order independently.

Example:

Seller A

Product Amount = NGN 2,000,000

Fee = 5%

TOFA Revenue = NGN 100,000

Seller B:

Product Amount = NGN 3,000,000

Fee = 2%

TOFA Revenue = NGN 60,000

Combined TOFA transaction revenue:

NGN 160,000

where both Seller Orders are in NGN.

The fact that the Buyer made one Payment does not mean there is one Seller transaction fee.

17.20 Product Analytics

Endpoint

GET /seller/analytics/products

Query

dateFrom
dateTo
sortBy
page
limit

Recommended sorting:

views
orders
sales
rating
conversion

Response

```
{
  "success": true,

  "data": [
    {
      "productId": "prod_101",

      "productName": "Premium Cashew Nuts",

      "views": 4200,

      "orders": 35,

      "unitsOrdered": 1250,

      "sales": {
        "currency": "USD",
        "amount": 45000
      },

      "rfqCount": 18,

      "averageRating": 4.8,
```



```
    "totalReviews": 92,  
    "conversionRate": 8.3  
  }  
],  
  
  "pagination": {  
    "page": 1,  
    "limit": 20,  
    "total": 45,  
    "totalPages": 3  
  }  
}
```

17.21 Product Conversion

Recommended:

$$\frac{\text{Unique Buyers Who Purchased Product}}{\text{Unique Buyers Who Viewed Product}} \times 100$$

This avoids repeated views excessively distorting conversion.

17.22 Seller Order Analytics

Endpoint

GET /seller/analytics/orders

Response

```
{  
  "success": true,  
  
  "data": {
```

```
"total": 120,

"byStatus": {
  "pending": 8,
  "confirmed": 10,
  "processing": 7,
  "shipped": 12,
  "delivered": 8,
  "completed": 68,
  "cancelled": 7
},

"completionRate": 56.67,

"cancellationRate": 5.83
}
}
```

The exact statuses must come from the existing Order module.

Analytics must not introduce another Order lifecycle.

17.23 Seller RFQ Analytics

Endpoint

GET /seller/analytics/rfqs

Response

```
{
  "success": true,

  "data": {
    "direct": {
      "received": 45,
      "responded": 40,
      "accepted": 16
    },
  },
}
```

```
"market": {
  "responded": 30,
  "accepted": 8
},

"quotes": {
  "submitted": 70,
  "accepted": 24,
  "expired": 15,
  "rejected": 20,
  "other": 11
},

"quoteAcceptanceRate": 34.29,

"ordersCreatedFromQuotes": 22
}
}
```

17.24 RFQ-to-Order Conversion

Recommended:

$$\frac{\text{Orders Created From Accepted Quotes}}{\text{Accepted Quotes}} \times 100$$

Direct RFQ and Market RFQ should remain distinguishable.

17.25 Seller Customer Analytics

Endpoint

GET /seller/analytics/customers

Response

```
{
  "success": true,

  "data": {
    "totalCustomers": 82,

    "newCustomers": 21,

    "returningCustomers": 61,

    "byCountry": [
      {
        "country": "Nigeria",
        "customers": 35
      },
      {
        "country": "Ghana",
        "customers": 22
      }
    ]
  }
}
```

Availability may depend on [analytics_level](#).

17.26 Customer Privacy

Analytics must not bypass:

view_buyer_info

Aggregated information may be displayed without exposing restricted personal information.

Allowed:

35 customers from Nigeria

Not automatically allowed:

Buyer Name

Email

Phone Number

17.27 Search Analytics

Endpoint

GET /seller/analytics/search

Response

```
{
  "success": true,

  "data": {
    "searchImpressions": 42000,

    "searchClicks": 6800,

    "clickThroughRate": 16.19,

    "topSearchTerms": [
      {
        "query": "cashew nuts",
        "impressions": 4800,
        "clicks": 1200
      }
    ]
  }
}
```

Search events come from Search & Discovery.

17.28 Seller Review Analytics

Endpoint

GET /seller/analytics/reviews

Response

```
{
  "success": true,

  "data": {
    "averageRating": 4.7,

    "totalReviews": 230,

    "ratingDistribution": {
      "5": 165,
      "4": 42,
      "3": 15,
      "2": 5,
      "1": 3
    },

    "productReviews": 190,

    "sellerReviews": 40
  }
}
```

Review data comes from the Review module.

17.29 Subscription Usage Analytics

Endpoint

GET /seller/analytics/subscription-usage

Response

```
{
  "success": true,
```

```
"data": {
  "plan": {
    "id": "plan_growth",
    "name": "Growth"
  },

  "usage": {
    "products": {
      "used": 42,
      "limit": 100,
      "remaining": 58
    },

    "featuredProducts": {
      "used": 2,
      "limit": 5,
      "remaining": 3
    }
  },

  "features": {
    "productPriority": 20,
    "marketRfqPriority": 20,
    "analyticsLevel": "advanced",
    "reportExport": true,
    "transactionFeePercentage": 3
  }
}
```

17.30 Buyer Analytics Overview

Endpoint

GET /buyer/analytics/overview

Response

```
{
  "success": true,
```

```
"data": {
  "orders": {
    "total": 38,
    "completed": 25,
    "active": 8,
    "cancelled": 5
  },

  "spendByCurrency": [
    {
      "currency": "USD",
      "amount": 85000
    },
    {
      "currency": "NGN",
      "amount": 12000000
    }
  ],

  "rfqs": {
    "directCreated": 15,
    "marketCreated": 12,
    "quotesReceived": 48,
    "ordersCreatedFromQuotes": 14
  },

  "suppliers": {
    "uniqueSuppliersPurchasedFrom": 18
  }
}
```

17.31 Buyer Spending Analytics

Endpoint

GET /buyer/analytics/spending

Query

dateFrom
dateTo
currency
groupBy

Response

```
{
  "success": true,

  "data": {
    "currency": "USD",

    "totalSpend": 85000,

    "trend": [
      {
        "period": "2026-08",
        "amount": 25000
      }
    ],

    "byCategory": [
      {
        "categoryId": "cat_agriculture",
        "categoryName": "Agriculture",
        "amount": 42000
      }
    ]
  }
}
```

17.32 Buyer Spend Rule

Buyer Spend uses the actual Order Payment amount and currency.

Example:

Buyer pays NGN
→ Buyer Spend = NGN

Buyer pays USD
→ Buyer Spend = USD

Seller Product currency must not replace Buyer Payment currency in Buyer Analytics.

17.33 Buyer RFQ Analytics

Endpoint

GET /buyer/analytics/rfqs

Response

```
{
  "success": true,

  "data": {
    "direct": {
      "created": 15,
      "quotesReceived": 15,
      "accepted": 9
    },

    "market": {
      "created": 12,
      "quotesReceived": 33,
      "awarded": 8
    },

    "ordersCreatedFromQuotes": 14
  }
}
```

17.34 Buyer Supplier Analytics

Endpoint

GET /buyer/analytics/suppliers

Response

```
{
  "success": true,

  "data": [
    {
      "sellerId": "seller_456",

      "storeName": "Agro Supplies Ltd",

      "orders": 8,

      "spend": {
        "currency": "USD",
        "amount": 22000
      },

      "completedOrders": 7
    }
  ]
}
```

17.35 Admin Analytics Overview

Endpoint

GET /admin/analytics/overview

Query

dateFrom
dateTo
country

Response

```
{
```

```
"success": true,

"data": {
  "users": {
    "total": 25000,
    "buyers": 16000,
    "sellers": 9000,
    "newUsers": 1800
  },

  "products": {
    "total": 82000,
    "active": 69000
  },

  "orders": {
    "total": 14500,
    "completed": 10500,
    "cancelled": 850
  },

  "gmV": {
    "reportingCurrency": "USD",
    "amount": 4500000
  },

  "tofaRevenue": {
    "reportingCurrency": "USD",
    "transactionFees": 185000,
    "subscriptionRevenue": 92000,
    "otherRevenue": 0,
    "totalRevenue": 277000
  },

  "rfqs": {
    "directCreated": 4200,
    "marketCreated": 3800,
    "quotesSubmitted": 11000,
    "ordersCreatedFromQuotes": 2900
  },

  "subscriptions": {
    "activePaidSubscriptions": 3200
  }
}
```

```
}  
}
```

17.36 GMV Definition

GMV means:

Gross Merchandise Value

Recommended TOFA definition:

Total Product/Commercial Value of qualifying marketplace Orders

Logistics should be reported separately rather than automatically included in Product GMV.

17.37 Booked and Completed GMV

Admin may distinguish:

Booked GMV

from:

Completed GMV

Example:

```
{  
  "bookedGmv": {  
    "currency": "USD",  
    "amount": 5000000  
  },  
  
  "completedGmv": {  
    "currency": "USD",
```

```
"amount": 4200000
}
}
```

17.38 GMV vs TOFA Revenue

These must remain separate.

GMV

Value of Products/commercial transactions processed through the marketplace.

TOFA Revenue

Actual amount TOFA earns.

Current major sources include:

Transaction Fees

Subscription Revenue

Additional revenue sources may be introduced later.

17.39 TOFA Transaction Revenue

TOFA's transaction revenue is percentage-based according to the Seller's Subscription.

However, Analytics does not calculate this from the current Plan.

Instead:

Qualifying Transaction

↓

Stored transactionFeeAmount

↓

Analytics

Recommended:

TOFA Transaction Revenue

=

SUM(qualifying transactionFeeAmount)

17.40 TOFA Revenue Analytics

Endpoint

GET /admin/analytics/revenue

Query

dateFrom

dateTo

currency

country

Response

```
{
  "success": true,

  "data": {
    "reportingCurrency": "USD",

    "transactionRevenue": 185000,

    "subscriptionRevenue": 92000,

    "otherRevenue": 0,

    "totalRevenue": 277000
  }
}
```

17.41 Transaction Revenue Breakdown by Subscription Plan

Admin may need to understand how much transaction revenue each Subscription Plan generates.

Endpoint

GET /admin/analytics/revenue/by-subscription

Response

```
{
  "success": true,

  "data": [
    {
      "planId": "plan_basic",

      "planName": "Basic",

      "transactionFeePercentageSnapshot": 5,

      "qualifyingTransactions": 5200,

      "transactionValue": {
        "reportingCurrency": "USD",
        "amount": 1500000
      },

      "transactionRevenue": {
        "reportingCurrency": "USD",
        "amount": 75000
      }
    },

    {
      "planId": "plan_growth",

      "planName": "Growth",

      "qualifyingTransactions": 2800,
```



```
"transactionRevenue": {  
  "reportingCurrency": "USD",  
  "amount": 65000  
}  
}  
]  
}
```

Where a Plan's fee changed during the selected period, do not assume one percentage applies to every historical transaction.

The actual stored transaction fee remains authoritative.

17.42 Effective Transaction Fee Analytics

Admin may also monitor:

Effective Transaction Fee Rate

Recommended:

$$\frac{\text{Total Transaction Fee Revenue}}{\text{Total Eligible Product Transaction Value}} \times 100$$

Example:

Eligible Product Value = USD 10,000,000

Transaction Fee Revenue = USD 350,000

Effective Fee Rate = 3.5%

This is a reporting metric.

It does not replace individual Plan percentages.

17.43 Admin Logistics Analytics

Endpoint

GET /admin/analytics/logistics

Response

```
{
  "success": true,

  "data": {
    "ordersByDeliveryType": {
      "integrated_logistics": 4200,
      "seller_arranged": 3100,
      "buyer_arranged": 1800
    },

    "integratedLogisticsValue": {
      "reportingCurrency": "USD",
      "amount": 280000
    }
  }
}
```

Unknown off-platform Logistics costs must not be invented.

17.44 Admin Order Analytics

Endpoint

GET /admin/analytics/orders

Query

dateFrom

dateTo
country
status
source

Order source may include the existing:

cart
direct_rfq
market_rfq

Response

```
{  
  "success": true,  
  
  "data": {  
    "totalOrders": 14500,  
  
    "bySource": {  
      "cart": 9000,  
      "direct_rfq": 2800,  
      "market_rfq": 2700  
    },  
  
    "byStatus": {  
      "completed": 10500,  
      "cancelled": 850,  
      "other": 3150  
    },  
  
    "averageOrderValue": {  
      "reportingCurrency": "USD",  
      "amount": 310.34  
    }  
  }  
}
```

17.45 Admin Payment Analytics

Endpoint

GET /admin/analytics/payments

Query

dateFrom
dateTo
paymentMethod
currency
paymentFor
status

Payment Analytics must support the Payment module's dynamic:

paymentFor

Examples:

order
subscription
future payment purposes

17.46 Payment Analytics Response

```
{
  "success": true,

  "data": {
    "payments": {
      "total": 15200,
      "successful": 13900,
      "pendingVerification": 420,
      "failed": 880
    },

    "byPaymentFor": [
      {
        "paymentFor": "order",
        "count": 12800
      }
    ]
  }
}
```

```

    },
    {
      "paymentFor": "subscription",
      "count": 2400
    }
  ],

  "byMethod": [
    {
      "paymentMethod": "paystack",
      "count": 5200
    },
    {
      "paymentMethod": "flutterwave",
      "count": 4100
    },
    {
      "paymentMethod": "bank_transfer",
      "count": 2600
    },
    {
      "paymentMethod": "papss",
      "count": 900
    }
  ]
}

```

Configured methods should appear dynamically.

17.47 Subscription Revenue

Subscription revenue must come from successful Subscription Payments.

Do not calculate:

Plan Price × Subscribers

because:

- Plan prices may change;
- currencies may differ;
- Payments may fail;
- billing periods may differ;
- promotions may exist.

Recommended:

Subscription Revenue

=

Successful Payments

where paymentFor = subscription

17.48 Admin Subscription Analytics

Endpoint

GET /admin/analytics/subscriptions

Response

```
{
  "success": true,

  "data": {
    "activeSubscriptions": 9000,

    "activePaidSubscriptions": 3200,

    "newSubscriptions": 480,

    "expiredSubscriptions": 210,

    "byPlan": [
      {
        "planId": "plan_basic",
        "planName": "Basic",
        "activeSubscribers": 5800
      },
      {
        "planId": "plan_growth",
```

```
    "planName": "Growth",
    "activeSubscribers": 2100
  },
  "revenueByCurrency": [
    {
      "currency": "USD",
      "amount": 85000
    },
    {
      "currency": "NGN",
      "amount": 45000000
    }
  ]
}
```

Plan names are dynamic.

17.49 Admin RFQ Analytics

Endpoint

GET /admin/analytics/rfqs

Response

```
{
  "success": true,

  "data": {
    "direct": {
      "created": 4200,
      "responded": 3500,
      "convertedToOrder": 1900
    },

    "market": {
      "created": 3800,
```

```
    "quotesReceived": 7500,  
    "awarded": 1700,  
    "convertedToOrder": 1000  
  },  
  
  "totalOrdersCreatedFromQuotes": 2900  
}
```

17.50 Admin Product Analytics

Endpoint

GET /admin/analytics/products

Response

```
{  
  "success": true,  
  
  "data": {  
    "totalProducts": 82000,  
  
    "activeProducts": 69000,  
  
    "topCategories": [  
      {  
        "categoryId": "cat_agriculture",  
        "categoryName": "Agriculture",  
        "products": 15000,  
        "orders": 5200  
      }  
    ],  
  
    "topProducts": [  
      {  
        "productId": "prod_101",  
        "productName": "Premium Cashew Nuts",  
        "orders": 850,  
        "views": 52000  
      }  
    ]  
  }  
}
```



```
]
}
}
```

17.51 Admin Seller Analytics

Endpoint

GET /admin/analytics/sellers

Response

```
{
  "success": true,

  "data": {
    "totalSellers": 9000,

    "verifiedSellers": 7200,

    "activeSellingSellers": 5400,

    "topSellers": [
      {
        "sellerId": "seller_456",

        "storeName": "Agro Supplies Ltd",

        "completedOrders": 350,

        "sales": {
          "reportingCurrency": "USD",
          "amount": 520000
        },

        "averageRating": 4.8
      }
    ]
  }
}
```

17.52 Admin Buyer Analytics

Endpoint

GET /admin/analytics/buyers

Response

```
{
  "success": true,

  "data": {
    "totalBuyers": 16000,

    "activeBuyers": 7200,

    "buyersWithOrders": 6100,

    "buyersWithRfqs": 4800,

    "repeatBuyers": 3200
  }
}
```

17.53 Admin Country Analytics

Endpoint

GET /admin/analytics/countries

Response

```
{
  "success": true,

  "data": [
    {
```

```
    "country": "Nigeria",
    "buyers": 8500,
    "sellers": 4200,
    "orders": 7200,
    "rfqs": 3100
  },
  {
    "country": "Ghana",
    "buyers": 3400,
    "sellers": 1800,
    "orders": 2800,
    "rfqs": 1500
  }
]
```

17.54 Admin Category Analytics

Endpoint

GET /admin/analytics/categories

Response

```
{
  "success": true,

  "data": [
    {
      "categoryId": "cat_agriculture",

      "categoryName": "Agriculture",

      "products": 15000,

      "orders": 5200,

      "rfqs": 1800,

      "gm": {
        "reportingCurrency": "USD",
```

```
    "amount": 1250000
  }
}
]
```

17.55 Marketplace Funnel Analytics

Endpoint

GET /admin/analytics/funnel

Response

```
{
  "success": true,

  "data": {
    "productFlow": {
      "productViews": 1200000,
      "checkoutStarted": 85000,
      "ordersCreated": 62000,
      "paymentsSuccessful": 58000,
      "ordersCompleted": 51000
    },

    "rfqFlow": {
      "rfqsCreated": 8000,
      "quotesSubmitted": 11000,
      "quotesAccepted": 3400,
      "ordersCreated": 2900,
      "ordersCompleted": 2400
    }
  }
}
```

Where possible, funnel relationships should use attributable records rather than unrelated period totals.

17.56 Pending Manual Payments

A manual Payment that has:

Proof Uploaded

but remains:

Pending Verification

must not count as successful revenue.

Once Admin confirms Payment:

Payment Successful



Analytics Updated

17.57 LC and TT

For RFQ Orders using:

Letter of Credit

Telegraphic Transfer

uploading Payment proof/documentation does not itself equal successful Payment.

Analytics follows the Payment status defined by the Payment module.

17.58 PAPSS

PAPSS is treated as a Payment option under the existing Payment module.

Analytics may report:

PAPSS Transaction Count

PAPSS Payment Volume

PAPSS Success Rate

No separate PAPSS lifecycle is required.

17.59 Payment Method Performance

Endpoint

GET /admin/analytics/payment-methods

Response

```
{
  "success": true,

  "data": [
    {
      "paymentMethod": "paystack",
      "attempts": 5500,
      "successful": 5200,
      "failed": 300,
      "successRate": 94.55
    },
    {
      "paymentMethod": "flutterwave",
      "attempts": 4400,
      "successful": 4100,
      "failed": 300,
      "successRate": 93.18
    }
  ]
}
```

Payment methods appear dynamically based on Payment configuration.

17.60 Analytics Date Range

Recommended common filters:

Today

Last 7 Days

Last 30 Days

This Month

Last Month

This Year

Custom

Frontend converts presets into:

dateFrom

dateTo

Backend processes actual dates.

17.61 Report Export

Where permitted by:

report_export

Users may export reports.

Endpoint

POST /reports/export

Payload

```
{
  "reportType": "seller_sales",

  "format": "xlsx",

  "filters": {
    "dateFrom": "2026-08-01",
    "dateTo": "2026-08-31",
    "currency": "USD"
  }
}
```

Recommended formats:

csv
xlsx
pdf

17.62 Report Structure

Report

Field	Type	Required
id	UUID/String	Yes
requestedBy	UUID/String	Yes
reportType	String	Yes
format	Enum	Yes
filters	JSON	Yes
status	Enum	Yes
fileUrl	String/null	No

expiresAt	Timestamp/null	No
errorMessage	String/null	No
createdAt	Timestamp	Yes
completedAt	Timestamp/null	No

Recommended statuses:

processing
completed
failed
expired

17.63 Report Generation

Small reports may return immediately.

Large reports should be asynchronous.

Response:

```
{
  "success": true,

  "message": "Your report is being generated.",

  "data": {
    "reportId": "report_101",
    "status": "processing"
  }
}
```

Toast

Your report is being generated. We'll notify you when it is ready.

17.64 Report Ready Email

Subject

Your TOFA Marketplace Report Is Ready

Body

Hello {FirstName},

Your requested TOFA Marketplace report has been generated successfully.

Report: {ReportName}

Period: {DateFrom} – {DateTo}

Format: {Format}

You can now sign in to your TOFA Marketplace account to download the report.

Please note that generated report files may only be available for a limited period.

Best Regards,

Traders of Africa Team

17.65 Report Ready Notification

Title

Your report is ready

Message

Your {ReportName} for {DateFrom} – {DateTo} is ready to download.

Action:

[View Report](#)

17.66 Analytics Access Restriction

If Seller requests an unavailable Analytics feature:

```
{  
  "success": false,  
  
  "code": "ANALYTICS_ACCESS_REQUIRED",  
  
  "message": "This analytics feature is not available on your current subscription."  
}
```

Frontend:

Upgrade your plan to access this analytics feature.

CTA:

View Plans

Do not hardcode which named Plan is required.

17.67 Report Export Restriction

If:

report_export = false

response:

```
{  
  "success": false,  
  
  "code": "REPORT_EXPORT_NOT_AVAILABLE",  
  
  "message": "Report export is not available on your current subscription."  
}
```

17.68 Analytics Event Architecture

Recommended:

Marketplace Modules



Business Events



Analytics Processing



Aggregated Analytics



Analytics APIs

Analytics should consume existing business events rather than introduce duplicate transactional actions.

17.69 Events Consumed

Examples:

USER_REGISTERED

SELLER_APPLICATION_APPROVED

PRODUCT_CREATED

PRODUCT_VIEWED

SEARCH_PERFORMED

SEARCH_RESULT_CLICKED

DIRECT_RFQ_CREATED

MARKET_RFQ_CREATED

QUOTE_SUBMITTED

QUOTE_ACCEPTED

ORDER_CREATED

ORDER_PAID

ORDER_SHIPPED

ORDER_DELIVERED

ORDER_COMPLETED

ORDER_CANCELLED

PAYMENT_SUCCESSFUL

PAYMENT_FAILED

PAYMENT_VERIFIED

REVIEW_PUBLISHED

SUBSCRIPTION_ACTIVATED

SUBSCRIPTION_UPGRADED

SUBSCRIPTION_RENEWED

SUBSCRIPTION_EXPIRED

Reuse event names already established by the relevant modules.

17.70 Analytics Idempotency

Duplicate events must not double-count metrics.

Example:

ORDER_COMPLETED

received twice must still count as:

1 Completed Order

Events should use an:

eventId

or equivalent idempotency mechanism.

17.71 Analytics Rebuild

Analytics is derived data.

Authorized Admin/System processes should be able to rebuild Analytics from source records if:

- events were missed;
- an Analytics bug is fixed;
- a metric is introduced;
- aggregates become inconsistent.

Endpoint

POST /admin/analytics/rebuild

Payload

```
{  
  "entityType": "seller",  
  
  "entityId": "seller_456",  
  
  "dateFrom": "2026-08-01",  
  
  "dateTo": "2026-08-31"  
}
```

Response

```
{  
  "success": true,  
  
  "message": "Analytics rebuild has been queued.",  
  
  "data": {  
    "jobId": "analytics_job_101",  
    "status": "processing"  
  }  
}
```

This endpoint should be highly restricted.

17.72 Analytics Freshness

Not all Analytics must be real-time.

Possible near-real-time metrics:

Product Views

Orders

Payments

RFQs

Possible aggregated metrics:

Top Categories

Country Performance

Marketplace GMV

Search Trends

Historical Reports

Frontend may show:

Last updated: 10 minutes ago

where appropriate.

17.73 Historical Product Data

Historical Order reports should use Order snapshots where applicable.

Example:

`OrderItem.productNameSnapshot`

If a Seller renames or deletes a Product later, historical reports remain understandable.

17.74 Deleted Products

Deleting a Product must not erase historical Analytics relating to completed transactions.

The Product may disappear from current marketplace Search while remaining represented in historical transaction reports.

17.75 Cancelled Orders

Cancelled Orders:

do not count toward Completed Product Sales

but may count toward:

Total Orders

Cancellation Rate

Booked Order Analytics

depending on the metric.

17.76 Admin Analytics Permissions

Recommended:

analytics.view_overview

analytics.view_users

analytics.view_sellers

analytics.view_buyers

analytics.view_products

analytics.view_orders

analytics.view_payments

analytics.view_rfqs

analytics.view_revenue

analytics.view_subscriptions

analytics.view_logistics

analytics.export_reports

analytics.rebuild

Use the existing Admin permission architecture.

17.77 Analytics Audit Events

Recommended:

ANALYTICS_REPORT_VIEWED

REPORT_EXPORT_REQUESTED

REPORT_EXPORT_COMPLETED

REPORT_EXPORT_FAILED

REPORT_DOWNLOADED

ANALYTICS_REBUILD_REQUESTED

ANALYTICS_REBUILD_COMPLETED

ANALYTICS_REBUILD_FAILED

17.78 Financial Precision

Financial calculations should use appropriate monetary/decimal handling.

Every financial value should retain:

amount

currency

Do not rely on floating-point arithmetic for authoritative financial calculations.

17.79 Source-of-Truth Rule

If Analytics temporarily disagrees with a transactional module:

Transactional Module

is authoritative.

Example:

Analytics:

Completed Orders = 50

Order Module:

Completed Orders = 51

Order Management remains correct.

Analytics should synchronize or rebuild.

17.80 Backend Implementation Guide

Backend should logically separate:

Analytics Query Service

Analytics Aggregation Service

Report Generation Service

Backend should:

- consume existing business data/events;
- aggregate marketplace metrics;
- maintain currency integrity;
- use historical FX information;
- use stored transaction fees;
- respect Subscription Analytics entitlements;
- respect Admin permissions;
- support report generation;
- make aggregation idempotent;

- support Analytics rebuilds.

Backend should not:

- calculate historical TOFA fees from current Subscription Plans;
 - combine unrelated currencies;
 - create new Order statuses;
 - create new Payment statuses;
 - create new RFQ lifecycles;
 - modify source transactions.
-

17.81 Frontend Implementation Guide

Frontend should:

- send date filters;
- display returned metrics;
- display currencies clearly;
- render charts;
- respect Analytics entitlements;
- show locked features;
- request report exports;
- display report status;
- provide download actions when ready.

Frontend should not:

- calculate authoritative GMV;
 - calculate TOFA revenue;
 - combine currencies;
 - recalculate transaction fees;
 - recalculate historical FX;
 - determine Payment status;
 - determine Order status;
 - determine Subscription entitlement.
-

17.82 Important Consistency Rules

Subscription

Analytics reads:

analytics_access
analytics_level
report_export

from Subscription entitlements.

Transaction Fees

Analytics uses:

transactionFeeAmount

stored against qualifying transactions.

It does not use the Seller's current Subscription percentage.

Orders

Existing Order statuses and Order sources remain authoritative.

Payments

Payment Analytics supports dynamic:

paymentFor

including Orders, Subscriptions and future payment purposes.

Logistics

Logistics is not automatically included in Product GMV or TOFA's percentage-based transaction fee.

RFQs

Direct RFQ and Market RFQ remain separate.

Search

Product views, Search impressions and clicks come from Search & Discovery.

Reviews

Ratings come from Reviews.

FX

Historical Analytics uses transaction-associated/reporting FX information rather than current FX.

17.83 Module Summary

The Analytics & Reporting Module establishes that:

- Analytics is derived from existing marketplace modules;
- transactional modules remain the source of truth;
- Seller, Buyer and Admin Analytics are separated;
- Seller Analytics access is controlled through dynamic Subscription entitlements;
- Analytics is not hardcoded to Subscription Plan names;
- Seller Analytics covers Products, views, Sales, Orders, RFQs, Reviews, Search, customers and Subscription usage;
- Buyer Analytics covers spending, Orders, RFQs and suppliers;
- Admin Analytics covers marketplace-wide Users, Sellers, Buyers, Products, Orders, Payments, RFQs, Subscriptions, GMV and revenue;
- Buyer spending uses Buyer Payment Currency;
- Seller Sales uses the Seller's applicable commercial currency;
- different currencies are not blindly combined;
- Admin may use a Marketplace Reporting Currency for normalized reporting;
- Marketplace Reporting Currency does not change transactional currencies;
- historical Analytics does not use today's FX rate to recalculate old transactions;
- GMV and TOFA Revenue remain separate;
- Product GMV does not automatically include Logistics;
- TOFA's transaction revenue comes from percentage-based transaction fees defined by Subscription Plans;
- percentages such as 5%, 3% and 2% are Plan configuration values rather than Analytics rules;
- transaction fee percentages are snapshotted when transaction financials are locked;
- Analytics aggregates stored `transactionFeeAmount`;
- Seller Subscription upgrades or downgrades do not alter historical TOFA transaction revenue;
- Admin Plan fee changes do not alter historical transactions;

- multi-Seller Checkout produces independent Seller transaction fees;
- TOFA transaction fee is calculated from Product/commercial value and excludes Logistics by default;
- Cart, Direct RFQ and Market RFQ transactions follow the same transaction fee reporting principle;
- Subscription revenue comes from successful Subscription Payments;
- pending manual Payments do not count as successful revenue;
- LC and TT proof submission does not equal successful Payment until confirmed;
- PAPSS and other configured Payment methods can be analyzed without separate lifecycles;
- Payment Analytics supports Orders, Subscriptions and future **paymentFor** types;
- Direct RFQ and Market RFQ Analytics remain distinguishable;
- report exports can be controlled through Subscription entitlement and Admin permission;
- large reports can be generated asynchronously;
- historical Product/Order information remains available after Product changes or deletion;
- duplicate events must not double-count Analytics;
- Analytics can be rebuilt from source records;
- frontend renders Analytics but does not calculate authoritative financial metrics;
- Analytics never overrides Order, Payment, Subscription, Product, RFQ, Review or Search source data.

MODULE 18. DISPUTE MANAGEMENT

18. DISPUTE MANAGEMENT MODULE

18.1 Overview

The Dispute Management Module allows Buyers and Sellers to formally report and resolve problems relating to transactions completed through TOFA Marketplace.

The module provides a structured process for:

- raising a dispute;
- identifying the affected Order;
- identifying the affected Seller Order where a Checkout contains multiple Sellers;
- selecting the reason for the dispute;
- providing a description;
- uploading supporting evidence;
- allowing the other party to respond;
- allowing authorized TOFA Admin Users to review the dispute;
- requesting additional evidence;
- recording the resolution;
- initiating any required refund or financial adjustment through the appropriate existing module;
- maintaining a complete dispute history.

The Dispute module does not replace:

- Order Management;
- Payment Management;
- Delivery Management;
- Message Center;
- Reviews;
- Notifications.

Instead, it references these modules where necessary.

18.2 Core Principle

A Dispute must always be connected to an existing marketplace transaction.

Recommended:

Order
↓
Seller Order
↓
Dispute

For an Order involving only one Seller:

Buyer
↓
Order
↓
Seller
↓
Dispute

For a multi-Seller Checkout:

Checkout
↓
Seller Order A
Seller Order B
Seller Order C
↓
Dispute against affected Seller Order

A problem with Seller A must not automatically create a dispute against Sellers B and C.

18.3 Who Can Raise a Dispute

A Dispute may be raised by:

buyer
seller

Admin does not normally create a Dispute on behalf of a User.

Admin manages and resolves submitted Disputes.

18.4 Dispute Eligibility

A User may raise a Dispute only where:

- the User is directly involved in the affected Order;
- the Order exists;
- the dispute concerns the relevant Seller Order;
- the Order has reached a stage where the selected dispute reason is applicable;
- the dispute is within the permitted dispute window;
- another active Dispute for the same issue is not already being processed.

The backend is responsible for validating eligibility.

18.5 Dispute Window

TOFA should configure how long Users have to raise certain disputes.

Recommended configuration:

disputeWindowDays

Example:

7 days after delivery

The exact number should be configurable rather than permanently hardcoded.

Certain exceptional dispute reasons may be allowed outside the normal window through Admin intervention.

18.6 Dispute Reasons

Recommended Buyer dispute reasons:

Product not received

Wrong product received

Product significantly different from description

Damaged product

Incorrect quantity

Missing items

Delivery issue

Payment issue

Other

Recommended Seller dispute reasons:

Payment issue

Buyer failed to fulfil agreed obligation

Buyer-arranged logistics issue

Delivery acceptance issue

Other

These reasons should preferably be configuration values so TOFA can modify them later.

18.7 Delivery Responsibility

Disputes involving Delivery must respect the Delivery method stored against the Order.

Possible Delivery methods already established include:

Integrated Logistics

Seller-Arranged Logistics

Buyer-Arranged Logistics

The Dispute module must not create another Delivery flow.

It should read the existing Delivery information from the Order/Delivery modules.

18.8 Buyer-Arranged Logistics

Where:

deliveryMethod = buyer_arranged

the Buyer is responsible for arranging the Logistics provider.

The Seller still updates the Order to **shipped** when the goods are handed over to the Buyer's nominated provider, as already established in the Order and Delivery modules.

Relevant handover information is stored against the Order.

A Delivery dispute must therefore consider:

Seller's handover evidence

Buyer-provided logistics information

Tracking information where available

Delivery communication/evidence

The Dispute module must not automatically assume Seller responsibility for events occurring after a valid handover to the Buyer's Logistics provider.

18.9 Seller-Arranged Logistics

Where:

deliveryMethod = seller_arranged

the Seller provides the applicable Delivery details when updating the Order to **shipped**.

A dispute may reference:

providerName
trackingId
trackingUrl
deliveryContact
estimatedDelivery
notes

where those fields exist on the Order.

18.10 Integrated Logistics

Where TOFA's integrated third-party Logistics option was selected, the Dispute module may reference:

Logistics Provider

Quote

Tracking Information

Shipment Events

Delivery Status

from the existing Delivery module.

TOFA integration with the Logistics provider does not mean the Dispute module should duplicate Logistics records.

18.11 Dispute Structure

Dispute

Field	Type	Required	Description
id	UUID/String	Yes	Dispute ID
disputeNumber	String	Yes	Human-readable reference
orderId	UUID/String	Yes	Parent Order
sellerOrderId	UUID/String	Yes	Affected Seller Order
raisedBy	UUID/String	Yes	User raising Dispute
raisedByType	Enum	Yes	buyer or seller
buyerId	UUID/String	Yes	Buyer
sellerId	UUID/String	Yes	Seller
reason	String	Yes	Dispute reason
description	Text	Yes	User explanation
status	Enum	Yes	Current dispute status
resolutionType	Enum/null	No	Final resolution
resolutionNotes	Text/null	No	Admin resolution explanation
assignedAdminId	UUID/String/null	No	Admin handling Dispute
resolvedBy	UUID/String/null	No	Admin who resolved it
resolvedAt	Timestamp/null	No	Resolution time
closedAt	Timestamp/null	No	Closure time
createdAt	Timestamp	Yes	Creation time
updatedAt	Timestamp	Yes	Last update

18.12 Dispute Number

Every Dispute should receive a human-readable reference.

Example:

DSP-2026-000123

Users and Admins should use this reference when discussing the case.

18.13 Dispute Status

Recommended statuses:

open

under_review

awaiting_buyer

awaiting_seller

resolved

closed

open

Dispute has been submitted.

under_review

TOFA is reviewing the case.

awaiting_buyer

Additional information or evidence has been requested from the Buyer.

awaiting_seller

Additional information or evidence has been requested from the Seller.

resolved

TOFA has made a resolution.

Any resulting financial or Order action should be initiated through the appropriate module.

closed

The dispute process is complete.

18.14 Dispute Status vs Order Status

Dispute status and Order status are separate.

Example:

Order Status:
delivered

Dispute Status:
under_review

Do not introduce Order statuses such as:

order_disputed

unless the existing Order module is intentionally updated to support such a status.

Recommended approach:

Order
hasActiveDispute = true

or derive this from the Dispute record.

This preserves the existing Order lifecycle.

18.15 Create Dispute

Endpoint

POST /disputes

Payload

```
{
  "sellerOrderId": "seller_order_123",

  "reason": "Product significantly different from description",

  "description": "The goods delivered do not match the specification agreed in the order.",

  "evidence": [
    "file_123",
    "file_124"
  ]
}
```

Backend determines:

orderId
buyerId
sellerId
raisedBy
raisedByType

from the authenticated User and Seller Order.

Frontend should not be trusted to supply these authoritative relationships.

18.16 Create Dispute Response

```
{
  "success": true,

  "message": "Your dispute has been submitted successfully.",
}
```

```
"data": {  
  "id": "dispute_123",  
  "disputeNumber": "DSP-2026-000123",  
  "status": "open"  
}
```

Toast

Your dispute has been submitted successfully. TOFA will review the case.

18.17 Evidence

Users should be able to upload supporting evidence.

Recommended supported evidence:

Images

PDF documents

Invoices

Receipts

Delivery documents

Product photographs

Screenshots

Other relevant documents

File restrictions should follow the platform's general upload security rules.

18.18 Dispute Evidence Structure

DisputeEvidence

Field	Type	Required
id	UUID/String	Yes
disputeId	UUID/String	Yes
uploadedBy	UUID/String	Yes
fileType	String	Yes
fileUrl	String	Yes
fileName	String	Yes
description	String/null	No
createdAt	Timestamp	Yes

Evidence must remain attached to the dispute even after resolution for Audit purposes.

18.19 Evidence Security

Only:

Buyer involved in Dispute

Seller involved in Dispute

Authorized Admin

should have access to private Dispute evidence.

Evidence must not be publicly accessible through predictable URLs.

18.20 Dispute Conversation

The Dispute should have its own structured response thread.

This is separate from the normal Message Center.

The Message Center remains for normal User-to-User communication.

Dispute communication exists specifically for the case.

Example:

Buyer Submission

Seller Response

Admin Request for Information

Buyer Evidence

Admin Resolution

18.21 Dispute Message Structure

DisputeMessage

Field	Type	Required
id	UUID/String	Yes
disputeId	UUID/String	Yes
senderId	UUID/String	Yes
senderType	Enum	Yes
message	Text	Yes
attachments	Array	No
createdAt	Timestamp	Yes

Recommended **senderType**:

buyer

seller

admin

18.22 Respond to Dispute

Endpoint

POST /disputes/:disputeId/messages

Payload

```
{
  "message": "The goods were supplied according to the specifications on the order.",
  "attachments": [
    "file_456"
  ]
}
```

The authenticated User becomes the sender automatically.

18.23 Get My Disputes

Endpoint

GET /disputes

Filters

status

dateFrom

dateTo

page

limit

The endpoint returns only Disputes where the authenticated User is the Buyer or Seller.

18.24 Get Dispute Details

Endpoint

GET /disputes/:disputeld

Response should include:

Dispute details

Dispute number

Order reference

Affected products

Buyer

Seller

Reason

Description

Status

Evidence

Dispute messages

Delivery information where relevant

Payment information where relevant

Resolution information

Sensitive information must respect User access rules.

18.25 Product Snapshot

Dispute details must use the Product information stored on the Order.

Do not depend on the current Product record.

Example:

OrderItem.productNameSnapshot

OrderItem.productImageSnapshot

OrderItem.variantSnapshot

OrderItem.quantity

OrderItem.unitPrice

OrderItem.totalPrice

If the Seller later edits the Product, the Dispute must still show what was actually ordered.

18.26 Multi-Product Seller Order

One Seller Order may contain several Products.

The User should identify the affected Order item(s).

Recommended:

disputedItems

Example:

{


```
"disputedItems": [  
  {  
    "orderItemId": "item_123",  
    "quantityAffected": 10  
  }  
]  
}
```

This is important because a Dispute may concern only part of the Order.

18.27 Disputed Item Structure

DisputeItem

Field	Type	Required
id	UUID/String	Yes
disputeId	UUID/String	Yes
orderItemId	UUID/String	Yes
quantityAffected	Decimal/Integer	Yes
createdAt	Timestamp	Yes

The quantity must not exceed the quantity purchased.

18.28 Buyer Dispute Submission Email

Subject

Your TOFA Dispute Has Been Submitted – {DisputeNumber}

Body

Hello {FirstName},

Your dispute regarding Order {OrderNumber} has been submitted successfully.

Dispute Reference: {DisputeNumber}

Order Reference: {OrderNumber}

Reason: {DisputeReason}

Status: Open

TOFA will review the information provided and may contact you if additional information is required.

You can follow the progress of the dispute from your TOFA Marketplace account.

Best Regards,
Traders of Africa Team

18.29 Seller Notification When Buyer Raises Dispute

Subject

A Dispute Has Been Raised on Order {OrderNumber}

Body

Hello {SellerName},

A Buyer has raised a dispute relating to Order {OrderNumber}.

Dispute Reference: {DisputeNumber}

Reason: {DisputeReason}

Please sign in to your TOFA Marketplace account to review the dispute and provide any required response or supporting evidence.

Providing accurate and timely information will help TOFA review the case.

Best Regards,
Traders of Africa Team

Notification

A dispute has been raised on Order {OrderNumber}. Review the case and provide your response.

18.30 Seller-Raised Dispute Email

Subject

Your TOFA Dispute Has Been Submitted – {DisputeNumber}

Body

Hello {SellerName},

Your dispute regarding Order {OrderNumber} has been submitted successfully.

Dispute Reference: {DisputeNumber}

Reason: {DisputeReason}

Status: Open

TOFA will review the case and may request additional information.

You can track the dispute from your TOFA Marketplace account.

Best Regards,
Traders of Africa Team

18.31 Buyer Notification for Seller-Raised Dispute

Subject

A Dispute Has Been Raised on Order {OrderNumber}

Body

Hello {FirstName},

A dispute has been raised relating to Order {OrderNumber}.

Dispute Reference: {DisputeNumber}

Reason: {DisputeReason}

Please sign in to your TOFA Marketplace account to review the case and provide any requested information or supporting evidence.

Best Regards,
Traders of Africa Team

18.32 Admin Dispute List

Endpoint

GET /admin/disputes

Permission

disputes.view

Filters

search

status

reason

raisedByType

buyerId

sellerId

assignedAdminId

dateFrom

dateTo

page

limit

Search should support:

Dispute Number

Order Number

Buyer

Seller

18.33 Admin Dispute Details

Endpoint

GET /admin/disputes/:disputeId

Permission

disputes.view

Admin view should provide the information required to investigate the case.

Recommended sections:

Dispute Summary

Order Details

Product Details

Buyer Details

Seller Details

Payment Details

Delivery Details

Evidence

Dispute Conversation

Timeline

Resolution

Information should come from the appropriate existing modules.

18.34 Assign Dispute

Endpoint

PATCH /admin/disputes/:disputeId/assign

Permission

disputes.assign

Payload

```
{  
  "adminId": "admin_123"  
}
```

Response

```
{  
  "success": true,  
  "message": "Dispute assigned successfully."  
}
```

Toast

Dispute assigned successfully.

18.35 Start Review

Endpoint

PATCH /admin/disputes/:disputeId/status

Permission

disputes.manage

Payload

```
{  
  "status": "under_review"  
}
```

Admin cannot arbitrarily set any status.

Backend must enforce valid Dispute status transitions.

18.36 Request Information From Buyer

Endpoint

POST /admin/disputes/:disputeId/request-information

Payload

```
{  
  "from": "buyer",  
  
  "message": "Please provide clear photographs showing the condition of the goods received."  
}
```

Backend changes:

status = awaiting_buyer

18.37 Buyer Additional Information Email

Subject

Additional Information Required – Dispute {DisputeNumber}

Body

Hello {FirstName},

TOFA requires additional information to continue reviewing your dispute regarding Order {OrderNumber}.

Dispute Reference: {DisputeNumber}

Information Required:

{RequestMessage}

Please sign in to your TOFA Marketplace account and provide the requested information and supporting evidence.

Best Regards,
Traders of Africa Team

Notification

Additional information is required for Dispute {DisputeNumber}.

18.38 Request Information From Seller

The same endpoint may use:

```
{  
  "from": "seller",  
  
  "message": "Please provide proof that the goods were handed over to the logistics provider."  
}
```

Backend changes:

status = awaiting_seller

18.39 Seller Additional Information Email

Subject

Additional Information Required – Dispute {DisputeNumber}

Body

Hello {SellerName},

TOFA requires additional information to continue reviewing the dispute regarding Order {OrderNumber}.

Dispute Reference: {DisputeNumber}

Information Required:

{RequestMessage}

Please sign in to your TOFA Marketplace account and provide the requested information and supporting evidence.

Best Regards,
Traders of Africa Team

18.40 Resolution Types

Recommended:

buyer_favour

seller_favour

partial_resolution

mutual_resolution

no_action

These describe the Dispute outcome.

They must not directly represent Payment status.

18.41 Resolve Dispute

Endpoint

POST /admin/disputes/:disputeId/resolve

Permission

disputes.resolve

Example — Seller Favoured

```
{  
  "resolutionType": "seller_favour",  
  
  "resolutionNotes": "Evidence confirms that the goods were supplied according to the agreed  
order specification.",  
  
  "financialAction": {  
    "type": "none"  
  }  
}
```

```
}
```

18.42 Buyer-Favour Resolution

Example:

```
{  
  "resolutionType": "buyer_favour",  
  
  "resolutionNotes": "The evidence confirms that the delivered goods materially differ from the  
agreed specification.",  
  
  "financialAction": {  
    "type": "refund",  
    "amount": 500000,  
    "currency": "NGN"  
  }  
}
```

The Dispute module records the resolution.

The actual Refund must be processed through the Payment/Refund mechanism.

18.43 Partial Resolution

A partial resolution may be required where only part of an Order is affected.

Example:

100 units ordered

10 units damaged

Resolution may authorize:

Refund for 10 units

rather than the entire Order.

Example:

```
{
  "resolutionType": "partial_resolution",

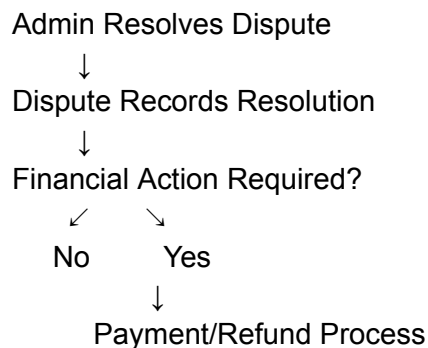
  "resolutionNotes": "Ten units were confirmed damaged.",

  "financialAction": {
    "type": "partial_refund",
    "amount": 150000,
    "currency": "NGN"
  }
}
```

18.44 Financial Action Principle

Dispute resolution and Payment movement are separate responsibilities.

Recommended:



The Dispute module must not create a second Payment system.

18.45 Refund Amount Currency

Refunds must use the appropriate original Payment currency.

Example:

Buyer Paid = NGN

Refund = NGN

Do not refund:

USD

simply because the Seller's Product was listed in USD.

The Payment module remains authoritative for actual refund processing and FX treatment.

18.46 Logistics Refund

If a resolution includes Logistics, the system must determine whether the Logistics amount was actually part of the TOFA Payment.

Example:

Integrated Logistics

If:

Product = NGN 500,000

Logistics = NGN 25,000

Buyer Payment = NGN 525,000

a valid resolution may include all or part of the Logistics amount.

Buyer-Arranged Logistics

If Logistics was paid outside TOFA:

TOFA cannot automatically refund money it did not collect.

The resolution record may acknowledge the issue, but the platform Payment refund must only relate to funds actually processed through TOFA.

18.47 Transaction Fee and Refund

Where a refund affects an Order for which TOFA recorded a transaction fee, the corresponding financial adjustment must be reflected in the Order/Payment financial records.

Example:

Product Value = NGN 1,000,000

TOFA Fee = 5%

Original TOFA Fee = NGN 50,000

If the entire Product amount is refunded, TOFA should not continue reporting the original NGN 50,000 as final transaction revenue unless TOFA's business policy explicitly makes that fee non-refundable.

The Payment/Order financial records should store the resulting adjustment.

Analytics then reads the adjusted financial values.

18.48 Partial Refund Transaction Fee

Example:

Product Value = NGN 1,000,000

Fee = 5%

Refunded Product Value = NGN 200,000

Where TOFA's fee is proportionally reversible:

Original Fee = NGN 50,000

Fee Adjustment = NGN 10,000

Final TOFA Fee = NGN 40,000

The applicable business policy should be implemented by the financial layer.

Analytics must consume the final stored values rather than independently recalculating them.

18.49 Resolution Email — Buyer Favoured

Subject

Dispute Resolved – {DisputeNumber}

Body

Hello {FirstName},

TOFA has completed the review of your dispute regarding Order {OrderNumber}.

Dispute Reference: {DisputeNumber}

Resolution:
{ResolutionSummary}

Where a refund or other financial adjustment is applicable, it will be processed according to the payment method and applicable payment process.

You can view the complete resolution from your TOFA Marketplace account.

Best Regards,
Traders of Africa Team

18.50 Resolution Email — Seller Favoured

Subject

Dispute Resolved – {DisputeNumber}

Body

Hello {FirstName},

TOFA has completed the review of the dispute regarding Order {OrderNumber}.

Dispute Reference: {DisputeNumber}

Resolution:
{ResolutionSummary}

You can view the complete resolution from your TOFA Marketplace account.

Best Regards,
Traders of Africa Team

Both Buyer and Seller should receive the resolution notification.

18.51 Dispute Resolution Notification

Title

Dispute resolved

Message

Dispute {DisputeNumber} for Order {OrderNumber} has been resolved.

Action:

[View Resolution](#)

18.52 Close Dispute

Once:

Resolution Recorded

Required Financial Action Completed or Appropriately Initiated

Required Order/Delivery Actions Completed

the Dispute may move to:

closed

The Dispute record remains permanently available for historical and Audit purposes.

18.53 Duplicate Dispute Prevention

Backend should prevent Users from repeatedly opening equivalent active Disputes for:

Same Seller Order

Same Order Item

Same Reason

while an existing relevant Dispute remains active.

Response:

```
{  
  "success": false,  
  "code": "ACTIVE_DISPUTE_EXISTS",  
  "message": "An active dispute already exists for this issue."  
}
```

Toast

You already have an active dispute for this issue.

18.54 Dispute Timeline

Every Dispute should maintain a chronological timeline.

Example:

27 Aug 2026, 10:15 AM

Buyer opened dispute

27 Aug 2026, 11:02 AM

Dispute assigned to Admin

27 Aug 2026, 1:20 PM

TOFA requested information from Seller

27 Aug 2026, 3:45 PM

Seller submitted evidence

28 Aug 2026, 9:30 AM

Dispute under review

28 Aug 2026, 2:10 PM

Dispute resolved

This helps Users and Admins understand the complete history.

18.55 Dispute Audit Events

Recommended:

DISPUTE_CREATED

DISPUTE_ASSIGNED

DISPUTE_REVIEW_STARTED

DISPUTE_MESSAGE_SENT

DISPUTE_EVIDENCE_ADDED

DISPUTE_BUYER_INFORMATION_REQUESTED

DISPUTE_SELLER_INFORMATION_REQUESTED

DISPUTE_BUYER_RESPONDED

DISPUTE_SELLER_RESPONDED

DISPUTE_RESOLVED

DISPUTE_REFUND_REQUESTED

DISPUTE_CLOSED

Audit records should capture:

actorId

actorType

action

timestamp

previousValue where applicable

newValue where applicable

18.56 Dispute Permissions

Recommended Admin Permissions:

disputes.view

disputes.assign

disputes.manage

disputes.request_information

disputes.resolve

disputes.close

The existing Admin Permission architecture should enforce these Permissions.

Do not create a separate Dispute Admin authentication system.

18.57 User Dashboard

Recommended Dispute list display:

Dispute Reference

Order Reference

Seller / Buyer

Reason

Date Raised

Status

Last Updated

Actions:

View Dispute

Where a response is required:

Action Required

should be clearly visible.

18.58 Order Details Integration

Order details should provide:

Report a Problem

or:

Raise Dispute

only where the authenticated User is eligible.

If a Dispute already exists:

View Dispute

should replace the creation action where appropriate.

Frontend may receive:

```
{
  "dispute": {
    "eligible": true,
    "hasActiveDispute": false,
    "disputelid": null
  }
}
```

Backend remains responsible for final eligibility validation.

18.59 Admin Dashboard

Recommended Dispute dashboard metrics:

Open Disputes

Under Review

Awaiting Buyer

Awaiting Seller

Resolved

Average Resolution Time

These may later feed into the Analytics module.

18.60 Analytics Integration

Analytics may consume:

DISPUTE_CREATED

DISPUTE_RESOLVED

DISPUTE_CLOSED

Recommended Admin metrics:

Total Disputes

Dispute Rate

Disputes by Reason

Disputes by Seller

Buyer-Favour Resolution Rate

Seller-Favour Resolution Rate

Average Resolution Time

Refund Value Resulting From Disputes

Analytics must not independently determine dispute outcomes.

18.61 Dispute Rate

Recommended:

Orders With Disputes

÷

Eligible Orders

×

100

This should be calculated over the selected reporting period.

18.62 Seller Performance

Dispute information may contribute to future Seller performance indicators.

However:

Dispute Created

must not automatically mean:

Seller At Fault

Only resolved outcomes should be considered when determining fault-related Seller performance.

18.63 Review Relationship

Dispute resolution does not automatically create, modify or delete a Review.

The Buyer may still submit a Review according to the existing Review module's eligibility rules.

Admin Review moderation remains separate.

18.64 Message Center Relationship

Normal Message Center conversations remain separate from Dispute conversations.

The Dispute view may optionally provide:

View Conversation With Seller

where the User has access.

However, the formal Dispute response and evidence should remain inside the Dispute record so that the resolution history is auditable.

18.65 Notification Relationship

The Dispute module emits events.

The existing Notification module handles:

In-App Notifications

Email Notifications

The Dispute module should not create another Notification infrastructure.

18.66 Order Relationship

The Dispute module references:

Order

Seller Order

Order Items

Order Status

Order Financial Snapshot

Delivery Method

Shipping Details

It must not duplicate them.

18.67 Payment Relationship

The Dispute module may determine that a resolution requires:

Full Refund

Partial Refund

No Financial Action

The Payment module remains responsible for actual Payment/refund execution.

18.68 Delivery Relationship

The Dispute module reads:

Delivery Method

Logistics Provider

Tracking Information

Shipment Details

Handover Information

Delivery Events

from the existing Order/Delivery architecture.

It does not create a second Logistics lifecycle.

18.69 Subscription Relationship

Subscription level should not prevent a Buyer or Seller from raising a legitimate Dispute.

Dispute access is a transactional protection mechanism and should not be restricted to paid Subscription Plans.

18.70 Search Relationship

A Dispute does not automatically remove a Seller or Product from Search.

If TOFA determines that a Product or Seller requires restriction:

Product Management

Seller Verification/User Management

should perform the appropriate action.

The Dispute module records the case but does not silently alter Search eligibility.

18.71 Backend Implementation Guide

Backend should:

- validate that the User belongs to the affected transaction;
- connect Disputes to Seller Orders;
- support disputes against specific Order items;
- preserve Product snapshots;
- validate dispute eligibility;
- enforce dispute windows;
- prevent duplicate active disputes;
- support evidence;
- maintain a structured Dispute conversation;
- maintain a Dispute timeline;
- allow authorized Admins to manage cases;
- enforce valid status transitions;
- integrate with Payment for refunds;
- integrate with Notifications;
- preserve all historical Dispute information;
- maintain Audit Logs.

Backend should not:

- create another Order lifecycle;
- create another Delivery lifecycle;
- create another Payment system;
- calculate refund FX independently from Payment;
- automatically blame a Seller because a Dispute exists;
- delete historical evidence after resolution.

18.72 Frontend Implementation Guide

Frontend should provide a clear Dispute flow.

Recommended creation flow:

Order Details



Raise Dispute



Select Affected Product(s)



Select Reason



Describe Problem



Upload Evidence



Review



Submit

Dispute details should show:

Dispute Number

Order Number

Affected Products

Reason

Description

Current Status

Timeline

Evidence

Messages

Resolution

When action is required:

TOFA requires additional information from you.

should be prominently displayed.

Frontend must not:

- determine dispute eligibility independently;
 - determine refund amounts;
 - change Payment status;
 - change Order status;
 - change Delivery status;
 - determine the dispute outcome.
-

18.73 Important Consistency Rules

Orders

Dispute status does not replace Order status.

Multi-Seller Orders

Disputes attach to the affected Seller Order.

Products

Use Order Product snapshots when investigating historical transactions.

Delivery

Use the Delivery method and Shipping information already stored against the Order.

Buyer-Arranged Logistics

Seller responsibility should be evaluated based on the agreed handover point and available evidence.

Payments

Refunds are executed through the Payment module.

Currency

Refunds follow the original Buyer Payment currency and Payment module rules.

Transaction Fees

Any fee adjustment resulting from a refund is recorded by the financial layer and consumed by Analytics.

Messaging

Formal Dispute communication remains inside the Dispute case.

Notifications

Use the existing Notification infrastructure.

Reviews

Dispute resolution does not automatically create or modify Reviews.

Subscriptions

Dispute protection is not restricted by Subscription tier.

Analytics

Analytics reads Dispute outcomes but does not determine them.

18.74 Module Summary

The Dispute Management Module establishes that:

- Buyers and Sellers can raise disputes relating to marketplace transactions;
- every Dispute is connected to an existing Order and affected Seller Order;
- multi-Seller Checkout does not create one dispute against every Seller;
- Users can identify specific affected Products and quantities;
- Disputes use Product information snapshotted on the Order;
- dispute reasons are structured and can be configurable;
- dispute eligibility and dispute windows are enforced by backend;
- Users can upload supporting evidence;
- Dispute communication is separate from the normal Message Center;
- TOFA Admin can request additional information from either party;
- Dispute status is separate from Order status;
- the existing Order lifecycle is not duplicated;
- the existing Delivery lifecycle is not duplicated;
- Buyer-, Seller- and integrated-Logistics scenarios use existing Delivery records;

- Buyer-arranged Logistics considers the Seller's handover to the Buyer's nominated provider;
- Seller-arranged Logistics uses Shipping details already provided when the Seller marks the Order as shipped;
- Admin can resolve a Dispute fully or partially;
- a Dispute resolution may require no financial action, a full refund or a partial refund;
- actual refunds are processed through the Payment module;
- refunds use the appropriate original Buyer Payment currency;
- TOFA cannot automatically refund off-platform Logistics amounts it did not collect;
- transaction fee adjustments resulting from refunds are handled by the financial layer;
- Analytics consumes final stored transaction fee values;
- dispute creation does not automatically mean the Seller is at fault;
- a Dispute does not automatically remove a Product or Seller from Search;
- Subscription level does not restrict legitimate dispute access;
- all Dispute actions and evidence remain auditable;
- Users receive email and in-app notifications for important Dispute events;
- Admin access is controlled through the existing Permission architecture.

MODULE 19. ADMIN MANAGEMENT

19. ADMIN MANAGEMENT & PERMISSIONS MODULE

19.1 Overview

The Admin Management & Permissions Module controls administrative access to TOFA Marketplace.

This module expands the basic Admin creation and Admin authentication information already introduced in Module 1.

Module 1 remains responsible for the core distinction between normal Marketplace Users and Admin Users, while this module provides the complete structure for:

- Super Admin management;
- Admin User management;
- Admin account lifecycle;
- Admin Roles;
- Permissions;
- Role and Permission assignment;
- Admin access control;
- Admin security;
- Admin account activation and deactivation;
- Admin password management;
- Admin profile management;
- administrative access to marketplace modules;
- Admin action traceability;
- Audit integration.

The core principle is:

Admin
↓
Role
↓
Permissions
↓
Allowed Administrative Actions

A Role is used to group Permissions.

The Role name itself must not determine what an Admin can do.

Example:

Do not implement:

```
if role == "finance"
    allow payment verification
```

Instead:

```
Admin
  ↓
Assigned Role
  ↓
payments.verify
```

The Permission determines whether the Admin can perform the action.

19.2 Admin Types

The system supports:

Super Admin

Admin

Super Admin

Super Admin has overall administrative authority.

Super Admin can:

- create Admin Users;
- update Admin Users;
- deactivate Admin Users;
- reactivate Admin Users;
- create Admin Roles;
- update Admin Roles;
- manage Role Permissions;
- view Permissions;
- access all administrative functions;

- manage sensitive administrative configuration;
- review Audit records where permitted by the Audit module.

Recommended field:

isSuperAdmin = true

Admin

Admin Users are created by Super Admin.

An Admin's access is determined by:

Assigned Role

+

Permissions assigned to that Role

Example:

Role:

Finance Officer

Permissions:

payments.view

payments.verify

analytics.view_revenue

subscriptions.view

The Admin can only perform administrative actions permitted by those Permissions.

19.3 Admin Creation Rule

Normal Users cannot register themselves as Admins.

There must be no public:

POST /admin/signup

endpoint.

Admin Users are created only by Super Admin.

Recommended flow:

Super Admin creates Admin



Admin record created



status = pending



Role assigned



Invitation email sent



Admin creates password



Account activated



Admin can login

Super Admin should not need to know or create the Admin's final password.

19.4 Admin Table Structure

Table: **admins**

Field	Type	Required	Notes
id	uuid	Yes	Primary key
firstName	varchar	Yes	Admin first name
lastName	varchar	Yes	Admin last name
email	varchar	Yes	Unique
phoneNumber	varchar	No	Admin phone number
passwordHash	varchar	No	Null until setup completed
roleId	uuid	No	References admin_roles
isSuperAdmin	boolean	Yes	Default false
status	enum	Yes	Admin account status
lastLoginAt	timestamp	No	Last successful login

failedLoginAttempts	integer	Yes	Default 0
lockedUntil	timestamp	No	Temporary login lock
createdBy	uuid	No	Super Admin that created account
deactivatedBy	uuid	No	Admin/Super Admin that deactivated
deactivatedAt	timestamp	No	Deactivation date
deactivationReason	text	No	Reason for deactivation
createdAt	timestamp	Yes	Creation date
updatedAt	timestamp	Yes	Last update

Passwords must never be returned through APIs.

19.5 Admin Status

Recommended statuses:

pending

active

inactive

pending

Admin account has been created but setup has not been completed.

The Admin cannot login until setup is completed.

active

Admin account is active.

The Admin can login and perform actions permitted by their Permissions.

inactive

Admin account has been deactivated.

The Admin cannot login or perform Admin actions.

Admin accounts should not be hard deleted because historical marketplace and Audit records may reference them.

19.6 Admin Role Structure

Table: `admin_roles`

Field	Type	Required	Notes
id	uuid	Yes	Primary key
name	varchar	Yes	Unique Role name
description	text	No	Role description
status	enum	Yes	active / inactive
createdBy	uuid	Yes	Admin that created Role
createdAt	timestamp	Yes	Creation date
updatedAt	timestamp	Yes	Last update

Example Role names may include:

Operations Manager

Finance Officer

Marketplace Manager

Customer Support

Verification Officer

Dispute Officer

These are examples only.

Roles must remain dynamically configurable.

19.7 Role Principle

The backend must not determine access using Role names.

Do not implement:

```
if role.name == "Operations Manager"  
    allow order management
```

Instead:

```
Admin  
↓  
Role  
↓  
orders.manage
```

This allows TOFA to change Role names or create new Roles without changing backend authorization logic.

19.8 Permission Structure

Table: **permissions**

Field	Type	Required	Notes
id	uuid	Yes	Primary key
code	varchar	Yes	Unique Permission code
name	varchar	Yes	Human-readable name
description	text	Yes	What the Permission allows
module	varchar	Yes	Related marketplace module
status	enum	Yes	active / inactive
createdAt	timestamp	Yes	Creation date
updatedAt	timestamp	Yes	Last update

Recommended Permission naming convention:

module.action

Examples:

users.view

users.update

users.disable

users.activate

sellers.view

sellers.verify

sellers.reject

products.view

products.update

products.activate

products.deactivate

products.archive

categories.view

categories.create

categories.update

categories.activate

categories.deactivate

categories.archive

orders.view

orders.manage

orders.cancel

payments.view

payments.verify

payments.reject

rfqs.view

rfqs.manage

reviews.view

reviews.moderate

subscriptions.view

subscriptions.manage

subscription_plans.view

subscription_plans.create

subscription_plans.update

subscription_plans.activate

subscription_plans.deactivate

analytics.view_overview

analytics.view_users

analytics.view_sellers

analytics.view_buyers

analytics.view_products

analytics.view_orders

analytics.view_payments

analytics.view_rfqs

analytics.view_revenue

analytics.view_subscriptions

analytics.view_logistics

analytics.export_reports

analytics.rebuild

disputes.view

disputes.assign

disputes.manage

disputes.request_information

disputes.resolve

disputes.close

admins.view

admins.create

admins.update

admins.activate

admins.deactivate

roles.view

roles.create

roles.update

roles.deactivate

permissions.view

permissions.assign

Additional Permissions can be added as new Marketplace modules are introduced.

19.9 Role Permission Structure

Table: `admin_role_permissions`

Field	Type	Required
id	uuid	Yes
roleId	uuid	Yes
permissionId	uuid	Yes
createdAt	timestamp	Yes

This creates the relationship:

Admin Role
↓
Multiple Permissions

Example:

Finance Officer

payments.view
payments.verify
analytics.view_revenue
subscriptions.view

19.10 Super Admin Access

Super Admin is the exception to normal Role Permission resolution.

Recommended backend logic:

```
if admin.isSuperAdmin == true  
    allow administrative action
```

Super Admin does not need every Permission individually attached.

However, Super Admin actions should still be logged for Audit purposes.

19.11 Create Admin

Endpoint

POST /admin/admin-users

Authentication

Super Admin only

Payload

```
{
  "firstName": "Joy",
  "lastName": "Okafor",
  "email": "joy@tofa.com",
  "phoneNumber": "+2348012345678",
  "roleId": "role_operations"
}
```

Backend Processing

1. Authenticate Super Admin.
2. Validate payload.
3. Confirm email is not already registered as an Admin.
4. Validate selected Role.
5. Create Admin record.
6. Set:

status = pending

7. Generate one-time setup token.
8. Set token expiry.
9. Send Admin invitation email.
10. Create Audit event.

Recommended setup token expiry:

24 hours

19.12 Create Admin Response

```
{
  "success": true,

  "message": "Admin account created successfully.",

  "data": {
    "id": "admin_102",

    "firstName": "Joy",

    "lastName": "Okafor",

    "email": "joy@tofa.com",

    "status": "pending",

    "role": {
      "id": "role_operations",
      "name": "Operations Manager"
    }
  }
}
```

Toast

Admin account created successfully. An invitation has been sent to the Admin.

19.13 Admin Invitation Email

Subject

You've Been Invited to TOFA Marketplace Admin

Body

Hello {FirstName},

You have been granted administrative access to the Traders of Africa Marketplace.

Role: {RoleName}

Please use the link below to complete your Admin account setup and create your password.

{SetupLink}

For security purposes, this link will expire in 24 hours and can only be used once.

If you were not expecting this invitation, please contact the TOFA team.

Best Regards,
Traders of Africa Team

19.14 Complete Admin Setup

Endpoint

POST /admin/auth/complete-setup

Payload

```
{  
  "token": "ADMIN_SETUP_TOKEN",  
  "password": "AdminPass123"  
}
```

Validation Rules

Token must be valid

Token must not be expired

Token must not already be used

Password must be at least 8 characters

Password must satisfy configured complexity rules

19.15 Complete Admin Setup Backend Processing

1. Validate setup token.
2. Retrieve Admin.

3. Confirm:

status = pending

4. Validate password.
5. Hash password.
6. Store passwordHash.
7. Mark setup token as used.
8. Set:

status = active

9. Store Audit event.

19.16 Complete Admin Setup Response

```
{  
  "success": true,  
  "message": "Your Admin account has been activated successfully."  
}
```

Toast

Your Admin account has been activated successfully. You can now sign in.

19.17 Admin Login

All active Admin Users can login.

Admin Login is not restricted to Super Admin.

Endpoint

POST /admin/auth/login

Payload

```
{  
  "email": "joy@tofa.com",  
  "password": "AdminPass123"  
}
```

19.18 Admin Login Backend Processing

1. Validate credentials.
2. Retrieve Admin.
3. Confirm account status.
4. Verify password.
5. Check temporary account lock.
6. Load Admin Role.
7. Load effective Permissions.
8. Generate authentication token.
9. Reset failed login attempts.
10. Update `lastLoginAt`.
11. Return Admin details and Permissions.

19.19 Admin Login Response

```
{
  "success": true,

  "message": "Login successful.",

  "data": {
    "token": "JWT_TOKEN",

    "admin": {
      "id": "admin_102",

      "firstName": "Joy",

      "lastName": "Okafor",

      "phoneNumber": "+2348012345678",

      "email": "joy@tofa.com",

      "status": "active",

      "isSuperAdmin": false,

      "role": {
        "id": "role_operations",
```



```

    "name": "Operations Manager"
  },

  "permissions": [
    "users.view",
    "sellers.view",
    "orders.view",
    "orders.manage"
  ],

  "lastLoginAt": "2026-08-28T08:00:00Z",

  "createdAt": "2026-08-01T10:00:00Z"
}
}
}

```

19.20 Admin Login Security

Recommended:

Rule	Value
Maximum failed login attempts	5
Temporary account lock	15 minutes
Password hashing	secure hashing
Admin setup token	one-time
Admin setup token expiry	24 hours
Session/JWT expiry	configurable

After 5 consecutive failed login attempts:

`lockedUntil = currentTime + 15 minutes`

Response should not expose sensitive authentication details.

19.21 Get Current Admin

Endpoint

GET /admin/auth/me

Authentication

Admin JWT required.

Response

```
{
  "success": true,

  "data": {
    "id": "admin_102",

    "firstName": "Joy",

    "lastName": "Okafor",

    "phoneNumber": "+2348012345678",

    "email": "joy@tofa.com",

    "status": "active",

    "isSuperAdmin": false,

    "role": {
      "id": "role_operations",
      "name": "Operations Manager"
    },

    "permissions": [
      "users.view",
      "sellers.view",
      "orders.view",
      "orders.manage"
    ],

    "lastLoginAt": "2026-08-28T08:00:00Z"
  }
}
```

```
}
```

Frontend should use **permissions** to determine which pages and actions are displayed.

Backend remains authoritative.

19.22 Get Admin Users

Endpoint

GET /admin/admin-users

Required Permission

admins.view

Super Admin automatically has access.

Query Parameters

search

status

roleId

page

limit

Response

```
{
  "success": true,

  "data": [
    {
      "id": "admin_102",

      "firstName": "Joy",

      "lastName": "Okafor",

      "email": "joy@tofa.com",

      "phoneNumber": "+2348012345678",
```

```
"status": "active",

"isSuperAdmin": false,

"role": {
  "id": "role_operations",
  "name": "Operations Manager"
},

"lastLoginAt": "2026-08-28T08:00:00Z",

"createdAt": "2026-08-01T10:00:00Z"
},
],

"pagination": {
  "page": 1,
  "limit": 20,
  "total": 12,
  "totalPages": 1
}
}
```

19.23 Get Admin Details

Endpoint

GET /admin/admin-users/:adminId

Required Permission

admins.view

Response

```
{
  "success": true,

  "data": {
    "id": "admin_102",
```

```
"firstName": "Joy",

"lastName": "Okafor",

"email": "joy@tofa.com",

"phoneNumber": "+2348012345678",

"status": "active",

"isSuperAdmin": false,

"role": {
  "id": "role_operations",
  "name": "Operations Manager"
},

"permissions": [
  "users.view",
  "sellers.view",
  "orders.view",
  "orders.manage"
],

"lastLoginAt": "2026-08-28T08:00:00Z",

"createdAt": "2026-08-01T10:00:00Z"
}
}
```

19.24 Update Admin Profile

An Admin should be able to update their own basic profile information.

Endpoint

PATCH /admin/profile

Payload

```
{
  "firstName": "Joy",
  "lastName": "Okafor",
```

```
"phoneNumber": "+2348012345678"
}
```

Only provided fields should be updated.

An Admin must not use this endpoint to change:

roleId

permissions

isSuperAdmin

status

Response

```
{
  "success": true,
  "message": "Admin profile updated successfully."
}
```

Toast

Profile updated successfully.

19.25 Update Another Admin

Endpoint

PATCH /admin/admin-users/:adminId

Required Permission

admins.update

Payload

```
{
  "firstName": "Joy",
  "lastName": "Okafor",
}
```

```
"phoneNumber": "+2348012345678",  
"roleId": "role_marketplace_manager"  
}
```

Backend must prevent unauthorized Super Admin modification.

19.26 Admin Role Change Rule

When an Admin's Role changes:

Previous Role



New Role Assigned



Previous Role Permissions Stop Applying



New Role Permissions Become Effective

The Admin must not retain old Role Permissions unless the new Role also contains them.

For sensitive Role changes, backend may invalidate existing Admin sessions.

19.27 Update Admin Response

```
{  
  "success": true,  
  "message": "Admin account updated successfully."  
}
```

Toast

Admin account updated successfully.

19.28 Deactivate Admin

Endpoint

PATCH /admin/admin-users/:adminId/deactivate

Authentication

Super Admin by default.

TOFA may later allow another Admin with:

admins.deactivate

if business policy permits it.

Payload

```
{  
  "reason": "Left the organization"  
}
```

19.29 Deactivate Admin Backend Processing

1. Validate requesting Admin.
2. Retrieve target Admin.
3. Prevent unauthorized deactivation of protected Super Admin accounts.
4. Set:

status = inactive

5. Store:

deactivatedBy

deactivatedAt

deactivationReason

6. Invalidate existing Admin sessions.
7. Store Audit event.
8. Send deactivation email.

19.30 Deactivate Admin Response

```
{  
  "success": true,
```



```
"message": "Admin account deactivated successfully."
}
```

Toast

Admin account deactivated successfully.

19.31 Admin Deactivation Email

Subject

Your TOFA Marketplace Admin Access Has Been Deactivated

Body

Hello {FirstName},

Your administrative access to the Traders of Africa Marketplace has been deactivated.

You will no longer be able to access the Admin platform.

If you believe this action was taken in error or require clarification, please contact the appropriate TOFA administrator.

Best Regards,
Traders of Africa Team

19.32 Reactivate Admin

Endpoint

PATCH /admin/admin-users/:adminId/activate

Authentication

Super Admin or an Admin with the required Permission where permitted.

Response

```
{
  "success": true,
```

```
"message": "Admin account activated successfully."
}
```

Toast

Admin account activated successfully.

19.33 Admin Reactivation Email

Subject

Your TOFA Marketplace Admin Access Has Been Restored

Body

Hello {FirstName},

Your administrative access to the Traders of Africa Marketplace has been restored.

You can now sign in and access the Admin features permitted for your account.

Best Regards,
Traders of Africa Team

19.34 Create Admin Role

Endpoint

POST /admin/roles

Required Permission

roles.create

Super Admin automatically has access.

Payload

```
{
  "name": "Finance Officer",
```

```
"description": "Responsible for payment verification and financial administration.",

"permissionIds": [
  "perm_payments_view",
  "perm_payments_verify",
  "perm_analytics_revenue"
]
}
```

19.35 Create Admin Role Backend Processing

1. Validate Permission.
2. Validate Role name.
3. Ensure Role name is unique.
4. Validate all Permission IDs.
5. Create Role.
6. Create Role Permission relationships.
7. Store Audit event.

19.36 Create Role Response

```
{
  "success": true,

  "message": "Admin role created successfully.",

  "data": {
    "id": "role_finance",
    "name": "Finance Officer",
    "status": "active"
  }
}
```

Toast

Admin role created successfully.

19.37 Get Admin Roles

Endpoint

GET /admin/roles

Required Permission

roles.view

Query Parameters

status
search
page
limit

Response

```
{
  "success": true,

  "data": [
    {
      "id": "role_finance",

      "name": "Finance Officer",

      "description": "Responsible for payment verification and financial administration.",

      "status": "active",

      "permissions": [
        {
          "id": "perm_payments_view",
          "code": "payments.view",
          "name": "View Payments"
        },
        {
          "id": "perm_payments_verify",
          "code": "payments.verify",
          "name": "Verify Payments"
        }
      ]
    }
  ]
}
```

```
]
}
```

19.38 Get Admin Role Details

Endpoint

GET /admin/roles/:roleId

Required Permission

roles.view

Response

```
{
  "success": true,

  "data": {
    "id": "role_finance",

    "name": "Finance Officer",

    "description": "Responsible for payment verification and financial administration.",

    "status": "active",

    "permissions": [
      {
        "id": "perm_payments_view",
        "code": "payments.view",
        "name": "View Payments"
      },
      {
        "id": "perm_payments_verify",
        "code": "payments.verify",
        "name": "Verify Payments"
      }
    ]
  }
}
```

19.39 Update Admin Role

Endpoint

PATCH /admin/roles/:roleId

Required Permission

roles.update

Payload

```
{
  "name": "Senior Finance Officer",

  "description": "Responsible for financial administration and reporting.",

  "permissionIds": [
    "perm_payments_view",
    "perm_payments_verify",
    "perm_analytics_revenue",
    "perm_subscriptions_view"
  ]
}
```

Backend Processing

1. Validate Role.
2. Validate Permission.
3. Update supplied Role fields.
4. Validate Permission IDs.
5. Replace/update Role Permission relationships.
6. Recalculate effective Permissions for Admins using that Role.
7. Store Audit event.

Existing sessions should not be allowed to continue indefinitely with removed Permissions.

19.40 Update Role Response

```
{
  "success": true,
  "message": "Admin role updated successfully."
}
```

Toast

Admin role updated successfully.

19.41 Deactivate Admin Role

Roles referenced by historical Admin records should not normally be hard deleted.

Endpoint

PATCH /admin/roles/:roleId/status

Required Permission

roles.deactivate

Payload

```
{  
  "status": "inactive"  
}
```

An inactive Role cannot be newly assigned.

Admin Users currently assigned to an inactive Role must be handled deliberately.

Recommended rule:

Role cannot be deactivated while active Admins are assigned to it

unless those Admins are first reassigned.

Response

```
{  
  "success": true,  
  "message": "Admin role deactivated successfully."  
}
```

19.42 Get Permissions

Endpoint

GET /admin/permissions

Required Permission

permissions.view

Permissions should be grouped by Marketplace module.

Response

```
{
  "success": true,

  "data": [
    {
      "module": "payments",

      "permissions": [
        {
          "id": "perm_payments_view",
          "code": "payments.view",
          "name": "View Payments",
          "description": "Allows the Admin to view Marketplace payments."
        },

        {
          "id": "perm_payments_verify",
          "code": "payments.verify",
          "name": "Verify Payments",
          "description": "Allows the Admin to verify manual payments."
        }
      ]
    },

    {
      "module": "products",

      "permissions": [
        {
          "id": "perm_products_view",
```



```
    "code": "products.view",  
    "name": "View Products"  
  }  
]  
}  
]  
}
```

19.43 Permission Management Principle

Core Permission codes should normally be controlled by the backend/application because each Permission corresponds to an actual protected action.

Admin should not create arbitrary Permission codes that have no corresponding backend authorization rule.

Recommended:

Backend/Application

↓

Defines Available Permission Codes

↓

Admin

↓

Assigns Permissions to Roles

This prevents invalid Permissions such as:

payments.make_me_super_admin

from being created through the UI.

19.44 Assign Permissions to Role

Role Permissions can be managed through the Role update endpoint.

Alternatively, where a dedicated endpoint is preferred:

Endpoint

PUT /admin/roles/:roleId/permissions

Required Permission

permissions.assign

Payload

```
{
  "permissionIds": [
    "perm_orders_view",
    "perm_orders_manage",
    "perm_disputes_view"
  ]
}
```

The payload represents the complete intended Permission set for that Role.

Response

```
{
  "success": true,
  "message": "Role permissions updated successfully."
}
```

19.45 Admin Disable User

The User status lifecycle remains owned by Module 1.

User statuses remain:

active

inactive

disabled

deleted

An authorized Admin may disable a User.

Endpoint

PATCH /admin/users/:userId/disable

Required Permission

users.disable

Payload

```
{  
  "reason": "Fraudulent activities"  
}
```

Backend Processing

Set:

status = disabled

Store:

reason

disabledBy

disabledAt

Disabled User behaviour remains as defined in Module 1:

User may authenticate,
but cannot perform Marketplace actions.

19.46 Admin Disable User Response

```
{  
  "success": true,  
  
  "message": "User account disabled successfully.",  
  
  "data": {  
    "userId": "usr_112",  
    "status": "disabled"  
  }  
}
```

Toast

User account disabled successfully.

19.47 User Restriction Email

Subject

Your Traders of Africa Account Has Been Restricted

Body

Hello {FirstName},

Your Traders of Africa Marketplace account has been restricted.

Reason:

{Reason}

While your account is restricted, you may be unable to perform Marketplace actions.

If you believe this action was taken in error, please contact the Traders of Africa support team.

Best Regards,
Traders of Africa Team

19.48 Reactivate Disabled User

Endpoint

PATCH /admin/users/:userId/activate

Required Permission

users.activate

Backend Processing

Where the User satisfies the required account conditions:

status = active

The backend must not reactivate an account incorrectly where:

- Email verification is incomplete;
- the account was soft deleted by the User;
- other restrictions prevent reactivation.

Response

```
{  
  "success": true,  
  "message": "User account activated successfully."  
}
```

19.49 Admin Change Password

Authenticated Admins can change their own password.

Endpoint

PATCH /admin/auth/change-password

Payload

```
{  
  "oldPassword": "OldAdminPass123",  
  "newPassword": "NewAdminPass123"  
}
```

Rules

Old password must be correct

New password must be at least 8 characters

New password must satisfy configured password complexity

Response

```
{  
  "success": true,  
  "message": "Password changed successfully."  
}
```

Toast

Your password has been changed successfully.

19.50 Admin Password Changed Email

Subject

Your TOFA Admin Password Has Been Changed

Body

Hello {FirstName},

The password for your TOFA Marketplace Admin account has been changed successfully.

If you made this change, no further action is required.

If you did not make this change, please contact the appropriate TOFA administrator immediately.

Best Regards,
Traders of Africa Team

19.51 Admin Forgot Password

Endpoint

POST /admin/auth/forgot-password

Payload

```
{  
  "email": "joy@tofa.com"  
}
```

The API should not expose whether the Admin email exists.

Response

```
{  
  "success": true,  
  "message": "If an eligible Admin account exists for this email, password reset instructions have  
been sent."  
}
```

19.52 Admin Password Reset Email

Subject

TOFA Admin Password Reset

Body

Hello {FirstName},

We received a request to reset the password for your TOFA Marketplace Admin account.

Your password reset code is:

{OTP}

This code will expire in 10 minutes.

If you did not request this password reset, please ignore this email and contact the appropriate TOFA administrator if you notice any suspicious activity.

Best Regards,
Traders of Africa Team

19.53 Reset Admin Password

Endpoint

POST /admin/auth/reset-password

Payload

```
{  
  "otp": "123456",  
  "password": "NewAdminPass123"  
}
```

Response

```
{  
  "success": true,  
  "message": "Password reset successful."  
}
```

Toast

Password reset successfully.

19.54 Admin Permission Enforcement

Every protected Admin endpoint must enforce Permissions on the backend.

Example:

POST /admin/subscription-plans

requires:

subscription_plans.create

Example:

POST /admin/disputes/:disputeId/resolve

requires:

disputes.resolve

Example:

POST /admin/categories

requires:

categories.create

Frontend Permission checks are not sufficient security.

A User must not be able to bypass Admin restrictions by manually calling an endpoint.

19.55 Permission Denied Response

Recommended HTTP status:

403 Forbidden

Response:

```
{
  "success": false,

  "code": "ADMIN_PERMISSION_DENIED",

  "message": "You do not have permission to perform this action."
}
```

Toast

You do not have permission to perform this action.

19.56 Inactive Admin Response

If an inactive Admin tries to login:

```
{
  "success": false,

  "code": "ADMIN_ACCOUNT_INACTIVE",

  "message": "Your Admin account is currently inactive."
}
```

19.57 Pending Admin Response

If an Admin has not completed account setup:

```
{
  "success": false,

  "code": "ADMIN_SETUP_REQUIRED",

  "message": "Please complete your Admin account setup before signing in."
}
```

Where the original setup link has expired, an authorized Super Admin should be able to resend an invitation.

19.58 Resend Admin Invitation

Endpoint

POST /admin/admin-users/:adminId/resend-invitation

Authentication

Super Admin.

Conditions

Recommended:

Admin status must be pending

Backend should:

1. invalidate previous unused setup token;
2. generate a new setup token;
3. set a new expiry;
4. resend invitation email;
5. store Audit event.

Response

```
{  
  "success": true,  
  "message": "Admin invitation resent successfully."  
}
```

Toast

Admin invitation resent successfully.

19.59 Admin Account Security

Recommended security controls include:

- secure password hashing;
- login rate limiting;
- temporary lock after repeated failed attempts;
- secure JWT/session handling;
- setup token expiration;
- password reset OTP expiration;
- session invalidation after deactivation;
- session invalidation after sensitive security changes where appropriate;
- backend Permission validation;
- logging of sensitive Admin actions.

Future MFA support can be added without changing the Role and Permission architecture.

19.60 Admin Activity Traceability

Sensitive administrative actions should identify the Admin that performed them.

Examples:

Payment

verifiedBy
verifiedAt

Seller Verification

verifiedBy
verifiedAt

Dispute Resolution

resolvedBy
resolvedAt

User Disable

disabledBy
disabledAt
reason

Subscription Plan Update

updatedBy
updatedAt

This allows TOFA to trace important administrative activity.

19.61 Admin Audit Events

Recommended events include:

ADMIN_CREATED

ADMIN_INVITATION_SENT

ADMIN_INVITATION_RESENT

ADMIN_SETUP_COMPLETED

ADMIN_LOGIN_SUCCESSFUL

ADMIN_LOGIN_FAILED

ADMIN_ACCOUNT_LOCKED

ADMIN_PROFILE_UPDATED

ADMIN_UPDATED

ADMIN_ROLE_CHANGED

ADMIN_DEACTIVATED

ADMIN_REACTIVATED

ADMIN_PASSWORD_CHANGED

ADMIN_PASSWORD_RESET

ADMIN_ROLE_CREATED

ADMIN_ROLE_UPDATED

ADMIN_ROLE_DEACTIVATED

ADMIN_ROLE_PERMISSIONS_UPDATED

USER_DISABLED_BY_ADMIN

USER_REACTIVATED_BY_ADMIN

Other Marketplace modules may produce additional Admin-related Audit events such as:

PAYMENT_VERIFIED_BY_ADMIN

SELLER_VERIFIED_BY_ADMIN

SELLER_REJECTED_BY_ADMIN

PRODUCT_DEACTIVATED_BY_ADMIN

DISPUTE_RESOLVED_BY_ADMIN

SUBSCRIPTION_PLAN_UPDATED_BY_ADMIN

The Audit Log module should later centralize storage and viewing of these records.

19.62 Frontend Admin Access Control

Frontend should retrieve the authenticated Admin from:

GET /admin/auth/me

and use:

permissions[]

to control:

- Admin navigation;
- available pages;
- buttons;
- form actions;
- settings;
- sensitive actions.

Example:

If the Admin does not have:

payments.verify

the frontend should not display:

Verify Payment

However, backend must still validate the Permission.

19.63 Permission-Based Navigation Example

Example Admin:

```
{
  "permissions": [
    "orders.view",
    "orders.manage",
    "disputes.view"
  ]
}
```

Frontend may display:

Dashboard

Orders

Disputes

but hide:

Payments

Subscription Plans

Admin Users

Roles

where the Admin lacks those Permissions.

19.64 Role Changes and Active Sessions

Permissions may change while an Admin is logged in.

Backend should not rely indefinitely on Permission data stored only in an old token.

Recommended approaches include:

- resolving current Permissions from backend on protected actions;
- using short-lived access tokens;
- refreshing Permission information regularly;
- invalidating sessions after sensitive Role changes.

The implementation may depend on the final authentication architecture.

19.65 Admin and User Separation

Normal Marketplace Users and Admin Users should remain logically separate.

Recommended:

users

for:

buyers
sellers

and:

admins

for administrative accounts.

A normal User account should not become an Admin merely by changing:

userType

Admin access requires an Admin record and Admin authentication.

19.66 User Type vs Admin Role

Normal User types remain:

buyer
seller

Admin Roles belong only to Admin accounts.

Do not use:

userType = finance

or:

userType = admin

for Admin authorization.

Similarly, Admin Roles should not contain:

buyer
seller

as administrative access roles.

19.67 Admin and Seller Verification Relationship

Seller Verification remains owned by the Seller/Business Verification module.

Admin Management only determines whether an Admin can perform the verification action.

Example:

sellers.verify

The Verification module remains responsible for:

- verification requirements;
- evidence;
- approval;
- rejection;
- verification status.

19.68 Admin and Product Management Relationship

Product Management remains responsible for Product lifecycle and Product actions.

Admin Management determines whether an Admin can perform applicable Admin Product actions.

Example:

products.view

products.update

products.deactivate

Admin Management must not introduce a separate Product status lifecycle.

19.69 Admin and Category Management Relationship

Category Management remains responsible for Category records.

Admin Permissions may include:

categories.view

categories.create

categories.update

categories.archive

Admin Management controls authorization only.

19.70 Admin and Payment Relationship

Payment Management remains responsible for Payment status and verification.

Admin Management determines whether the Admin can perform actions such as:

payments.view

payments.verify

payments.reject

Admin Management must not independently update Payment financial logic.

19.71 Admin and Order Relationship

Order Management remains responsible for Order lifecycle.

Admin Management may provide Permissions such as:

orders.view

orders.manage

orders.cancel

where the Order module permits the corresponding Admin action.

Admin Permission does not create a new Order action that does not already exist in Order Management.

19.72 Admin and Subscription Relationship

Subscription Management remains responsible for:

- Subscription Plans;
- Plan prices;
- entitlements;
- transaction fee percentages;
- User Subscriptions.

Admin Management controls authorization through Permissions such as:

subscription_plans.view

subscription_plans.create

subscription_plans.update

subscription_plans.activate

subscription_plans.deactivate

19.73 Admin and Analytics Relationship

Analytics remains responsible for reporting.

Admin access may be controlled using:

analytics.view_overview

analytics.view_revenue

analytics.view_payments

analytics.export_reports

An Admin may therefore have access to operational Analytics without access to sensitive financial Analytics.

Example:

Operations Admin

analytics.view_overview
analytics.view_orders

but not:

analytics.view_revenue

19.74 Admin and Dispute Relationship

The Dispute module remains responsible for Dispute lifecycle.

Admin Permissions may include:

disputes.view

disputes.assign

disputes.manage

disputes.request_information

disputes.resolve

disputes.close

Example:

A Customer Support Admin may have:

disputes.view

disputes.request_information

but not:

disputes.resolve

This allows TOFA to separate case investigation from final resolution authority.

19.75 Admin and Notification Relationship

The existing Notification module handles delivery of:

Email Notifications

In-App Notifications

Admin Management defines the event that triggers the Notification where applicable.

Examples:

Admin Created

Admin Deactivated

Admin Reactivated

Password Changed

It should not create a separate Notification infrastructure.

19.76 Admin and Audit Relationship

Admin Management emits Admin activity events.

The Audit Log module should later provide:

- centralized Audit storage;
- Admin activity history;
- filtering;
- investigation;
- historical changes;
- sensitive action tracking.

Admin Management itself should still preserve relevant fields such as:

createdBy

updatedBy

deactivatedBy

where appropriate.

19.77 Admin Dashboard Recommendations

The Admin dashboard may show only the information the Admin has Permission to access.

Example:

Operations Admin

May see:

Orders

Products

Seller Verification

Disputes

Finance Admin

May see:

Payments

Subscription Revenue

Transaction Fee Revenue

Financial Analytics

Customer Support

May see:

Users

Orders

Disputes

Message Support Information

The frontend must derive these views from Permissions rather than Role names.

19.78 Super Admin Dashboard

Super Admin may access:

- all Admin management;
- Role management;
- Permission assignment;
- Users;
- Sellers;
- Products;
- Categories;
- Orders;
- Payments;
- RFQs;
- Reviews;
- Subscriptions;
- Analytics;
- Disputes;
- system settings;
- Audit Logs;
- future administrative modules.

19.79 Backend Implementation Guide

Backend should:

- keep Admin accounts separate from normal Users;
- allow Admin creation only through Super Admin;
- use secure invitation/setup flow;
- store Admin Roles separately;
- store Permissions separately;
- assign Permissions to Roles;
- resolve effective Permissions dynamically;
- recognize Super Admin separately;
- validate Permissions on protected Admin endpoints;
- invalidate Admin sessions after deactivation;
- protect sensitive Super Admin operations;
- prevent hard deletion of historically referenced Admin accounts;
- maintain Admin security controls;
- emit Audit events;
- preserve the Admin responsible for important administrative actions.

Backend should not:

- hardcode access based on Role names;
- use normal User `userType` as Admin authorization;
- trust frontend Permission checks;
- allow public Admin signup;
- allow unauthorized Admin Role changes;
- allow inactive Admins to access protected APIs;
- duplicate the business logic of Product, Payment, Order, Subscription, Dispute or other modules.

19.80 Frontend Implementation Guide

Frontend should:

- provide Admin Login;
- support Admin account setup;
- support forgot/reset password;
- retrieve current Admin;
- render navigation according to Permissions;
- hide unavailable actions;
- provide Admin User management to authorized Users;
- provide Role management to authorized Users;
- group Permissions by module;
- clearly display Admin status;
- display confirmation before sensitive actions such as deactivation;
- show loading, empty and error states;
- refresh Permission information after Role changes where appropriate.

Frontend should not:

- determine authoritative access based on Role name;
- assume hidden buttons are sufficient security;
- create Admin accounts without backend approval;
- directly modify Permissions outside authorized endpoints.

19.81 Important Consistency Rules

- Admin Users are separate from normal Marketplace Users.
- Normal Buyers and Sellers cannot self-register as Admins.
- Super Admin creates Admin Users.
- Admins create their own password through secure setup.
- Admin Role names are dynamic.
- Role names do not determine backend access.
- Permissions determine Admin actions.

- Roles group Permissions.
- Super Admin has full administrative authority.
- Every sensitive Admin endpoint must enforce Permission checks.
- Frontend Permission checks are for UX only.
- Admin status is separate from User status.
- Admin accounts should not be hard deleted where historical records reference them.
- Deactivated Admin sessions should be invalidated.
- Role changes should take effect without requiring backend code changes.
- Permission codes should correspond to actual backend capabilities.
- Arbitrary Permissions should not be created through the Admin UI.
- Each Marketplace module continues to own its own business logic.
- Admin Management only determines whether an Admin is authorized to perform an action.
- Sensitive Admin actions should identify the Admin who performed them.
- Audit events should be emitted for important Admin activity.

19.82 Module Summary

The Admin Management & Permissions Module establishes that:

- TOFA has separate Admin accounts from normal Marketplace User accounts;
- Super Admin creates Admin Users;
- there is no public Admin registration;
- newly created Admins begin with **pending** status;
- Admins receive secure setup invitations;
- Admins create their own passwords;
- active Admins use a dedicated Admin authentication flow;
- Admin login is available to all active Admins and is not Super Admin only;
- Admin accounts support **pending**, **active** and **inactive** statuses;
- Admin accounts are not hard deleted where historical references exist;
- Super Admin has platform-wide administrative authority;
- normal Admin access is Permission-based;
- Admin Role names do not determine access;
- Roles are dynamic records;
- Roles group Permissions;
- Permissions use a consistent **module.action** naming convention;
- Permission codes correspond to real backend capabilities;
- Admins can only perform actions permitted by their effective Permissions;
- frontend uses Permissions to determine navigation and available actions;
- backend always performs authoritative Permission checks;
- Super Admin can create, update, deactivate and reactivate Admin accounts;
- authorized Admins can manage Roles and Role Permissions;
- Admins can update their own profiles;

- Admins can change and reset passwords;
- repeated failed Admin logins can trigger a temporary account lock;
- Admin deactivation invalidates active access;
- Admin Role changes immediately affect effective access;
- normal User `userType` is not used for Admin authorization;
- Seller Verification remains owned by its existing module;
- Product Management remains owned by its existing module;
- Category Management remains owned by its existing module;
- Order Management remains owned by its existing module;
- Payment Management remains owned by its existing module;
- Subscription Management remains owned by its existing module;
- Analytics remains owned by its existing module;
- Dispute Management remains owned by its existing module;
- this module provides authorization for Admin actions performed within those modules;
- important Admin actions are traceable;
- Admin-related Audit events are emitted for the future centralized Audit Log module.

MODULE 20. CANCELLATION, RETURN

20. CANCELLATION, RETURN & REFUND MANAGEMENT MODULE

20.1 Overview

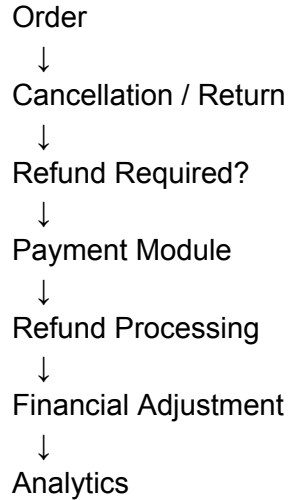
The Cancellation, Return & Refund Management Module manages what happens when a Marketplace transaction cannot proceed as originally agreed or when goods need to be returned and money needs to be refunded.

This module covers:

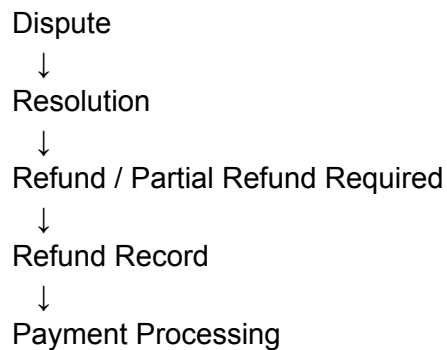
- Order cancellation requests;
- cancellation eligibility;
- Buyer cancellation;
- Seller cancellation;
- Admin cancellation intervention;
- Return requests;
- Return eligibility;
- Return reasons;
- Return evidence;
- Seller response to Return requests;
- Return shipment tracking;
- Return receipt confirmation;
- full Refunds;
- partial Refunds;
- Refund processing;
- Refund status tracking;
- Refund adjustments;
- transaction fee reversals;
- logistics Refund treatment;
- multi-seller Order handling;
- RFQ-originated Orders;
- Dispute-originated Refunds;
- Payment integration;
- Analytics integration;
- Notifications;
- Admin oversight.

This module must not duplicate the existing Order, Payment, Delivery or Dispute lifecycles.

The relationship is:



A Refund can also originate from a Dispute resolution:



20.2 Core Principles

The following principles apply:

- Cancellation does not automatically mean Refund.
- Return does not automatically mean Refund.
- Refund cannot be marked successful simply because it was requested.
- Payment Module remains the source of truth for actual money movement.
- Order Module remains the source of truth for Order lifecycle.
- Delivery Module remains the source of truth for shipment/delivery information.
- Dispute Module remains the source of truth for formal transaction disputes.
- This module coordinates Cancellation, Return and Refund workflows.
- Financial history must never be overwritten.
- Partial Refunds must preserve the original Payment and Order values.

- Multi-seller transactions must be handled at Seller Order level.
- Refunds must use the original Buyer payment currency unless the Payment provider requires another supported process.
- Historical FX snapshots must be preserved.
- TOFA transaction fee adjustments must use stored transaction financial snapshots.

20.3 Cancellation vs Return vs Refund

Cancellation

Cancellation stops an Order or Seller Order from continuing before the transaction has been fully completed.

Example:

Buyer places Order

↓

Seller has not shipped

↓

Buyer requests cancellation

Return

Return means goods already shipped or delivered are being sent back according to Marketplace policy.

Example:

Order delivered

↓

Buyer identifies eligible issue

↓

Return requested

↓

Seller/Admin approves

↓

Goods returned

Refund

Refund means money previously paid by the Buyer is returned in full or in part.

A Refund may result from:

- approved cancellation;
- approved Return;
- Dispute resolution;
- Payment correction;
- authorized Admin financial action.

These concepts must remain separate.

20.4 Cancellation Scope

Cancellation should operate primarily at:

Seller Order

level.

This is important for multi-seller checkout.

Example:

Checkout Payment

↓

Seller Order A

Seller Order B

Seller Order C

If Seller Order B is cancelled, Seller Orders A and C should not automatically be cancelled.

20.5 Cancellation Eligibility

Cancellation eligibility depends on the current Order state and fulfilment progress.

Recommended default:

A Buyer may request cancellation before the affected Seller Order has been shipped.

Once the Seller Order has been handed over for delivery, normal Buyer cancellation should no longer be available.

The Buyer may instead use:

- Return flow, where eligible; or
- Dispute flow, where appropriate.

Cancellation eligibility should also consider:

- Order status;
- Payment status;
- shipment status;
- Seller Order status;
- Order source;
- custom RFQ terms;
- whether cancellation already exists;
- whether Refund processing has already started.

20.6 Cancellation Reasons

Recommended Buyer reasons:

ordered_by_mistake

no_longer_needed

incorrect_delivery_information

duplicate_order

seller_delay

other

Recommended Seller reasons:

out_of_stock

unable_to_fulfil

pricing_error

buyer_requirement_cannot_be_met

operational_issue

other

Admin may use:

fraud_risk

policy_violation

payment_issue

compliance_issue

dispute_resolution

other

Reason definitions may be configurable through System Settings.

20.7 Cancellation Request Table

Table: **order_cancellations**

Field	Type	Required	Notes
id	uuid	Yes	Primary key
cancellationNumber	varchar	Yes	Human-readable unique reference
orderId	uuid	Yes	Parent Order
sellerOrderId	uuid	Yes	Affected Seller Order
requestedBy	uuid	Yes	User/Admin ID
requestedByType	enum	Yes	buyer / seller / admin
reason	varchar	Yes	Cancellation reason
description	text	No	Additional explanation
status	enum	Yes	Cancellation lifecycle
refundRequired	boolean	Yes	Whether money must be returned

refundId	uuid	No	Linked Refund
reviewedBy	uuid	No	Admin/User where applicable
reviewedAt	timestamp	No	Review timestamp
cancelledAt	timestamp	No	Final cancellation timestamp
createdAt	timestamp	Yes	Creation timestamp
updatedAt	timestamp	Yes	Last update

Example Cancellation Number:

CAN-2026-000123

20.8 Cancellation Status

Recommended statuses:

pending

approved

rejected

completed

pending

Cancellation request exists and requires processing or approval.

approved

Cancellation has been approved but related financial processing may still be pending.

rejected

Cancellation request was rejected.

completed

Cancellation and all required associated processing have been completed.

Cancellation status must not replace the Order status.

The Order Module should update the appropriate Seller Order state through its existing lifecycle when cancellation is finalized.

20.9 Request Buyer Cancellation

Endpoint

POST /orders/:sellerOrderId/cancellation

Authentication

Buyer must own the affected Seller Order.

Payload

```
{
  "reason": "ordered_by_mistake",
  "description": "The order was placed twice accidentally."
}
```

20.10 Buyer Cancellation Backend Processing

1. Authenticate Buyer.
2. Retrieve Seller Order.
3. Confirm Buyer ownership.
4. Check Cancellation eligibility.
5. Confirm Seller Order has not passed the cancellable stage.
6. Check for existing active Cancellation.
7. Create Cancellation record.
8. Determine whether immediate cancellation is allowed or approval is required.
9. Determine whether Refund is required.
10. Update related Order state through Order Management where appropriate.
11. Trigger Refund workflow where required.
12. Emit Audit event.
13. Send Notifications.

20.11 Cancellation Request Response

```
{
  "success": true,
```

```
"message": "Cancellation request submitted successfully.",

"data": {
  "id": "cancel_123",
  "cancellationNumber": "CAN-2026-000123",
  "sellerOrderId": "seller_order_123",
  "status": "pending",
  "refundRequired": true
}
```

Toast

Cancellation request submitted successfully.

20.12 Seller Cancellation

A Seller may request or initiate cancellation where the Seller cannot fulfil the Order.

Endpoint

POST /seller/orders/:sellerOrderId/cancellation

Payload

```
{
  "reason": "out_of_stock",
  "description": "The required quantity is no longer available."
}
```

Seller cancellation should be recorded because repeated Seller-initiated cancellations may later contribute to Seller performance Analytics.

Seller cancellation must not automatically be treated as fraud or misconduct.

20.13 Seller Cancellation Notification to Buyer

Subject

Update on Your TOFA Marketplace Order

Body

Hello {BuyerFirstName},

The Seller has requested cancellation of part of your TOFA Marketplace Order.

Order Number: {OrderNumber}

Seller: {SellerName}

Reason: {Reason}

If payment has already been completed and a Refund is required, the applicable Refund process will be initiated.

You can view the latest status from your TOFA Marketplace account.

Best Regards,
Traders of Africa Team

20.14 Admin Cancellation

Admin cancellation should be exceptional and permission-controlled.

Endpoint

POST /admin/orders/:sellerOrderId/cancellation

Required Permission

orders.cancel

Payload

```
{  
  "reason": "fraud_risk",  
  "description": "Transaction suspended following internal review."  
}
```

Admin cancellation must:

- record the Admin;
- record the reason;
- preserve Order history;
- trigger Refund processing where applicable;
- emit an Audit event.

20.15 Duplicate Cancellation Prevention

Only one active Cancellation should exist for the same Seller Order at a time.

Recommended error:

```
{
  "success": false,
  "code": "ACTIVE_CANCELLATION_EXISTS",
  "message": "An active cancellation request already exists for this order."
}
```

20.16 Get Cancellation Requests

Buyer/Seller Endpoint

GET /cancellations

Query Parameters

status
sellerOrderId
page
limit

Users only receive Cancellation records they are authorized to view.

Response

```
{
  "success": true,

  "data": [
    {
      "id": "cancel_123",
      "cancellationNumber": "CAN-2026-000123",

```

```
    "orderId": "order_100",
    "sellerOrderId": "seller_order_123",
    "requestedByType": "buyer",
    "reason": "ordered_by_mistake",
    "status": "completed",
    "refundRequired": true,
    "refundId": "refund_123",
    "createdAt": "2026-08-28T10:00:00Z"
  }
],

"pagination": {
  "page": 1,
  "limit": 20,
  "total": 1,
  "totalPages": 1
}
}
```

20.17 Get Cancellation Details

Endpoint

GET /cancellations/:cancellationId

Response

```
{
  "success": true,

  "data": {
    "id": "cancel_123",
    "cancellationNumber": "CAN-2026-000123",
    "orderId": "order_100",
    "sellerOrderId": "seller_order_123",
    "requestedByType": "buyer",
    "reason": "ordered_by_mistake",
    "description": "The order was placed twice accidentally.",
    "status": "completed",
    "refundRequired": true,
    "refundId": "refund_123",
    "createdAt": "2026-08-28T10:00:00Z",
    "cancelledAt": "2026-08-28T11:00:00Z"
  }
}
```

```
}  
}
```

20.18 Return Eligibility

Return is primarily intended for goods that have already been shipped or delivered.

Return eligibility may depend on:

- Order completion/delivery state;
- Product type;
- Product condition;
- reason for Return;
- elapsed time since delivery;
- Seller/Marketplace policy;
- whether goods are returnable;
- whether a Dispute already controls the issue.

Recommended configurable field:

returnWindowDays

Example:

7 days after confirmed delivery

The value should be configured through System Settings rather than hardcoded.

20.19 Return Reasons

Recommended reasons:

wrong_product

damaged_product

significantly_not_as_described

incorrect_quantity

missing_items

defective_product

other

A simple:

changed_mind

Return may be supported only where TOFA's applicable Return policy permits it.

20.20 Return Request Table

Table: returns

Field	Type	Required	Notes
id	uuid	Yes	Primary key
returnNumber	varchar	Yes	Unique reference
orderId	uuid	Yes	Parent Order
sellerOrderId	uuid	Yes	Affected Seller Order
buyerId	uuid	Yes	Buyer
sellerId	uuid	Yes	Seller
reason	varchar	Yes	Return reason
description	text	Yes	Buyer explanation
status	enum	Yes	Return lifecycle
returnMethod	varchar	No	Return delivery method
returnProviderName	varchar	No	Provider
returnTrackingId	varchar	No	Tracking reference
returnTrackingUrl	varchar	No	Tracking URL
approvedBy	uuid	No	Seller/Admin

approvedAt	timestamp	No	Approval time
receivedAt	timestamp	No	Seller receipt time
refundId	uuid	No	Linked Refund
createdAt	timestamp	Yes	Creation
updatedAt	timestamp	Yes	Last update

Example:

RET-2026-000123

20.21 Return Item Table

Table: **return_items**

Field	Type	Required	Notes
id	uuid	Yes	Primary key
returnId	uuid	Yes	Parent Return
orderId	uuid	Yes	Order Item snapshot
quantity	decimal	Yes	Quantity being returned
reason	varchar	No	Item-specific reason
createdAt	timestamp	Yes	Creation

Return quantities cannot exceed quantities purchased minus quantities already returned/refunded where applicable.

20.22 Return Evidence

Return requests may include:

- product images;
- damaged packaging images;
- delivery documents;
- videos where supported;
- PDF documents;

- other supporting files.

The platform's shared File/Upload service should be used.

Return records should store file references rather than duplicating file storage logic.

20.23 Return Status

Recommended statuses:

requested

approved

rejected

awaiting_return

in_transit

received

completed

cancelled

requested

Buyer submitted Return request.

approved

Return was approved.

rejected

Return request was rejected.

awaiting_return

Buyer must send the goods back.

in_transit

Return shipment has been handed over.

received

Seller has confirmed receipt.

completed

Return workflow and associated Refund processing have been completed.

cancelled

Return request was withdrawn or cancelled before completion where permitted.

20.24 Create Return Request

Endpoint

POST /returns

Payload

```
{
  "sellerOrderId": "seller_order_123",

  "reason": "damaged_product",

  "description": "Two cartons were damaged when the shipment arrived.",

  "items": [
    {
      "orderItemId": "order_item_101",
      "quantity": 2
    }
  ],

  "evidence": [
    "file_123",
    "file_124"
  ]
}
```

20.25 Create Return Backend Processing

1. Authenticate Buyer.
2. Retrieve Seller Order.
3. Confirm Buyer ownership.
4. Check delivery/fulfilment state.
5. Check Return window.
6. Validate reason.
7. Validate Return items.
8. Validate quantities.
9. Check existing active Return for same affected items.
10. Create Return.
11. Create Return Items.
12. Link evidence.
13. Notify Seller.
14. Emit Audit event.

20.26 Create Return Response

```
{
  "success": true,

  "message": "Return request submitted successfully.",

  "data": {
    "id": "return_123",
    "returnNumber": "RET-2026-000123",
    "sellerOrderId": "seller_order_123",
    "status": "requested"
  }
}
```

Toast

Return request submitted successfully.

20.27 Return Request Email to Seller

Subject

New Return Request for Order {OrderNumber}

Body

Hello {SellerName},

A Buyer has submitted a Return request for an Order from your store.

Order Number: {OrderNumber}

Return Number: {ReturnNumber}

Reason: {Reason}

Please sign in to your TOFA Marketplace account to review the request and supporting information.

Best Regards,
Traders of Africa Team

20.28 Seller Responds to Return

Endpoint

PATCH /seller/returns/:returnId/respond

Payload — Approve

```
{  
  "decision": "approve",  
  "notes": "Return approved. Please send the affected goods back."  
}
```

Payload — Reject

```
{  
  "decision": "reject",  
  "notes": "The submitted evidence does not show the reported damage."  
}
```

Seller rejection should not prevent the Buyer from using the Dispute process where the Buyer believes the issue remains unresolved.

20.29 Approved Return Email

Subject

Your TOFA Marketplace Return Request Has Been Approved

Body

Hello {BuyerFirstName},

Your Return request has been approved.

Return Number: {ReturnNumber}

Order Number: {OrderNumber}

Please follow the Return instructions displayed in your TOFA Marketplace account.

Your Refund, where applicable, will be processed according to the approved Return terms and after the required Return conditions have been satisfied.

Best Regards,
Traders of Africa Team

20.30 Rejected Return Email

Subject

Update on Your TOFA Marketplace Return Request

Body

Hello {BuyerFirstName},

Your Return request has been reviewed and was not approved.

Return Number: {ReturnNumber}

Reason/Response:

{Response}

If you believe the issue remains unresolved and qualifies for formal review, you may raise a Dispute through TOFA Marketplace.

Best Regards,
Traders of Africa Team

20.31 Submit Return Shipment Information

Where the Buyer is responsible for sending the approved Return:

Endpoint

PATCH /returns/:returnId/shipment

Payload

```
{  
  "providerName": "ABC Transport",  
  "trackingId": "RET938483",  
  "trackingUrl": null,  
  "deliveryContact": "+2348012345678",  
  "notes": "Returned two damaged cartons."  
}
```

Backend sets:

status = in_transit

20.32 Confirm Return Receipt

Endpoint

PATCH /seller/returns/:returnId/received

Authentication

Affected Seller.

Backend Processing

1. Retrieve Return.
2. Confirm Seller ownership.
3. Confirm Return is in appropriate state.
4. Set **receivedAt**.
5. Set:

status = received

6. Trigger applicable Refund processing.
7. Notify Buyer.
8. Emit Audit event.

20.33 Refund Structure

Refunds must be stored separately from Payments.

Original Payment history must remain unchanged.

Table: **refunds**

Field	Type	Required	Notes
id	uuid	Yes	Primary key
refundNumber	varchar	Yes	Unique reference
paymentId	uuid	Yes	Original Payment
orderId	uuid	Yes	Parent Order
sellerOrderId	uuid	Yes	Affected Seller Order
cancellationId	uuid	No	Source Cancellation
returnId	uuid	No	Source Return
disputeId	uuid	No	Source Dispute
sourceType	enum	Yes	cancellation / return / dispute / payment_adjustment / admin
refundType	enum	Yes	full / partial
amount	decimal	Yes	Refund amount

currency	varchar	Yes	Original Buyer payment currency
status	enum	Yes	Refund lifecycle
reason	text	Yes	Refund reason
gateway	varchar	No	Payment gateway/method
gatewayRefundReference	varchar	No	Provider reference
requestedBy	uuid	Yes	User/Admin/System
approvedBy	uuid	No	Where approval required
processedAt	timestamp	No	Processing time
completedAt	timestamp	No	Completion time
failedAt	timestamp	No	Failure time
failureReason	text	No	Failure explanation
createdAt	timestamp	Yes	Creation
updatedAt	timestamp	Yes	Last update

Example:

REF-2026-000123

20.34 Refund Status

Recommended statuses:

pending

approved

processing

successful

failed

cancelled

pending

Refund record exists and is awaiting approval/processing.

approved

Refund has been approved.

processing

Refund has been submitted for financial processing.

successful

Payment provider/manual financial process confirms money has been returned.

failed

Refund attempt failed.

cancelled

Refund request was cancelled before financial processing where permitted.

A Refund must never become **successful** merely because an Admin clicked Approve.

20.35 Refund Types

full

partial

Full Refund

The entire eligible amount for the affected Seller Order is returned.

Partial Refund

Only part of the eligible amount is returned.

Example:

Seller Order Product Amount = NGN 1,000,000

Approved Refund = NGN 200,000

Original Seller Order remains historically:

NGN 1,000,000

The Refund is stored separately:

NGN 200,000

Net reporting becomes derived from original amount minus Refund adjustments.

20.36 Create Refund

Refund creation should normally be initiated by an approved business event rather than arbitrary User action.

Sources include:

approved Cancellation

approved Return

Dispute resolution

authorized Admin Payment correction

Internal/Admin Endpoint

POST /admin/refunds

Required Permission

refunds.create

Payload

```
{
  "sellerOrderId": "seller_order_123",
  "sourceType": "return",
  "returnId": "return_123",
  "refundType": "partial",
  "amount": 200000,
  "reason": "Refund for two damaged cartons."
}
```

Backend should derive:

- paymentId;
- orderId;
- Buyer payment currency;
- financial snapshots;

from the original transaction.

The client must not be trusted to provide financial values that can be derived from authoritative transaction records.

20.37 Refund Validation

Backend must verify:

Refund amount > 0

and:

Total successful/pending refundable amount
≤
Maximum refundable amount

The system must prevent cumulative Refunds from exceeding the original eligible amount.

Example:

Original refundable amount = NGN 1,000,000

Previous successful Refund = NGN 200,000

Maximum additional Refund = NGN 800,000

20.38 Refund Currency

Refunds should normally use the currency in which the Buyer originally paid TOFA.

Example:

Buyer paid = USD 10,000

Seller settlement currency = NGN

Buyer Refund:

USD

The platform must not recalculate the Refund using the current FX rate.

The original transaction's FX snapshot must be preserved for Seller settlement reversal and reporting.

20.39 Multi-Seller Refunds

Example:

Buyer Checkout Payment = USD 30,000

Allocation:

Seller A = USD equivalent 10,000

Seller B = USD equivalent 12,000

Seller C = USD equivalent 8,000

If Seller B requires a Refund:

Refund applies only to Seller B allocation.

The original Checkout Payment remains recorded as:

USD 30,000

The Refund is stored as a separate financial adjustment.

Seller A and Seller C Orders remain unaffected.

20.40 Logistics Refund Treatment

Logistics Refund depends on how delivery was arranged.

Integrated Logistics

If TOFA collected the logistics amount, the applicable logistics amount may be refundable according to policy and provider status.

Seller-Arranged Logistics

If logistics was included in the Seller commercial terms and collected through TOFA, applicable Refund treatment should follow the locked transaction terms and Return/Cancellation policy.

Buyer-Arranged Logistics

Where the Buyer paid an external logistics provider directly:

TOFA did not collect the logistics money.

Therefore TOFA cannot automatically Refund that amount.

The Buyer may need to resolve external logistics charges directly with the logistics provider.

20.41 Refund Amount Breakdown

Refund records should support a breakdown where applicable.

Recommended:

```
{  
  "productAmount": 180000,  
  "logisticsAmount": 20000,
```

```
"otherAmount": 0,  
"totalRefundAmount": 200000  
}
```

This improves:

- financial reconciliation;
- transaction fee adjustment;
- Analytics;
- reporting.

20.42 Transaction Fee Adjustment

TOFA transaction fee was defined in Subscription Management.

Example original transaction:

Product Amount = NGN 1,000,000

Transaction Fee Percentage = 5%

Original TOFA Fee = NGN 50,000

If product Refund:

NGN 200,000

and the applicable fee policy requires proportional reversal:

Fee Reversal

=

$200,000 \times 5\%$

=

NGN 10,000

Final net TOFA transaction fee:

NGN 40,000

Do not change the original:

transactionFeeAmount = NGN 50,000

Instead create a financial adjustment/reversal.

20.43 Financial Adjustment Table

Recommended shared financial structure:

Table: `financial_adjustments`

Field	Type	Required	Notes
id	uuid	Yes	Primary key
orderId	uuid	Yes	Related Order
sellerOrderId	uuid	Yes	Related Seller Order
paymentId	uuid	No	Related Payment
refundId	uuid	No	Related Refund
adjustmentType	enum	Yes	refund / transaction_fee_reversal / settlement_adjustment / other
amount	decimal	Yes	Adjustment amount
currency	varchar	Yes	Adjustment currency
direction	enum	Yes	debit / credit
reason	text	Yes	Explanation
createdAt	timestamp	Yes	Creation

Historical financial values must not be overwritten.

20.44 Process Refund

Endpoint

POST /admin/refunds/:refundId/process

Required Permission

refunds.process

Backend:

1. Retrieve Refund.
2. Validate Refund state.
3. Retrieve original Payment.
4. Determine original Payment method/provider.
5. Validate refundable amount.
6. Submit Refund to supported provider or manual workflow.
7. Set:

status = processing

8. Store provider reference.
9. Await confirmation.
10. Emit Audit event.

20.45 Automatic Gateway Refund

Where a Payment provider supports Refund APIs:

TOFA Backend

↓

Payment Provider

↓

Refund Request

↓

Provider Confirmation/Webhook

↓

Refund status = successful

Provider webhooks must be verified.

Processing should be idempotent to prevent duplicate Refunds.

20.46 Manual Refund

Some Payment methods may require manual financial processing.

Example:

Bank Transfer

LC

TT

Other manually reconciled method

Admin approval alone does not make the Refund successful.

The Refund remains:

approved

or:

processing

until TOFA confirms actual financial processing.

20.47 Confirm Manual Refund

Endpoint

PATCH /admin/refunds/:refundId/confirm

Required Permission

refunds.confirm

Payload

```
{  
  "reference": "BANK-REF-938483",  
  "notes": "Refund transferred to Buyer account."  
}
```

Backend sets:

status = successful

only after authorized confirmation.

20.48 Failed Refund

Where provider processing fails:

status = failed

Store:

failedAt

failureReason

The original Refund record should not be deleted.

Authorized Admins may retry where safe.

20.49 Retry Failed Refund

Endpoint

POST /admin/refunds/:refundId/retry

Required Permission

refunds.process

Backend must use idempotency and provider references to prevent duplicate financial processing.

20.50 Get User Refunds

Endpoint

GET /refunds

Query Parameters

status
orderId
page
limit

Buyer only receives Refunds associated with their transactions.

Response

```
{
  "success": true,

  "data": [
    {
      "id": "refund_123",
      "refundNumber": "REF-2026-000123",
      "orderId": "order_100",
      "sellerOrderId": "seller_order_123",
      "sourceType": "return",
      "refundType": "partial",
      "amount": 200000,
      "currency": "NGN",
      "status": "successful",
      "createdAt": "2026-08-28T10:00:00Z",
      "completedAt": "2026-08-28T12:00:00Z"
    }
  ],

  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "totalPages": 1
  }
}
```

20.51 Get Refund Details

Endpoint

GET /refunds/{refundId}

Response

```
{
  "success": true,

  "data": {
    "id": "refund_123",
    "refundNumber": "REF-2026-000123",
    "paymentId": "payment_123",
    "orderId": "order_100",
    "sellerOrderId": "seller_order_123",
    "sourceType": "return",
    "returnId": "return_123",
    "refundType": "partial",

    "breakdown": {
      "productAmount": 180000,
      "logisticsAmount": 20000,
      "otherAmount": 0,
      "totalRefundAmount": 200000
    },

    "currency": "NGN",
    "status": "successful",
    "reason": "Refund for two damaged cartons.",
    "completedAt": "2026-08-28T12:00:00Z"
  }
}
```

20.52 Admin Get Refunds

Endpoint

GET /admin/refunds

Required Permission

refunds.view

Query Parameters

search
status
sourceType
refundType
paymentId
orderId
sellerOrderId
dateFrom
dateTo
currency
page
limit

20.53 Admin Get Refund Details

Endpoint

GET /admin/refunds/:refundId

Required Permission

refunds.view

Admin detail should include:

- Refund;
- source Cancellation/Return/Dispute;
- original Payment;
- Seller Order;
- Buyer;
- Seller;
- amount breakdown;
- financial adjustments;
- processing history;
- provider references;
- Audit information.

20.54 Refund Successful Email

Subject

Your TOFA Marketplace Refund Has Been Processed

Body

Hello {BuyerFirstName},

Your Refund has been processed successfully.

Refund Number: {RefundNumber}

Order Number: {OrderNumber}

Refund Amount: {Currency} {Amount}

Depending on your payment provider or financial institution, it may take additional time for the funds to appear in your account.

You can view the Refund details from your TOFA Marketplace account.

Best Regards,
Traders of Africa Team

In-App Notification

Title:

Refund processed

Message:

Your refund of {Currency} {Amount} for Order {OrderNumber} has been processed successfully.

20.55 Refund Failed Notification**Subject**

Update on Your TOFA Marketplace Refund

Body

Hello {BuyerFirstName},

We encountered an issue while processing your Refund.

Refund Number: {RefundNumber}

Order Number: {OrderNumber}

The Refund has not been marked as completed.

Our team will review the transaction and take the appropriate next step.

Best Regards,
Traders of Africa Team

20.56 Cancellation and RFQ Orders

Orders created from:

direct_rfq

market_rfq

must preserve the accepted Quote terms.

Cancellation eligibility may consider:

- accepted quantity;
- Seller fulfilment commitment;
- quote terms;
- delivery method;
- custom transaction conditions.

Cancellation must not change the original accepted Quote.

The Order stores the locked transaction snapshot.

20.57 Return and Dispute Relationship

Return is an operational resolution flow.

Dispute is a formal escalation flow.

Example:

Buyer requests Return

↓

Seller approves

↓
Return completed
↓
Refund processed

No Dispute is necessary.

Alternative:

Buyer requests Return
↓
Seller rejects
↓
Buyer disagrees
↓
Buyer raises Dispute

The Dispute may reference the Return.

Do not automatically create a Dispute for every Return.

20.58 Dispute-Originated Refund

Where Admin resolves a Dispute with:

`financialAction = refund`

or:

`financialAction = partial_refund`

the Dispute Module should create/request a Refund through this module.

The Dispute itself must not directly mark money as returned.

20.59 Refund Permissions

Recommended Admin Permissions:

`refunds.view`

refunds.create

refunds.approve

refunds.process

refunds.confirm

Where Return administration is required:

returns.view

returns.manage

Permissions remain governed by Module 19.

20.60 Analytics Relationship

Analytics should consume:

- Cancellation count;
- Cancellation rate;
- Buyer cancellation;
- Seller cancellation;
- Return count;
- Return rate;
- Refund amount;
- Refund rate;
- product Refund amount;
- logistics Refund amount;
- transaction fee reversal;
- net TOFA transaction revenue.

Analytics must not overwrite historical GMV.

Recommended:

Gross GMV

Refunds

Net GMV

as separate concepts.

20.61 Seller Performance

Seller performance may consider:

- Seller-initiated cancellations;
- eligible Returns;
- resolved Disputes;
- Refund rates.

However:

- every Return does not mean Seller fault;
- every Refund does not mean Seller fault;
- Buyer cancellation should not automatically harm Seller performance;
- unresolved Disputes should not automatically count against Seller.

Performance logic should use finalized outcomes.

20.62 Notification Relationship

Cancellation, Return and Refund events should use the existing Notification Module.

Important events include:

CANCELLATION_REQUESTED

CANCELLATION_APPROVED

CANCELLATION_REJECTED

CANCELLATION_COMPLETED

RETURN_REQUESTED

RETURN_APPROVED

RETURN_REJECTED

RETURN_SHIPPED

RETURN_RECEIVED

RETURN_COMPLETED

REFUND_CREATED

REFUND_PROCESSING

REFUND_SUCCESSFUL

REFUND_FAILED

20.63 Audit Events

Recommended Audit events:

CANCELLATION_CREATED

CANCELLATION_APPROVED

CANCELLATION_REJECTED

CANCELLATION_COMPLETED

RETURN_CREATED

RETURN_APPROVED

RETURN_REJECTED

RETURN_SHIPMENT_ADDED

RETURN_RECEIVED

RETURN_COMPLETED

REFUND_CREATED

REFUND_APPROVED

REFUND_PROCESSING_STARTED

REFUND_SUCCESSFUL

REFUND_FAILED

REFUND_RETRIED

TRANSACTION_FEE_REVERSED

FINANCIAL_ADJUSTMENT_CREATED

Module 21 should store the centralized Audit records.

20.64 Frontend Implementation Guide

Frontend should:

- show Cancel Order only when eligible;
- explain when cancellation is no longer available;
- provide cancellation reason selection;
- provide Return request flow;
- allow affected Order Items and quantities to be selected;
- allow evidence uploads;
- show Return status;
- show Return shipment instructions;
- allow Return tracking submission;
- show Refund status;
- display Refund amount and currency;
- clearly distinguish requested, processing and successful Refunds;
- show relevant timeline information;
- provide Admin Refund queues where authorized;
- show appropriate confirmation dialogs before sensitive actions.

Frontend must not:

- calculate authoritative Refund values;
- mark Refunds successful;
- decide financial eligibility without backend;
- use current FX rates to calculate historical Refunds.

20.65 Backend Implementation Guide

Backend should:

- validate Seller Order ownership;
- validate Cancellation eligibility;
- validate Return eligibility;

- prevent duplicate active requests;
- validate quantities;
- use Order Item snapshots;
- derive financial values from authoritative Order/Payment data;
- maintain separate Refund records;
- preserve original Payment history;
- process provider Refunds idempotently;
- verify provider webhooks;
- support manual Refund confirmation;
- preserve historical FX;
- create financial adjustment records;
- adjust transaction fees through reversals;
- support multi-seller allocation;
- emit Notification events;
- emit Audit events;
- expose appropriate Admin endpoints;
- enforce Admin Permissions.

20.66 Important Consistency Rules

- Cancellation, Return and Refund are separate concepts.
- Cancellation primarily applies at Seller Order level.
- Multi-seller Orders must not be entirely cancelled because one Seller Order is cancelled.
- Return applies to eligible goods after fulfilment has progressed.
- Refund represents actual financial reversal.
- Refund success requires financial confirmation.
- Payment remains the source of truth for money movement.
- Order remains the source of truth for Order lifecycle.
- Delivery remains the source of truth for shipment data.
- Dispute remains the source of truth for formal disputes.
- Refund records do not overwrite original Payments.
- Partial Refunds do not rewrite original Order amounts.
- Historical FX must not be replaced with current FX.
- Buyer Refund currency should normally be the original Buyer payment currency.
- Seller financial reversal uses the stored transaction FX snapshot.
- Transaction fee reversals must be stored separately.
- Buyer-arranged external logistics cannot automatically be refunded by TOFA.
- Seller performance should use finalized outcomes rather than raw Return/Dispute creation.
- Admin Refund actions require Permissions.
- All sensitive financial actions must be auditable.

20.67 Module Summary

The Cancellation, Return & Refund Management Module establishes that:

- Buyers, Sellers and authorized Admins can initiate appropriate cancellation workflows;
- cancellation eligibility depends on fulfilment progress;
- cancellation operates primarily at Seller Order level;
- multi-seller Orders remain independently manageable;
- Buyers can request eligible Returns;
- Returns identify affected Order Items and quantities;
- Return requests may include evidence;
- Sellers can approve or reject normal Return requests;
- unresolved Return disagreements may escalate to Disputes;
- approved Returns can lead to Refunds;
- Dispute resolutions can initiate Refunds;
- Refunds are separate financial records;
- Refunds may be full or partial;
- Refunds preserve original Payment history;
- Refund amounts cannot exceed eligible transaction amounts;
- Buyer Refunds normally use original payment currency;
- historical FX snapshots are preserved;
- logistics Refund treatment depends on delivery arrangement and who collected the money;
- transaction fee reversals use stored transaction financial snapshots;
- manual and provider Refunds remain processing until financially confirmed;
- Analytics consumes Refund and adjustment records;
- Notifications communicate Cancellation, Return and Refund events;
- Audit events record sensitive actions;
- Admin access is controlled by Module 19 Permissions.

MODULE 21. AUDIT LOG MANAGEMENT

21. AUDIT LOG MANAGEMENT MODULE

21.1 Overview

The Audit Log Management Module provides a centralized, immutable history of important activities performed across TOFA Marketplace.

The purpose of Audit Logs is to answer questions such as:

Who performed the action?

What action was performed?

When was it performed?

Which record was affected?

What changed?

Why was the action performed?

Where did the action originate?

Audit Logs are primarily intended for:

- accountability;
- operational investigation;
- financial reconciliation;
- security investigation;
- Admin oversight;
- compliance support;
- dispute investigation;
- change tracking;
- debugging important business events.

Audit Logs are not the same as normal application logs.

Application logs may contain technical information such as:

server errors

API latency

database errors
stack traces

Audit Logs record meaningful business and security actions.

21.2 Core Principles

Audit Logs must be:

Centralized

Immutable

Timestamped

Traceable

Secure

Permission-controlled

Once an Audit Log has been created, normal Admin Users must not be able to edit or delete it.

The system should never expose an endpoint such as:

DELETE /admin/audit-logs/:id

for normal administrative use.

21.3 Audit Scope

Audit events should be generated for important actions across:

- Authentication;
- Users;
- Admins;
- Seller Verification;
- Products;
- Categories;
- Cart/Checkout where materially important;
- Payments;

- Orders;
- Delivery;
- Direct RFQ;
- Market RFQ;
- Reviews/moderation;
- Subscriptions;
- Notifications where necessary;
- Search administration;
- Analytics administration;
- Disputes;
- Cancellations;
- Returns;
- Refunds;
- System Settings.

Not every read action needs to create an Audit Log.

The focus should be:

security-sensitive actions

financial actions

status changes

administrative actions

business-critical changes

21.4 Audit Log Table

Table: `audit_logs`

Field	Type	Required	Notes
id	uuid	Yes	Primary key
eventCode	varchar	Yes	Standard event identifier
actorId	uuid	No	User/Admin/System actor
actorType	enum	Yes	user / admin / system
actorEmail	varchar	No	Snapshot where appropriate

module	varchar	Yes	Source module
action	varchar	Yes	Action performed
entityType	varchar	No	Affected entity type
entityId	uuid	No	Affected entity
description	text	Yes	Human-readable event description
reason	text	No	Reason where applicable
oldValue	json/jsonb	No	Relevant previous values
newValue	json/jsonb	No	Relevant new values
metadata	json/jsonb	No	Additional safe context
ipAddress	varchar	No	Request IP where available
userAgent	text	No	Request client information
requestId	varchar	No	Correlation/request ID
createdAt	timestamp	Yes	Event timestamp

Audit records should not use:

updatedAt

because Audit Logs should not normally be updated.

21.5 Actor Types

Recommended:

user

admin

system

user

Marketplace Buyer or Seller performed the action.

admin

TOFA Admin/Super Admin performed the action.

system

Automated Marketplace process performed the action.

Examples:

Payment webhook confirmed

Subscription expired automatically

Refund webhook confirmed

Order auto-transition

Scheduled process

21.6 Actor Snapshot

Where useful, Audit records may preserve limited actor snapshot information.

Example:

```
{  
  "actorId": "admin_123",  
  "actorType": "admin",  
  "actorEmail": "finance@tofa.com"  
}
```

This helps historical investigation if the Admin account is later deactivated.

Sensitive profile information should not be unnecessarily duplicated.

21.7 Event Code Naming

Recommended naming:

ENTITY_ACTION

Examples:

USER_DISABLED

ADMIN_CREATED

SELLER_VERIFIED

PAYMENT_VERIFIED

ORDER_CANCELLED

DISPUTE_RESOLVED

REFUND_SUCCESSFUL

Event codes should be stable.

Frontend display labels can change without changing stored event codes.

21.8 Authentication Audit Events

Recommended events:

USER_SIGNUP

USER_EMAIL_VERIFIED

USER_LOGIN_SUCCESSFUL

USER_LOGIN_FAILED

USER_PASSWORD_RESET

USER_PASSWORD_CHANGED

USER_ACCOUNT_DELETED

ADMIN_LOGIN_SUCCESSFUL

ADMIN_LOGIN_FAILED

ADMIN_ACCOUNT_LOCKED

ADMIN_PASSWORD_RESET

ADMIN_PASSWORD_CHANGED

Sensitive authentication values must never be stored.

Do not store:

password

passwordHash

OTP

JWT

refreshToken

setupToken

21.9 User Management Audit Events

Recommended:

USER_PROFILE_UPDATED

USER_DISABLED

USER_REACTIVATED

USER_ACCOUNT_DELETED

USER_TYPE_CHANGED

Example:

```
{  
  "eventCode": "USER_DISABLED",
```



```
"actorId": "admin_100",
"actorType": "admin",
"module": "users",
"action": "disable",
"entityType": "user",
"entityId": "usr_123",
"description": "User account was disabled by an Admin.",
"reason": "Fraudulent activities",
"oldValue": {
  "status": "active"
},
"newValue": {
  "status": "disabled"
}
}
```

21.10 Admin Management Audit Events

Recommended:

ADMIN_CREATED

ADMIN_INVITATION_SENT

ADMIN_INVITATION_RESENT

ADMIN_SETUP_COMPLETED

ADMIN_PROFILE_UPDATED

ADMIN_UPDATED

ADMIN_ROLE_CHANGED

ADMIN_DEACTIVATED

ADMIN_REACTIVATED

ADMIN_ROLE_CREATED

ADMIN_ROLE_UPDATED

ADMIN_ROLE_DEACTIVATED

ADMIN_ROLE_PERMISSIONS_UPDATED

Example:

```
{
  "eventCode": "ADMIN_ROLE_CHANGED",

  "actorId": "admin_super_1",

  "actorType": "admin",

  "module": "admin_management",

  "action": "role_change",

  "entityType": "admin",

  "entityId": "admin_102",

  "description": "Admin role was changed.",

  "oldValue": {
    "roleId": "role_support"
  },

  "newValue": {
    "roleId": "role_operations"
  }
}
```

21.11 Seller Verification Audit Events

Recommended:

SELLER_VERIFICATION_SUBMITTED

SELLER_VERIFICATION_UPDATED

SELLER_VERIFIED

SELLER_VERIFICATION_REJECTED

SELLER_VERIFICATION_RESUBMITTED

Audit should identify:

sellerId

adminId where applicable

previous status

new status

reason

21.12 Product Audit Events

Recommended important events:

PRODUCT_CREATED

PRODUCT_UPDATED

PRODUCT_ACTIVATED

PRODUCT_DEACTIVATED

PRODUCT_ARCHIVED

PRODUCT_DELETED

Not every minor inventory movement necessarily needs the central Audit Log if a dedicated Inventory transaction history exists.

Business-critical changes should be logged.

Example:

```
{  
  "eventCode": "PRODUCT_DEACTIVATED",  
  "actorId": "admin_200",  
  "actorType": "admin",
```

```
"module": "products",
"action": "deactivate",
"entityType": "product",
"entityId": "prod_123",
"description": "Product was deactivated by an Admin.",
"reason": "Product violates marketplace policy.",
"oldValue": {
  "status": "active"
},
"newValue": {
  "status": "inactive"
}
}
```

21.13 Category Audit Events

Recommended:

CATEGORY_CREATED

CATEGORY_UPDATED

CATEGORY_ACTIVATED

CATEGORY_DEACTIVATED

CATEGORY_ARCHIVED

21.14 Payment Audit Events

Financial Audit events are critical.

Recommended:

PAYMENT_CREATED

PAYMENT_PROCESSING_STARTED

PAYMENT_SUCCESSFUL

PAYMENT_FAILED

PAYMENT_PROOF_UPLOADED

PAYMENT_VERIFIED

PAYMENT_REJECTED

PAYMENT_WEBHOOK_RECEIVED

Webhook events should not store the entire raw provider payload if it contains sensitive data.

Store only safe relevant metadata or keep raw secure provider logs separately where necessary.

21.15 Manual Payment Verification Audit

Example:

```
{
  "eventCode": "PAYMENT_VERIFIED",

  "actorId": "admin_finance_1",

  "actorType": "admin",

  "module": "payments",

  "action": "verify",

  "entityType": "payment",

  "entityId": "payment_123",

  "description": "Manual payment was verified.",

  "oldValue": {
    "status": "processing"
  },

  "newValue": {
    "status": "successful"
  },
}
```

```
"metadata": {  
  "paymentFor": "order",  
  "currency": "USD",  
  "amount": 10000  
}
```

Do not store:

- bank passwords;
- card information;
- secret provider credentials;
- sensitive proof-document content.

21.16 Order Audit Events

Recommended important events:

ORDER_CREATED

ORDER_CONFIRMED

ORDER_STATUS_CHANGED

ORDER_SHIPPED

ORDER_DELIVERED

ORDER_COMPLETED

ORDER_CANCELLED

If Order status changes:

```
{  
  "oldValue": {  
    "status": "confirmed"  
  },  
  "newValue": {  
    "status": "shipped"  
  }  
}
```

should be sufficient where appropriate.

21.17 Delivery Audit Events

Recommended:

DELIVERY_METHOD_SELECTED

DELIVERY_QUOTE_SELECTED

SHIPMENT_CREATED

SHIPMENT_UPDATED

TRACKING_UPDATED

DELIVERY_CONFIRMED

The Delivery Module remains the source of truth for full shipment history.

Audit Logs record important actions rather than replacing shipment event data.

21.18 RFQ Audit Events

Recommended:

DIRECT_RFQ_CREATED

DIRECT_RFQ_UPDATED

DIRECT_RFQ_QUOTE_SUBMITTED

DIRECT_RFQ_QUOTE_REFRESHED

DIRECT_RFQ_ACCEPTED

DIRECT_RFQ_EXPIRED

MARKET_RFQ_CREATED

MARKET_RFQ_RESPONSE_SUBMITTED

MARKET_RFQ_RESPONSE_UPDATED

MARKET_RFQ_RESPONSE_ACCEPTED

MARKET_RFQ_CLOSED

Audit must not replace the RFQ lifecycle.

21.19 Review Audit Events

Recommended administrative events:

REVIEW_CREATED

REVIEW_MODERATED

REVIEW_HIDDEN

REVIEW_RESTORED

Normal public review content remains stored in Review Management.

Audit should primarily record moderation actions and important state changes.

21.20 Subscription Audit Events

Recommended:

SUBSCRIPTION_PLAN_CREATED

SUBSCRIPTION_PLAN_UPDATED

SUBSCRIPTION_PLAN_ACTIVATED

SUBSCRIPTION_PLAN_DEACTIVATED

SUBSCRIPTION_ACTIVATED

SUBSCRIPTION_RENEWED

SUBSCRIPTION_CHANGED

SUBSCRIPTION_EXPIRED

SUBSCRIPTION_CANCELLED

For Plan changes, relevant old/new values may include:

```
{  
  "oldValue": {  
    "transaction_fee_percentage": 5  
  },  
  
  "newValue": {  
    "transaction_fee_percentage": 3  
  }  
}
```

Historical transaction financial snapshots must still remain unchanged.

21.21 Analytics Audit Events

Normal dashboard viewing does not necessarily require Audit records.

Sensitive actions should.

Recommended:

ANALYTICS_REPORT_EXPORT_REQUESTED

ANALYTICS_REPORT_GENERATED

ANALYTICS_REBUILD_STARTED

ANALYTICS_REBUILD_COMPLETED

ANALYTICS_REBUILD_FAILED

For highly sensitive financial reporting, TOFA may optionally log report downloads/access depending on internal policy.

21.22 Dispute Audit Events

Recommended:

DISPUTE_CREATED

DISPUTE_ASSIGNED

DISPUTE_REVIEW_STARTED

DISPUTE_MESSAGE_SENT

DISPUTE_EVIDENCE_ADDED

DISPUTE_INFORMATION_REQUESTED

DISPUTE_RESOLVED

DISPUTE_CLOSED

Resolution Audit should preserve:

resolutionType

resolvedBy

financialAction

where applicable.

21.23 Cancellation Audit Events

Recommended:

CANCELLATION_CREATED

CANCELLATION_APPROVED

CANCELLATION_REJECTED

CANCELLATION_COMPLETED

21.24 Return Audit Events

Recommended:

RETURN_CREATED

RETURN_APPROVED

RETURN_REJECTED

RETURN_SHIPMENT_ADDED

RETURN_RECEIVED

RETURN_COMPLETED

RETURN_CANCELLED

21.25 Refund Audit Events

Financially important events:

REFUND_CREATED

REFUND_APPROVED

REFUND_PROCESSING_STARTED

REFUND_SUCCESSFUL

REFUND_FAILED

REFUND_RETRIED

TRANSACTION_FEE_REVERSED

FINANCIAL_ADJUSTMENT_CREATED

Refund Audit records should identify:

refundId

paymentId

orderId

sellerOrderId

amount

currency

sourceType

where appropriate.

21.26 System Settings Audit Events

System Settings changes are sensitive and must be audited.

Recommended:

SYSTEM_SETTING_UPDATED

PAYMENT_METHOD_ACTIVATED

PAYMENT_METHOD_DEACTIVATED

CURRENCY_ACTIVATED

CURRENCY_DEACTIVATED

COUNTRY_ACTIVATED

COUNTRY_DEACTIVATED

MARKETPLACE_RULE_UPDATED

Old and new values should be stored for configuration changes where safe.

21.27 Creating Audit Logs

Audit Logs should normally be created internally by backend services.

There should not be a general frontend endpoint such as:

POST /audit-logs

because clients should not be trusted to create authoritative Audit history.

Instead:

Business Action

↓

Backend validates action

↓

Business transaction succeeds

↓

Audit event emitted

↓

Audit record created

Example:

Admin verifies Payment

↓

Payment Module updates Payment

↓

PAYMENT_VERIFIED event emitted

↓

Audit Service stores record

21.28 Transaction Consistency

For critical actions, the business change and Audit event should be transactionally reliable.

The system must avoid:

Payment changed successfully

but

Audit event permanently lost

Recommended implementation options include:

- database transaction where suitable;
- transactional outbox;
- reliable event queue;

- idempotent Audit consumer.

The final architecture may depend on TOFA infrastructure.

21.29 Idempotency

Event-driven Audit creation must be idempotent.

A duplicated event must not create duplicate Audit records.

Recommended event metadata:

eventId

A unique constraint may be maintained against the event ID where event-driven processing is used.

21.30 Audit Metadata

The **metadata** field may store additional safe context.

Example:

```
{
  "orderNumber": "ORD-2026-00122",
  "paymentNumber": "PAY-2026-00321",
  "currency": "USD",
  "amount": 10000
}
```

Metadata should not become an uncontrolled storage area for complete database objects.

Store only information useful for investigation.

21.31 Old and New Values

For update events, Audit Logs may store changed values.

Example:

```
{
  "oldValue": {
```

```
    "status": "active"
  },
  "newValue": {
    "status": "disabled"
  }
}
```

Avoid storing the entire entity where only one field changed.

Prefer:

changed fields only

This reduces unnecessary sensitive-data duplication.

21.32 Sensitive Data Exclusion

Audit Logs must never store:

passwords

password hashes

OTP codes

JWT tokens

refresh tokens

API secrets

private keys

full card details

CVV

gateway secret keys

bank authentication credentials

Sensitive uploaded documents should not normally be copied into Audit Logs.

Audit may instead reference:

fileId

where authorized investigation requires it.

21.33 IP Address and User Agent

Where available and permitted, Audit Logs may record:

ipAddress

userAgent

Example:

```
{  
  "ipAddress": "102.xxx.xxx.xxx",  
  "userAgent": "Mozilla/5.0 ..."  
}
```

These fields are primarily useful for:

- security investigations;
- suspicious Admin activity;
- account investigations.

Access should be restricted.

21.34 Request Correlation

Each API request may have a unique:

requestId

Example:

req_93hd83hd

Audit records can store this value.

This allows technical logs and business Audit records to be correlated during investigation.

21.35 Get Audit Logs

Endpoint

GET /admin/audit-logs

Required Permission

audit_logs.view

Query Parameters

search

eventCode

actorId

actorType

module

action

entityType

entityId

dateFrom

dateTo

page

limit

Example:

GET

/admin/audit-logs?module=payments&eventCode=PAYMENT_VERIFIED&page=1&limit=20

21.36 Get Audit Logs Response

```
{
  "success": true,

  "data": [
    {
      "id": "audit_123",

      "eventCode": "PAYMENT_VERIFIED",

      "actor": {
        "id": "admin_finance_1",
        "type": "admin",
        "email": "finance@tofa.com"
      },

      "module": "payments",

      "action": "verify",

      "entity": {
        "type": "payment",
        "id": "payment_123"
      },

      "description": "Manual payment was verified.",

      "metadata": {
        "paymentFor": "order",
        "amount": 10000,
        "currency": "USD"
      },

      "ipAddress": "102.xxx.xxx.xxx",

      "createdAt": "2026-08-28T10:00:00Z"
    }
  ],
}
```

```
"pagination": {  
  "page": 1,  
  "limit": 20,  
  "total": 215,  
  "totalPages": 11  
}  
}
```

21.37 Get Audit Log Details

Endpoint

GET /admin/audit-logs/:auditLogId

Required Permission

audit_logs.view

Response

```
{  
  "success": true,  
  
  "data": {  
    "id": "audit_123",  
  
    "eventCode": "USER_DISABLED",  
  
    "actor": {  
      "id": "admin_100",  
      "type": "admin",  
      "email": "admin@tofa.com"  
    },  
  
    "module": "users",  
  
    "action": "disable",  
  
    "entity": {  
      "type": "user",  
      "id": "usr_123"  
    },  
  },  
}
```

```
"description": "User account was disabled by an Admin.",
"reason": "Fraudulent activities",
"oldValue": {
  "status": "active"
},
"newValue": {
  "status": "disabled"
},
"metadata": {},
"ipAddress": "102.xxx.xxx.xxx",
"userAgent": "Mozilla/5.0 ...",
"requestId": "req_93hd83hd",
"createdAt": "2026-08-28T10:00:00Z"
}
}
```

21.38 Entity Audit History

Admins may need to see Audit history for a specific record.

Endpoint

GET /admin/audit-logs/entity/:entityType/:entityId

Required Permission

audit_logs.view

Example:

GET /admin/audit-logs/entity/payment/payment_123

Response returns chronological Audit history for that entity.

This is useful on Admin detail pages.

Example:

Payment Created

Proof Uploaded

Admin Viewed

Payment Verified

Only meaningful audited actions should appear.

21.39 Admin Activity History

Endpoint

GET /admin/audit-logs/admin/:adminId

Required Permission

audit_logs.view

This allows authorized investigators to review actions performed by a specific Admin.

Query Parameters

module

eventCode

dateFrom

dateTo

page

limit

21.40 Audit Log Export

Authorized Admins may export Audit records for investigation or compliance purposes.

Endpoint

POST /admin/audit-logs/export

Required Permission

audit_logs.export

Payload

```
{  
  "format": "xlsx",  
  
  "filters": {  
    "module": "payments",  
    "dateFrom": "2026-08-01",  
    "dateTo": "2026-08-31"  
  }  
}
```

Supported formats may include:

CSV

XLSX

PDF

Large exports should be asynchronous.

21.41 Audit Export Response

```
{  
  "success": true,  
  
  "message": "Audit log export is being prepared.",  
  
  "data": {  
    "reportId": "report_audit_123",  
    "status": "processing"  
  }  
}
```

The existing Notification infrastructure may notify the Admin when the report is ready.

21.42 Audit Export Security

Audit exports may contain sensitive operational information.

Therefore:

- export requires explicit Permission;
- generated files should use protected access;
- download links should expire;
- exports should themselves create an Audit event.

Recommended event:

AUDIT_LOG_EXPORTED

21.43 Audit Permissions

Recommended Permissions:

audit_logs.view

audit_logs.export

TOFA may later introduce more granular Permissions such as:

audit_logs.view_financial

audit_logs.view_security

if stronger separation is required.

Super Admin automatically has access.

21.44 Frontend Audit Dashboard

Recommended Admin interface:

Audit Logs

Search

Module Filter

Event Filter

Actor Filter

Date Range

Entity Type

Export

Each row may show:

Timestamp

Actor

Event

Module

Entity

Description

Clicking a row opens full Audit details.

21.45 Audit Timeline

For an entity detail page, frontend may display:

28 Aug 2026 10:00

Payment created

28 Aug 2026 10:15

Payment proof uploaded by Buyer

28 Aug 2026 11:05

Payment verified by Finance Admin

This timeline is derived from Audit Logs.

21.46 Audit and Analytics Difference

Audit Logs and Analytics serve different purposes.

Audit

Answers:

Who did what and when?

Analytics

Answers:

How is the Marketplace performing?

Example:

Audit:

Admin Victor verified Payment PAY-123 at 10:15.

Analytics:

Total successful Payments this month = 1,500.

Analytics should not be built directly by repeatedly scanning Audit Logs where proper transactional/analytics data already exists.

21.47 Audit and Notification Difference

Audit records system activity.

Notification informs Users/Admins.

Example:

Refund successful

may create:

Audit Event

+

Buyer Notification

They are independent.

Failure to send a Notification should not remove the Audit history of the Refund.

21.48 Audit and Message Center

Private Message Center content should not be copied into Audit Logs.

Audit may record safe metadata for important administrative actions where necessary.

Example:

MESSAGE_REPORTED

But not:

full private conversation

unless a specific authorized moderation/investigation process requires access through the Message Center itself.

21.49 Audit and Disputes

Dispute has its own case timeline.

Audit provides system-level traceability.

Example:

Dispute timeline may show:

Buyer submitted evidence

Admin requested Seller response
Dispute resolved

Audit additionally identifies technical/admin context such as:

Admin ID
IP
requestId
old/new state

The two may reference the same event but serve different purposes.

21.50 Audit Retention

Audit Logs should be retained for a configurable period according to TOFA's legal, financial, security and operational requirements.

Recommended System Setting concept:

auditLogRetentionPeriod

However, financial and security records may require longer retention than general operational Audit records.

The final retention period should be approved based on applicable legal/compliance requirements.

21.51 Archived Users and Admins

Audit Logs must remain valid when:

- User deletes their account;
- Admin is deactivated;
- Seller is disabled;
- Product is deleted;
- Role is deactivated.

Do not delete Audit history simply because the referenced entity is no longer active.

Where appropriate, limited snapshot data helps maintain historical context.

21.52 System-Generated Audit Example

Example:

```
{
  "eventCode": "SUBSCRIPTION_EXPIRED",

  "actorId": null,

  "actorType": "system",

  "module": "subscriptions",

  "action": "expire",

  "entityType": "subscription",

  "entityId": "sub_123",

  "description": "Subscription expired automatically.",

  "oldValue": {
    "status": "active"
  },

  "newValue": {
    "status": "expired"
  },

  "createdAt": "2026-08-28T00:00:00Z"
}
```

21.53 Failed Actions

Not every failed business action needs a permanent Audit Log.

However, security-sensitive failures should be recorded.

Examples:

ADMIN_LOGIN_FAILED

ADMIN_PERMISSION_DENIED

ADMIN_ACCOUNT_LOCKED

SUSPICIOUS_PAYMENT_WEBHOOK

REPEATED_LOGIN_FAILURE

High-volume technical failures may belong in security/application logs rather than the main business Audit table.

21.54 Permission Denied Audit

Sensitive unauthorized Admin attempts may be audited.

Example:

```
{
  "eventCode": "ADMIN_PERMISSION_DENIED",

  "actorId": "admin_102",

  "actorType": "admin",

  "module": "payments",

  "action": "verify",

  "entityType": "payment",

  "entityId": "payment_123",

  "description": "Admin attempted to verify a Payment without the required Permission.",

  "metadata": {
    "requiredPermission": "payments.verify"
  }
}
```

This is particularly useful for sensitive financial and security actions.

21.55 Audit Record Creation Failure

Failure to write Audit records for critical actions must be monitored.

The architecture should provide:

- retry;
- dead-letter/error handling where event queues are used;
- monitoring;
- reconciliation.

Audit reliability should be treated as a backend infrastructure concern.

21.56 Audit Reconciliation

For critical financial/admin actions, periodic reconciliation may compare source records against Audit records.

Examples:

successful Refund
but no REFUND_SUCCESSFUL Audit

or:

verified manual Payment
but no PAYMENT_VERIFIED Audit

The system can flag these inconsistencies for technical investigation.

21.57 Audit Database Indexing

Recommended indexes include:

eventCode

actorId

actorType

module

entityType + entityId

createdAt

requestId

Composite indexes may be added based on actual query patterns.

Audit tables can become large and should be designed for efficient historical querying.

21.58 Audit Storage Scaling

As TOFA grows, Audit volume may become significant.

Backend architecture may later support:

- partitioning by date;
- archive storage;
- dedicated Audit database;
- cold storage;
- search indexing.

The API contract does not need to change when the storage implementation changes.

21.59 Backend Implementation Guide

Backend should:

- centralize Audit record creation;
- use stable event codes;
- capture actor information;
- capture affected entity;
- capture relevant changed values;
- exclude secrets;
- make records immutable;
- use reliable event delivery;
- support idempotency;
- support request correlation;
- index commonly filtered fields;
- enforce Audit viewing Permissions;
- support entity history;
- support Admin activity history;
- support secure exports;
- preserve Audit history for inactive/deleted entities;
- monitor Audit creation failures.

Backend should not:

- allow clients to directly create authoritative Audit Logs;
- expose update/delete Audit endpoints;
- store passwords or tokens;
- store unnecessary full entity snapshots;
- use Audit Logs as a replacement for business tables;
- use Audit Logs as the primary Analytics database.

21.60 Frontend Implementation Guide

Frontend should:

- provide Audit Log list for authorized Admins;
- support search;
- support filters;
- support date range;
- show actor;
- show event;
- show module;
- show entity;
- show timestamp;
- provide detail view;
- show old/new values clearly;
- support entity Audit timeline;
- support Admin activity history;
- support export where permitted;
- hide Audit navigation without Permission;
- never allow editing/deleting Audit records.

21.61 Important Consistency Rules

- Audit Logs are immutable.
- Audit Logs are not normal application logs.
- Audit Logs do not replace transactional business data.
- Audit Logs do not replace Analytics.
- Audit Logs do not replace Dispute timelines.
- Audit Logs do not replace Delivery tracking.
- Backend creates authoritative Audit records.
- Frontend cannot create arbitrary Audit history.
- Sensitive values must never be stored.
- Old/new values should contain relevant changed fields only.
- Admin actions must identify the responsible Admin.

- System actions use `actorType = system`.
- Audit records remain after Users/Admins/entities become inactive or deleted.
- Critical Audit event delivery must be reliable.
- Duplicate events must not create duplicate records.
- Audit access requires Permission.
- Audit exports require separate Permission.
- Audit exports should themselves be audited.
- Financial Audit records require particular care.
- Audit retention should be configurable and policy-driven.

21.62 Module Summary

The Audit Log Management Module establishes that:

- TOFA maintains centralized Audit history for important Marketplace actions;
- Audit records identify who performed an action, what changed and when;
- Users, Admins and System processes can be Audit actors;
- Audit Logs use stable event codes;
- important events across Authentication, Users, Admin, Verification, Products, Categories, Payments, Orders, Delivery, RFQs, Reviews, Subscriptions, Analytics, Disputes, Cancellations, Returns, Refunds and Settings can be recorded;
- Admin financial actions are auditable;
- Refund and transaction fee adjustments are auditable;
- sensitive security events may be audited;
- Audit records cannot normally be edited or deleted;
- secrets, passwords, OTPs and tokens must never be stored;
- changed fields may be stored using old/new values;
- request IDs can correlate Audit records with technical logs;
- authorized Admins can search and filter Audit history;
- authorized Admins can view an entity's Audit timeline;
- authorized Admins can review Admin activity;
- Audit exports are protected and auditable;
- Audit history survives entity deactivation/deletion;
- event processing should be reliable and idempotent;
- Audit storage can scale independently as TOFA grows;
- Module 19 controls Admin access to Audit functionality;
- the next System Settings module can centralize configurable operational rules referenced throughout the Marketplace.

MODULE 22. SYSTEM SETTINGS

22. SYSTEM SETTINGS & MARKETPLACE CONFIGURATION MODULE

22.1 Overview

The System Settings & Marketplace Configuration Module provides a centralized way for authorized TOFA Admins to manage operational settings that should be configurable without changing application code.

Several Marketplace modules depend on configurable values such as:

- supported countries;
- supported currencies;
- supported languages;
- payment methods;
- default Marketplace language;
- dispute window;
- return window;
- refund operational rules;
- RFQ configuration;
- logistics configuration;
- subscription defaults;
- seller settlement configuration;
- payout processing configuration;
- file upload limits;
- notification settings;
- security-related operational settings;
- Audit retention settings.

This module becomes the source of truth for platform-wide configuration.

It does not replace the business logic of the modules consuming these settings.

The relationship is:

System Settings



Marketplace Modules



Operational Behaviour

Example:

`returnWindowDays = 7`

is configured here.

Return Management reads and applies the value.

Similarly:

`settlementHoldDays = 2`

is configured here.

Seller Settlement Management reads and applies the value.

22.2 Core Principles

System configuration must follow these principles:

- operational values should not be unnecessarily hardcoded;
- configuration should be centrally manageable;
- only authorized Admins can modify protected settings;
- backend remains authoritative;
- sensitive configuration must not be exposed publicly;
- all settings must be validated before saving;
- configuration changes must be audited;
- configuration changes should normally apply prospectively;
- historical transactions must not be recalculated because current settings change;
- Marketplace modules remain responsible for their own business logic;
- System Settings provides configuration rather than duplicated workflows.

22.3 Setting Categories

Recommended categories:

general

marketplace

localization

country

currency

payment

order

rfq

delivery

dispute

return

refund

subscription

settlement

payout

notification

security

file_upload

audit

Additional categories may be introduced as the Marketplace grows.

22.4 System Settings Table

Table: **system_settings**

Field	Type	Required	Notes
id	uuid	Yes	Primary key
key	varchar	Yes	Unique stable setting key

category	varchar	Yes	Setting category
value	json/jsonb	Yes	Current value
valueType	enum	Yes	string / number / boolean / array / object
description	text	No	Setting explanation
isPublic	boolean	Yes	Whether public/client APIs may retrieve it
isEditable	boolean	Yes	Whether authorized Admins may modify it
updatedBy	uuid	No	Admin that last changed it
createdAt	timestamp	Yes	Creation
updatedAt	timestamp	Yes	Last update

Example:

```
{
  "key": "settlementHoldDays",
  "category": "settlement",
  "value": 2,
  "valueType": "number",
  "description": "Number of days after Order completion before an eligible Seller Settlement can be processed.",
  "isPublic": false,
  "isEditable": true
}
```

22.5 Setting Key Rules

Setting keys should be stable and unique.

Examples:

marketplaceName

defaultLanguage

defaultCountry

defaultCurrency

returnWindowDays

disputeWindowDays

settlementHoldDays

automaticSettlementEnabled

minimumPayoutAmount

adminSetupTokenExpiryHours

passwordResetOtpExpiryMinutes

maxLoginAttempts

loginLockMinutes

auditLogRetentionDays

Backend services should reference stable setting keys rather than database IDs.

22.6 Public vs Private Settings

Some settings may safely be exposed to the frontend.

Examples:

supportedCountries

supportedCurrencies

supportedLanguages

returnWindowDays

disputeWindowDays

Other settings should remain private.

Examples:

settlementHoldDays

automaticSettlementEnabled

minimumPayoutAmount

security thresholds

internal operational controls

provider configuration

Public settings should only include information Users need to interact with the Marketplace.

API responses must never expose:

- API secrets;
- private keys;
- gateway secret keys;
- payout provider secrets;
- webhook secrets;
- database credentials;
- cloud credentials.

Secrets should be stored using appropriate secure secret-management infrastructure rather than normal System Settings records.

22.7 Get Public Marketplace Configuration

Endpoint

GET /config

No Admin authentication required.

Only safe public settings are returned.

Response

```
{  
  "success": true,  
  "data": {  
    "marketplace": {  
      "name": "Traders of Africa",
```



```
    "defaultLanguage": "en"  
  },
```

```
  "countries": [  
    {  
      "code": "NG",  
      "name": "Nigeria"  
    },  
    {  
      "code": "GH",  
      "name": "Ghana"  
    }  
  ],
```

```
  "currencies": [  
    {  
      "code": "NGN",  
      "name": "Nigerian Naira"  
    },  
    {  
      "code": "GHS",  
      "name": "Ghanaian Cedi"  
    },  
    {  
      "code": "USD",  
      "name": "US Dollar"  
    }  
  ],
```

```
  "languages": [  
    {  
      "code": "en",  
      "name": "English"  
    },  
    {  
      "code": "fr",  
      "name": "French"  
    }  
  ],
```

```
  "policies": {  
    "returnWindowDays": 7,  
    "disputeWindowDays": 7  
  }
```

```
}  
}
```

Private Settlement configuration should not normally appear here.

22.8 Admin Get System Settings

Endpoint

GET /admin/settings

Required Permission

settings.view

Query Parameters

category
search
page
limit

Example:

GET /admin/settings?category=settlement

Response

```
{  
  "success": true,  
  "data": [  
    {  
      "id": "setting_123",  
      "key": "settlementHoldDays",  
      "category": "settlement",  
      "value": 2,  
      "valueType": "number",  
      "description": "Number of days after Order completion before an eligible Settlement can be  
processed.",  
      "isPublic": false,  
      "isEditable": true,  
    }  
  ]  
}
```

```
    "updatedAt": "2026-08-28T10:00:00Z"
  }
]
}
```

22.9 Get Setting Details

Endpoint

GET /admin/settings/:key

Required Permission

settings.view

Response

```
{
  "success": true,
  "data": {
    "key": "settlementHoldDays",
    "category": "settlement",
    "value": 2,
    "valueType": "number",
    "description": "Settlement hold period after eligible Order completion.",
    "isPublic": false,
    "isEditable": true
  }
}
```

22.10 Update System Setting

Endpoint

PATCH /admin/settings/:key

Required Permission

settings.update

Payload

```
{  
  "value": 3  
}
```

Backend Processing

1. Authenticate Admin.
2. Validate Permission.
3. Retrieve setting.
4. Confirm `isEditable = true`.
5. Validate the value against setting rules.
6. Preserve previous value for Audit.
7. Update the setting.
8. Store `updatedBy`.
9. Emit Audit event.
10. Invalidate relevant cache.
11. Notify affected services where event-driven configuration refresh is used.

Response

```
{  
  "success": true,  
  "message": "System setting updated successfully.",  
  "data": {  
    "key": "settlementHoldDays",  
    "value": 3  
  }  
}
```

Toast

System setting updated successfully.

22.11 Historical Configuration Rule

Configuration changes must not rewrite historical transactions.

Example:

Transaction created:

transactionFeePercentage = 5%

Later:

Seller Subscription fee changes to 3%

The old transaction remains:

5%

Likewise, historical values such as:

Accepted RFQ Quote

Delivery Quote

Buyer Payment Currency

Historical FX

Transaction Fee

Refund

Completed Payout

must remain unchanged.

Settlement configuration should also respect historical states.

Example:

Order completed when settlementHoldDays = 2

If TOFA later changes:

settlementHoldDays = 3

recommended behaviour is that already-calculated **eligibleAt** values remain unchanged.

The new rule applies to future eligibility calculations unless TOFA explicitly performs a controlled migration.

22.12 Country Management

Supported Marketplace countries should be configurable.

Table: **countries**

Field	Type	Required
id	uuid	Yes
code	varchar	Yes
name	varchar	Yes
phoneCode	varchar	No
status	enum	Yes
createdAt	timestamp	Yes
updatedAt	timestamp	Yes

Recommended status:

active
inactive

Example:

```
{
  "code": "NG",
  "name": "Nigeria",
  "phoneCode": "+234",
  "status": "active"
}
```

22.13 Get Countries

Endpoint

GET /countries

Returns active supported countries.

Response

```
{
  "success": true,
  "data": [
    {
      "code": "NG",
      "name": "Nigeria",
      "phoneCode": "+234"
    },
    {
      "code": "GH",
      "name": "Ghana",
      "phoneCode": "+233"
    }
  ]
}
```

22.14 Admin Manage Countries

Create Country

POST /admin/countries

Required Permission:

settings.countries.manage

Payload:

```
{
  "code": "KE",
  "name": "Kenya",
  "phoneCode": "+254"
}
```

Update Country

PATCH /admin/countries/:countryId

Change Country Status

PATCH /admin/countries/:countryId/status

Payload:

```
{
  "status": "inactive"
}
```

Historical records referencing a country remain valid.

22.15 Currency Management

Supported currencies must be dynamic.

Table: **currencies**

Field	Type	Required
id	uuid	Yes
code	varchar	Yes
name	varchar	Yes
symbol	varchar	No
decimalPlaces	integer	Yes
status	enum	Yes
createdAt	timestamp	Yes
updatedAt	timestamp	Yes

Example:

```
{
```



```
"code": "NGN",
"name": "Nigerian Naira",
"symbol": "₦",
"decimalPlaces": 2,
"status": "active"
}
```

22.16 Get Supported Currencies

Endpoint

GET /currencies

Response

```
{
  "success": true,
  "data": [
    {
      "code": "NGN",
      "name": "Nigerian Naira",
      "symbol": "₦"
    },
    {
      "code": "USD",
      "name": "US Dollar",
      "symbol": "$"
    }
  ]
}
```

22.17 Currency Rules

If a currency becomes inactive, it cannot normally be selected for new transactions.

Historical:

- Orders;
- Payments;
- Settlements;
- Refunds;

- Payouts;

using that currency remain valid.

No historical financial values should be automatically converted.

22.18 Admin Manage Currencies

Create

POST /admin/currencies

Update

PATCH /admin/currencies/:currencyId

Change Status

PATCH /admin/currencies/:currencyId/status

Required Permission:

settings.currencies.manage

22.19 Language Configuration

Initial supported Marketplace languages include:

English

French

Arabic

Swahili

Portuguese

The architecture should support additional languages later.

Table: languages

Field	Type	Required
id	uuid	Yes
code	varchar	Yes
name	varchar	Yes
nativeName	varchar	No
direction	enum	Yes
status	enum	Yes

Direction:

ltr
rtl

Arabic uses:

rtl

22.20 Payment Method Configuration

Payment Management remains responsible for Payment processing.

This module controls which configured Payment methods are available.

Examples may include:

bank_transfer

flutterwave

paystack

transactworld

papss

RFQ transactions may additionally use agreed methods such as:

LC

TT

where applicable.

Table: `payment_method_configs`

Field	Type	Required
id	uuid	Yes
code	varchar	Yes
displayName	varchar	Yes
status	enum	Yes
supportedCurrencies	json/jsonb	Yes
paymentContext	json/jsonb	No
sortOrder	integer	No
createdAt	timestamp	Yes
updatedAt	timestamp	Yes

Provider secrets must not be stored here.

22.21 Payment Method Availability

The Payment Module may determine available Payment methods using:

selected currency

transaction type

Buyer country

provider availability

Admin configuration

System Settings supplies configuration.

Payment Management remains responsible for actual processing.

22.22 RFQ Configuration

Recommended configurable RFQ values:

defaultQuoteValidityDays

maximumRFQAttachments

RFQExpiryReminderEnabled

Where a global response cap exists:

maximumMarketRFQResponses

may be configured.

However, Subscription-specific Market RFQ response limits remain owned by Subscription entitlements.

22.23 Dispute Configuration

Recommended:

disputeWindowDays

maximumDisputeEvidenceFiles

disputeAutoCloseDays

Example:

```
{  
  "disputeWindowDays": 7,  
  "maximumDisputeEvidenceFiles": 10  
}
```

Dispute Management remains responsible for interpreting and enforcing these rules.

22.24 Return Configuration

Recommended:

returnWindowDays

maximumReturnEvidenceFiles

Example:

```
{  
  "returnWindowDays": 7,  
  "maximumReturnEvidenceFiles": 10  
}
```

Return Management remains responsible for Return eligibility.

22.25 Refund Configuration

Recommended:

manualRefundRequiresApproval

refundNotificationEnabled

automaticRefundEnabled

maximumAutomaticRefundAmount

These settings should be treated as sensitive financial configuration.

Highly sensitive Refund configuration may be restricted to Super Admin or Finance-authorized roles.

Refund Management remains responsible for Refund lifecycle.

22.26 Subscription Configuration

Subscription Management remains the source of truth for Plans and entitlements.

System Settings may define:

defaultFreePlanId

subscriptionExpiryReminderDays

Plan-specific settings such as:

transaction_fee_percentage

max_products

max_images_per_product

market_rfq_response_limit

analytics_access

remain inside Subscription Plans.

They should not be duplicated here.

22.27 Delivery Configuration

Recommended:

integratedLogisticsEnabled

sellerArrangedLogisticsEnabled

buyerArrangedLogisticsEnabled

Delivery provider API configuration remains owned by the Delivery integration.

System Settings only controls high-level availability.

22.28 Seller Settlement Configuration

Seller Settlement Management, defined in Module 25, owns the settlement and payout lifecycle.

This module only stores the operational rules used by that lifecycle.

Recommended Seller Settlement settings:

settlementHoldDays

automaticSettlementEnabled

settlementProcessingEnabled

minimumPayoutAmount

Additional optional settings:

settlementProcessingDays

settlementProcessingTime

settlementRequiresVerifiedSeller

settlementRequiresVerifiedPayoutAccount

22.29 settlementHoldDays

Defines the number of days after an Order reaches the qualifying completion state before the Seller Settlement becomes eligible for payout.

Example:

```
{
  "key": "settlementHoldDays",
  "category": "settlement",
  "value": 2,
  "valueType": "number"
}
```

Example flow:

Order completed:

28 August

settlementHoldDays:

2

eligibleAt:
30 August

This does not override:

- active Disputes;
- pending Refunds;
- compliance holds;
- Return holds;
- invalid payout accounts.

22.30 Settlement Hold Rule

The Settlement Module should calculate and store:

eligibleAt

when the qualifying Order event occurs.

Recommended:

eligibleAt
=
orderCompletedAt
+
settlementHoldDays

Once stored, **eligibleAt** should not normally change if the global setting changes later.

This preserves predictable historical transaction processing.

22.31 **automaticSettlementEnabled**

Controls whether an eligible Seller Settlement can automatically proceed into payout processing.

Example:

{

```
"key": "automaticSettlementEnabled",  
"category": "settlement",  
"value": false,  
"valueType": "boolean"  
}
```

If:

false

flow may be:

Settlement eligible

↓

Payout created as pending

↓

Finance/Admin approval

↓

Processing

If:

true

flow may be:

Settlement eligible

↓

Final validation

↓

Payout automatically approved/processed according to configured controls

Enabling automatic settlement must not bypass:

- Settlement holds;
- payout account validation;
- Refund checks;
- Dispute checks;
- duplicate payout prevention;
- financial limits;
- provider availability.

22.32 Recommended Initial Value

For an early Marketplace rollout, recommended:

`automaticSettlementEnabled = false`

This allows Finance/Admin review before Seller funds are released.

Automation can be introduced after transaction reconciliation and operational controls are proven stable.

22.33 `settlementProcessingEnabled`

This acts as an operational master switch for Seller payout processing.

Example:

```
{
  "key": "settlementProcessingEnabled",
  "category": "settlement",
  "value": true,
  "valueType": "boolean"
}
```

If temporarily set to:

`false`

the system should stop initiating new payouts.

It must not:

- cancel existing successful payouts;
- reverse completed payouts;
- delete eligible Settlements.

Eligible Settlements remain available until processing is resumed.

This can be useful during:

- provider downtime;
- reconciliation issues;

- maintenance;
- temporary banking restrictions.

22.34 minimumPayoutAmount

Defines the minimum amount that may be paid out in a given currency if TOFA chooses to enforce payout thresholds.

Because different currencies have different values, one global numeric setting may be insufficient.

Recommended structure:

```
{
  "key": "minimumPayoutAmounts",
  "category": "payout",
  "value": {
    "NGN": 5000,
    "GHS": 50,
    "USD": 10
  },
  "valueType": "object"
}
```

If TOFA does not need a minimum payout threshold initially, this may be disabled or set appropriately.

22.35 Minimum Payout Behaviour

Example:

Seller eligible amount = NGN 3,000

Minimum payout = NGN 5,000

The Settlement remains payable but payout may wait until a permitted batching/next-payout model exists.

However, Module 25 currently recommends:

1 Seller Order
=

1 Settlement
=
1 Payout

for v1.

Therefore, if TOFA wants to keep v1 simple, `minimumPayoutAmounts` should not be enabled initially unless there is a clear operational reason.

22.36 Settlement Processing Days

Optional setting:

`settlementProcessingDays`

Example:

```
{  
  "key": "settlementProcessingDays",  
  "category": "settlement",  
  "value": [  
    "monday",  
    "tuesday",  
    "wednesday",  
    "thursday",  
    "friday"  
  ],  
  "valueType": "array"  
}
```

This may be useful where Finance processes payouts only on banking/business days.

If omitted, Settlement may process whenever eligible.

22.37 Settlement Processing Time

Optional:

`settlementProcessingTime`

Example:

```
{
  "key": "settlementProcessingTime",
  "category": "settlement",
  "value": "14:00",
  "valueType": "string"
}
```

The applicable timezone must be explicit in backend configuration.

This should only be introduced if TOFA has a fixed payout processing schedule.

22.38 settlementRequiresVerifiedSeller

Recommended:

```
{
  "key": "settlementRequiresVerifiedSeller",
  "category": "settlement",
  "value": true,
  "valueType": "boolean"
}
```

If enabled, Seller Settlement requires the Seller to satisfy the Marketplace's applicable verification requirements.

Seller Verification remains responsible for verification status.

Settlement Management simply consumes it.

22.39 settlementRequiresVerifiedPayoutAccount

Recommended:

```
{
  "key": "settlementRequiresVerifiedPayoutAccount",
  "category": "payout",
  "value": true,
  "valueType": "boolean"
}
```

If true:

Payout account status must equal verified

before payout processing.

This is strongly recommended.

22.40 Payout Configuration

Additional payout configuration may include:

manualPayoutRequiresApproval

payoutRetryEnabled

maximumAutomaticPayoutAmount

Example:

```
{
  "manualPayoutRequiresApproval": true,
  "payoutRetryEnabled": true,
  "maximumAutomaticPayoutAmount": {
    "NGN": 5000000,
    "USD": 5000
  }
}
```

These are operational controls.

Payout Management still owns the actual payout state.

22.41 Automatic Payout Limit

If TOFA later enables automatic settlement, it may be useful to limit automatic payouts.

Example:

automaticSettlementEnabled = true

maximumAutomaticPayoutAmount.NGN = 5,000,000

Then:

Payout = NGN 3,000,000

→ may auto-process

while:

Payout = NGN 10,000,000

→ requires Admin approval

This provides an additional financial control.

22.42 Payout Approval Configuration

Potential:

payoutApprovalMode

Allowed values:

single_approval

maker_checker

single_approval

One authorized Admin approves/processes according to Permission design.

maker_checker

One Admin approves and another Admin processes/confirms.

Module 19 Permissions still determine which Admins can perform each action.

This setting only defines operational policy.

22.43 Recommended Initial Payout Approval Mode

For high-value B2B transactions, recommended:

maker_checker

where operationally feasible.

Example:

Finance Reviewer

↓

Approves payout

↓

Finance Processor

↓

Processes payout

The same Admin should not complete both stages where strict separation of duties is required.

22.44 Settlement Hold Configuration

System-generated Settlement hold types may be configured/enabled.

Recommended default blocking conditions:

active_dispute

approved_return

pending_refund

fraud_review

seller_compliance_issue

invalid_payout_account

payment_reversal

These should preferably be recognized backend hold codes rather than arbitrary Admin-created strings.

22.45 Hold Behaviour

System Settings may define whether a hold type is blocking.

Example conceptual configuration:

```
{
  "active_dispute": {
    "blocksSettlement": true
  },
  "pending_refund": {
    "blocksSettlement": true
  },
  "invalid_payout_account": {
    "blocksSettlement": true
  }
}
```

For initial implementation, these core blocking rules may remain application-defined rather than fully Admin-editable, because financial safety rules should not be accidentally disabled.

22.46 Settlement Currency Rules

System Settings should not override the transaction's stored Settlement currency.

Currency configuration only determines whether a currency is currently supported for new transactions.

Existing Settlement records continue using their stored:

settlementCurrency

even if that currency later becomes inactive.

22.47 Historical FX and Settings

FX configuration may change over time.

Settlement Management must use:

historical transaction FX snapshot

not current Marketplace FX configuration.

System Settings may define FX providers or supported conversion pairs elsewhere, but cannot alter existing transaction snapshots.

22.48 Payout Provider Configuration

High-level payout provider availability may be configured.

Example:

paystack_transfer
flutterwave_transfer
bank_manual

However:

- provider API keys;
- secret keys;
- private credentials;
- signing secrets;

must live in secure secret infrastructure.

System Settings may only store safe operational values such as:

providerEnabled

supportedCurrencies

supportedCountries

22.49 File Upload Configuration

Recommended:

maxImageSizeMb

maxDocumentSizeMb

allowedImageTypes

allowedDocumentTypes

maximumFilesPerUpload

Example:

```
{
  "maxImageSizeMb": 10,
  "maxDocumentSizeMb": 20,
  "allowedImageTypes": [
    "image/jpeg",
    "image/png",
    "image/webp"
  ],
  "allowedDocumentTypes": [
    "application/pdf"
  ]
}
```

Subscription-specific Product image limits remain controlled by Subscription entitlements.

22.50 Security Configuration

Operational security values may include:

userOtpExpiryMinutes

adminOtpExpiryMinutes

maxLoginAttempts

loginLockMinutes

adminSetupTokenExpiryHours

Security settings must have safe backend constraints.

Example:

maxLoginAttempts
minimum = 3
maximum = 10

Admins should not be able to weaken security beyond approved limits.

22.51 Audit Configuration

Potential:

auditLogRetentionDays

auditExportExpiryMinutes

Financial and security Audit records may require separate retention policies.

Legal/compliance requirements should determine actual retention.

22.52 Setting Validation

Each setting must have validation rules.

Recommended metadata:

minimumValue

maximumValue

allowedValues

validationSchema

Examples:

returnWindowDays:

1–90

settlementHoldDays:

0–30

payoutApprovalMode:

single_approval | maker_checker

22.53 Settlement Setting Validation

Recommended validation examples:

settlementHoldDays

integer

minimum = 0

maximum = 30

automaticSettlementEnabled

boolean

settlementProcessingEnabled

boolean

minimumPayoutAmounts

object

currency code → non-negative decimal

maximumAutomaticPayoutAmount

object

currency code → positive decimal

payoutApprovalMode

single_approval

maker_checker

22.54 Configuration Caching

Frequently accessed settings may be cached.

Recommended flow:

Admin updates setting

↓

Database updated

↓
Audit event emitted
↓
Cache invalidated
↓
Affected service reads updated setting

Financially sensitive values should have short or explicitly invalidated caches.

22.55 Configuration Versioning

For high-impact operational settings, the system may maintain configuration version/history.

Recommended for settings such as:

settlementHoldDays

automaticSettlementEnabled

payoutApprovalMode

returnWindowDays

disputeWindowDays

Module 21 already preserves Audit old/new values.

A separate version table is optional unless configuration rollback becomes necessary.

22.56 Admin Permissions

Recommended Permissions:

settings.view

settings.update

settings.countries.manage

settings.currencies.manage

settings.languages.manage

settings.payment_methods.manage

settings.settlement.manage

settings.payout.manage

Sensitive Settlement/Payout configuration should be limited to appropriately authorized roles or Super Admin.

22.57 Settlement Configuration Permission

Recommended:

settings.settlement.manage

for:

settlementHoldDays

automaticSettlementEnabled

settlementProcessingEnabled

settlementProcessingDays

settlementProcessingTime

settlementRequiresVerifiedSeller

22.58 Payout Configuration Permission

Recommended:

settings.payout.manage

for:

minimumPayoutAmounts

maximumAutomaticPayoutAmount

payoutApprovalMode

payoutRetryEnabled

settlementRequiresVerifiedPayoutAccount

22.59 Audit Events

Recommended:

SYSTEM_SETTING_UPDATED

COUNTRY_CREATED

COUNTRY_UPDATED

COUNTRY_ACTIVATED

COUNTRY_DEACTIVATED

CURRENCY_CREATED

CURRENCY_UPDATED

CURRENCY_ACTIVATED

CURRENCY_DEACTIVATED

LANGUAGE_ACTIVATED

LANGUAGE_DEACTIVATED

PAYMENT_METHOD_ACTIVATED

PAYMENT_METHOD_DEACTIVATED

SETTLEMENT_CONFIGURATION_UPDATED

PAYOUT_CONFIGURATION_UPDATED

MARKETPLACE_CONFIGURATION_UPDATED

Module 21 stores Audit records.

22.60 Settlement Setting Audit Example

```
{
  "eventCode": "SETTLEMENT_CONFIGURATION_UPDATED",
  "actorId": "admin_super_1",
  "actorType": "admin",
  "module": "system_settings",
  "action": "update",
  "entityType": "system_setting",
  "entityId": "setting_settlement_hold_days",
  "description": "Seller settlement hold period was updated.",
  "oldValue": {
    "settlementHoldDays": 2
  },
  "newValue": {
    "settlementHoldDays": 3
  }
}
```

22.61 High-Risk Configuration Changes

Changes to financial settings should require stronger frontend confirmation.

Example:

You are changing the Seller Settlement Hold Period
from 2 days to 0 days.

This affects future Seller payout eligibility.

Continue?

Potentially high-risk settings include:

automaticSettlementEnabled

settlementProcessingEnabled

settlementHoldDays

maximumAutomaticPayoutAmount

payoutApprovalMode

22.62 Configuration Change Scope

Before updating a high-impact setting, backend/frontend should clearly define whether it affects:

future transactions only

or:

existing pending records

Recommended default:

prospective application

For Settlement:

- existing calculated `eligibleAt` values remain;
- new qualifying Orders use the new hold period.

For automatic processing:

- changing `automaticSettlementEnabled` may affect currently eligible but unprocessed Settlements after the setting takes effect;
- it must never reprocess already successful payouts.

22.63 Settlement Configuration and Module 25

Module 22 stores configuration.

Module 25 owns:

- Settlement creation;
- eligibility;
- hold records;
- net Settlement calculation;

- Seller payout accounts;
- payout creation;
- payout processing;
- payout confirmation;
- failed payout retry;
- adjustments;
- Seller payout history;
- reconciliation.

The responsibilities must not be duplicated.

Example:

Module 22:

settlementHoldDays = 2

Module 25:

Order completed

↓

Read settlementHoldDays

↓

eligibleAt calculated

22.64 Frontend Implementation Guide

Admin frontend should:

- group settings by category;
- include Settlement and Payout sections;
- display descriptions and operational impact;
- display current values;
- validate inputs;
- require confirmation for sensitive financial changes;
- restrict settings using Permissions;
- clearly distinguish public/private settings;
- show active/inactive status where applicable;
- show success/error toasts;
- refresh cached configuration after updates.

Frontend should not:

- expose provider secrets;

- determine Settlement eligibility;
- alter existing Seller Settlements directly through System Settings;
- automatically recalculate historical transactions.

22.65 Backend Implementation Guide

Backend should:

- centralize configurable operational values;
- validate every setting;
- separate public/private configuration;
- secure sensitive configuration;
- enforce Admin Permissions;
- Audit all configuration changes;
- invalidate caches;
- preserve historical financial snapshots;
- preserve previously calculated Settlement eligibility where required;
- expose configuration consistently to internal services;
- provide safe defaults;
- enforce financial safety boundaries;
- prevent unsafe combinations.

22.66 Configuration Dependency Validation

Some settings depend on others.

Example:

If:

`automaticSettlementEnabled = true`

backend should verify:

`settlementProcessingEnabled = true`

and appropriate payout processing configuration exists.

If:

`payoutApprovalMode = maker_checker`

Admin architecture should support separate approval/processing Permissions.

If no valid payout provider/manual workflow exists:

`automaticSettlementEnabled`

should not be enabled.

22.67 Safe Default Configuration

Recommended initial values for a controlled launch:

```
{
  "settlementHoldDays": 2,
  "automaticSettlementEnabled": false,
  "settlementProcessingEnabled": true,
  "settlementRequiresVerifiedSeller": true,
  "settlementRequiresVerifiedPayoutAccount": true,
  "payoutApprovalMode": "maker_checker",
  "payoutRetryEnabled": true
}
```

`minimumPayoutAmounts` and `maximumAutomaticPayoutAmount` may remain disabled until TOFA defines those policies.

22.68 Important Consistency Rules

- System Settings does not replace Marketplace module business logic.
- Historical financial transactions must not be recalculated when settings change.
- Subscription entitlements remain owned by Subscription Management.
- Payment processing remains owned by Payment Management.
- Delivery processing remains owned by Delivery Management.
- Dispute lifecycle remains owned by Dispute Management.
- Cancellation/Return/Refund lifecycle remains owned by Module 20.
- Settlement/Payout lifecycle remains owned by Module 25.
- Permission management remains owned by Module 19.
- Audit remains owned by Module 21.
- Settlement settings configure Module 25 but do not perform Settlement actions.
- `settlementHoldDays` should normally apply prospectively.
- Existing Settlement `eligibleAt` should not be casually recalculated.
- Automatic Settlement must never bypass active holds.

- Payout processing must never bypass payout account verification where required.
- Automatic payout limits should be currency-specific where used.
- High-risk financial settings require stronger Admin protection.
- Provider secrets must not be stored as ordinary System Settings.
- Configuration changes must be auditable.

22.69 Module Summary

The System Settings & Marketplace Configuration Module establishes that:

- TOFA Marketplace operational configuration is centrally managed;
- authorized Admins can manage supported settings;
- countries are dynamically configurable;
- currencies are dynamically configurable;
- languages are dynamically configurable;
- Payment method availability is configurable;
- Return and Dispute windows are configurable;
- RFQ operational defaults are configurable;
- file and security operational limits can be configured;
- Seller Settlement configuration is centrally managed;
- `settlementHoldDays` controls the standard waiting period before payout eligibility;
- automatic Seller Settlement can be enabled or disabled;
- Seller payout processing can be temporarily enabled/disabled;
- verified Seller and verified payout-account requirements can be configured;
- payout approval policy can support single approval or maker-checker controls;
- optional minimum and automatic payout limits can be configured by currency;
- Settlement processing days/times may be configured if TOFA adopts scheduled payouts;
- financial configuration changes apply according to defined scope;
- historical transaction and payout data remain unchanged;
- settings are validated;
- unsafe settings can be blocked by backend safety rules;
- public and private configuration are separated;
- provider secrets remain outside ordinary configuration storage;
- configuration changes are audited;
- Module 19 controls Admin Permissions;
- Module 21 records configuration Audit history;
- Module 25 consumes Settlement/Payout configuration while retaining ownership of the financial lifecycle.

MODULE 23. REFERRAL & REWARD MANAGEMENT

23. REFERRAL & REWARD MANAGEMENT MODULE

23.1 Overview

The Referral & Reward Management Module manages TOFA Marketplace referrals and User reward points.

Module 1 already introduced the following User fields:

referral

referralCode

totalPoints

This module defines the complete lifecycle behind those fields.

It covers:

- User referral codes;
- referral attribution;
- referral relationships;
- referral qualification;
- reward points;
- reward rules;
- reward transactions;
- reward history;
- Admin reward configuration;
- fraud/abuse prevention;
- reward reversals;
- referral Analytics;
- Notifications.

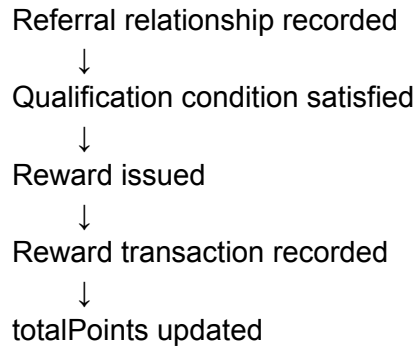
The core relationship is:

User A shares referral code

↓

User B signs up using code

↓



23.2 Core Principles

Referral and Reward Management must follow these principles:

- every User receives a unique referral code;
- referral attribution occurs during eligible signup/onboarding;
- Users cannot refer themselves;
- referral relationships must not be repeatedly changed after qualification;
- rewards are issued only when configured qualification conditions are met;
- every point increase/decrease must have a transaction record;
- `totalPoints` is a derived/current balance;
- reward history must remain immutable;
- reward values must not be hardcoded throughout the application;
- reward rules should be configurable;
- fraud and duplicate reward claims must be prevented;
- reversed business events may reverse previously awarded points where policy requires it.

23.3 User Referral Code

Each User receives:

`referralCode`

during account creation.

Example:

VIC83AF2

Requirements:

- unique;
- non-sensitive;
- safe to share;
- case-insensitive when entered where appropriate;
- cannot expose internal User ID directly.

The code should normally remain stable.

23.4 Referral Relationship

The existing Module 1:

referral

field may store the referral code used at signup for simple reference.

However, a dedicated Referral table is required for proper lifecycle tracking.

Table: referrals

Field	Type	Required	Notes
id	uuid	Yes	Primary key
referrerId	uuid	Yes	User who referred
referredUserId	uuid	Yes	New User
referralCode	varchar	Yes	Code used
status	enum	Yes	Referral lifecycle
qualifiedAt	timestamp	No	Qualification date
rewardedAt	timestamp	No	Reward date
createdAt	timestamp	Yes	Creation
updatedAt	timestamp	Yes	Last update

23.5 Referral Status

Recommended:

pending

qualified

rewarded

invalid

pending

Referral relationship exists but reward conditions have not yet been satisfied.

qualified

Referral has met required qualification conditions.

rewarded

Applicable referral reward has been issued.

invalid

Referral was invalidated due to:

- self-referral;
- duplicate/abusive account;
- fraudulent activity;
- failed eligibility;
- administrative investigation.

23.6 Signup Referral Processing

Existing signup payload remains:

```
{  
  "email": "user@email.com",  
  "password": "Password123",  
  "referralCode": "ABC123",  
  "termsOfUse": true  
}
```

`referralCode` remains optional.

Backend Processing

If referral code is supplied:

1. normalize code;
2. retrieve referrer;
3. validate code;
4. prevent self-referral where identifiable;
5. create User;
6. create Referral record;
7. set:

status = pending

8. store referral code used on User record where existing Module 1 structure requires it.

Signup must not fail simply because an optional referral code is invalid unless TOFA deliberately chooses strict validation.

Recommended behaviour:

Return clear validation before account creation or allow User to remove invalid code.

23.7 Referral Qualification

TOFA should not necessarily reward a referral merely because an email address was entered.

Recommended qualification events can include:

email_verified

company_verified

first_successful_trade

TOFA may configure which event qualifies a referral.

For a B2B Marketplace, a stronger qualification event such as:

company_verified

or:

first_successful_trade

is generally more resistant to referral abuse than signup alone.

The exact rule should be configurable.

23.8 Referral Qualification Setting

Recommended configuration:

referralQualificationEvent

Example:

company_verified

Referral Management listens for the configured business event.

Example:

Referred User becomes company verified

↓

Referral found

↓

status = qualified

↓

Reward rule evaluated

↓

Points awarded

23.9 Reward Rule Structure

Rewards should not be hardcoded into frontend/backend business flows.

Table: **reward_rules**

Field	Type	Required	Notes
id	uuid	Yes	Primary key

eventCode	varchar	Yes	Reward-triggering event
name	varchar	Yes	Display name
description	text	No	Explanation
points	integer	Yes	Points awarded
status	enum	Yes	active / inactive
maxPerUser	integer	No	Optional lifetime cap
maxPerPeriod	integer	No	Optional period cap
periodType	enum	No	day / week / month / lifetime
createdBy	uuid	Yes	Admin
createdAt	timestamp	Yes	Creation
updatedAt	timestamp	Yes	Last update

23.10 Initial Reward Events

Based on the reward examples introduced in Module 1, initial events may include:

REVIEW_SUBMITTED

SUCCESSFUL_TRADE

REFERRAL_QUALIFIED

Example initial values may be:

Event	Example Points
Leave eligible Review	+10
Successful Trade	+20
Qualified Referral	+30

These values are examples/configuration, not permanent hardcoded rules.

23.11 Reward Transaction Table

Every points movement must create a transaction.

Table: **reward_transactions**

Field	Type	Required	Notes
id	uuid	Yes	Primary key
userId	uuid	Yes	Reward owner
eventCode	varchar	Yes	Trigger
transactionType	enum	Yes	credit / debit
points	integer	Yes	Positive absolute value
balanceBefore	integer	Yes	Previous balance
balanceAfter	integer	Yes	New balance
sourceType	varchar	No	review / order / referral / admin / other
sourceId	uuid	No	Related entity
description	text	Yes	User-readable explanation
idempotencyKey	varchar	Yes	Prevent duplicate award
createdBy	uuid	No	Admin for manual action
createdAt	timestamp	Yes	Creation

Reward transactions should not normally be edited or deleted.

23.12 Reward Balance

The User record may continue storing:

totalPoints

for fast access.

However:

reward_transactions

is the authoritative transaction history.

The balance must reconcile with transaction history.

Example:

Starting balance = 0

Review = +10

Successful Trade = +20

Referral = +30

totalPoints = 60

23.13 Idempotency

The same event must not award points twice.

Example:

Order order_123 completed

Idempotency key:

SUCCESSFUL_TRADE:order_123:user_123

If the event is delivered twice, only one Reward transaction is created.

23.14 Successful Trade Reward

Reward should only be issued when the existing Order Module reaches the configured qualifying state.

Recommended:

Order completed

rather than:

Order created

Example flow:

Seller Order completed

↓

Reward Service receives event

↓

Reward Rule retrieved

↓

Eligibility checked

↓

Reward transaction created

↓

totalPoints updated

↓

Notification sent

The exact beneficiary can be configured if TOFA intends to reward:

- Buyer;
- Seller;
- both.

23.15 Review Reward

Review Management remains responsible for Review eligibility and submission.

Reward Management only consumes the eligible Review event.

Points should not be awarded repeatedly for:

- editing the same Review;
- deleting and recreating an equivalent Review;
- duplicate Review attempts.

Recommended idempotency source:

reviewId

23.16 Referral Reward

When a Referral reaches the configured qualification state:

Referral
↓
qualified
↓
REFERRAL_QUALIFIED
↓
Reward transaction
↓
Referrer receives points
↓
Referral status = rewarded

Depending on future policy, the referred User may also receive points.

If so, this should be represented as a separate Reward rule/transaction.

23.17 Get My Referral Information

Endpoint

GET /referrals/me

Authentication

User JWT required.

Response

```
{  
  "success": true,  
  
  "data": {  
    "referralCode": "VIC83AF2",  
  
    "referralLink": "https://marketplace.example/signup?ref=VIC83AF2",  
  
    "statistics": {  
      "totalReferrals": 12,  
      "pending": 3,  
    }  
  }  
}
```

```
    "qualified": 2,  
    "rewarded": 7  
  }  
}  
}
```

The production Marketplace domain should generate the actual referral link.

23.18 Get My Referrals

Endpoint

GET /referrals

Query Parameters

status
page
limit

Response

```
{  
  "success": true,  
  
  "data": [  
    {  
      "id": "ref_123",  
  
      "referredUser": {  
        "id": "usr_200",  
        "displayName": "ABC Agro Ltd"  
      },  
  
      "status": "rewarded",  
  
      "qualifiedAt": "2026-08-20T10:00:00Z",  
  
      "rewardedAt": "2026-08-20T10:01:00Z",  
  
      "createdAt": "2026-08-10T08:00:00Z"  
    }  
  ]  
}
```

```
],  
  
  "pagination": {  
    "page": 1,  
    "limit": 20,  
    "total": 12,  
    "totalPages": 1  
  }  
}
```

Do not expose unnecessary personal information about referred Users.

23.19 Get Reward Balance

Endpoint

GET /rewards/balance

Response

```
{  
  "success": true,  
  
  "data": {  
    "totalPoints": 120  
  }  
}
```

23.20 Get Reward History

Endpoint

GET /rewards/history

Query Parameters

transactionType
eventCode
dateFrom
dateTo
page

limit

Response

```
{
  "success": true,

  "data": [
    {
      "id": "reward_txn_123",
      "eventCode": "REFERRAL_QUALIFIED",
      "transactionType": "credit",
      "points": 30,
      "balanceBefore": 90,
      "balanceAfter": 120,
      "description": "Reward for a qualified referral.",
      "createdAt": "2026-08-20T10:01:00Z"
    }
  ],

  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 15,
    "totalPages": 1
  }
}
```

23.21 Reward Reversal

Some previously awarded points may need to be reversed.

Examples:

- fraudulent referral discovered;
- qualifying transaction reversed;
- eligible event invalidated.

Do not delete the original Reward transaction.

Create a debit transaction.

Example:

Original Reward
+30

Reversal:

-30

Transaction:

```
{  
  "eventCode": "REFERRAL_REWARD_REVERSED",  
  "transactionType": "debit",  
  "points": 30,  
  "sourceType": "referral",  
  "sourceId": "ref_123",  
  "description": "Referral reward reversed."  
}
```

23.22 Negative Reward Balance

Recommended rule:

totalPoints should not normally fall below 0

If a reversal exceeds available points because points have already been redeemed in a future reward-redemption system, TOFA will need a separate liability/negative-balance policy.

Until Reward Redemption is introduced, prevent negative balances.

23.23 Manual Admin Reward Adjustment

Authorized Admins may adjust points for exceptional operational reasons.

Endpoint

POST /admin/rewards/adjust

Required Permission

rewards.adjust

Payload

```
{
  "userId": "usr_123",
  "transactionType": "credit",
  "points": 50,
  "reason": "Customer service reward adjustment."
}
```

Backend must create a Reward transaction.

Never directly change:

users.totalPoints

without a corresponding transaction.

23.24 Manual Adjustment Response

```
{
  "success": true,

  "message": "Reward points adjusted successfully.",

  "data": {
    "userId": "usr_123",
    "points": 50,
    "transactionType": "credit",
    "newBalance": 170
  }
}
```

Toast

Reward points adjusted successfully.

23.25 Admin Reward Rules

Get Rules

GET /admin/reward-rules

Required Permission:

rewards.rules.view

Create Rule

POST /admin/reward-rules

Required Permission:

rewards.rules.manage

Payload

```
{  
  "eventCode": "REFERRAL_QUALIFIED",  
  "name": "Qualified Referral",  
  "description": "Points awarded when an eligible referral qualifies.",  
  "points": 30,  
  "maxPerPeriod": 20,  
  "periodType": "month"  
}
```

23.26 Update Reward Rule

Endpoint

PATCH /admin/reward-rules/:ruleId

Required Permission

rewards.rules.manage

Payload

```
{  
  "points": 40,  
  "status": "active"  
}
```

}

Changing a Reward rule applies prospectively.

Previous Reward transactions remain unchanged.

23.27 Fraud Prevention

Referral abuse prevention should consider:

- self-referrals;
- repeated accounts;
- suspicious account patterns;
- repeated business registration information;
- duplicate verified businesses;
- referral farming;
- automated account creation.

The exact fraud system may evolve separately.

Referral Management should support:

status = invalid

where a Referral is confirmed invalid.

23.28 Referral Attribution Rule

Recommended:

first valid referral attribution wins

Once a User has a valid Referral relationship, they should not repeatedly replace the referrer.

This prevents referral hijacking.

Admin correction may be permitted before qualification where there is a genuine operational error.

Such correction must be audited.

23.29 Reward Notification

In-App Notification

Title:

You've earned reward points

Message:

You earned {Points} TOFA points for {RewardReason}. Your new balance is {TotalPoints} points.

23.30 Referral Reward Email

Subject

You've Earned TOFA Reward Points

Body

Hello {FirstName},

Good news! You have earned {Points} TOFA reward points for a qualified referral.

Your current reward balance is:

{TotalPoints} points

You can view your referral and reward activity from your TOFA Marketplace account.

Best Regards,

Traders of Africa Team

23.31 Reward Redemption

This module currently defines earning and tracking points.

It does not assume that points automatically represent money.

Until TOFA defines a redemption model:

1 point $\neq$ fixed monetary value

unless explicitly configured later.

Future Reward Redemption may support:

- transaction discounts;
- Subscription discounts;
- promotional benefits;
- marketplace benefits.

Redemption should not be invented before TOFA approves the commercial model.

23.32 Admin Get Referral Analytics

Endpoint

GET /admin/referrals/analytics

Required Permission

referrals.analytics.view

Response

```
{
  "success": true,

  "data": {
    "totalReferrals": 1250,
    "pendingReferrals": 220,
    "qualifiedReferrals": 810,
    "rewardedReferrals": 780,
    "invalidReferrals": 30,
    "totalReferralPointsIssued": 23400
  }
}
```

Detailed business Analytics may also be consumed by Module 17.

23.33 Admin Get Referrals

Endpoint

GET /admin/referrals

Required Permission

referrals.view

Query Parameters

search
status
referrerId
referredUserId
dateFrom
dateTo
page
limit

23.34 Admin Get Reward Transactions

Endpoint

GET /admin/rewards/transactions

Required Permission

rewards.view

Query Parameters

userId
eventCode
transactionType
sourceType
dateFrom
dateTo
page
limit

23.35 Admin Permissions

Recommended:

referrals.view

referrals.manage

referrals.analytics.view

rewards.view

rewards.adjust

rewards.rules.view

rewards.rules.manage

These should be added to Module 19's Permission registry.

23.36 Audit Events

Recommended:

REFERRAL_CREATED

REFERRAL_QUALIFIED

REFERRAL_REWARDED

REFERRAL_INVALIDATED

REFERRAL_CORRECTED

REWARD_POINTS_CREDITED

REWARD_POINTS_DEBITED

REWARD_POINTS_REVERSED

REWARD_RULE_CREATED

REWARD_RULE_UPDATED

REWARD_RULE_ACTIVATED

REWARD_RULE_DEACTIVATED

ADMIN_REWARD_ADJUSTMENT

Module 21 stores the Audit records.

23.37 Analytics Relationship

Module 17 may consume:

- total Referrals;
- Referral conversion rate;
- qualified Referrals;
- Referral-generated Buyers;
- Referral-generated Sellers;
- Referral-generated transactions;
- total points issued;
- reward activity.

Referral Management remains the source of truth for Referral relationships and Reward transactions.

23.38 Review Relationship

Review Management determines whether a Review is valid and eligible.

Referral & Reward Management does not create Reviews.

It consumes the qualifying event and awards points according to the active Reward rule.

23.39 Order Relationship

Order Management determines successful/completed trade.

Reward Management does not determine whether an Order is successful.

It consumes the completed Order event and applies the Reward rule.

If an Order is later financially reversed and TOFA policy requires reward reversal, a debit Reward transaction is created.

23.40 Seller Verification Relationship

If Referral qualification uses:

company_verified

Seller Verification emits the qualifying event.

Referral Management updates the Referral.

It does not perform company verification itself.

23.41 Notification Relationship

Reward and Referral Notifications use the existing Notification Module.

Referral Management defines business events.

Notification Management handles delivery.

23.42 Frontend Implementation Guide

Frontend should:

- show User referral code;
- provide copy referral code;
- provide shareable referral link;
- display Referral statistics;
- display Referral status;
- display Reward balance;
- display Reward history;
- clearly distinguish credits and debits;
- show points earned after eligible actions;
- provide Admin Reward rule management where authorized;
- provide Admin Reward adjustments where authorized.

Frontend should not:

- calculate authoritative point balance;
- award points locally;
- determine Referral qualification;
- allow Referral attribution to be arbitrarily changed.

23.43 Backend Implementation Guide

Backend should:

- generate unique Referral codes;
- validate Referral codes;
- prevent self-referral;
- create Referral relationships;
- prevent duplicate attribution;
- listen for qualifying events;
- use configurable Reward rules;
- create immutable Reward transactions;
- update `totalPoints` transactionally;
- use idempotency keys;
- prevent duplicate rewards;
- support Reward reversals;
- prevent unauthorized manual adjustments;
- emit Notifications;
- emit Audit events;
- provide reconciliation between Reward transactions and `totalPoints`.

23.44 Important Consistency Rules

- Every User receives a unique Referral code.
- Referral codes are shareable and non-sensitive.
- Referral code at signup remains optional.
- Users cannot refer themselves.
- Referral attribution should not be repeatedly changed.
- Referral qualification is separate from signup.
- Reward values are configurable.
- Reward transactions are the authoritative Reward history.
- `totalPoints` represents current balance.
- Every balance change requires a transaction.
- Duplicate events must not create duplicate Rewards.
- Previous Rewards are not edited when Reward rules change.
- Reversals use debit transactions.
- Points do not automatically equal money.
- Reward Redemption should not be assumed until commercially defined.
- Review Management owns Review eligibility.
- Order Management owns successful trade status.
- Seller Verification owns company verification.
- Module 19 controls Admin Permissions.
- Module 21 stores Audit history.

23.45 Module Summary

The Referral & Reward Management Module establishes that:

- every User has a unique Referral code;
- Users can invite other Marketplace Users;
- valid Referral relationships are recorded;
- Referrals move through pending, qualified, rewarded or invalid states;
- Referral qualification uses configurable business events;
- TOFA can reward eligible Marketplace behaviour;
- Reward rules are configurable;
- Review, successful trade and Referral qualification can trigger Rewards;
- every points movement creates a Reward transaction;
- `totalPoints` maintains the current User balance;
- duplicate Rewards are prevented through idempotency;
- fraudulent Referrals may be invalidated;
- previously awarded points may be reversed through debit transactions;
- authorized Admins may perform documented Reward adjustments;
- Users can view Referral information, Referral history, Reward balance and Reward history;
- Admins can manage Reward rules according to Permissions;
- Referral and Reward activity can feed Analytics;
- Notifications communicate Reward events;
- all important administrative and Reward actions are auditable.

MODULE 24. WISHLIST / SAVED ITEMS

24. WISHLIST / SAVED ITEMS MANAGEMENT MODULE

24.1 Overview

The Wishlist / Saved Items Management Module allows authenticated TOFA Marketplace Users to save Products they are interested in and return to them later.

For a B2B Marketplace, the preferred User-facing term may be:

Saved Products

rather than:

Wishlist

because Buyers may use the feature to shortlist Products for future sourcing, comparison or RFQ activity.

This module covers:

- saving Products;
- removing saved Products;
- viewing saved Products;
- checking whether a Product is saved;
- bulk removal;
- handling inactive/deleted Products;
- saved-item counts;
- optional Notifications;
- Analytics integration.

This module does not reserve inventory.

Saving a Product does not:

- create an Order;
- add the Product to Cart;
- create an RFQ;
- lock a price;

- lock inventory;
- guarantee availability.

24.2 Core Principles

Saved Products must follow these principles:

- User must be authenticated;
- a Product can only be saved once per User;
- saving a Product does not affect inventory;
- saving does not lock Product price;
- saving does not create a commercial commitment;
- Product Management remains the source of truth for Product data;
- inactive/deleted Products should not remain purchasable simply because they were saved;
- Sellers may also save Products because Sellers can buy from other Sellers.

Do not restrict this feature based on:

userType = buyer

because TOFA Users may participate as Buyers even when they are Sellers.

24.3 Saved Product Table

Table: **saved_products**

Field	Type	Required	Notes
id	uuid	Yes	Primary key
userId	uuid	Yes	User
productId	uuid	Yes	Saved Product
createdAt	timestamp	Yes	Date saved

Recommended unique constraint:

UNIQUE(userId, productId)

This prevents duplicate saves.

24.4 Save Product

Endpoint

POST /saved-products

Authentication

User JWT required.

Payload

```
{  
  "productId": "prod_123"  
}
```

24.5 Save Product Backend Processing

1. Authenticate User.
2. Retrieve Product.
3. Confirm Product exists.
4. Confirm Product is eligible to be viewed/saved.
5. Check whether Product is already saved.
6. If not, create Saved Product record.
7. Return success.

The endpoint should preferably be idempotent.

If the Product is already saved, it may simply return the existing saved state rather than generating an error.

24.6 Save Product Response

```
{  
  "success": true,  
  
  "message": "Product saved successfully.",  
  
  "data": {  
    "productId": "prod_123",  
    "isSaved": true  
  }  
}
```

Toast

Product saved successfully.

24.7 Remove Saved Product

Endpoint

DELETE /saved-products/:productId

Authentication

User JWT required.

Backend Processing

1. Authenticate User.
2. Find User/Product relationship.
3. Delete Saved Product relationship.
4. Return success.

Removing a Saved Product does not affect the Product itself.

24.8 Remove Saved Product Response

```
{
  "success": true,

  "message": "Product removed from saved items.",

  "data": {
    "productId": "prod_123",
    "isSaved": false
  }
}
```

Toast

Product removed from saved items.

24.9 Get Saved Products

Endpoint

GET /saved-products

Authentication

User JWT required.

Query Parameters

Existing Product search/filter concepts may be reused where useful:

search

categoryId

sellerId

countryOfOrigin

minPrice

maxPrice

currency

availability

sortBy

page

limit

Do not create a separate Product search architecture specifically for Saved Products.

24.10 Get Saved Products Response

```
{  
  "success": true,
```



```
"data": [  
  {  
    "id": "saved_123",  
  
    "savedAt": "2026-08-20T10:00:00Z",  
  
    "product": {  
      "id": "prod_123",  
  
      "name": "Premium Nigerian Cocoa Beans",  
  
      "slug": "premium-nigerian-cocoa-beans",  
  
      "productType": "simple",  
  
      "price": 2500,  
  
      "currency": "USD",  
  
      "origin": "Nigeria",  
  
      "status": "active",  
  
      "inventoryStatus": "in_stock",  
  
      "primaryImage": {  
        "url": "https://cdn.example.com/product.jpg"  
      },  
  
      "seller": {  
        "id": "seller_123",  
        "storeName": "ABC Agro Export"  
      },  
  
      "rating": {  
        "average": 4.7,  
        "count": 25  
      }  
    }  
  }  
],  
  
"pagination": {  
  "page": 1,
```

```
"limit": 20,  
"total": 15,  
"totalPages": 1  
}  
}
```

Product information should be retrieved from current Product/Seller data rather than copied permanently into `saved_products`.

24.11 Saved Product Price

Saved Products do not lock price.

Example:

Product saved:
USD 2,500

Seller later updates Product:

USD 2,700

Saved Products should display:

USD 2,700

because the Product has not yet become a transaction.

Historical price locking happens later through the appropriate Cart/Checkout/RFQ/Order process.

24.12 Saved Product Inventory

Saving does not reserve inventory.

Example:

Buyer saves Product

↓

Product inventory = 100

↓

Other Buyers purchase Product

↓

Inventory = 0

Saved Product may then display:

Out of Stock

The User does not receive reserved inventory.

24.13 Saved Variable Product

For variable Products, default saved behaviour should save the Product itself rather than locking a particular variant unless TOFA explicitly wants variant-level saving.

Recommended v1:

userId + productId

When the User later opens the Product, they select the current available variant.

This keeps Saved Products simple and avoids stale variant references.

24.14 Check Saved Status

Product APIs may include:

```
{  
  "isSaved": true  
}
```

when an authenticated User requests Product details.

Alternatively:

Endpoint

GET /saved-products/:productId/status

Response

```
{
  "success": true,

  "data": {
    "productId": "prod_123",
    "isSaved": true
  }
}
```

For frontend efficiency, including **isSaved** in authenticated Product list/detail responses is preferred where practical.

24.15 Product List Integration

Authenticated Product list responses may include:

```
{
  "id": "prod_123",
  "name": "Premium Nigerian Cocoa Beans",
  "isSaved": true
}
```

This allows frontend to render:

Saved icon state

without making one API request per Product.

Avoid N+1 API requests.

24.16 Saved Product Count

Endpoint

GET /saved-products/count

Response

```
{
  "success": true,
```

```
"data": {  
  "count": 15  
}  
}
```

This may be used in User navigation.

24.17 Clear Saved Products

Users may remove all Saved Products.

Endpoint

DELETE /saved-products

Response

```
{  
  "success": true,  
  "message": "All saved products have been removed."  
}
```

Confirmation

Frontend should confirm:

Remove all saved products?

before calling the endpoint.

24.18 Bulk Remove Saved Products

Optional endpoint:

DELETE /saved-products/bulk

Payload

```
{  
  "productIds": [  

```

```
"prod_123",  
"prod_456"  
]  
}
```

Response

```
{  
  "success": true,  
  "message": "Selected products removed from saved items."  
}
```

24.19 Deleted Product Behaviour

If Product status becomes:

deleted

it should no longer be displayed as an active purchasable Saved Product.

Recommended behaviour:

- keep the relationship temporarily where useful for cleanup/history;
- exclude deleted Products from normal Saved Product results;
- periodically clean orphaned Saved Product relationships where appropriate.

Saved Product history is not financial history and does not require indefinite retention.

24.20 Archived Product Behaviour

If Product becomes:

archived

it should not be purchasable.

Frontend may either:

- hide it from normal Saved Product results; or
- display it as unavailable.

Recommended B2B UX:

Display it temporarily as:

Currently unavailable

where the Product still exists and may return.

24.21 Inactive Product Behaviour

If Product becomes:

inactive

Saved Product may display:

Currently unavailable

The User cannot proceed to Cart/RFQ while the Product is not eligible for purchase.

If Product becomes active again, the saved relationship remains useful.

24.22 Out-of-Stock Product Behaviour

Out-of-stock Products may remain in Saved Products.

Display:

Out of Stock

Do not automatically remove them.

This allows Buyers to revisit them if inventory returns.

24.23 Seller Account Restriction

If the Seller becomes ineligible to trade:

- Saved Product remains non-purchasable;
- Product eligibility rules from Product/Search/User Verification modules apply;
- Saved Products must not bypass Seller restrictions.

24.24 Add Saved Product to Cart

Saved Product Management does not create a new Cart lifecycle.

Frontend may provide:

Add to Cart

which calls the existing Cart endpoint.

Example:

Saved Product

↓

Add to Cart

↓

Cart Module

Cart Module validates:

- Product status;
- Seller eligibility;
- price;
- inventory;
- selected variant;
- quantity.

Saved Product Management does not perform those validations itself.

24.25 Create RFQ from Saved Product

Frontend may provide:

Request Quote

on a Saved Product.

This uses the existing Direct RFQ flow.

Example:

Saved Product

↓
Request Quote
↓
Direct RFQ

Saved Product Management does not create its own RFQ endpoint.

24.26 Share Saved Product

Users may share the normal Product URL.

No separate Saved Product sharing architecture is required.

The recipient sees the Product according to normal Product visibility rules.

24.27 Saved Product Ownership

Saved Products are private to the User.

A Seller must not be able to see:

which specific Users saved my Product

unless TOFA deliberately introduces such a feature later with appropriate privacy consideration.

Seller Analytics may receive aggregate metrics such as:

Product saves = 125

without exposing individual Users.

24.28 Product Saved Count

The platform may calculate:

savedCount

for Product Analytics.

Example:

```
{  
  "productId": "prod_123",  
  "savedCount": 125  
}
```

This is an engagement metric.

It must not be treated as:

- purchase;
- RFQ;
- conversion;
- inventory reservation.

24.29 Analytics Relationship

Analytics may consume:

PRODUCT_SAVED

PRODUCT_UNSAVED

events.

Possible Seller Product Analytics:

Product Views

Product Saves

RFQs

Orders

Conversion

Saved Product Management remains the source of truth for current save relationships.

Analytics remains responsible for aggregated reporting.

24.30 Search Relationship

Search may optionally use aggregate save popularity as a weak ranking signal.

Example:

Product with meaningful Buyer engagement

may receive a small quality signal.

However:

- save count must never override relevance;
- save count must not override Product eligibility;
- save count must not override Seller eligibility;
- Subscription priority rules remain as defined in Search Management.

24.31 Notifications

Basic Save/Unsave actions do not require emails.

Do not send an email every time a User saves a Product.

In-app toast is sufficient.

Potential future Notifications may include:

Saved Product back in stock

Saved Product price changed

Saved Product becoming unavailable

These should only be introduced if TOFA deliberately enables Saved Product alerts.

24.32 Optional Saved Product Alerts

If TOFA later enables alerts, User preference should be respected.

Potential table:

Table: **saved_product_preferences**

Field	Type	Required
-------	------	----------

id	uuid	Yes
userId	uuid	Yes
productId	uuid	Yes
priceChangeAlert	boolean	Yes
backInStockAlert	boolean	Yes
createdAt	timestamp	Yes
updatedAt	timestamp	Yes

This is optional and does not need to be implemented in the initial version.

24.33 Product Back-in-Stock Notification

If future alert enabled:

Subject

A Product You Saved Is Back in Stock

Body

Hello {FirstName},

A Product you saved on Traders of Africa Marketplace is available again.

Product: {ProductName}

Current Price: {Currency} {Price}

You can view the Product from your TOFA Marketplace account.

Please note that Product availability may change based on Seller inventory.

Best Regards,
Traders of Africa Team

This email is optional and should not be part of initial Save behaviour unless enabled.

24.34 Saved Product and Authentication

Unauthenticated visitors may view public Products but cannot persist Saved Products to a TOFA account.

Recommended frontend behaviour:

User clicks Save

↓

Not authenticated

↓

Prompt Login/Register

↓

Authentication completed

↓

Return to Product

↓

Save Product

This can follow the same authentication return-flow pattern used elsewhere in the Marketplace.

24.35 User Account Deletion

When User status becomes:

deleted

Saved Product relationships may be removed according to data-retention policy because they are non-financial User preference data.

They must not prevent account deletion.

24.36 No Admin Management Requirement

Saved Products generally do not require an Admin CRUD interface.

Admins should not normally:

- add Products to a User's Saved Products;
- remove Products from a User's Saved Products;
- view individual private Saved Product lists.

Operational Analytics may use aggregate save data.

This keeps the feature simple and privacy-conscious.

24.37 Audit Requirements

Normal Save/Unsave activity does not need to create high-cost central Audit records.

These are ordinary engagement events.

They may instead emit Analytics events:

PRODUCT_SAVED

PRODUCT_UNSAVED

Central Audit may only be needed for exceptional administrative/system actions if introduced later.

24.38 Frontend Implementation Guide

Frontend should:

- display Save icon/button on eligible Products;
- show saved state;
- support Save from Product list;
- support Save from Product detail;
- provide Saved Products page;
- show current Product information;
- show current price;
- show availability;
- show Seller;
- allow removal;
- allow clear all;
- allow Add to Cart using Cart Module;
- allow Request Quote using RFQ Module;
- handle inactive/archived Products clearly;
- show Out of Stock without removing the Product;
- prompt authentication for unauthenticated Save attempts;
- avoid one status API call per Product.

24.39 Backend Implementation Guide

Backend should:

- require authentication;
- validate Product existence;
- enforce unique User/Product relationship;
- make Save operation idempotent;
- provide paginated Saved Products;
- join current Product information;
- enforce current Product/Seller visibility;
- include saved state efficiently in authenticated Product responses;
- support remove and clear operations;
- emit lightweight Analytics events;
- avoid unnecessary central Audit events;
- prevent Saved Products from bypassing Product eligibility.

24.40 Important Consistency Rules

- Sellers can save Products because Sellers can also act as Buyers.
- Saving does not reserve inventory.
- Saving does not lock price.
- Saving does not create an Order.
- Saving does not create an RFQ.
- Saving does not add to Cart automatically.
- One User cannot save the same Product multiple times.
- Current Product data is displayed.
- Current Product eligibility applies.
- Inactive Products cannot be purchased through Saved Products.
- Out-of-stock Products may remain saved.
- Product Management owns Product status.
- Cart owns Cart validation.
- RFQ owns Quote requests.
- Search owns ranking.
- Analytics owns aggregated engagement reporting.
- Individual Saved Product lists should remain private.
- Normal Save/Unsave actions do not require email.
- Normal Save/Unsave actions do not require central Audit Logs.

24.41 Module Summary

The Wishlist / Saved Items Management Module establishes that:

- authenticated Users can save Products for later;
- both Buyers and Sellers can use Saved Products;

- each Product can only be saved once per User;
- Users can remove individual Saved Products;
- Users can clear Saved Products;
- Saved Product lists are paginated;
- Product information remains current;
- Product prices are not locked when saved;
- inventory is not reserved;
- inactive and archived Products cannot bypass Product eligibility;
- out-of-stock Products may remain saved;
- Users can proceed from Saved Products to existing Cart or RFQ flows;
- Saved Products do not create separate Cart/RFQ business logic;
- unauthenticated Users are prompted to authenticate before saving;
- Seller Analytics may receive aggregate save metrics;
- Sellers should not receive individual User Saved Product lists;
- normal Save/Unsave actions require only lightweight frontend feedback;
- future price-change/back-in-stock alerts may be added separately if TOFA chooses;
- the feature remains intentionally lightweight and does not interfere with transactional Marketplace lifecycles.

MODULE 25. SELLER SETTLEMENT & PAYOUT

25. SELLER SETTLEMENT & PAYOUT MANAGEMENT MODULE

25.1 Overview

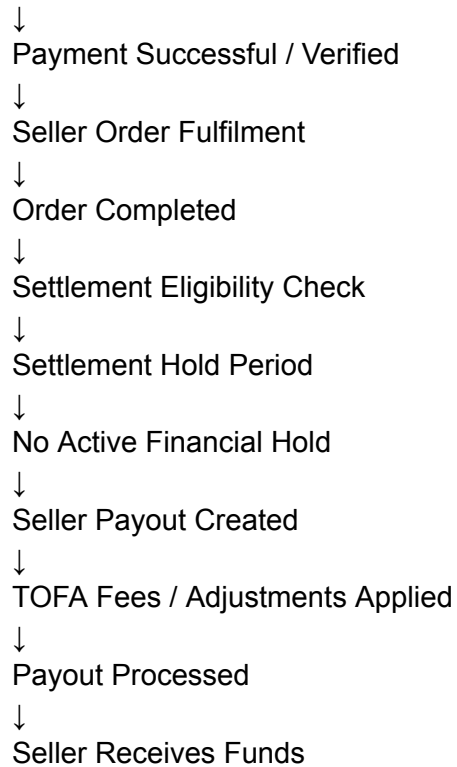
The Seller Settlement & Payout Management Module manages how TOFA releases eligible transaction funds to Sellers after a Buyer has paid successfully and the Seller has fulfilled the applicable transaction obligations.

This module covers:

- Seller settlement eligibility;
- Seller payout account management;
- payout amount calculation;
- settlement hold periods;
- multi-seller payment allocation;
- transaction fee deduction;
- logistics amount treatment;
- Refund and Dispute holds;
- full and partial settlement;
- settlement adjustments;
- payout creation;
- payout approval;
- payout processing;
- manual payout confirmation;
- automatic payout processing;
- failed payout handling;
- payout retries;
- Seller payout history;
- Admin settlement management;
- reconciliation;
- cross-currency settlement;
- historical FX preservation;
- Notifications;
- Audit Logs;
- Analytics integration.

The relationship is:

Buyer Payment



This module does not replace the Payment Module.

The Payment Module remains responsible for:

Buyer → TOFA

financial collection.

This module is responsible for:

TOFA → Seller

settlement.

25.2 Core Principles

Seller settlement must follow these principles:

- successful Buyer payment does not automatically mean immediate Seller payout;
- TOFA must confirm Seller Order eligibility before settlement;
- settlement should operate primarily at Seller Order level;

- multi-seller transactions must be settled independently;
- Seller payout amount must be calculated from stored transaction snapshots;
- transaction fees must not be recalculated using current Subscription rules;
- FX must not be recalculated using current exchange rates;
- Refunds and financial adjustments must reduce the amount payable where applicable;
- active Disputes may place settlement on hold;
- failed payouts must not be marked successful;
- every payout must have a traceable lifecycle;
- historical financial records must never be overwritten;
- Admin settlement actions must be permission-controlled and audited.

25.3 Settlement Terminology

Settlement

Settlement is the process of determining what amount TOFA owes a Seller for an eligible Seller Order.

Payout

Payout is the actual transfer of the approved settlement amount to the Seller.

Example:

Seller Order becomes eligible

↓

Settlement calculated

↓

Payout created

↓

Money transferred

Settlement Hold

A Settlement Hold temporarily prevents payout.

Examples:

- active Dispute;
- Refund being processed;
- fraud review;
- compliance hold;
- incomplete delivery;

- incorrect Seller bank details.

25.4 Settlement Boundary

Settlement should primarily occur at:

sellerOrderId

level.

Example:

Checkout

↓

Single Buyer Payment

↓

Seller Order A

Seller Order B

Seller Order C

Each Seller Order must have an independent settlement lifecycle.

Seller A should not wait for Seller C if Seller A has already fulfilled all settlement conditions.

25.5 Multi-Seller Example

Buyer pays:

NGN 30,000,000

Allocation:

Seller A = NGN 12,000,000

Seller B = NGN 10,000,000

Seller C = NGN 8,000,000

If:

Seller A Order = completed

Seller B Order = dispute open

Seller C Order = still in transit

then:

Seller A → eligible for settlement

Seller B → settlement on hold

Seller C → not yet eligible

The original Buyer Payment remains:

NGN 30,000,000

25.6 Settlement Eligibility

Recommended default rule:

A Seller Order becomes eligible for settlement when:

1. the associated Buyer Payment is successful or verified;
2. the Seller Order has reached the required fulfilment state;
3. the Order has reached the configured completion condition;
4. the settlement hold period has elapsed;
5. there is no active Dispute that blocks settlement;
6. there is no pending Refund that affects the payable amount;
7. there is no fraud/compliance hold;
8. the Seller has valid payout details;
9. the Seller remains eligible to receive settlement.

25.7 Recommended Default Settlement Point

Recommended standard flow:

Buyer pays

↓

Payment successful

↓

Seller fulfils



Goods delivered



Order completed



Settlement hold period



Eligibility check



Payout

The system should not normally settle immediately when:

Payment successful

because the Seller has not yet completed fulfilment obligations.

25.8 Settlement Hold Period

Recommended System Setting:

settlementHoldDays

Example:

2

Flow:

Order completed on Monday



settlementHoldDays = 2



Eligible Wednesday

The value should be configurable through Module 22.

It must not be hardcoded in multiple services.

25.9 Settlement Hold Configuration

Recommended settings:

settlementHoldDays

settlementProcessingEnabled

automaticSettlementEnabled

minimumPayoutAmount

Potential additional settings:

settlementProcessingDays

settlementProcessingTime

if TOFA later processes payouts only on certain banking days.

25.10 Order Completion and Settlement

Order Management remains responsible for defining:

completed

Settlement Management consumes that event.

It must not independently set Orders as completed simply to make them payable.

Example:

ORDER_COMPLETED

↓

Settlement Service

↓

Calculate eligibleAt

25.11 Delivery Method Considerations

Settlement eligibility may depend on Delivery method.

Integrated Logistics

TOFA can use integrated delivery status and confirmation.

Example:

Delivered
↓
Order completed
↓
Settlement hold

Seller-Arranged Logistics

Use the Seller Order delivery information and Buyer/Order completion process.

Buyer-Arranged Logistics

TOFA may not control the logistics provider.

The Seller may have fulfilled their responsibility once goods are handed over according to the agreed transaction terms.

Settlement must follow the existing Order completion rules rather than inventing a separate logistics rule.

25.12 Seller Payout Account

A Seller must have valid settlement details before payout.

Table: **seller_payout_accounts**

Field	Type	Required	Notes
id	uuid	Yes	Primary key
sellerId	uuid	Yes	Seller
accountType	enum	Yes	bank_account / other_supported_type
accountName	varchar	Yes	Beneficiary

accountNumberEncrypted	text	Yes	Securely stored
bankCode	varchar	No	Bank identifier
bankName	varchar	Yes	Bank name
country	varchar	Yes	Account country
currency	varchar	Yes	Settlement currency
providerRecipientId	varchar	No	External provider reference
isDefault	boolean	Yes	Default payout account
status	enum	Yes	pending / verified / invalid / inactive
verifiedAt	timestamp	No	Verification
createdAt	timestamp	Yes	Creation
updatedAt	timestamp	Yes	Last update

Sensitive banking information must be protected.

25.13 Payout Account Status

Recommended:

pending

verified

invalid

inactive

Only:

verified

accounts should normally receive automatic payout.

25.14 Add Seller Payout Account

Endpoint

POST /seller/payout-accounts

Payload

```
{
  "accountType": "bank_account",
  "accountName": "ABC Agro Limited",
  "accountNumber": "0123456789",
  "bankCode": "058",
  "bankName": "Guaranty Trust Bank",
  "country": "NG",
  "currency": "NGN",
  "isDefault": true
}
```

25.15 Backend Processing

1. Authenticate Seller.
2. Confirm Seller account eligibility.
3. Validate payout details.
4. Validate supported country/currency.
5. Verify bank account where provider supports verification.
6. Encrypt sensitive data.
7. Create account.
8. Set status.
9. Emit Audit event.
10. Notify Seller where appropriate.

25.16 Add Payout Account Response

```
{
  "success": true,
  "message": "Payout account added successfully.",
  "data": {
    "id": "payout_account_123",
    "bankName": "Guaranty Trust Bank",
    "maskedAccountNumber": "*****6789",
    "currency": "NGN",
  }
}
```

```
    "status": "verified",
    "isDefault": true
  }
}
```

Toast

Payout account added successfully.

25.17 Get Seller Payout Accounts

Endpoint

GET /seller/payout-accounts

Response

```
{
  "success": true,
  "data": [
    {
      "id": "payout_account_123",
      "accountName": "ABC Agro Limited",
      "maskedAccountNumber": "*****6789",
      "bankName": "Guaranty Trust Bank",
      "country": "NG",
      "currency": "NGN",
      "status": "verified",
      "isDefault": true
    }
  ]
}
```

Never return the full account number unnecessarily.

25.18 Update Payout Account

Endpoint

PATCH /seller/payout-accounts/:payoutAccountId

Bank account changes may require re-verification.

Sensitive payout account changes should not immediately affect a payout already being processed.

25.19 Set Default Payout Account

Endpoint

PATCH /seller/payout-accounts/:payoutAccountId/default

Backend must ensure only one default account per relevant Seller/currency where applicable.

25.20 Settlement Record

Recommended table:

Table: **seller_settlements**

Field	Type	Required	Notes
id	uuid	Yes	Primary key
settlementNumber	varchar	Yes	Unique
sellerId	uuid	Yes	Seller
orderId	uuid	Yes	Parent Order
sellerOrderId	uuid	Yes	Settlement boundary
paymentId	uuid	Yes	Original Buyer Payment
status	enum	Yes	Settlement lifecycle
grossProductAmount	decimal	Yes	Product/commercial amount
logisticsAmount	decimal	Yes	Seller-owned eligible logistics amount
transactionFeePercentage	decimal	Yes	Historical snapshot
transactionFeeAmount	decimal	Yes	Historical fee
refundAmount	decimal	Yes	Refund deduction

adjustmentAmount	decimal	Yes	Other net adjustments
netSettlementAmount	decimal	Yes	Amount payable
settlementCurrency	varchar	Yes	Seller settlement currency
eligibleAt	timestamp	No	When payout can begin
holdReason	text	No	If on hold
payoutId	uuid	No	Linked payout
createdAt	timestamp	Yes	Creation
updatedAt	timestamp	Yes	Last update

Example:

SET-2026-000123

25.21 Settlement Status

Recommended statuses:

pending

on_hold

eligible

processing

settled

cancelled

pending

Settlement record exists but Order has not yet met payout eligibility conditions.

on_hold

Settlement would otherwise progress but is blocked.

eligible

All required conditions are satisfied.

processing

A payout is being processed.

settled

Seller payout has been successfully completed.

cancelled

Settlement record was cancelled because the underlying payable obligation no longer exists.

25.22 Settlement Amount Calculation

Recommended:

Net Settlement

=

Gross Product Amount

+

Seller-Owned Logistics Amount

-

Transaction Fee

-

Refunds

+

/-

Settlement Adjustments

Example:

Gross Product Amount = NGN 10,000,000

Seller-Owned Logistics = NGN 500,000

Transaction Fee = NGN 300,000

Refund Deduction = NGN 0

Adjustment = NGN 0

Net Settlement:

NGN 10,200,000

25.23 Transaction Fee Snapshot

The transaction fee must come from the stored transaction snapshot.

Existing Order/financial record may contain:

subscriptionId

subscriptionPlanId

transactionFeePercentage

feeBaseAmount

transactionFeeAmount

sellerNetProductAmount

Settlement must use these stored values.

Do not perform:

Current Seller Subscription

↓

Current transaction fee

↓

Recalculate old Order

That would be incorrect.

25.24 Logistics Settlement Treatment

Logistics amount belongs to the Seller only where the commercial structure says so.

Integrated Logistics

If TOFA separately pays the logistics provider, the logistics amount should not automatically be included in Seller payout.

Seller-Arranged Logistics

If:

Buyer paid logistics to TOFA

and:

Seller is responsible for arranging delivery

then applicable logistics amount may form part of Seller settlement.

Buyer-Arranged Logistics

If Buyer pays logistics externally:

Seller Settlement Logistics Amount = 0

unless some separate charge is payable through TOFA.

25.25 Cross-Currency Settlement

Example:

Buyer pays USD 10,000

Seller agreed settlement currency = NGN

Historical FX snapshot:

USD 1 = NGN 1,600

Seller commercial equivalent:

NGN 16,000,000

Settlement must use the locked transaction FX snapshot.

If current FX becomes:

USD 1 = NGN 1,700

the old Seller Order must not be recalculated.

25.26 Settlement Currency

Recommended:

Seller settlement currency = listing / accepted quote commercial currency

unless the transaction contract explicitly defines another settlement currency.

Buyer payment currency may differ.

Example:

Buyer Payment Currency = USD

Seller Settlement Currency = NGN

25.27 Settlement Creation

Settlement record should normally be created automatically when the Seller Order financial obligation becomes known.

Recommended point:

Order successfully created after payment terms are locked

or:

Payment becomes successful

depending on architecture.

Recommended approach:

Create the Settlement record once Buyer Payment becomes successful/verified and the Seller Order financial snapshot is final.

Initial status:

pending

25.28 Settlement Eligibility Processing

When the Order becomes completed:

1. retrieve Settlement;
2. calculate:

$\text{eligibleAt} =$
 $\text{orderCompletedAt} + \text{settlementHoldDays}$

3. check Disputes;
4. check Refunds;
5. check financial holds;
6. check Seller payout account;
7. move to:

eligible

when all conditions are satisfied.

25.29 Active Dispute Hold

If an active Dispute exists for the Seller Order:

$\text{settlement.status} = \text{on_hold}$

Recommended hold reason:

active_dispute

The hold should generally affect only the associated Seller Order.

25.30 Dispute Resolution and Settlement

If Dispute resolves in Seller's favour:

Dispute resolved

↓

No Refund required

↓

Hold removed

↓

Settlement recalculated if necessary

↓

Eligible

If Dispute results in partial Refund:

Original Seller Amount

↓

Refund adjustment

↓

Reduced Settlement

If Dispute results in full Refund:

Net settlement may become 0

If no payable amount remains, Settlement may be:

cancelled

with reason.

25.31 Return Hold

If an approved Return is in progress before settlement:

settlement.status = on_hold

until Return outcome is known.

This prevents TOFA from settling the Seller and then immediately needing to recover the same funds.

25.32 Refund Before Seller Settlement

If Refund occurs before payout:

Example:

Original Net Seller Settlement = NGN 9,700,000

Refund impact = NGN 2,000,000

Transaction fee reversal = NGN 60,000

Settlement is recalculated using stored adjustments.

Do not overwrite original financial records.

25.33 Refund After Seller Settlement

If Seller has already been paid and a later Refund is approved:

Do not modify the completed payout.

Create:

settlement adjustment

or:

seller receivable

Example:

Seller already received = NGN 9,700,000

Later Seller liability = NGN 1,940,000

Create a negative settlement adjustment.

That amount may be deducted from a future Seller payout according to TOFA policy.

25.34 Settlement Adjustment Table

Table: **seller_settlement_adjustments**

Field	Type	Required
id	uuid	Yes
sellerId	uuid	Yes
settlementId	uuid	No
sellerOrderId	uuid	Yes
adjustmentType	enum	Yes
amount	decimal	Yes
currency	varchar	Yes
direction	enum	Yes
sourceType	varchar	Yes
sourceId	uuid	No
reason	text	Yes
status	enum	Yes
createdAt	timestamp	Yes

Possible **adjustmentType**:

refund_recovery

transaction_fee_reversal

manual_credit

manual_debit

dispute_adjustment

correction

Direction:

credit

debit

25.35 Future Payout Deduction

If Seller owes TOFA due to post-settlement Refund:

Example:

Seller next settlement = NGN 5,000,000

Outstanding debit = NGN 1,000,000

New payout:

NGN 4,000,000

The original payout history remains unchanged.

25.36 Negative Seller Balance

The system may maintain:

seller settlement balance

or derive it from unsettled adjustments.

If Seller owes TOFA more than the next payout:

Next payout = NGN 500,000

Outstanding debit = NGN 1,000,000

Then:

Payout amount = 0

Remaining Seller liability = NGN 500,000

Do not create a negative bank payout.

25.37 Payout Record

Table: **seller_payouts**

Field	Type	Required	Notes
id	uuid	Yes	Primary key
payoutNumber	varchar	Yes	Unique
sellerId	uuid	Yes	Seller
settlementId	uuid	Yes	Settlement
payoutAccountId	uuid	Yes	Destination account
amount	decimal	Yes	Payout amount
currency	varchar	Yes	Currency
status	enum	Yes	Payout lifecycle
method	varchar	Yes	bank_transfer / provider
provider	varchar	No	Payout provider
providerReference	varchar	No	External reference
approvedBy	uuid	No	Admin
approvedAt	timestamp	No	Approval
processingAt	timestamp	No	Processing start
completedAt	timestamp	No	Success
failedAt	timestamp	No	Failure

failureReason	text	No	Failure
createdAt	timestamp	Yes	Creation
updatedAt	timestamp	Yes	Last update

Example:

PAYOUT-2026-000123

25.38 Payout Status

Recommended:

pending

approved

processing

successful

failed

cancelled

pending

Payout created but not yet approved/processed.

approved

Authorized to be processed.

processing

Transfer request submitted.

successful

Financial institution/provider confirms payout.

failed

Transfer failed.

cancelled

Payout cancelled before successful processing.

25.39 Settlement vs Payout Status

These must remain separate.

Example:

Settlement = eligible

Payout = pending

Later:

Settlement = processing

Payout = processing

Finally:

Settlement = settled

Payout = successful

25.40 Automatic vs Manual Payout

TOFA may support:

automatic payout

and/or:

manual finance-approved payout

The setting may be:

automaticSettlementEnabled

For initial Marketplace rollout, a controlled Admin approval workflow may be safer.

25.41 Create Payout Automatically

When Settlement becomes eligible:

Eligible Settlement

↓

Validate Payout Account

↓

Calculate final amount

↓

Create Payout

If automatic processing is disabled:

status = pending

Finance/Admin reviews it.

25.42 Admin Get Settlements

Endpoint

GET /admin/settlements

Required Permission

settlements.view

Query Parameters

search

status

sellerId

orderId

sellerOrderId

currency
eligibleFrom
eligibleTo
dateFrom
dateTo
page
limit

25.43 Admin Settlement List Response

```
{
  "success": true,
  "data": [
    {
      "id": "settlement_123",
      "settlementNumber": "SET-2026-000123",
      "seller": {
        "id": "seller_123",
        "storeName": "ABC Agro Limited"
      },
      "sellerOrderId": "seller_order_123",
      "grossProductAmount": 10000000,
      "transactionFeeAmount": 300000,
      "refundAmount": 0,
      "adjustmentAmount": 0,
      "netSettlementAmount": 9700000,
      "settlementCurrency": "NGN",
      "status": "eligible",
      "eligibleAt": "2026-08-30T12:00:00Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 215,
    "totalPages": 11
  }
}
```

25.44 Admin Get Settlement Details

Endpoint

GET /admin/settlements/:settlementId

Required Permission

settlements.view

Response

```
{
  "success": true,
  "data": {
    "id": "settlement_123",
    "settlementNumber": "SET-2026-000123",

    "seller": {
      "id": "seller_123",
      "storeName": "ABC Agro Limited"
    },

    "order": {
      "orderId": "order_123",
      "sellerOrderId": "seller_order_123"
    },

    "payment": {
      "paymentId": "payment_123",
      "status": "successful",
      "buyerPaymentCurrency": "USD"
    },

    "financials": {
      "grossProductAmount": 10000000,
      "logisticsAmount": 500000,
      "transactionFeePercentage": 3,
      "transactionFeeAmount": 300000,
      "refundAmount": 0,
      "adjustmentAmount": 0,
      "netSettlementAmount": 10200000,
      "currency": "NGN"
    },

    "status": "eligible",
```

```
"eligibleAt": "2026-08-30T12:00:00Z",
"holdReason": null,

"payout": null
}
}
```

25.45 Seller Get Settlements

Endpoint

GET /seller/settlements

Query Parameters

status
dateFrom
dateTo
currency
page
limit

Response

```
{
  "success": true,
  "data": [
    {
      "settlementNumber": "SET-2026-000123",
      "sellerOrderId": "seller_order_123",
      "grossProductAmount": 10000000,
      "transactionFeeAmount": 300000,
      "adjustments": 0,
      "netSettlementAmount": 9700000,
      "currency": "NGN",
      "status": "settled",
      "eligibleAt": "2026-08-30T12:00:00Z"
    }
  ]
}
```

25.46 Seller Get Settlement Details

Endpoint

GET /seller/settlements/:settlementId

Seller can only retrieve their own Settlement.

25.47 Seller Payout History

Endpoint

GET /seller/payouts

Query Parameters

status
currency
dateFrom
dateTo
page
limit

Response

```
{
  "success": true,
  "data": [
    {
      "payoutNumber": "PAYOUT-2026-000123",
      "settlementNumber": "SET-2026-000123",
      "amount": 9700000,
      "currency": "NGN",
      "status": "successful",
      "bankName": "Guaranty Trust Bank",
      "maskedAccountNumber": "*****6789",
      "completedAt": "2026-08-30T15:00:00Z"
    }
  ]
}
```

25.48 Admin Get Payouts

Endpoint

GET /admin/payouts

Required Permission

payouts.view

Query Parameters

search
status
sellerId
currency
provider
dateFrom
dateTo
page
limit

25.49 Approve Payout

Endpoint

POST /admin/payouts/:payoutId/approve

Required Permission

payouts.approve

Backend must revalidate:

- Settlement remains eligible;
- no new Dispute exists;
- no new Refund affects amount;
- payout account is valid;
- amount is still correct;
- payout has not already been processed.

Response

```
{
  "success": true,
  "message": "Seller payout approved successfully.",
  "data": {
    "payoutId": "payout_123",
    "status": "approved"
  }
}
```

Toast

Seller payout approved successfully.

25.50 Process Payout

Endpoint

POST /admin/payouts/:payoutId/process

Required Permission

payouts.process

Backend:

1. retrieve Payout;
2. validate state;
3. validate Settlement;
4. validate payout destination;
5. generate idempotency key;
6. submit transfer to provider/bank workflow;
7. store provider reference;
8. set:

status = processing

9. set Settlement:

status = processing

10. await financial confirmation.

25.51 Automatic Provider Payout

Flow:

TOFA Backend
↓
Payout Provider
↓
Transfer Request
↓
Provider Reference
↓
Webhook / Status Confirmation
↓
Payout successful

Provider webhooks must be verified.

Duplicate webhook delivery must not create duplicate payout effects.

25.52 Manual Payout

If settlement is processed manually by Finance:

Payout may move:

pending
↓
approved
↓
processing

Finance performs the transfer externally.

Only after actual transfer confirmation should the payout become:

successful

25.53 Confirm Manual Payout

Endpoint

PATCH /admin/payouts/:payoutId/confirm

Required Permission

payouts.confirm

Payload

```
{  
  "reference": "BANK-TRF-938483",  
  "notes": "Seller settlement transferred successfully."  
}
```

Backend:

payout.status = successful

settlement.status = settled

and records completion time.

25.54 Payout Failure

If provider transfer fails:

payout.status = failed

Store:

failedAt

failureReason

Settlement should normally return to:

eligible

unless another hold exists.

25.55 Retry Failed Payout

Endpoint

POST /admin/payouts/:payoutId/retry

Required Permission

payouts.process

Backend must:

- check provider status first where appropriate;
- ensure previous attempt did not actually succeed;
- use idempotency;
- avoid duplicate Seller payment.

25.56 Prevent Duplicate Payout

Only one successful payout should exist for a Settlement.

Recommended database/business constraints should prevent:

Settlement SET-123

↓

Payout A successful

↓

Payout B also successful

This is a critical financial control.

25.57 Settlement Hold Endpoint

Endpoint

POST /admin/settlements/:settlementId/hold

Required Permission

settlements.hold

Payload

```
{  
  "reason": "Compliance review required."  
}
```

Backend sets:

status = on_hold

and records reason.

25.58 Release Settlement Hold

Endpoint

POST /admin/settlements/:settlementId/release

Required Permission

settlements.hold

Backend revalidates eligibility before moving to:

eligible

Do not blindly release without checking current Dispute/Refund/financial conditions.

25.59 Automatic Hold Sources

The system may automatically place hold for:

active_dispute

approved_return

pending_refund

fraud_review

seller_compliance_issue

invalid_payout_account

payment_reversal

Store machine-readable hold code and optional description.

25.60 Settlement Hold Structure

Recommended fields:

holdCode

holdReason

heldAt

heldBy

releasedAt

releasedBy

For complex multiple holds, a dedicated table is preferable.

Table: **settlement_holds**

Field	Type	Required
id	uuid	Yes
settlementId	uuid	Yes
holdCode	varchar	Yes
reason	text	No
sourceType	varchar	Yes
sourceId	uuid	No
status	enum	Yes
heldBy	uuid	No

heldAt timestamp Yes

releasedBy uuid No

releasedAt timestamp No

Status:

active

released

Settlement becomes eligible only when no blocking active hold remains.

25.61 Partial Settlement

TOFA may eventually support partial settlement.

However, recommended initial implementation:

One Seller Order

↓

One final net settlement

↓

One payout

unless business needs require partial payout.

This significantly simplifies reconciliation.

Partial Refunds can still reduce the final net amount before payout.

25.62 Settlement Batching

Seller settlements may later be grouped into one payout.

Example:

Settlement A = NGN 2m

Settlement B = NGN 3m

Settlement C = NGN 1m

One payout:

NGN 6m

If TOFA adopts this model, use:

Table: payout_settlements

payoutId

settlementId

allocatedAmount

For initial implementation, one payout per settlement is simpler and more transparent.

25.63 Recommended v1 Settlement Model

Recommended v1:

1 Seller Order

=

1 Settlement

=

1 Payout

This makes:

- reconciliation easier;
- Refund tracing easier;
- Dispute tracing easier;
- Seller statements clearer;
- Admin investigation simpler.

Batch settlement can be introduced later without changing underlying Settlement records.

25.64 Minimum Payout Amount

If configured:

minimumPayoutAmount

Seller settlements below the minimum may remain eligible until enough payable value accumulates.

This feature is only necessary if transaction/payment fees make very small payouts inefficient.

If not operationally needed, do not enable it initially.

25.65 Seller Deactivation

If Seller becomes inactive/disabled after completing legitimate transactions:

Settlement must not automatically disappear.

TOFA must determine whether:

- settlement can proceed;
- payout requires compliance review;
- funds must be held.

A Seller account restriction should normally create:

compliance hold

rather than deleting Settlement records.

25.66 Seller Verification Status

Only properly eligible Sellers should receive payouts.

If verification expires or compliance information requires review:

Settlement may be placed on hold

according to Marketplace policy.

Seller Verification Module remains the owner of verification status.

25.67 Refund Interaction

Module 20 creates Refund records.

Settlement Management consumes successful/pending Refund impact.

Before payout:

Refund reduces net settlement

After payout:

Refund creates Seller debit/adjustment

Do not create another Refund lifecycle here.

25.68 Dispute Interaction

Dispute Management owns Dispute lifecycle.

Settlement Management only reacts to relevant Dispute states.

Example:

Dispute created

↓

Settlement hold

Then:

Dispute resolved

↓

Settlement outcome recalculated

25.69 Payment Interaction

Payment Management owns:

Buyer Payment status

Settlement Management must never create Seller Settlement for an unpaid Buyer transaction unless TOFA explicitly assumes credit risk.

Default:

Payment successful/verified required

25.70 Subscription Interaction

Subscription Management determines Seller transaction fee at transaction creation.

Settlement uses stored:

transactionFeePercentage

transactionFeeAmount

Subscription changes later do not affect old Settlements.

25.71 Analytics Interaction

Analytics may consume:

- gross Seller payable;
- pending Settlements;
- on-hold Settlements;
- eligible Settlements;
- successful payouts;
- failed payouts;
- total amount settled;
- settlement turnaround time;
- Seller outstanding balances;
- settlement adjustments;
- transaction fee deducted;
- Refund recovery;
- payout success rate.

25.72 Seller Analytics

Seller dashboard may show:

Pending Settlement

On Hold

Available for Payout

Paid

Adjustments

Example:

```
{
  "pendingSettlement": {
    "amount": 12000000,
    "currency": "NGN"
  },
  "onHold": {
    "amount": 3000000,
    "currency": "NGN"
  },
  "settled": {
    "amount": 55000000,
    "currency": "NGN"
  }
}
```

Do not add amounts from different currencies without grouping or normalized reporting.

25.73 Admin Settlement Summary

Endpoint

GET /admin/settlements/summary

Required Permission

settlements.view

Response

```
{
  "success": true,
  "data": {
    "pending": {
      "count": 120,
      "byCurrency": {
```

```
    "NGN": 45000000,  
    "USD": 120000  
  },  
  
  "onHold": {  
    "count": 15,  
    "byCurrency": {  
      "NGN": 8000000  
    }  
  },  
  
  "eligible": {  
    "count": 70,  
    "byCurrency": {  
      "NGN": 22000000  
    }  
  },  
  
  "settled": {  
    "count": 540,  
    "byCurrency": {  
      "NGN": 350000000,  
      "USD": 450000  
    }  
  }  
}  
}
```

25.74 Payout Successful Email

Subject

Your TOFA Marketplace Settlement Has Been Paid

Body

Hello {SellerName},

Your TOFA Marketplace settlement has been processed successfully.

Settlement Number: {SettlementNumber}

Payout Number: {PayoutNumber}

Order Number: {OrderNumber}

Amount Paid: {Currency} {Amount}

Destination Account: {MaskedAccountNumber}

You can view the full payout details from your TOFA Marketplace account.

Best Regards,
Traders of Africa Team

25.75 Payout Processing Notification

In-App Notification

Title:
Seller payout processing

Message:
Your payout of {Currency} {Amount} for Settlement {SettlementNumber} is being processed.

25.76 Payout Successful Notification

Title:
Settlement paid

Message:
Your payout of {Currency} {Amount} has been completed successfully.

25.77 Payout Failed Email

Subject

Update on Your TOFA Marketplace Settlement

Body

Hello {SellerName},

We encountered an issue while processing your TOFA Marketplace payout.

Settlement Number: {SettlementNumber}

Payout Number: {PayoutNumber}

Amount: {Currency} {Amount}

The payout has not been marked as completed.

Our team will review the payout and take the appropriate next step.

Best Regards,
Traders of Africa Team

25.78 Settlement Hold Notification

Where appropriate:

Subject

Update on Your TOFA Marketplace Settlement

Body

Hello {SellerName},

Your settlement for Order {OrderNumber} is currently on hold.

Settlement Number: {SettlementNumber}

Reason: {HoldReason}

You can view the latest status from your TOFA Marketplace account.

If additional action is required from you, the instructions will be displayed in your account.

Best Regards,
Traders of Africa Team

Some internal compliance holds may require less detailed public explanation.

25.79 Reconciliation

Settlement and payout records must be periodically reconciled with:

- Buyer Payments;
- Seller Orders;
- Refunds;
- transaction fees;
- settlement adjustments;
- provider transfer records;
- bank transfer records.

Examples of reconciliation checks:

successful payout
but
settlement not marked settled

settlement marked settled
but
no successful payout

payout amount
≠
net settlement amount

successful refund after payout
but
no Seller adjustment created

These should be flagged.

25.80 Payout Statement

Seller should be able to view a clear financial breakdown.

Endpoint

GET /seller/payouts/:payoutId/statement

Response

{


```
"success": true,
"data": {
  "payoutNumber": "PAYOUT-2026-000123",
  "settlementNumber": "SET-2026-000123",

  "order": {
    "orderNumber": "ORD-2026-000500"
  },

  "breakdown": {
    "grossProductAmount": 10000000,
    "sellerLogisticsAmount": 500000,
    "transactionFee": 300000,
    "refundDeduction": 0,
    "otherAdjustments": 0,
    "netPayout": 10200000
  },

  "currency": "NGN",

  "destination": {
    "bankName": "Guaranty Trust Bank",
    "maskedAccountNumber": "*****6789"
  },

  "status": "successful",
  "completedAt": "2026-08-30T15:00:00Z"
}
}
```

25.81 Payout Export

Seller may export their payout history where useful.

Admin may export settlement/payout reports.

Recommended reuse of Reporting infrastructure:

POST /reports/export

Possible **reportType**:

seller_settlements

seller_payouts

admin_settlements

admin_payouts

Do not create duplicate report generation infrastructure.

25.82 Admin Manual Adjustment

Endpoint

POST /admin/settlements/:settlementId/adjustments

Required Permission

settlements.adjust

Payload

```
{
  "adjustmentType": "manual_credit",
  "amount": 50000,
  "currency": "NGN",
  "reason": "Approved operational correction."
}
```

The Admin must not directly overwrite:

netSettlementAmount

without creating an adjustment record.

25.83 Adjustment Validation

Backend must verify:

- currency compatibility;

- amount > 0;
- Settlement state;
- Admin Permission;
- adjustment reason;
- no invalid overpayment;
- whether payout already occurred.

If payout already occurred, adjustment should affect future Seller balance rather than rewrite the completed payout.

25.84 Admin Permissions

Recommended permissions:

settlements.view

settlements.hold

settlements.adjust

payouts.view

payouts.approve

payouts.process

payouts.confirm

Potential:

payout_accounts.view

for authorized Finance/Operations staff.

These permissions should be added to Module 19.

25.85 Separation of Duties

For stronger financial control, TOFA may configure:

Admin A approves payout

Admin B confirms/processes payout

This is recommended for large-value transactions if operationally practical.

The permission model already supports this.

Example:

Finance Reviewer:

payouts.view

payouts.approve

Finance Processor:

payouts.view

payouts.process

payouts.confirm

25.86 Audit Events

Recommended:

SELLER_PAYOUT_ACCOUNT_CREATED

SELLER_PAYOUT_ACCOUNT_UPDATED

SELLER_PAYOUT_ACCOUNT_VERIFIED

SETTLEMENT_CREATED

SETTLEMENT_BECAME_ELIGIBLE

SETTLEMENT_PLACED_ON_HOLD

SETTLEMENT_HOLD_RELEASED

SETTLEMENT_ADJUSTMENT_CREATED

PAYOUT_CREATED

PAYOUT_APPROVED

PAYOUT_PROCESSING_STARTED

PAYOUT_SUCCESSFUL

PAYOUT_FAILED

PAYOUT_RETRIED

MANUAL_PAYOUT_CONFIRMED

SELLER_DEBIT_CREATED

SELLER_CREDIT_CREATED

Module 21 stores centralized Audit history.

25.87 Financial Audit Example

```
{
  "eventCode": "PAYOUT_APPROVED",

  "actorId": "admin_finance_123",

  "actorType": "admin",

  "module": "settlements",

  "action": "approve_payout",

  "entityType": "payout",

  "entityId": "payout_123",

  "description": "Seller payout was approved.",

  "metadata": {
    "settlementId": "settlement_123",
    "sellerId": "seller_123",
    "amount": 9700000,
    "currency": "NGN"
  }
}
```

25.88 Notifications

Recommended events:

SETTLEMENT_ELIGIBLE

SETTLEMENT_ON_HOLD

SETTLEMENT_HOLD_RELEASED

PAYOUT_CREATED

PAYOUT_PROCESSING

PAYOUT_SUCCESSFUL

PAYOUT_FAILED

Notification Module handles delivery.

25.89 Security Requirements

Settlement is a high-risk financial module.

Backend must implement:

- strict Admin Permissions;
- secure Seller payout account storage;
- masked account responses;
- bank account verification where available;
- idempotent payout processing;
- provider webhook verification;
- payout reference uniqueness;
- duplicate payout prevention;
- sensitive action Audit;
- rate limiting for sensitive endpoints;
- revalidation before payout approval/processing;
- secure provider credential management;
- no frontend-authoritative financial calculation.

25.90 Frontend Seller Implementation Guide

Seller frontend should show:

- payout account management;
- masked bank account details;
- account verification status;
- pending settlement;
- on-hold settlement;
- eligible settlement;
- paid settlements;
- payout history;
- payout statement;
- transaction fee deduction;
- Refund/adjustment deductions where relevant;
- clear status explanations.

Seller must not be able to:

- mark Settlement eligible;
- approve Payout;
- mark Payout successful;
- edit fee calculations;
- override adjustments.

25.91 Frontend Admin Implementation Guide

Admin frontend should provide:

- Settlement list;
- Settlement detail;
- filters;
- on-hold queue;
- eligible queue;
- Payout queue;
- Payout detail;
- Seller account details in masked format;
- approve action;
- process action;
- manual confirmation where authorized;
- failed payout retry;
- Settlement adjustments;
- Audit timeline;
- financial breakdown.

Sensitive actions should use confirmation modals.

Example:

Approve payout of NGN 9,700,000 to ABC Agro Limited?

25.92 Backend Implementation Guide

Backend should:

- create Settlement records from authoritative transaction events;
- operate at Seller Order level;
- calculate amounts from historical financial snapshots;
- preserve historical FX;
- use configurable hold periods;
- automatically create/remove system holds;
- react to Dispute/Return/Refund events;
- validate Seller payout account;
- create Payout records;
- process payouts idempotently;
- prevent duplicate payouts;
- support automatic/manual processing;
- support failed payout retries;
- create post-settlement adjustments instead of mutating history;
- reconcile financial records;
- emit Audit events;
- emit Notification events;
- expose Seller/Admin reporting endpoints;
- enforce Permissions.

25.93 Important Consistency Rules

- Buyer Payment and Seller Payout are different financial flows.
- Payment Module owns Buyer-to-TOFA collection.
- Settlement Module owns TOFA-to-Seller payout.
- Successful Buyer payment does not immediately pay the Seller.
- Settlement operates primarily per Seller Order.
- Multi-seller Orders are settled independently.
- Order Management owns Order completion.
- Settlement consumes Order completion.
- Historical transaction fees must not be recalculated.
- Historical FX must not be recalculated.
- Refunds before payout reduce Settlement.
- Refunds after payout create Seller adjustments/debits.

- Completed payouts must never be rewritten.
- Active Disputes may hold Settlement.
- Return workflows may hold Settlement.
- Seller-arranged logistics may be included only where payable to Seller.
- Buyer-arranged external logistics is not Seller settlement income through TOFA.
- Integrated logistics should not automatically be paid to Seller.
- Settlement holds must be explicit and traceable.
- A failed payout is not a successful payout.
- Manual payout requires actual transfer confirmation.
- One Settlement must never be paid twice.
- Seller payout account data must be secured.
- Admin actions require Permissions.
- Financial actions must be audited.
- Analytics consumes Settlement/Payout records but does not own them.

25.94 Module Summary

The Seller Settlement & Payout Management Module establishes that:

- TOFA collects Buyer funds before Seller settlement;
- Seller Orders are settled independently;
- successful Buyer payment does not cause immediate Seller payout;
- Seller settlement is dependent on fulfilment and Order completion;
- a configurable settlement hold period may apply;
- active Disputes, Returns, Refunds, compliance issues and payout account problems may hold Settlement;
- Seller payout amount is calculated from locked transaction snapshots;
- transaction fee deductions use the historical Subscription fee;
- historical FX rates are preserved;
- Seller-owned logistics may be included where applicable;
- integrated/external logistics is treated according to who actually receives the logistics funds;
- Refunds before payout reduce Settlement;
- Refunds after payout create Seller debit adjustments;
- financial history is never rewritten;
- Sellers maintain verified payout accounts;
- Settlements move through pending, on_hold, eligible, processing, settled or cancelled states;
- Payouts move through pending, approved, processing, successful, failed or cancelled states;
- authorized Admins can approve/process payouts;
- manual payouts require actual transfer confirmation;
- failed payouts can be safely retried;
- duplicate payouts must be prevented;

- Sellers can view Settlement and payout history;
- Sellers can view clear payout breakdowns;
- Admins can manage holds and financial adjustments;
- Settlement data feeds Analytics;
- Notifications communicate Settlement/Payout events;
- Module 21 captures the Audit history;
- Module 19 controls Settlement/Payout Admin Permissions;
- Module 22 provides configurable Settlement settings.